

Capitulo 1 - Database System Concepts

Introduction

A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise.

Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information.

Database-system application

The earliest database systems arose in the 1960s in response to the computerized management of commercial data.

Database systems are used to manage collections of data that:

- are highly valuable
- are relatively large
- are accessed by multiple users and applications, often at the same time.

Modern database systems exploit commonalities in the structure of data to gain efficiency but also allow for weakly structured data and for data whose formats are highly variable.

...

Capitulo 3 - Database System Concepts

Review Terms

- Data-definition language
- Data-manipulation language
- Database schema
- Database instance
- Relation schema
- Relation instance
- Primary key
- Foreign key
 - Referencing relation
 - Referenced relation
- Null value
- Query language
- SQL query structure
 - **select** clause
 - **from** clause
 - **where** clause
- Multiset relational algebra
- **as** clause
- **order by** clause
- Table alias
- Correlation name (correlation variable, tuple variable)
- Set operations
 - **union**
 - **intersect**
 - **except**
- Aggregate functions
 - **avg, min, max, sum, count**
 - **group by**
 - **having**
- Nested subqueries
- Set comparisons
 - {<, <=, >, >=} { **some, all** }
 - **exists**
 - **unique**
- **lateral** clause
- **with** clause
- Scalar subquery
- Database modification
 - Delete
 - Insert
 - Update

SQL

Data-definition language (DDL): The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.

Data-manipulation language (DML): The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.

We define an SQL relation by using the **create table** command. The following command creates a relation department in the database:

```
create table department  
(dept name varchar (20),  
  building varchar (15),  
  budget numeric (12,2),  
  primary key (dept name));
```

The general form of the create table command is:

```
create table r  
  (A1 D1 ,  
   A2 D2 ,  
   ...,  
   An Dn ,  
   <integrity-constraint1 >,  
   ...,  
   <integrity-constraintk >);
```

where r is the name of the relation, each A_i is the name of an attribute in the schema of relation r, and D_i is the domain of attribute A_i ; that is, D_i specifies the type of attribute A_i

SQL supports a number of different integrity constraints:

- **primary key** (A_{j1} , A_{j2} , ... , A_{jm}): The primary-key specification says that attributes A_{j1} , A_{j2} , ... , A_{jm} form the primary key for the relation. The primary-key attributes are required to be nonnull and unique.
- **foreign key** (A_{k1} , A_{k2} , ... , A_{kn}) references s: The foreign key specification says that the values of attributes (A_{k1} , A_{k2} , ... , A_{kn}) for any tuple in the relation must correspond to values of the primary key attributes of some tuple in relation s. The definition of the course table has a declaration “foreign key (dept name) references department”.
- **not null**: The not null constraint on an attribute specifies that the null value is not allowed for that attribute;

To remove a relation from an SQL database, we use the

drop table

command. This one is more drastic than

delete from r;

The latter retains relation r, but deletes all tuples in r. The former deletes not only all tuples of r, but also the schema for r.

We use the alter table command to add attributes to an existing relation. The form of the alter table command is **alter table r add A D;** where r is the name of an existing relation, A is the name of the attribute to be added, and D is the type of the added attribute.

We can drop attributes from a relation by the command **alter table r drop A;**

Operaciones

El select de toda la vida:

```
select dept name  
from instructor;
```

Select que elimina los duplicados:

```
select distinct dept name  
from instructor;
```

SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed:

```
select all dept name  
from instructor;
```

The select clause may also contain arithmetic expressions involving the operators +, −, *, and / operating on constants or attributes of tuples. For example, the query:

```
select ID, name, dept name, salary * 1.1  
from instructor;
```

returns a relation that is the same as the instructor relation, except that the attribute salary is multiplied by 1.1.

The **where** clause allows us to select only those rows in the result relation of the from clause that satisfy a specified predicate.

```
select name  
from instructor  
where dept name = 'Comp. Sci.' and salary > 70000;
```

- The **select** clause is used to list the attributes desired in the result of a query.
- The **from** clause is a list of the relations to be accessed in the evaluation of the query.

- The **where** clause is a predicate involving attributes of the relation in the from clause.

A typical SQL query has the form:

```
select A1 , A2 , ... , An
from r1 , r2 , ... , rm
where P;
```

Each A_i represents an attribute, and each r_i a relation. P is a predicate. If the where clause is omitted, the predicate P is true.

Hence, SQL provides a way of renaming the attributes of a result relation.

```
select name as instructor name, course id
from instructor, teaches
where instructor.ID= teaches.ID;
```

The **as** clause is particularly useful in **renaming relations**. One reason to rename a relation is to replace a long relation name with a shortened version that is more convenient to use elsewhere in the query.

```
select T.name, S.course id
from instructor as T, teaches as S
where T.ID= S.ID;
```

SQL also permits a variety of functions on character strings, such as:

- **concatenating** (using **"//"**)
- converting strings **to uppercase** (using the function **upper(s)**)
- converting strings **to lowercase** (using the function **lower(s)**)
- **removing spaces at the end** of the string (using **trim(s)**)

Pattern matching can be performed on strings using the operator like. We describe patterns by using two special characters:

- **Percent (%)**: The % character matches any substring.
- **Underscore (_)**: The _ character matches any character

Patterns are case sensitive; that is, uppercase characters do not match lowercase characters, or vice versa.

- 'Intro%' matches any string beginning with "Intro".
- '%Comp%' matches any string containing "Comp" as a substring, for example, 'Intro. to Computer Science', and 'Computational Biology'.
- '___' matches any string of exactly three characters.
- '___%' matches any string of at least three characters.

SQL expresses patterns by using the like comparison operator.

The escape character is used immediately before a special pattern character to indicate that the special pattern character is to be treated like a normal character.

like 'ab\%cd%' **escape** '\ ' matches all strings beginning with "ab%cd".

like 'ab\\cd%' **escape** '\ ' matches all strings beginning with "ab\cd".

The asterisk symbol " *" can be used in the select clause to denote "all attributes."

```
select instructor.*
from instructor, teaches
where instructor.ID= teaches.ID
order by name;
```

The **order by** clause causes the tuples in the result of a query to appear in sorted order. To specify the sort order, we may specify **desc** for descending order or **asc** for ascending order.

Between:

```
select name
from instructor
where salary between 90000 and 100000;
```

instead of:

```
select name
from instructor
where salary <= 100000 and salary >= 90000;
```

Operaciones con conjuntos

Union: (select ...) **union** (select ...);

If we want to retain all duplicates, we must write **union all** in place of union.

Intersección: (select ...) **intersect** (select ...);

If we want to retain all duplicates, we must write **intersect all** in place of intersect.

Resta: (select ...) **except** (select ...);

Once again... If we want to retain duplicates, we must write **except all** in place of except

null y unknown

SQL uses the special keyword **null** in a predicate to test for a null value.

```
select name  
from instructor  
where salary is null;
```

SQL allows us to test whether the result of a comparison is unknown, rather than true or false, by using the clauses **is unknown** and **is not unknown**.

```
select name  
from instructor  
where salary > 10000 is unknown;
```

Funciones de agregación

Aggregate functions are functions that take a collection of values as input and return a single value. SQL offers five standard built-in aggregate functions:

- Average: **avg** → **select avg (salary)**
- Minimum: **min**
- Maximum: **max**
- Total: **sum**
- Count: **count** → **select count (distinct ID)**

Tuples with the same value on all attributes in the **group by** clause are placed in one group.

When an SQL query uses grouping, it is important to ensure that...

The only attributes that appear in the select statement without being aggregated are those that are present in the group by clause.

It is useful to state a condition that applies to groups rather than to tuples. For example, we might be interested in only those departments where the average salary of the instructors is more than \$42,000. To express such a query, we use the **having** clause.

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name  
having avg_salary > 42000;
```

Any attribute that is present in the **having** clause without being aggregated must appear in the group by clause, otherwise the query is erroneous.

Subqueries anidadas

select ... and course id in (select .. and year= 2018);

select ... and course id not in (select .. and year= 2018); SIMILAR AL **except**

✿ The in and not in operators can also be used on enumerated sets.

...

where name not in ('Mozart', 'Einstein');

✿ The phrase “greater than at least one” is represented in SQL by **> some**.

```
select name  
from instructor  
where salary > some (select salary  
                     from instructor  
                     where dept name = 'Biology');
```

✿ The construct **> all** corresponds to the phrase “greater than all”.

✿ The **exists/ not exists** construct returns the value true/false if the argument subquery is nonempty.

...

where exists (select * ...);

✿ The **unique/not unique** construct returns the value true/false if the argument subquery contains no duplicate tuples.

...

where unique(select * ...);

✿ The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the with clause occurs.

```
with max_budget (value) as  
  (select max(budget)  
   from department)  
select budget  
from department, max_budget  
where department.budget = max_budget.value;
```

✿ Scalar subqueries

-

Modificación de la bbdd

✿ A **delete** request is expressed in much the same way as a query.

```
delete from r  
where P;
```

where P represents a predicate and r represents a relation.

The delete statement first finds all tuples t in r for which $P(t)$ is true, and then deletes them from r

Note that a delete command operates on only one relation.

✿ The simplest **insert** statement is a request to insert one tuple

```
insert into course (course id, title, dept name, credits)  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

More generally, we might want to insert tuples on the basis of the result of a query:

```
insert into instructor  
select ... from ... where ...
```

✿ The **update** statement can be used to change a value in a tuple without changing all values in the tuple. We can choose the tuples to be updated by using a query.

```
update instructor  
set salary = salary * 1.05  
where salary < 70000;
```

SQL provides a **case** construct that we can use to perform both updates with a single update statement, avoiding the problem with the order of updates.

```
update instructor  
set salary = case  
    when salary <= 100000 then salary * 1.05  
    ...  
    when predn then resultn  
    else salary * 1.03  
end
```

Capitulo 4 - Database System Concepts

Review Terms

- Join types
 - Natural join
 - Inner join with **using** and **on**
 - Left, right and full outer join
 - Outer join with **using** and **on**
- View definition
 - Materialized views
 - View maintenance
 - View update
- Transactions
 - Commit work
 - Rollback work
 - Atomic transaction
- Constraints
 - Integrity constraints
 - Domain constraints
 - Unique constraint
 - Check clause
 - Referential integrity
 - ◊ Cascading deletes
 - ◊ Cascading updates
 - Assertions
- Data types
 - Date and time types
 - Default values
 - Large objects
 - ◊ clob
 - ◊ blob
 - User-defined types
 - distinct types
 - Domains
 - Type conversions
- Catalogs
- Schemas
- Indices
- Privileges
 - Types of privileges
 - ◊ **select**
 - ◊ **insert**
 - ◊ **update**
 - Granting of privileges
 - Revoking of privileges
 - Privilege to grant privileges
 - Grant option
- Roles
- Authorization on views
- Execute authorization
- Invoker privileges
- Row-level authorization
- Virtual private database (VPD)

Join expressions

⌘ **Natural join:** It considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations.

student natural join takes
considers only those pairs of tuples where both the tuple from student and the tuple from takes have the same value on the common attribute, ID.

```
select A1, A2, ..., An
from r1 natural join r2 natural join ... natural join rm
where P;
```

⌘ **Join conditions:** The **on** condition allows a general predicate over the relations being joined.

```
select *
from student join takes on student.ID = takes.ID;
```

⌘ **Outer joins:** It works in a manner similar to the join operations we have already studied, but it preserves those tuples that would be lost in a join by creating tuples in the result containing null values.

- **left outer join:** preserves tuples only in the relation named before
- **right outer join:** preserves tuples only in the relation named after
- **full outer join:** preserves tuples in both relations.

¿Cómo funciona el outer join?

Primero se hace un inner join y después se agregan las tuplas correspondientes del lado que debe estar y se completa con valores null.

```
select *
from student left outer join takes on student.ID = takes.ID;
```

on and **where** behave differently for outer join. The reason for this is that outer join adds null-padded tuples only for those tuples that do not contribute to the result of the corresponding "inner" join. The **on** condition is part of the outer join specification, but a **where** clause is not.

```
select *
from student left outer join takes on true
where student.ID = takes.ID;
```

La primera query incluye la tupla: (70557, Snow, Physics, 0, null, null, null, null, null, null) , mientras que la segunda no.

⌘ **Inner joins:** Operations that do not preserve nonmatched tuples (intersección para los panas)

✿ **Join types and conditions:** Normal **joins** are called **inner joins** in SQL. The keyword **inner** is, however, optional. The default join type, when the join clause is used without the outer prefix, is the inner join.

```
select *  
from student inner join takes using (ID);
```

Vistas

The **with** clause allows us to assign a name to a subquery for use as often as desired, but in one particular query only. On the other hand, **views** present a way to extend this concept beyond a single query.

✿ **Definición:** Con el comando create view → **create view v as <query expression>;**

When we define a view, the database system stores the definition of the view itself, rather than the result of evaluation of the query expression that defines the view. Wherever a view relation appears in a query, it is replaced by the stored query expression. Thus, whenever we evaluate the query, the view relation is recomputed.

Views, once created, remain available until explicitly dropped.

```
create view departments_total_salary(dept_name, total_salary) as  
select dept name, sum (salary)  
from instructor  
group by dept name;
```

✿ **Materialized views:** Certain database systems allow view relations to be stored, but they make sure that, if the actual relations used in the view definition change, the view is kept up-to-date. The process of keeping the materialized view up-to-date is called **materialized view maintenance**. Applications that use a view frequently or demand fast response may benefit if the view is materialized.

✿ **Update of a view:** An SQL view is said to be **updatable** (i.e., inserts, updates, or deletes can be applied on the view) if the following conditions are all satisfied by the query defining the view:

- The **from** clause has only one database relation.
- The **select** clause contains only attribute names of the relation and does not have any expressions, aggregates, or distinct specification.
- Any attribute not listed in the **select** clause can be set to null; that is, it does not have a not null constraint and is not part of a primary key.
- The query does not have a **group by** or **having** clause.

Views can be defined with a **with check option** clause at the end of the view definition.

Transactions

A transaction consists of a sequence of query and/or update statements. The SQL standard specifies that a transaction begins implicitly when an SQL statement is executed.

✿ **Commit:** commits the current transaction; that is, it makes the updates performed by the transaction become permanent in the database.

✿ **Rollback:** causes the current transaction to be rolled back; that is, it undoes all the updates performed by the SQL statements in the transaction.

Transaction rollback is useful if some error condition is detected during execution of a transaction. Once a transaction has executed **commit**, its effects can no longer be undone by **rollback**. The database provides an abstraction of a transaction as being **atomic**, that is, indivisible. How to turn off automatic commit depends on the specific SQL implementation, although many databases support the command `set autocommit off`.

However, several other databases, such as MySQL and PostgreSQL, support a **begin** statement which starts a transaction containing all subsequent SQL statements, but do not support the **end** statement; instead, the transaction must be ended by either a **commit** or a **rollback** command

Integrity constraints

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus, integrity constraints guard against accidental damage to the database.

✿ We study another form of integrity constraint, called **functional dependencies**, that is used primarily in the process of schema design.

Integrity constraints are usually identified as part of the database schema design process and declared as part of the create table command used to create relations. However, integrity constraints can also be added to an existing relation by using the command `alter table-name add constraint`, where constraint can be any constraint on the relation.

✿ **Constraints on a single relation:** In addition to the primary-key constraint, there are a number of other ones that can be included in the create table command. Such as: **not null**, **unique**, **check(<predicate>)**.

✿ **Not null constraint:** The not null constraint prohibits the insertion of a null value for the attribute, and is an example of a domain constraint. Any database modification that would cause a null to be inserted in an attribute declared to be not null generates an error diagnostic.

✿ **Unique constraint:** The unique specification says that attributes $A_{j1}, A_{j2}, \dots, A_{jm}$ form a superkey; that is, no two tuples in the relation can be equal on all the listed attributes.

✿ **Check clause:** The clause `check(P)` specifies a predicate P that must be satisfied by every tuple in a relation. A common use of the check clause is to ensure that attribute values satisfy specified conditions, in effect creating a powerful type system. For instance, a clause `check(budget > 0)` in the create table command for relation *department* would ensure that the value of **budget** is nonnegative.

```
create table section
(
    ...,
    time_slot_id varchar(4),
    primary key (course_id, sec_id, semester, year),
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```

A check clause is satisfied if it is not false, so clauses that evaluate to **unknown** are not violations. If null values are not desired, a separate not null constraint must be specified

✿ **Referential integrity:** We wish to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation. **Foreign keys** are a form of a referential integrity constraint where the referenced attributes form a primary key of the referenced relation.

Foreign keys can be specified as part of the SQL create table statement by using the **foreign key clause**:

```
foreign key (dept_name) references department
```

By default, in SQL a foreign key references the primary-key attributes of the referenced table. The specified list of attributes must, however, be declared as a superkey of the referenced relation, using either a primary key constraint or a unique constraint.

Note that the foreign key must reference a compatible set of attributes, that is, the number of attributes must be the same and the data types of corresponding attributes must be compatible.

```
dept_name varchar(20) references department
```

However, a **foreign key clause** can specify that if a delete or update action on the referenced relation violates the constraint, then, instead of rejecting the action, the system must take steps to change the tuple in the referencing relation to restore the constraint.

```
create table course
(
    ... foreign key (dept_name) references department
    on delete cascade
    on update cascade, ... );
```

✿ **Names to constraints:** To name a constraint, we precede the constraint with the keyword **constraint** and the name we wish to assign it.

```
salary numeric(8,2), constraint minsalary check (salary > 29000)
```

If we decide we no longer want this constraint:

```
alter table instructor drop constraint minsalary;
```

✿ **Constraint violation during a transaction:** Transactions may consist of several steps, and integrity constraints may be violated temporarily after one step, but a later step may remove the violation. To handle such situations, the SQL standard allows a clause **initially deferred** to be added to a constraint specification; the constraint would then be checked at the end of a transaction and not at intermediate steps. A constraint can alternatively be specified as **deferrable**, which means it is checked immediately.

```
set constraints constraint_list deferred
```

✿ **Complex check conditions:** The predicate in the check clause can be an arbitrary predicate that can include a subquery.

```
check (time_slot_id in (select time_slot_id from time_slot))
```

Complex check conditions can be useful when we want to ensure the integrity of data, but they may be costly to test.

✿ **Assertions:** Is a predicate expressing a condition that we wish the database always to satisfy. Consider the following constraints, which can be expressed using assertions.

- For each tuple in the *student* relation, the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully.
- An instructor cannot teach in two different classrooms in a semester in the same time slot.

```
create assertion credits_earned_constraint check  
(not exists (select ID  
            from student  
            where tot_cred <> (select coalesce(sum(credits), 0)  
                               from takes natural join course  
                               where student.ID= takes.ID  
                               and grade is not null and grade<> 'F' )))  
  
create assertion <assertion-name> check <predicate>;
```

SQL data types and schemas

In addition to the basic data types, SQL standard supports data types such as dates and times:

→ **date:** A calendar date containing a (four-digit) year, month, and day of the month.

- **time**: The time of day, in hours, minutes, and seconds. A variant, **time(p)**, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time-zone information along with the time by specifying **time with timezone**.
- **timestamp**: A combination of date and time. A variant, **timestamp(p)**, can be used to specify the number of fractional digits for seconds (the default here being 6). Time-zone information is also stored if **with timezone** is specified.

Date and time values can be specified like this:

```
date '2018-04-25'
time '09:30:00'
timestamp '2018-04-25 10:29:01.45'
```

✿ **Type conversion and formatting functions**: We can use an expression of the form: **cast (e as t)** to convert an expression *e* to the type *t*.

Database systems offer a variety of formatting functions, and details vary among the leading systems. MySQL offers a **format** function. Oracle and PostgreSQL offer a set of functions, **to_char**, **to_number**, and **to_date**. SQL Server offers a **convert** function. We can choose how null values are output in a query result using the **coalesce** function.

```
select ID, coalesce(salary, 0) as salary
from instructor
```

✿ **Default values**: SQL allows a **default** value to be specified for an attribute.

```
create table student
(..., tot_cred numeric(3,0) default 0,
primary key (ID));
```

✿ **Large object types**: SQL provides **large-object data types** for character data (**clob**) and binary data (**blob**). The letters “lob” in these data types stand for “Large Object.” For example, we may declare attributes:

```
book_review clob(10KB)
image blob(10MB)
movie blob(2GB)
```

For result tuples containing large objects (multiple megabytes to gigabytes), it is inefficient or impractical to retrieve an entire large object into memory. Instead, an application would usually use an SQL query to retrieve a “locator” for a large object and then use the locator to manipulate the object from the host language in which the application itself is written.

✿ **User defined types**: The first form is called **distinct types**. The other form, called **structured data types**, allows the creation of complex data types with nested record structures, arrays, and multisets.

The create type clause can be used to define new types.


```
create type Dollars as numeric(12,2) final;  
create type Pounds as numeric(12,2) final;
```

SQL provides **drop type** and **alter type** clauses to drop or modify types that have been created earlier.

Differences between domains and types:

- Domains can have constraints, such as not null, specified on them, and can have default values defined for variables of the domain type, whereas user-defined types cannot have constraints or default values specified on them.
- Domains are not strongly typed.

✿ **Generating unique key values:** Database systems offer automatic management of unique key-value generation. Since this feature works only for numeric key-value data types, we change the type of ID to number, and write:

```
ID number(5) generated always as identity;
```

When the **always** option is used, any **insert** statement must avoid specifying a value for the automatically generated key. In **PostgreSQL**, we can define the type of ID as **serial**, which tells PostgreSQL to automatically generate identifiers; in MySQL we use **auto_increment** in place of **generated always as identity**, while in SQL Server we can use just **identity**.

✿ **Create table extensions:** SQL provides a **create table like** extension to support this task: **create table temp_instructor like instructor;** The above statement creates a new table temp_instructor that has the same schema as instructor.

✿ **Schemas, catalogs and environments:** Consider how files are named in a file system. In database systems users had to coordinate to make sure they did not try to use the same name for different relations. Database systems provide a three-level hierarchy for naming relations. The top level of the hierarchy consists of **catalogs**, each of which can contain **schemas**.

The default catalog and schema are part of an SQL **environment** that is set up for each connection. The environment additionally contains the user identifier. We can create and drop schemas by means of **create schema** and **drop schema** statements.

Index definition

An **index** on an attribute of a relation is a data structure that allows the database system to find those tuples in the relation that have a specified value for that attribute efficiently, without scanning through all the tuples of the relation. Indices form part of the physical schema of the database, as opposed to its logical schema.

Indices are important for efficient processing of transactions. We create an index with the create index command, which takes the form:

```
create index <index-name> on <relation-name> (<attribute-list>);
```


If we wish to declare that the search key is a candidate key, we add the attribute **unique** to the index definition.

Authorization

We may assign a user several forms of authorizations on parts of the database.

- Authorization to read data.
- Authorization to insert new data.
- Authorization to update data.
- Authorization to delete data.

Each of these types of authorizations is called a **privilege**. When a user submits a query or an update, the SQL implementation first checks if the query or update is authorized. In addition to authorizations on data, users may also be granted authorizations on the database schema, allowing them, for example, to create, modify, or drop relations. A user who has some form of authorization may be allowed to pass on (**grant**) this authorization to other users, or to withdraw (**revoke**) an authorization that was granted earlier. The database administrator may authorize new users, restructure the database, and so on; (**superuser**).

✿ **Granting and revoking of privileges:** The SQL standard includes the **privileges** select, insert, update, and delete (**all privileges** for all the allowable privileges).

The grant statement is used to confer authorization. The basic form of this statement is:

```
grant <privilege list>  
on <relation name or view name>  
to <user/role list>;
```

For update you can specify the attribute → **grant update (budget) on...**

✿ **Roles:** A set of roles is created in the database. Authorizations can be granted to roles, in exactly the same fashion as they are granted to individual users. Each database user is granted a set of roles (which may be empty) that she is authorized to perform. Roles can be created in SQL as follows:

```
create role instructor;
```

Roles can be granted to users, as well as to other roles:

```
create role dean;  
grant instructor to dean;  
grant dean to Satoshi;
```

✿ **Authorization on views:** A user who creates a view does not necessarily receive all privileges on that view. For example, a user who creates a view cannot be given update authorization on a view without having update authorization on the relations used to define the view. The **execute** privilege can be granted on a function or procedure, enabling a user to execute the function or procedure.

✿ **Authorization on schemas:** SQL includes a **references** privilege that permits a user to declare foreign keys when creating relations:

grant references (dept name) **on** department **to** Mariano;

Initially, it may appear that there is no reason ever to prevent users from creating foreign keys referencing another relation. However, recall that foreign-key constraints restrict deletion and update operations on the referenced relation. The **references privilege** on *department* is also required to create a **check** constraint on a relation *r* if the constraint has a subquery referencing department.

✿ **Transfer of privileges:** If we wish to grant a privilege and to allow the recipient to pass the privilege on to other users, we append the **with grant option** clause to the appropriate grant command. The passing of a specific authorization from one user to another can be represented by an **authorization graph**.

✿ **Revoking of privileges:** Revocation of a privilege from a user/role may cause other users/roles also to lose that privilege (**cascading revocation**). The revoke statement may specify **restrict** in order to prevent cascading revocation:

revoke select on department **from** Amit, Satoshi **restrict**;

Capitulo 5 - Database System Concepts

Advanced SQL

A database programmer must have access to a general-purpose programming language for at least two reasons:

- Not all queries can be expressed in SQL, since SQL does not provide the full expressive power of a general-purpose language.
- Nondeclarative actions—such as printing a report, interacting with a user, or sending the results of a query to a graphical user interface—cannot be done from within SQL.

There are two approaches to accessing SQL from a general-purpose programming language:

- **Dynamic SQL:** Dynamic SQL allows the program to construct an SQL query as a character string at runtime, submit the query, and then retrieve the result into program variables a tuple at a time.
- **Embedded SQL:** under embedded SQL, the SQL statements are identified at compile time using a preprocessor, which translates requests expressed in embedded SQL into function calls. At runtime, these function calls connect to the database using an API that provides dynamic SQL facilities but may be specific to the database that is being used.

✿ **JDBC:** Is a standard that defines an application program interface (API) that Java programs can use to connect to database servers. The first step in accessing a database from a Java program is to open a connection to the database. A connection is opened using the `getConnection()` method of the `DriverManager` class (within `java.sql`). This method takes three parameters:

- 1st parameter: String that specifies the URL
- 2nd parameter: database user identifier (string)
- 3rd parameter: Password (string)

Connections and statements: Once the connection is open, the program can use it to send SQL statements. A `Statement` object is not the SQL statement itself, but rather an object that allows the Java program to invoke methods that ship an SQL statement given as an argument. To execute a statement, we invoke either the `executeQuery()` method or the `executeUpdate()` method.

Exceptions: The `try { ... } catch { ... }` construct permits us to catch any exceptions (error conditions) that arise when JDBC calls are made and take appropriate action.

Programmers must take care to ensure that programs close all such resources. Failure to do so may cause the database system's resource pools to become exhausted, rendering the system inaccessible or inoperative until a time-out period expires. This protects us from leaving connections or statements unclosed.

Retrieving the result of a query: Execute a query by using `stmt.executeQuery()`. It retrieves the set of tuples in the result into a `ResultSet` object `rset` and fetches them one tuple at a time. The method `getString()` can retrieve any of the basic SQL data types.

Prepared statements: We can create a prepared statement in which some values are replaced by "?", thereby specifying that actual values will be provided later. The `prepareStatement()` method of the `Connection` class defines a query that may contain parameter values. The method returns an object of class `PreparedStatement`. Prepared statements allow for more efficient execution in cases where the same query can be compiled once and then run multiple times with different parameter values.

SQL injection: ?

Programmers must pass user-input strings to the database only through parameters of prepared statements; creating SQL queries by concatenating strings with user-input values is an extremely serious security risk and should never be done in any program.

Callable statements: Allows invocation of SQL stored procedures and functions:

```
CallableStatement cStmt2 = conn.prepareCall("{call some procedure(?,?)}");
```

Metadata: The interface `ResultSet` has a method, `getMetaData()`, that returns a `ResultSetMetaData` object that contains metadata about the result set. The `DatabaseMetaData` interface in turn has a very large number of methods to get metadata about the database and the database system to which the application is connected.

The method `getColumns()` takes four arguments: a catalog name (null signifies that the catalog name is to be ignored), a schema name pattern, a table name pattern, and a column name pattern.

The `getTables()` method allows you to get a list of all tables in the database. The first three parameters to `getTables()` are the same as for `getColumns()`. The fourth parameter can be used to restrict the types of tables returned.

✿ **Access from python:**

```
import psycopg2
def PythonDatabaseExample(userid, passwd)
    try:
        conn = psycopg2.connect( host="db.yale.edu", port=5432, dbname="univdb",
            user=userid, password=passwd)
        cur = conn.cursor()
        try:
            cur.execute("insert into instructor values(%s, %s, %s, %s)",
                ("77987","Kim","Physics",98000))
            conn.commit();
        except Exception as sqle:
            print("Could not insert tuple. ", sqle)
            conn.rollback()
        cur.execute( "select dept name, avg (salary) " " from instructor group by dept
            name"))
        for dept in cur:
```

```

        print dept[0], dept[1]
except Exception as sqle:
    print("Exception : ", sqle)

```

✿ **ODBC (open database connectivity):** defines an API that applications can use to open a connection with a database, send queries and updates, and get back results.

the program can send SQL commands to the database by using `SQLExecDirect`.

`SQLFetch`: SQL SUCCESS cuando encuentra resultado

`SQLBindCol`

A negative value returned for the length field indicates that the value is null

`SQLSetConnectOption(conn, SQL AUTOCOMMIT, 0)`

`SQLTransact(conn, SQL COMMIT)`

✿ **Embedded SQL:** A language in which SQL queries are embedded is referred to as a host language, and the SQL structures permitted in the host language constitute embedded SQL.

`EXEC SQL <embedded SQL statement>;`

Variables of the host language can be used within embedded SQL statements, but they must be preceded by a colon (:) to distinguish them from SQL variables.

The exact syntax for embedded SQL requests depends on the language in which SQL is embedded.

Functions and procedures

Procedures and functions allow “business logic” to be stored in the database and executed from SQL statements.

Most databases implement nonstandard versions of this syntax (SQL standard).

✿ **Declaring and invoking SQL functions and procedures:** Suppose that we want a function that, given the name of a department, returns the count of the number of instructors in that department. This function can be used in a query that returns names and budgets of all departments with more than 12 instructors:

```

select dept name, budget
from department
where dept count(dept name) > 12;

```

```

create function instructor_of (dept_name varchar(20))
returns table (
    ID varchar (5),
    name varchar (20),
    dept_name varchar (20),
    salary numeric (8,2))
return table
(select ID, name, dept_name, salary
from instructor
where instructor.dept_name = instructor_of.dept_name);

```

Performance problems have been observed on many database systems when invoking complex user-defined functions within a query. Programmers should therefore take performance into consideration when deciding whether to use user-defined functions in a query.

Table functions: functions that can return tables as results.

La función de la imagen puede ser usada en una query de la siguiente manera:

```
select *  
from table(instructor of('Finance'));
```

Table-valued functions can be thought of as **parameterized views** that generalize the regular notion of views by allowing parameters.

The dept count function could instead be written as a procedure:

```
create procedure dept_count_proc(in dept_name varchar(20),  
                                out d_count integer)  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name= dept_count_proc.dept_name  
end
```

The keywords **in** and **out** indicate, respectively, parameters that are expected to have values assigned to them and parameters whose values are set in the procedure in order to return results.

Procedures can be invoked either from an SQL procedure or from embedded SQL by the **call** statement:

```
declare d_count integer;  
call dept_count_proc('Physics', d_count);
```

✿ **Language Constructs for Procedures and Functions:** SQL supports constructs that give it almost all the power of a general-purpose programming language. (**Persistent storage module** deals with these constructs)

Variables are declared using a **declare** statement and can have any valid SQL data type. A compound statement is of the form **begin ... end**.

The syntax for while statements and repeat statements is:

```
while boolean expression do  
    sequence of statements;  
end while  
  
repeat  
    sequence of statements;  
until boolean expression  
end repeat
```

There is also a for loop that permits iteration over all the results of a query:

```
declare n integer default 0;  
for r as  
    select budget from department  
    where dept_name = 'Music'  
do  
    set n = n - r.budget  
end for
```

The statement `leave` can be used to exit the loop.

The conditional statements supported by SQL include if-then-else statements by using this syntax:

```
if boolean expression  
    then statement or compound statement  
elseif boolean expression  
    then statement or compound statement  
else statement or compound statement  
end if
```

The SQL procedural language also supports the signaling of exception conditions and declaring of handlers that can handle the exception, as in this code:

```
declare out_of_classroom_seats condition  
declare exit handler for out_of_classroom_seats  
begin  
    sequence of statements  
end
```

The statements between the **begin** and the **end** can raise an exception by executing **signal** out of classroom seats. The handler says that if the condition arises, the action to be taken is to exit the enclosing begin end statement.

Triggers

after update of takes on grade

Triggers can be activated before the event (insert, delete, or update) instead of after the event. Triggers that execute before an event can serve as extra constraints that can prevent invalid updates, inserts, or deletes.

create trigger *credits_earned* **after update of takes on grade**

Recursive Queries

Capitulo 8 - Database System Concepts

Complex Data Types

Semi-structured Data

Traditional relational database designs mandate schemas with a fixed number of attributes, where each attribute must contain an atomic value. **Schema modifications** (e.g., adding an attribute) are rare events that often require **expensive alterations** to application source code. While this structured architecture is optimal for standard corporate workflows, modern fields such as fast-evolving web environments require **storage of semi-structured data**, where the **underlying schema shifts frequently**, and individual data items of the same type can have entirely different sets of attributes.

✿ Motivation and Non-atomic Attributes

The primary motivations for semi-structured data paradigms are **flexible data modeling** and **data exchange**:

- **Data Modeling Flexibility (flexible schema)**: Some database systems allow each tuple to potentially have a different set of attributes; such a representation is referred to as a **wide column** data representation. The **set of attributes is not fixed** in such a representation; each tuple may have a different set of attributes, and new attributes may be added as needed. A more restricted form of this representation is to have a **fixed but very large number of attributes**, with each tuple using only those attributes that it needs, leaving the rest with null values; such a representation is called a **sparse column** representation.
- **Multivalued data types**: Many data representations allow **attributes to contain non-atomic values**. Many databases allow the storage of **sets, multisets, or arrays** as attribute values. Some representations allow attributes to store key-value maps, which store key-value pairs. A key-value map, often just called a map. Array databases exist.
- **Nested data types**: Many data representations allow attributes to be structured, directly modeling composite attributes in the E-R model. These representations also support multivalued data types. All of these data types represent a **hierarchy of data types**.
- **Knowledge representation**: The **Resource Description Format (RDF)** data representation is one such representation that has found very wide acceptance.

✿ JSON (JavaScript Object Notation)

JSON is a textual representation of **complex data types** that is widely used to transmit data between applications and to store complex data. It represents data using **two primary structures**:

1. **Objects**: An unordered collection of key-value pairs enclosed in curly braces `{}`. Keys must be strings, and values can be any valid data type.
2. **Arrays**: An ordered list of values enclosed in square brackets `[]`.

✿ Textual Representation Example

The textbook contrasts standard tables with a nested JSON block. For instance, a student profile record with nested arrays of structural interests and historical course marks is represented as:

JSON

```
{
  "ID": "00128",
  "name": "Zhang",
  "dept_name": "Comp. Sci.",
  "tot_cred": 102,
  "interests": ["Databases", "Data Mining", "AI"],
  "courses_taken": [
    {"course_id": "CS-101", "sec_id": "1", "semester": "Fall", "year": 2017, "grade": "A"},
    {"course_id": "CS-347", "sec_id": "1", "semester": "Fall", "year": 2017, "grade": "A-"}
  ]
}
```

✿ Storage and Querying Extensibility in SQL

Modern relational database management systems (DBMS) provide robust native mechanisms to store, index, and query JSON documents directly inside a table column.

- **Data Types:** Most platforms provide two distinct JSON storage implementations. For instance, PostgreSQL supports a raw textual JSON type (which preserves exact formatting and duplicate keys but requires re-parsing on every read) and a decompressed, binary-serialized JSONB format (which strips whitespace and duplicate keys but enables highly efficient indexed querying).
- **Path Expressions and Extraction:** SQL dialect extensions use specialized operators to extract scalar fields or nested values from a JSON document. In standard compliance formats, path expressions use a dot notation syntax (e.g., column.path.to.attribute). In PostgreSQL specifically:
 - The -> operator extracts a nested component as a JSON object/array wrapper.
 - The ->> operator extracts the target component explicitly as a scalar text value.
- **JSON Constraints and Generation:** Engines provide functions like json_object() or json_array() to construct JSON structures from standard relational columns dynamically. Constraints such as IS JSON can be appended to standard check clauses to validate document integrity before insertion.

✿ **XML (Extensible Markup Language)**

XML was designed as a generalized framework for structuring text documents and serializing data for distributed exchange. Unlike HTML, which uses a fixed set of predefined tags for layout rendering, XML tags are completely user-defined and specify the explicit structural semantics of the nested data elements.

✿ Structural Breakdown: Elements, Attributes, and Nesting

An XML document is composed of hierarchical nodes:

- **Elements:** Marked by an opening tag (e.g., <instructor>) and a closing tag (e.g., </instructor>). Elements can contain plain text, other nested child elements, or remain empty.

- **Attributes:** Key-value metadata specifications embedded directly within an element's opening tag (e.g., <instructor id="10101">).
- **Nesting Limitations & Well-formedness:** To be structurally well-formed, an XML document must possess exactly one singular root element, all nested tags must close in precise inverse order of their opening sequence, and attribute values must be enclosed in quotation marks.

✿ Querying XML: XPath and XQuery (No lo estudio pero es interesante)

Querying XML documents requires specialized functional abstraction languages:

- **XPath:** A syntax used to navigate through elements and attributes in an XML tree structure using path expressions.
 - /root/child specifies absolute path navigation.
 - //element specifies a deep search for the target element anywhere at any depth level in the document tree.
 - @attribute extracts metadata attributes.
 - Slotted bracket conditions (e.g., /instructor[salary > 70000]) act as localized predicates.
- **XQuery:** A highly robust, declarative query language built on top of XPath expressions. It uses FLWOR expressions (For, Let, Where, Order by, Return) to iterate, filter, and transform XML structural outputs.

✿ SQL/XML Integration Functions

The SQL language itself has been extended to support XML in several ways:

- XML data can be stored as an XML data type.
- SQL queries can generate XML data from relational data.
- SQL queries can extract data from an XML data type value.

✿ **Graph Data Models: RDF and knowledge graph**

When data objects exhibit highly complex, many-to-many relationships that evolve dynamically, representing them as rigid tables can lead to an excess of foreign keys and multi-table joins. Graph Data Models treat data as a network of nodes and directed edges.

✿ RDF (Resource Description Framework)

Under the RDF data model, information is strictly represented as a collection of structural assertions called triples:

(ID, attribute-name, value) or (ID1, relationship-name, ID2), where ID, ID1 and ID2 are identifiers of entities; entities are also referred to as resources in RDF.

—(Subject, predicate, object)—

10101	instance-of	instructor .
10101	name	"Srinivasan"
10101	salary	"6500" .
00128	instance-of	student .
00128	name	"Zhang" .
00128	tot_cred	"102" .
comp_sci	instance-of	department

Knowledge graph:

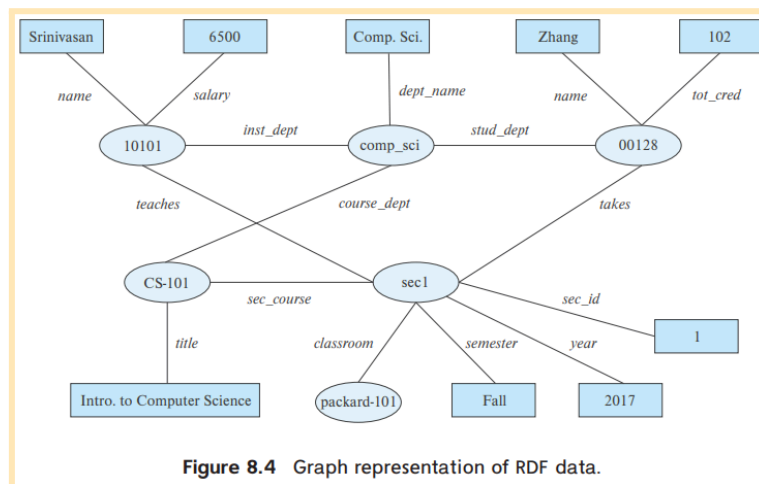


Figure 8.4 Graph representation of RDF data.

✿ SPARQL

Query language designed to query RDF data. Based on triple patterns. It executes pattern matching against graph triples by declaring variables (denoted with a leading question mark, ?var).

The following query retrieves names of all students who have taken a section whose course is titled "Intro. to Computer Science".

```
select ?name
where {
  ?cid title "Intro. to Computer Science" .
  ?sid course ?cid .
  ?id takes ?sid .
  ?id name ?name .
}
```

Object Orientation

The **object-relational data model** extends the relational data model by providing a **richer type system**, including complex data types and object orientation. **Three approaches** are used in practice for integrating object orientation with database systems:

1. Build an object-relational database system, which adds **object-oriented features** to a **relational database system**.
2. Automatically **convert data** from the native object-oriented type system of the programming language to a relational representation for storage
3. Build an object-oriented database system that allows **direct access to data** from an object-oriented programming language using the native type system of the language.

✿ Object-Relational Database Systems

Object extensions to SQL allow the creation of structured user-defined types, references to such types, and tables containing tuples of such types. The SQL standard includes extensions of the SQL data-definition and query language to deal with new data types and

with object orientation, including support for collection-valued attributes, inheritance, and tuple references. Such extensions attempt to preserve the relational foundations—in particular, the declarative access to data—while extending the modeling power.

✿ User-Defined Types

SQL allows definition of structured types. For example:

```
-----:
create type Person (
    ID varchar(20) primary key,
    name varchar(20),
    address varchar(20))
    ref from(ID);
create table people of Person;
```

Similarly, nested table/multiset collection types can be declared as attributes:

```
-----:
create type interest as table ( topic varchar(20), degree of interest int );
create table users ( ID varchar(20), name varchar(20), interests interest );
-----:
```

User-defined types can also have methods associated with them. Only a few database systems, such as Oracle, support this feature.

✿ Type Inheritance

Consider a definition of the type Person and table people. We may want to store extra information in the database about people who are students and people who are teachers. Methods of a structured type are inherited by its subtypes, just as attributes are. However, a subtype can redefine the effect of a method.

```
create type Student under Person
    (degree varchar(20)) ;
create type Teacher under Person
    (salary integer);
```

✿ Table Inheritance

Table inheritance allows a table to be declared as a subtable of another table and corresponds to the E-R notion of specialization/generalization. Several database systems support table inheritance, but in different ways. In PostgreSQL, we could create a table people and then create tables students and teachers as subtables of people.

We can insert a tuple into the student table providing values for the inherited attributes as well as local ones:

-----:

insert into student values ('00128', 'Zhang', '235 Coyote Run', 'Ph.D.');

-----:

When we declare students and teachers as subtables of people, every tuple present in students or teachers becomes implicitly present in people. Thus, if a query uses the table people, it will find not only tuples directly inserted into that table but also tuples inserted into its subtables. However, only those attributes that are present in people can be accessed by that query. SQL permits us to find tuples that are in people but not in its subtables by using only people in place of people in a query.

System-generated identifiers can be retrieved using ref(r) where r is a table name or table alias used in a query. Using the ID or name of a person, we could retrieve the reference to the tuple in a subquery, or navigate using deref:

-----:

select deref(head).name from departments;

-----:

✿ Object-Relational Mapping

Object-relational mapping (ORM) systems allow a programmer to define a mapping between tuples in database relations and objects in the programming language. An object, or a set of objects, can be retrieved based on a selection condition on its attributes; relevant data are retrieved from the underlying database based on the selection conditions, and one or more objects are created from the retrieved data, based on the prespecified mapping between objects and relations.

Textual Data

Textual data consists of unstructured text. The term information retrieval generally refers to the querying of unstructured textual data. In the traditional model used in the field of information retrieval, textual information is organized into documents. In a database, a text-valued attribute can be considered a document. In the context of the web, each web page can be considered to be a document.

Information-retrieval systems use a simpler data model than do database systems but provide more powerful querying capabilities within the restricted model.

✿ Keyword Queries

Information retrieval systems support the ability to retrieve documents with some desired information. The desired documents are typically described by a set of keywords. Documents have associated with them a set of keywords; typically, all the words in the documents are considered keywords. A keyword query retrieves documents whose set of keywords contains all the keywords in the query.

The query that a user has in mind usually cannot be stated precisely; hence, information-retrieval systems order answers on the basis of potential relevance.

✿ Relevance Ranking

✿ Ranking Using TF-IDF

The word **term** refers to a **keyword occurring in a document**, or given as part of a query. Given a particular term t , the system must determine how relevant a particular document d is to the term. One approach is to use the **number of occurrences of the term in the document as a measure of its relevance** (term frequency). However, just counting the number of occurrences of a term is usually not a good indicator because it depends on the length of the document, and a document containing 10 occurrences of a term may not be 10 times as relevant as a document containing one occurrence. Relevance ranking makes use of detailed statistical concepts like **term frequency** and **inverse document frequency**.

$$TF(d, t) = \log \left(1 + \frac{n(d, t)}{n(d)} \right) \quad IDF(t) = \frac{1}{n(t)}$$

where $n(d)$ = number of documents and $n(d, t)$ = amount of occurrences of t in d .

Information-retrieval systems define a set of words, called **stop words**, containing 100 or so of the most common words, and **ignore these words** when indexing a document.

Proximity: If the terms **occur close to each other** in the document, the document will be ranked higher than if they occur far apart.

✿ PageRank

In the context of web link structures, if a page i has N_i links out to other pages, a user browsing page i has an **equal probability of following any link**, expressed as $T[i, j] = 1/N_i$. Then the PageRank $P[j]$ for each page j can be defined as:

$$P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^N (T[i, j] * P[i])$$

where **delta** is a constant between 0 and 1, usually set to 0.15, and N is the number of pages.

Two metrics are used to measure how well an information-retrieval system is able to answer queries: The first,

1. **Precision**: measures what percentage of the retrieved documents are actually **relevant to the query**.
2. **Recall**: measures what percentage of the documents relevant to the query **were retrieved**.

Spatial Data

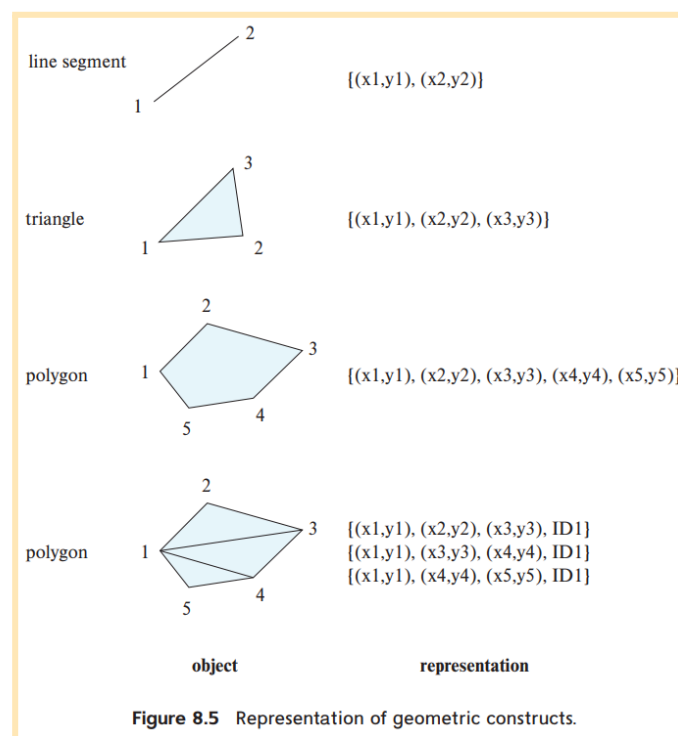
Spatial data support in database systems is important for efficiently storing, indexing, and querying of data on the basis of spatial locations. Two types of spatial data are particularly important:

- **Geographic data:** such as road maps, land-usage maps, topographic elevation maps, political maps showing boundaries, and land-ownership maps. Geographic information systems are special-purpose database systems tailored for storing geographic data. Geographic data is based on a round-earth coordinate system, with latitude, longitude, and elevation.
- **Geometric data:** which include spatial information about how objects—such as buildings, cars, or aircraft—are constructed. Geometric data is based on a two-dimensional or three-dimensional Euclidean space, with X, Y, and Z coordinates.

Geographic and geometric data types are supported by many database systems, such as Oracle Spatial and Graph, the PostGIS extension of PostgreSQL, SQL Server, and the IBM DB2 Spatial Extender.

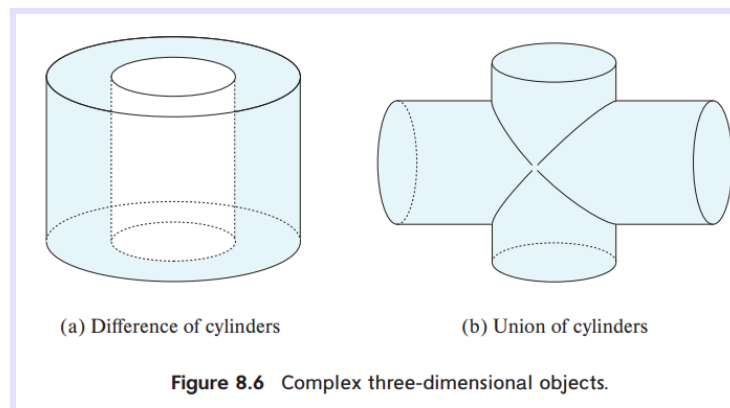
✿ Representation of Geometric Information

A polygon can be represented by listing its vertices in order, specifying the boundary of a polygonal region.



This division process is called triangulation, and any polygon can be triangulated. The complex polygon can be given an identifier, and each of the triangles into which it is divided carries the identifier of the polygon. Circles and ellipses can be represented by corresponding types or approximated by polygons.

Design databases also store nonspatial information about objects, such as the material from which the objects are constructed. Various spatial operations must be performed on a design, such as retrieving a part of the design that corresponds to a particular region of interest. Spatial-index structures are useful for such tasks. Spatial-index structures are multidimensional, dealing with two- and three-dimensional data, rather than dealing with just the simple one-dimensional ordering provided by B+-trees.



✿ Geographic data

Geographic databases for public-utility information have become very important as the network of buried cables and pipes has grown. Without detailed maps, work carried out by one utility may damage the structure of another utility, resulting in large-scale disruption of service. Geographic databases, coupled with accurate location-finding systems using GPS help avoid such problems.

In general, queries on spatial data may have a combination of spatial and nonspatial requirements. For instance, we may want to find the nearest restaurant that has vegetarian selections and that charges less than \$10 for a meal.

✿ Representation of Geographic Data

- **Raster data:** Such data consist of bitmaps or pixel maps, in two or more dimensions. Raster data are often represented as tiles, each covering a fixed-size area. A larger area can be displayed by displaying all the tiles that overlap with the area. Raster data can be three-dimensional—for example, the temperature at different altitudes at different regions.
- **Vector data:** Vector data are constructed from basic geometric objects. In the context of geographic data, points are usually represented by latitude and longitude, and where the height is relevant, additionally by elevation.

Topographical information: information about the elevation (height) of each point on a surface, can be represented in raster form.

The **triangulated irregular network (TIN)** (from triangulation) representation is a compact representation which is particularly useful for generating three-dimensional views of an area.

✿ Spatial queries

- **Region queries:** Deal with spatial regions. Such a query can ask for objects that lie partially or fully inside a specified region. A query to find all retail shops within the geographic boundaries of a given town is an example.
- **Nearness queries:** Request objects that lie near a specified location. A query to find all restaurants that lie within a given distance of a given point is an example of a nearness query. The **nearest-neighbor query** requests the object that is nearest to a specified point.
- **Spatial graph queries:** request information based on spatial graphs such as road maps.

Capitulo 9 - Database System Concepts

Application Development

Practically all use of databases occurs from within application programs, making almost all user interaction with databases indirect. This chapter details the tools, architectural layers, performance mechanics, and security strategies essential to designing interactive database applications.

Application Programs and User Interfaces

- **User Authentication and Interface Initialization:** User-centric database applications initially require identification and authentication (e.g., via a username and password). Upon validation, the application uses this identity to query and extract personalized data fields (such as a student's name or registered courses) from the database engine to generate the layout.
- **Interaction Interfaces:** The program serves as an intermediary providing specialized forms and interfaces that enable users to submit updates or invoke queries (e.g., course registrations or instructor lookups) without writing raw database code.
- **Enterprise Automation Domains:** Core application workflows are built to manage and automate highly structured institutional tasks, spanning retail sales, purchasing workflows, accounting registers, corporate payroll systems, human-resource tracking, and inventory control.
- **Primary Interface Modalities:** The two dominant presentation standards used to expose database capabilities to contemporary end-users are web interfaces and standalone mobile application layouts.

Web Fundamentals

This section outlines the basic infrastructure and components that underpin the World Wide Web.

✿ Uniform Resource Locators

- **Definition and Structure:** A Uniform Resource Locator (URL) is a globally unique name assigned to each resource or document accessible across the web.
- **Protocol Segments:** For an illustrative address such as "*http://www.acm.org/sigmod*", the initial component (http) designates the HyperText Transfer Protocol, which functions as the standardized data transfer protocol for HTML documents. The variant https denotes the cryptographically secure execution of HTTP, which is established as the default preferred standard.
- **Server and Path Mapping:** The secondary segment specifies the target host network machine executing a web server instance, while the remaining trailing parameters map to the explicit logical file paths or unique internal document identifiers on that server.

✿ HyperText Markup Language

HTML serves as the standard formatting structure for web text layout engines. Documents must be structurally enclosed within an `<html>` tag block, with the renderable contents isolated inside a `<body>` tag block.

- **Tabular Elements**: Structured data grids are defined via the `<table>` wrapper tag. Rows within a table are defined using the `<tr>` tag. Structural column headers are declared with the `<th>` cell tag, whereas regular database record fields are bound inside standard `<td>` data cell tags.
- **Data Transmission Actions**: Interactive inputs are compiled inside a `<form>` component tag, which specifies how variables are routed via HTTP methods:
 - ◆ The **GET** Method: Encodes client input parameters natively as an appended plain-text string directly inside the requested URL string. For example, submitting a query variable `q` containing the string "silberschatz" results in a URL formatted to transmit the parameter directly to the host.
 - ◆ The **POST** Method: Bundles user variables within the body portion of the HTTP request payload, hiding them from the plain-text URL line.

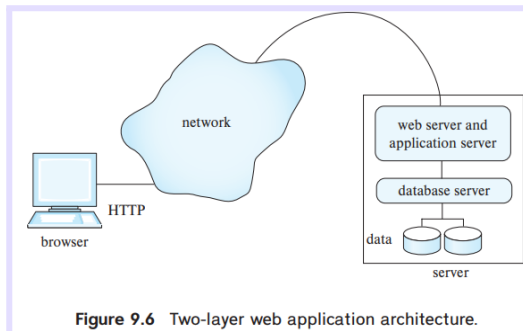
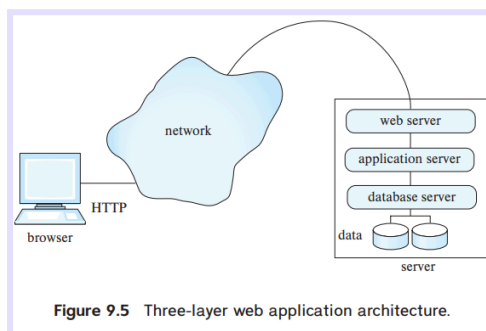
✿ Web Servers and Sessions

A **web server** is a continuous network process running on a server machine that actively listens for incoming HTTP requests from remote client browsers and responds by returning formatted document structures.

The **common gateway interface (CGI)** standard defines how the web server communicates with application programs.

Modern database applications generally decouple infrastructure responsibilities into explicit layers:

- **Three-Layer (Three-Tier) Architecture**: The client browser interfaces with a front-end web server, which routes functional processing demands to an intermediate application server, which in turn queries a separate database engine back-end.
- **Two-Layer (Two-Tier) Architecture**: The web server and application tier are unified onto a single platform layer directly querying the back-end database tier.



Core HTTP network connections are **stateless** by design: each request-response cycle is treated as an entirely independent transaction; a connection can be broken immediately after satisfying a standalone request to keep system ports open for concurrent users.

Because meaningful database interaction requires maintaining **contextual continuity across multiple pages** (e.g., keeping a user logged in without requiring re-authentication on every single action), systems use sessions to preserve stateful user contexts throughout a connected lifetime.

To implement sessions in spite of connections getting closed, extra information has to be stored at the client and returned with each request in a session; this extra information is usually maintained in the form of a cookie at the client; a **cookie** is simply a small piece of text containing identifying information and with an associated name.

Servlets

Servlets represent compiled server-side Java application components executed within an application server or web container environment to process client requests and dynamically output HTML structures.

Recall that the interaction between a browser and a web/application server is stateless. That is, each time the browser makes a request to the server, the browser needs to connect to the server, request some information, then disconnect from the server. The servlet API provides a method of tracking a session and storing information pertaining to it (`getSession()`). Then the user has to authenticate.

To create a new session, the server would internally carry out the following tasks:

1. Set a cookie (called, for example, `sessionId`) with a session identifier as its associated value at the client browser
2. Create a new session object
3. Associate the session identifier value with the session object.

Servlets import core packages including `java.io.*`, `javax.servlet.*`, and `javax.servlet.http.*`. Classes extend `HttpServlet` and override processing methods such as `doGet` or `doPost`.

Application routers direct unauthorized traffic toward a primary login view. Upon submitting credentials, a separate authentication checker validates the password. If verified, the engine issues an HTTP redirection command to route the user's browser toward the main system form; otherwise, it forces a redirect back to the login view.

Alternative Server-Side Frameworks

In contrast to pure servlets that embed text output strings inside imperative code, **frameworks** provide streamlined approaches to decouple business logic from the user interface.

Server-Side Scripting: Scripting languages provide constructs that can be embedded within HTML documents. Before delivering a web page, the server executes the scripts embedded within the HTML contents of the page. i.e: Java server pages (JSP), PHP, Django.

Client-Side Code and Web Services

Modern applications distribute processing responsibilities between server-side logic and client-side scripts to deliver highly responsive interfaces.

✿ Client-Side Code

Languages designed to be executed on the client's web browser. The primary motivation for such scripting languages is flexible interaction with the user, providing features beyond the limited interaction power provided by HTML and HTML forms.

Browsers download script files that execute locally on the user's CPU, modifying the interface structure in real time without triggering full-page browser reloads. i.e: input validation `<input type="number" name="credits" size="2" min="1" max="15" >`, responsive user interfaces, interfacing with web services (dynamic web pages)

Asynchronous Operations: Scripts can issue background network requests to fetch raw structured data from the server in the background (e.g., retrieving a list of states immediately after a user selects a country) and update specific drop-down fields or panels on the fly once the payload arrives.

✿ Web Services

A web service functions as a remote Application Programming Interface (API) exposed over standard web networks.

Two approaches are widely used to implement web services:

1. **Representation State Transfer (REST):** Web service function calls are executed by a standard HTTP request to a URL at an application server.
2. **JSON:** The requestor parses the returned page to access the returned data.

Communication Protocol: Service calls package requests into structured formats transmitted over standard HTTP. These are parsed and executed by a remote application server, which returns raw structured data payloads back to the invoking client application.

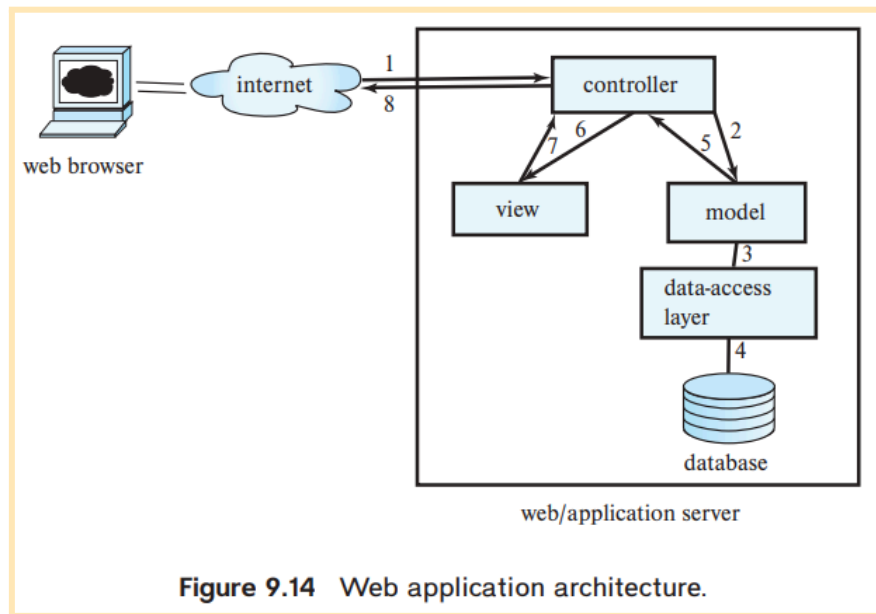
Application Architectures

To manage structural complexity, large-scale systems segment application software into decoupled architectural layers.

- **Presentation Layer:** Manages the direct display layout and governs user-system interaction. A single system often maintains multiple variants of this layer to tailor content layouts for different devices, such as desktop web browsers or small mobile phone screens. The presentation architecture is often structured using the **Model-View-Controller (MVC)** paradigm to isolate code responsibilities:
 - ◆ **Model:** Maps directly to the underlying business logic and data schema states.
 - ◆ **View:** Dictates the exact visual presentation layout of the data fields. A single model can support multiple views tailored to different software applications or user devices.
 - ◆ **Controller:** Acts as the coordinator that intercepts incoming user actions or system events, updates the state of the model accordingly, and selects the next view to return to the user.
- **Business-Logic Layer:** Encapsulates the core programmatic rules, constraints, and operational workflows of the enterprise. In addition, the business logic includes workflows, which describe how a particular task that involves multiple participants is handled.
- **Data access Layer:** provides the interface between the business-logic layer and the underlying database.

En esta parte habla de que en muchos casos se necesitaban implementar **object-oriented databases** pero que no tuvieron éxito comercial; se empezaron a utilizar bdd relacionales y a implementar **object-relational mappings(ORM)** para mapear objetos a datos de una bdd relacional

The **Hibernate** system is widely used for mapping from Java objects to relations. Hibernate provides an implementation of the Java Persistence API (JPA). In Hibernate, the mapping from each Java class to one or more relations is specified in a **mapping file**.



Application Performance

In this section, we discuss how developers try to **reduce overhead** and ensure that requests are served with **low response times**.

- **Caching Tiers**: Costs can be further reduced by **caching the final web page** that is sent in response to a request (as materialized views). High-volume database applications introduce rapid in-memory caching layers to avoid executing redundant, resource-intensive disk queries.
- **Connection pooling (Managing connections)**: The connection pool manager creates a **pool of open ODBC/JDBC connections**. Instead of opening a new connection to the database, the code servicing a user request asks for a connection from the connection pool and returns the connection to the pool when the code (servlet) completes its processing. Specialized key-value memory platforms, such as **memcached** and **Redis**, are deployed to store frequent query outputs or session variables directly in RAM, minimizing back-end database load.
- **Parallel processing**: using a large number of application servers running in parallel, each handling a fraction of the requests.

Application Security

Database-backed applications are exposed to severe malicious exploits if input processing and access controls are poorly constructed.

- **SQL Injection Attacks**: Occurs when unvalidated user inputs are concatenated directly into raw SQL query strings. Attackers manipulate form fields by embedding SQL commands that alter the statement's logic, enabling unauthorized data extraction or table destruction. This risk is mitigated by using Prepared Statements and Parameterized Queries.
- **Cross-Site Scripting (XSS)**: Happens when an application accepts malicious JavaScript input from one user, stores it in the database, and later outputs it unfiltered to other users. This allows unauthorized scripts to execute in victims' browsers, potentially stealing session cookies.
- **Cross-Site Request Forgery (CSRF)**: Exploits a browser's tendency to automatically append session cookies to requests. Attackers trick a logged-in user into loading a malicious page that secretly fires an unauthorized transaction request to the application server.

2FA = Application level authentication

- Lack of end-user information
- Lack of fine-grained authorization

It is worth noting that even with two-factor authentication, users may still be vulnerable to man-in-the-middle attacks. In such attacks, a user attempting to connect to the application is diverted to a fake web site, which accepts the password (including second factor passwords) from the user and uses it immediately to authenticate to the original application.

The **Security Assertion Markup Language (SAML)** is a protocol for exchanging authentication and authorization information between different security domains, to provide cross-organization single sign-on.

An **audit trail** is a log of all changes (inserts, deletes, and updates) to the application data, along with information such as which user performed the change and when the change was performed.

Encryption and Its Applications

Leakage of such personal information allows a criminal to impersonate someone else and get access to service or goods; such impersonation is referred to as **identity theft**.

In a **symmetric-key** encryption technique, the encryption key is also used to decrypt data. In contrast, in **public-key** (also known as **asymmetric-key**) encryption techniques, there are two different keys.

A good encryption technique has the following properties:

- It is relatively simple for authorized users to encrypt and decrypt data.

- It depends not on the secrecy of the algorithm, but rather on a parameter of the algorithm called the encryption key, which is used to encrypt data.
- Its decryption key is extremely difficult for an intruder to determine, even if the intruder has access to encrypted data.

The **Advanced Encryption Standard (AES)** is a symmetric-key encryption algorithm that was adopted as an encryption standard by the U.S. government in 2000 and is now widely used. The standard is based on the **Rijndael algorithm** (named for the inventors V. Rijmen and J. Daemen). The algorithm operates on a 128-bit block of data at a time.

Many file systems and database systems today support encryption of data. Such encryption protects the data from someone who is able to access the data but is not able to access the decryption key.

Encryption at the database level has the advantage of requiring relatively low time and space overhead and does not require modification of applications.

✿ Encryption Support in Databases

Relational database storage engines support encryption at two different tiers:

1. **Disk-Block-Level Encryption:** The database system encrypts blocks immediately before writing them to disk and decrypts them upon retrieval into the buffer manager. This protects against the physical theft of hard drives or backup media, requiring zero application code modifications.
2. **Attribute-Level Encryption:** Specific sensitive columns (such as credit card numbers) are encrypted using distinct keys, minimizing computational overhead.

Limitations of Attribute-Level Encryption

- Database systems typically do not allow primary key and foreign key attributes to be encrypted.
- Database systems cannot build standard indices on encrypted attributes, preventing range queries and fast lookups.

Decryption keys are often managed within a secured, encrypted file or relation called a **wallet**. To access encrypted data columns, an application must provide a master key to unlock the wallet. While applications can encrypt data on the client side *before* transmitting it to the database, this approach requires extensive application code modifications and prevents the database engine from sorting or filtering those values.

✿ Encryption and Authentication

To prevent network sniffers from capturing passwords, applications use challenge–response protocols. The database issues a random challenge string, the client encrypts it using their password as a key, and the server validates the response locally, ensuring passwords never travel over the network.

✿ Digital Signatures

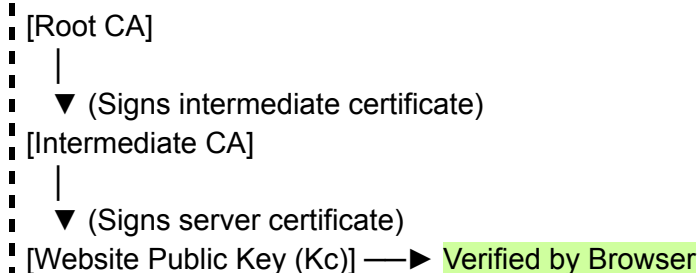
Digital signatures reverse the roles of public-key components. A sender encrypts data using their private key, allowing anyone to verify its authenticity using the corresponding public key. This guarantees data integrity and ensures **nonrepudiation**; the sender cannot falsely deny authoring the document, since only their private key could generate the signature.

✿ Digital Certificates

To prevent attackers from broadcasting fraudulent public keys, applications rely on **digital certificates**. Public keys are cryptographically signed by a trusted third-party **Certification Authority (CA)**. Standard browsers come pre-installed with the root public keys of major CAs to allow automated verification. To manage volume, a multi-level tree hierarchy is used, where parent authorities sign intermediate certificates.

El servidor de la página te entrega un certificado digital que contiene su clave pública. Tu navegador utiliza esa clave pública para encriptar los datos que le mandás (como contraseñas o tarjetas de crédito), asegurando que incluso si un sitio malicioso intercepta la comunicación, no pueda leer nada porque carece de la clave privada real del servidor. Para garantizar que esa clave pública de verdad pertenece a la página que querés visitar, la clave es firmada y validada por una Agencia de Certificación

A digital certificate includes the owner's public key (K_C) and an encrypted text block (E) signed by the CA's key (K_A). Browsers recursively decrypt these blocks using CA public keys up the chain until a trusted root CA key is reached.



In the **secure HTTPS protocol**:

1. A website delivers its digital certificate to the browser upon connection.
2. Once verified, the browser uses the validated public key to encrypt a newly generated, one-time symmetric session key and transmits it to the server. Only the legitimate website possessing the corresponding private key can decrypt this package.
3. The session then switches to symmetric encryption using this shared key to ensure high-performance, secure data transfer.

Capitulo 10 - Database System Concepts

Big Data

Motivating Examples

✿ Sources and Uses of Big Data

Traditional database applications focused on structured data generated within an enterprise, such as banking transactions or retail sales. **Big Data** is characterized by data sets whose volume, velocity, and variety are far greater than traditional datasets.

Sources of Big Data:

- **Web logs**: Tracking user clicks, page visits, and session timelines across large websites.
- **Social media networks**: Capturing user interactions, posts, likes, photos, and connection graphs.
- **Scientific applications**: Storing astronomical observations, genomic sequences, and high-energy physics simulation runs.
- **Internet of Things (IoT)**: Real-time sensor readings from smart devices, environmental monitors, fleet vehicles, and industrial machinery.

The Three Vs of Big Data:

- **Volume**: The sheer size of the data, often measured in petabytes (10^{15} bytes) or exabytes (10^{18} bytes), which exceeds the storage capacity of a single standard machine.
- **Velocity**: The rapid rate at which new data is generated and needs to be analyzed or processed, frequently arriving as continuous streams.
- **Variety**: The diverse formats of data, including structured tables, semi-structured formats (**JSON**, **XML**), and completely unstructured blocks like raw text documents, audio, and video recordings.

✿ Querying Big Data

Traditional **relational databases** rely on upfront **schema definition**, **declarative SQL queries**, and **strict ACID** transaction properties.

Building **data management systems that can scale** to a large volume/velocity of data requires **parallel storage and processing of data**. Querying millions of files or multi-petabyte datasets requires **distributed computational structures** because a single centralized CPU cannot provide acceptable response times.

Analytical requirements shift from processing small, precise transactional records to analyzing structural patterns, computing multi-attribute aggregations, and scanning vast arrays of historical logs. **Big Data query engines** often **trade off** strict transactional consistency to guarantee massive horizontal scalability and fault tolerance across a cluster of commodity servers.

Big Data Storage Systems

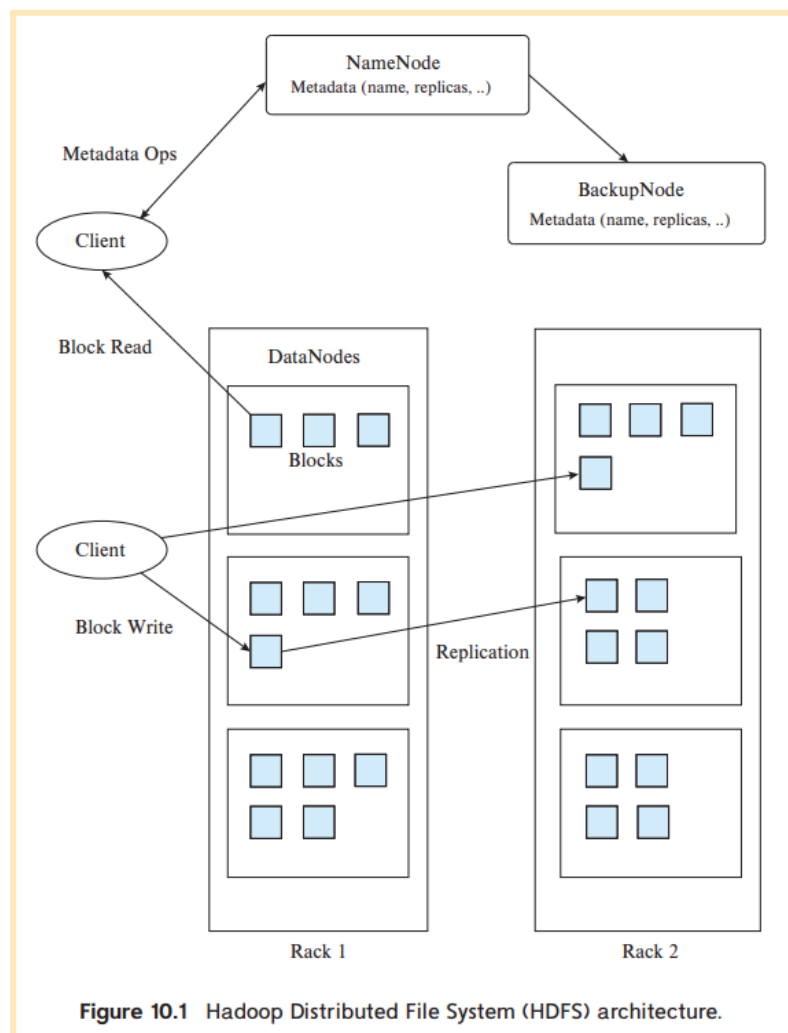
Applications on Big Data have extremely high scalability requirements. The following systems have been developed and deployed to address the data management requirements of modern applications:

- Distributed File Systems.
- Sharding across multiple databases.
- Key-Value Storage Systems.
- Parallel and Distributed Databases.

✿ Distributed file systems

A distributed file system(DFS) stores files across a large collection of machines while giving a single-file-system view to clients. As with any file system, there is a system of file names and directories, which clients can use to identify and access files.

- Hadoop Distributed File System (HDFS) architecture elements:
 - ◆ **NameNode**: A single centralized coordinator server that maintains the file system directory tree, tracks file metadata, and monitors block locations across the cluster.
 - ◆ **DataNode**: Worker nodes that store the raw data blocks on local standard disk drives and execute read/write requests issued by clients.
- **Fault Tolerance**: To protect against frequent hardware failures in commodity clusters, each block is replicated (typically three times) across distinct DataNodes situated on different network server racks.



✿ Sharding

The term sharding refers to the partitioning of data across multiple databases or machines.

Sharding is the architectural practice of dividing a large database table horizontally into smaller, self-contained pieces called shards. Each individual shard is assigned to a specific storage node within the distributed environment.

→ Partitioning strategies:

- ◆ Range-based sharding: Tuples are distributed according to contiguous ranges of a chosen sharding key attribute. This creates a risk of data hotspots if queries frequently access a specific active range.
- ◆ Hash-based sharding: A cryptographic hash function is applied to the sharding key to assign records uniformly across available nodes. This strategy minimizes hotspots but makes range-based scanning highly inefficient.

✿ Key-Value storage systems

Key-Value Stores represent highly simplified data management systems that model all information as a collection of unique keys mapped to values. These systems discard structural constraints, data schemas, foreign key relationships, and complex relational join operations.

→ Data access is strictly limited to low-latency operations:

- ◆ put(key, value): Inserts or updates a key-value pair.
- ◆ get(key): Retrieves the value associated with a specific key.
- ◆ delete(key): Removes a key-value entry from the storage system.

→ High scalability is achieved because partitioning can be executed easily using simple hashing algorithms over the primary key space.

→ Key-value stores are also called NoSQL systems, to emphasize that they do not support SQL, and the lack of support for SQL was initially viewed as something positive, rather than a limitation.

→ In MongoDB (as in many other databases), partitioning is done based on the value of a specified attribute, called the partitioning attribute or shard key.

✿ Parallel and distributed databases

Parallel databases are databases that run on multiple machines.

Unlike simple key-value platforms, Parallel and Distributed Databases seek to preserve structured relational data models and full query expressiveness across scale-out architectures.

→ Architectural variations:

- ◆ Shared-memory: Multiple processors share a single, global main memory space and a centralized disk infrastructure.
- ◆ Shared-disk: Each processor possesses its own private memory space, but all nodes access a common, interconnected disk array.
- ◆ Shared-nothing: Every independent node contains its own private processor, main memory, and local disk space. All coordination and data exchange occur

through messages sent over a high-speed network. The shared-nothing model is the **primary framework** for large-scale Big Data architectures.

✿ Replication and consistency

Distributing and replicating data across independent nodes introduces **coordination challenges when updates occur**. A key requirement here is **consistency**, that is, **all live replicas of a data item have the same value**, and each read sees the **latest version** of the data item.

- **The CAP Theorem**: States that a distributed data system can simultaneously **guarantee** at most two of the following three properties:
 - ◆ **Consistency (C)**: Every read operation returns the most recent write or an error across all nodes.
 - ◆ **Availability (A)**: Every non-failing node returns a non-error response for every received request.
 - ◆ **Partition Tolerance (P)**: The system continues to operate despite arbitrary message loss or network connection failures between nodes.
- **Consistency Models**:
 - ◆ **Strong consistency**: All replicas reflect identical data values simultaneously. This model requires complex, high-overhead coordination protocols like **Two-Phase Commit (2PC)**.
 - ◆ **Eventual consistency**: Replicas can **temporarily diverge** during updates, but the system guarantees that all storage locations will eventually converge to an identical state if no new modifications are submitted.

The MapReduce Paradigm

✿ Why MapReduce?

Processing multi-terabyte datasets using traditional database techniques requires **moving massive volumes of raw data from disk storage nodes across network switches to a centralized processing unit**. This dynamic causes **severe network bottlenecks**, stalling compute performance.

- The **MapReduce Framework** solves this problem by **moving the computational code directly to the storage nodes holding the required data blocks**. This approach is called data locality.
- It provides an **abstract execution model** that automatically manages parallel task distribution, network data sorting, node communication failures, and internal cluster coordination.

✿ MapReduce By Example 1: Word Count

- The **Word Count** application **calculates the total occurrences of each word** across an immense corpus of text files.
- The **Map Function**:
 - ◆ Receives an input text line as a key-value pair.
 - ◆ Tokenizes the raw text into individual words.
 - ◆ Emits a continuous sequence of intermediate key-value records where the key is the string word and the value is the integer 1.
 - ◆ Formally: `map(String key, Integer value) -> List<Pair<String, Integer>>`

→ The **Shuffle and Sort Phase**:

- ◆ An internal automated framework collects all intermediate key-value pairs generated by the map tasks.
- ◆ It groups all values that share an identical key into a combined list structure.

→ The **Reduce Function**:

- ◆ Receives a specific word key alongside a collection of its emitted count values.
- ◆ Sums the total array of integer values.
- ◆ Emits a single final key-value pair containing the unique word and its total calculated count.
- ◆ Formally: `reduce(String key, Iterable<Integer> values) -> Pair<String, Integer>`

✿ MapReduce by Example 2: Log Processing

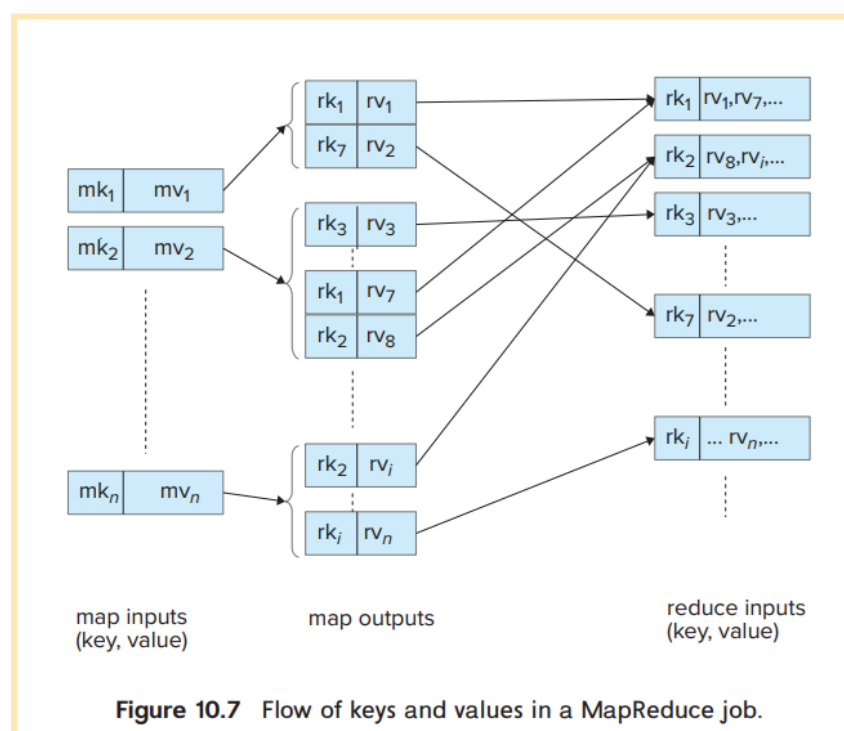
→ Organizations use MapReduce to **extract analytical data from massive web server access files** (access.log). Consider a task designed to calculate the total data bytes transferred for each distinct URL path.

→ **Map Task Implementation**:

- ◆ **Parses** individual lines of raw log text.
- ◆ **Extracts** the target URL string and the numeric bytes field from the log entry.
- ◆ Emits an **intermediate pair**: (URL_path, bytes_transferred).

→ **Reduce Task Implementation**:

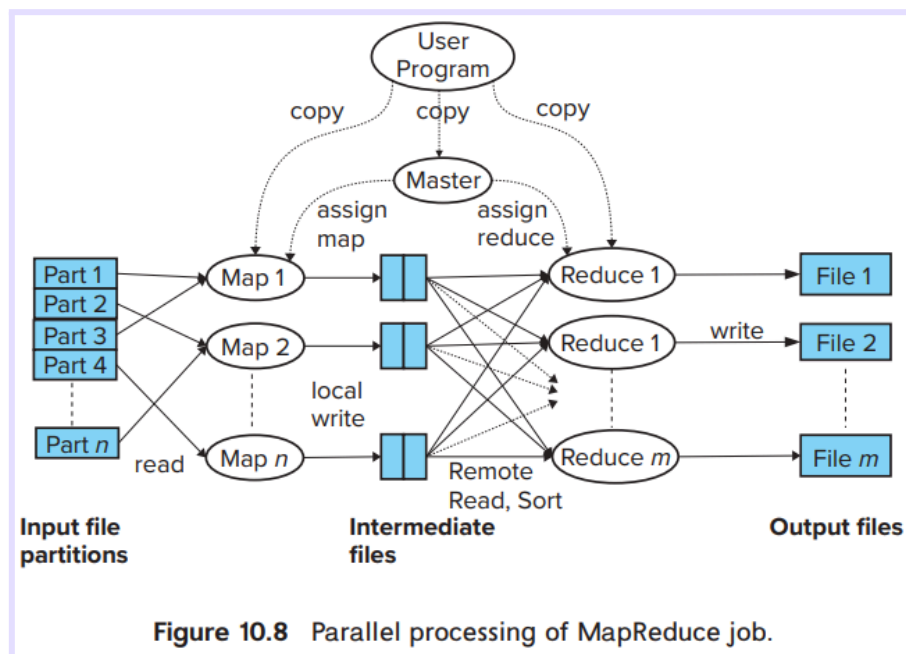
- ◆ Receives a specific URL_path key paired with a list of all byte sizes emitted across the logs.
- ◆ **Aggregates** the byte values to compute the total traffic volume for that specific path.
- ◆ Outputs the **absolute traffic sum** for each web resource.



❁ Parallel Processing of MapReduce Tasks

Execution of a **MapReduce job across a cluster** follows a structured parallel timeline:

1. **Input Splitting**: The framework divides the input data into fixed pieces (typically 64MB to 128MB), matching the underlying distributed file system block boundaries.
2. **Map Allocation**: A master coordinator schedules **Map Tasks** across nodes containing the local data splits, maximizing data locality.
3. **Local Writing**: Map outputs are written to local disk drives, sorted by their target reducer partition.
4. **Shuffling**: Reducer tasks pull their **assigned key partitions** from the map disks over the network using HTTP requests.
5. **Reduce Execution**: Reducers **process the grouped keys sequentially** and write the final outputs directly back into the distributed file system.



❁ MapReduce in Hadoop

Apache Hadoop provides an open-source framework that implements the distributed MapReduce execution model.

→ Core Components:

- ◆ **MapReduce Engine**: The compute layer that manages task scheduling, detects node failures, re-executes stalled operations, and tracks job lifecycles.
- ◆ **YARN (Yet Another Resource Negotiator)**: A centralized cluster resource manager that allocates memory and CPU containers to executing software tasks.

→ **Resilience Mechanism**: If a worker node crashes during a job, the master coordinator automatically detects the failure via missing heartbeats. It then schedules the lost map or reduce tasks on alternative nodes holding replicated copies of the raw data blocks.

✿ SQL on MapReduce

Writing raw Java or Python MapReduce programs is complex and error-prone for data analysts.

Systems like Apache Hive were created to compile declarative SQL queries into structured, multi-stage directed acyclic graphs (DAGs) of MapReduce jobs.

Hive Metadata Store: Maps traditional database concepts (tables, schemas, columns, types) onto raw text or binary files stored within HDFS (Hadoop Distributed File System) directories. This abstraction allows users to execute standard relational operations (SELECT, JOIN, GROUP BY) over unstructured distributed files without writing imperative code.

Beyond MapReduce: Algebraic Operations

✿ Motivation for Algebraic Operations

While MapReduce is powerful, it has major architectural limitations for complex data workflows:

- It forces all computations into a rigid, two-step Map and Reduce structure.
- Complex queries require chaining multiple MapReduce jobs together.
- Every stage in a chained MapReduce job must write its intermediate results completely to disk storage and reload them across the network for the next phase. This approach introduces massive performance penalties for iterative workflows like machine learning or multi-stage data graphs.

✿ Algebraic Operations in Spark

Apache Spark addresses these limitations by utilizing an in-memory data framework structured around Resilient Distributed Datasets (RDDs).

An RDD is an immutable, fault-tolerant collection of data objects partitioned across cluster nodes that can be manipulated in main memory.

→ Spark Operations Classification:

- ◆ **Transformations:** Lazy evaluations that build a directed acyclic graph (DAG) of computations instead of executing immediately (e.g., map(), filter(), join(), groupByKey()).
 - ◆ **Actions:** Operations that trigger the execution of the compiled DAG to return a final result or write data to external storage (e.g., count(), collect(), saveAsTextFile()).
- **Lineage Graphs:** Fault tolerance is achieved without aggressive data replication. Each RDD tracks the precise algebraic transformations used to construct it from the base storage layer. If a processing node crashes, Spark reconstructs only the lost partitions by replaying the lineage graph in memory.

Streaming Data

✿ Applications of Streaming Data

Many real-world datasets are unbounded, meaning new data arrives continuously over time and has no defined end point.

→ Streaming Data Applications:

- ◆ Financial trading: Tracking real-time stock price ticks to detect anomalies or execute high-frequency trades.
- ◆ Fraud detection: Analyzing credit card transaction feeds immediately to block fraudulent activities before authorization completes.
- ◆ Network telemetry: Monitoring continuous router data and system logs to identify infrastructure intrusions or performance issues.
- ◆ Sensors: vehicles, buildings, factories.
- ◆ E-commerce
- ◆ Social media

✿ Querying Streaming Data

Data stored in a database are sometimes referred to as data-at-rest, in contrast to streaming data.

One way to deal with the unbounded nature of streams is to define windows on the streams, where each window contains tuples with a certain timestamp range or a certain number of tuples.

Continuous stream processing requires a fundamental shift away from batch processing models. A stream query engine cannot wait for all data to arrive before returning a response; instead, it processes incoming records incrementally as they pass through memory.

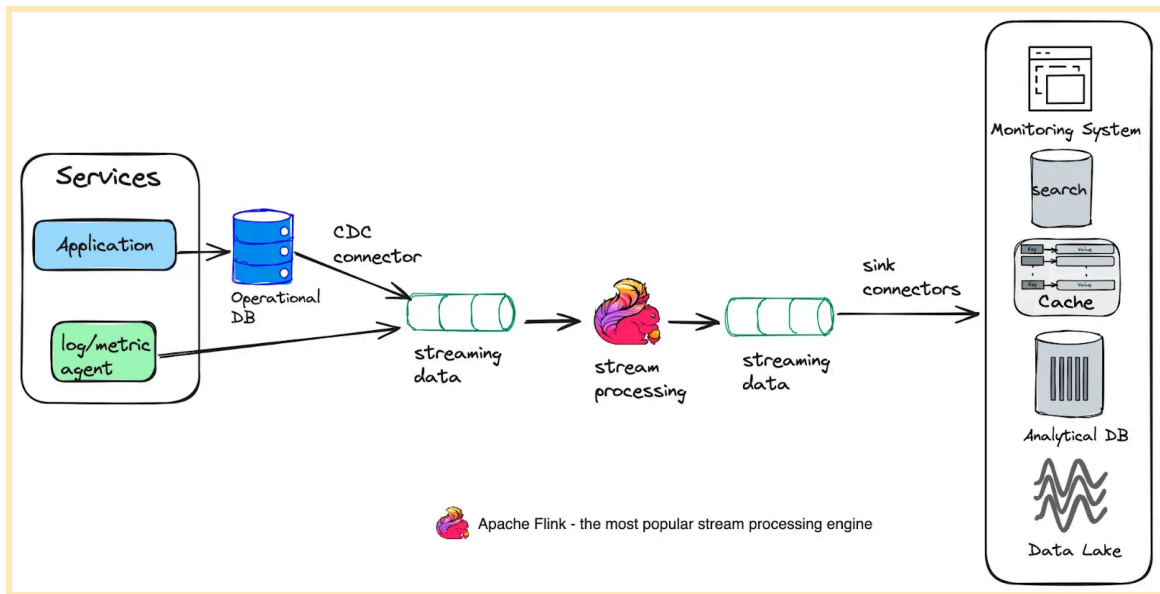
Streams would contain punctuations, that is, metadata tuples that state that all future tuples will have a timestamp greater than some value.

→ Approaches:

- ◆ Continuous queries: Incoming data streams are treated as inserts to a relation, queries that are running continuously.
- ◆ Stream query languages: Extending SQL or relational algebra (usage of windows)
- ◆ Algebraic operators on streams: allow users to write operators that are executed on each incoming tuple.
- ◆ Pattern matching: Define a pattern matching language and allow users to write multiple rules, each with a pattern and an action. Complex event processing (CEP) systems.

Many stream-processing systems keep data in-memory and do not provide persistence guarantees. On the other hand, the incoming data may also need to be stored in a database for later processing.

Lambda architecture: a copy of the input data is provided to the stream-processing system and another copy is provided to a database for storage and later processing.



✿ Stream Extensions to SQL

To handle continuous data feeds, SQL dialects add **temporal processing frameworks** called **Time Windows**.

→ **Window Classifications:**

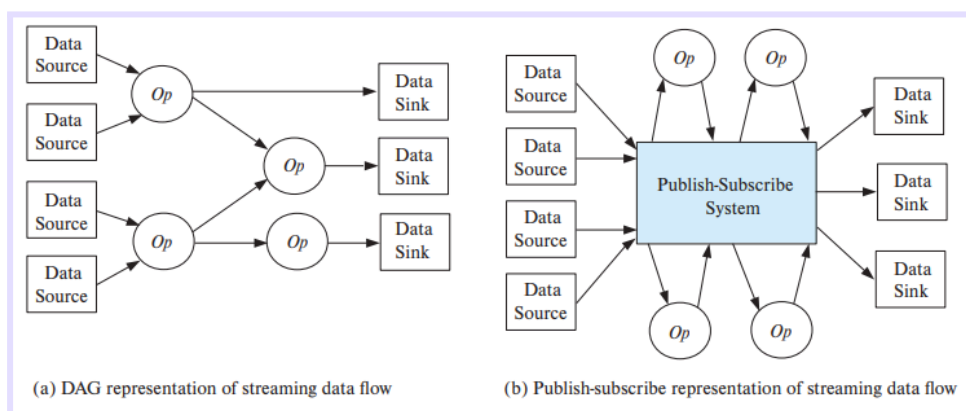
- ◆ **Tumbling Windows:** Fixed-size, non-overlapping time intervals. For example, a 5-minute tumbling window evaluates data from 10:00 to 10:05, and then opens a new interval from 10:05 to 10:10.
- ◆ **Sliding Windows:** Specified size (based on time, or number of tuples) **around each incoming tuple**.
- ◆ **Hopping Windows:** An hourly window computed every 20 minutes.
- ◆ **Session Windows:** User and a time-out interval, and contains a **sequence of operations** such that each operation occurs within the timeout interval from the previous operation.

```
SELECT stream_hop_start(rowtime, INTERVAL '1' MINUTE, INTERVAL '10' MINUTE) as window_start,
       sensor_id,
       AVG(temperature)
FROM sensor_stream
GROUP BY HOP(rowtime, INTERVAL '1' MINUTE, INTERVAL '10' MINUTE), sensor_id;
```

✿ **Algebraic Operations on Streams**

Stream processing engines like Apache Flink or Spark Structured Streaming apply functional **algebraic operators** directly to continuous data feeds.

The logical routing of tuples is done by creating a **directed acyclic graph (DAG)** with **operators as nodes**.



A **publish-subscribe system** allows publication of documents or other forms of data, with an **associated topic**. Whenever a document is published to a particular topic, a **copy of the document** is sent to all **subscribers** who have subscribed to that topic. A major benefit of the publish-subscribe approach is that operators can be added to the system, or **removed** from it, with **relative ease**.

The Apache Kafka system uses the publish-subscribe model to manage routing of tuples in streams. Tuples published for a topic are retained for a specified period of time.

Apache Spark allows streaming data sources to be used as inputs for such operations. Spark breaks up streams into **discretized streams**, where the stream data for a **particular time window** are treated as a data input to algebraic operators.

Graph Databases

Graph Databases treat data **relationships** as first-class citizens, moving away from traditional relational tables, primary-foreign key pairings, and row-scanning mechanics.

- Data is modeled using nodes, edges, and properties:
 - ◆ **Nodes**: Represent distinct **entities** or objects (e.g., a specific user, a product, or a business location).
 - ◆ **Edges**: Directed lines that explicitly connect nodes, representing **real-world relationships** (e.g., knows, purchased, or located_in).
 - ◆ **Properties**: Key-value pairs attached to either nodes or edges to store descriptive metadata (e.g., name: "Alice" on a node, or since: "2021" on an edge).

Graph based databases support query languages designed for easily expressing path queries (such as Cypher for Neo4j), which may be harder to express in SQL.

- **Index-Free Adjacency**: Relational databases must execute expensive index-lookup joins at runtime to connect tables. In contrast, **graph databases** store **direct physical pointers on disk between connected nodes and edges**. This design allows queries to traverse millions of relationships per second with constant time complexity ($O(1)$), regardless of the overall dataset size.

There are two popular approaches for parallel graph processing:

- **Map-reduce and algebraic frameworks**: Graphs can be represented as relations, and individual steps of many parallel graph algorithms can be represented as joins. Graphs can thus be stored in a parallel storage system, partitioned across multiple machines.
- **Bulk synchronous processing frameworks**: Framework that frames **graph algorithms as computations associated with vertices that operate in an iterative manner**. Each vertex (node) of the graph has data (state) associated with it. In the BSP framework programmers provide methods that are executed for each node of the graph. In each iteration, called a **superstep**, the **method associated with each node** is executed;

Summary

- Modern data management applications often deal with non-relational, distributed datasets whose immense scale surpasses the storage and generation capacity of single standard enterprises.
- Distributed file systems provide high-capacity file abstractions across standard hardware clusters using centralized coordinate directories like the HDFS NameNode alongside distributed DataNodes.
- The MapReduce pattern streamlines massively distributed batch architectures into simple user-defined map and reduce steps, automated via underlying data-shuffling operations.
- Unbounded streaming systems demand continuous real-time execution across sliding or tumbling intervals fed by scalable publish-subscribe brokers.
- Graph databases eliminate table-join overheads by storing rich network structures as direct node-to-node relationships, optimizing complex path traversal queries.

Capitulo 11 - Database System Concepts

Data Analytics

Acompañame a terminar de caer en la locura...

The term **data analytics** refers broadly to the processing of data to infer patterns, correlations, or models for prediction. The results of analytics are then used to drive business decisions.

Overview of Analytics

- These models take as input attributes of individuals, and output predictions (such as the likelihood of paying back a loan or clicking on an advertisement), which are then used to make business decisions.
- The cost of making wrong decisions can be very high.
- The field of data mining combines knowledge-discovery techniques invented by artificial intelligence researchers and statistical analysts with efficient implementation techniques that enable them to be used on extremely large databases.
- Decision-support systems come in various forms, including OLAP systems and data-mining systems.
- The term **business intelligence (BI)** is used in a broadly similar sense to **data analytics**. The term **decision support** is also used in a related but narrower sense, with a focus on reporting and aggregation, but not including machine learning/data mining.

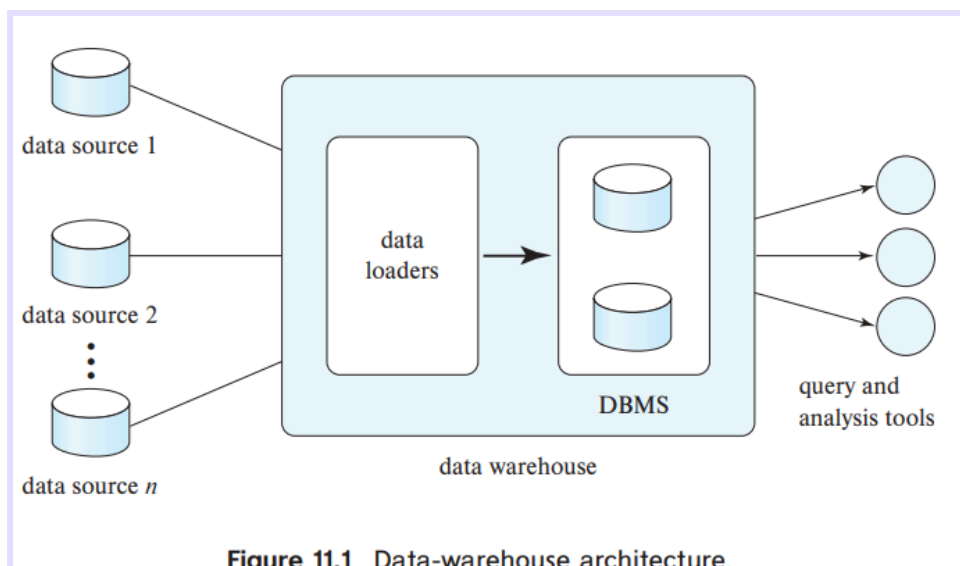
Data Warehousing

A **data warehouse** is a repository of information gathered from multiple sources, stored under a unified schema, at a single site. Once gathered, the data are stored for a long time, permitting access to historical data.

Warehouses are used for decision support and analysis on historical data, for instance, to predict trends.

✿ Components of a Data Warehouse

- **Data Gathering (When and How):** In a **source-driven architecture** for gathering data, the data sources transmit new information, either continually or periodically. In a **destination-driven architecture**, the data warehouse periodically sends requests for new data to the sources. Many data warehouses do not use synchronous replication because it can be expensive.



- **Schema Integration:** Part of the task of a warehouse is to perform schema integration and to convert data to the integrated schema before they are stored. Data stored in the warehouse can be thought of as a materialized view of the data at the sources.
- **Data Transformation and Cleansing:** The task of correcting and preprocessing data is called data cleansing. This includes correcting minor inconsistencies like misspelled names and addresses using fuzzy lookup (data matching a query based on similarity rather than an exact character-for-character match). Address lists collected from multiple sources may have duplicates that need to be eliminated in a merge-purge operation (also referred to as deduplication). Data may be transformed in ways other than cleansing, such as changing the units of measure.
- **Updating and Summarizing:** Updates on relations at the data sources must be propagated to the data warehouse, which is basically the view-maintenance problem. Raw data can be too large to store online, so systems can maintain summary data obtained by aggregation instead of the entire relation.
- **ETL vs. ELT:** The different steps involved in getting data into a data warehouse are called extract, transform, and load (ETL) tasks. In current generation data warehouses that support user-defined functions or MapReduce frameworks, data may be extracted, loaded into the warehouse, and then transformed; these steps are referred to as extract, load, and transform or ELT tasks.

❁ Warehouse Schemas

Warehouse schemas tend to be multidimensional, involving one or a few very large fact tables and several much smaller dimension tables.

- **Fact Tables:** Record information about individual events, such as sales, and are usually very large. The attributes in a fact table can be classified as either dimension attributes or measure attributes. The measure attributes store quantitative information, which can be aggregated upon. The dimension attributes are dimensions upon which measure attributes are grouped and viewed.
- **Dimension Tables:** To minimize storage requirements, dimension attributes are usually short identifiers that are foreign keys into other tables called dimension tables.

A fact table **sales** would have dimension attributes **item_id**, **store_id**, **customer_id**, and **date**, and measure attributes **number** and **price**. The attribute **store_id** is a foreign key into a dimension table **store**.

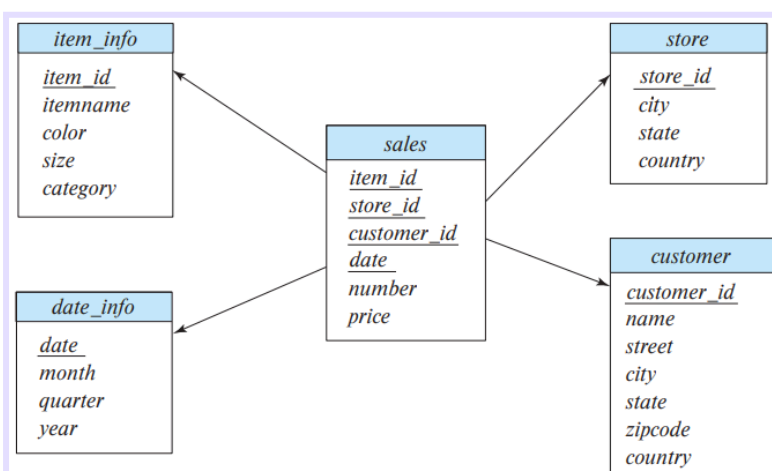


Figure 11.2 Star schema for a data warehouse.

→ **Star Schema:** A schema with a fact table, multiple dimension tables, and foreign keys from the fact table to the dimension tables is called a star schema.

- **Snowflake Schema:** More complex data-warehouse designs may have multiple levels of dimension tables; for instance, an item table may have a foreign key into another table giving manufacturer details. Such schemas are called snowflake schemas.

✿ Database Support for Data Warehouses

A transaction-processing database needs to support many small queries, which may involve updates in addition to reads. In contrast, data warehouses typically need to process far fewer queries, but each query accesses a much larger amount of data.

Records are typically never updated once they are added to a data warehouse relation. Thus, data warehouses do not need to pay any overhead for concurrency control.

Databases traditionally store all attributes of a tuple together sequentially in a file, which is referred to as row-oriented storage. In contrast, in column-oriented storage, each attribute of a relation is stored in a separate file, with values from successive tuples stored at successive positions in the file.

Since small deletes and updates do not occur in a data warehouse, large sequences of attribute values can be stored and compressed together as one unit, allowing for better compression.

Selection operations on a single attribute have significantly lower cost in a column store since only the relevant attributes need to be accessed.

✿ Data Lakes

The term data lake is used to refer to a repository where data can be stored in multiple formats, including structured records and unstructured file formats.

Unlike data warehouses, data lakes do not require up-front effort to preprocess data, but they do require more effort when creating queries.

Since data may be stored in many different formats, querying tools also need to be quite flexible. Apache Hadoop and Apache Spark are popular tools for querying data lakes, since they support querying of both unstructured and structured data.

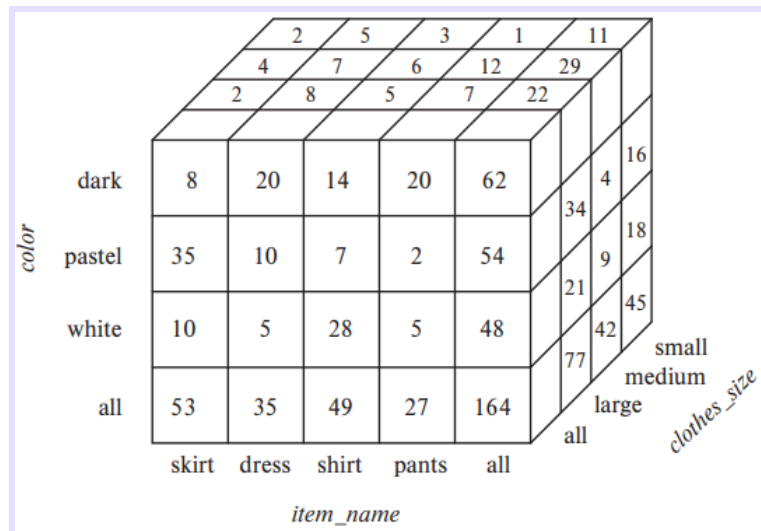
Online Analytical Processing

Online Analytical processing (OLAP) tools help analysts view data summarized in different ways, so that they can gain insight into the functioning of an organization.

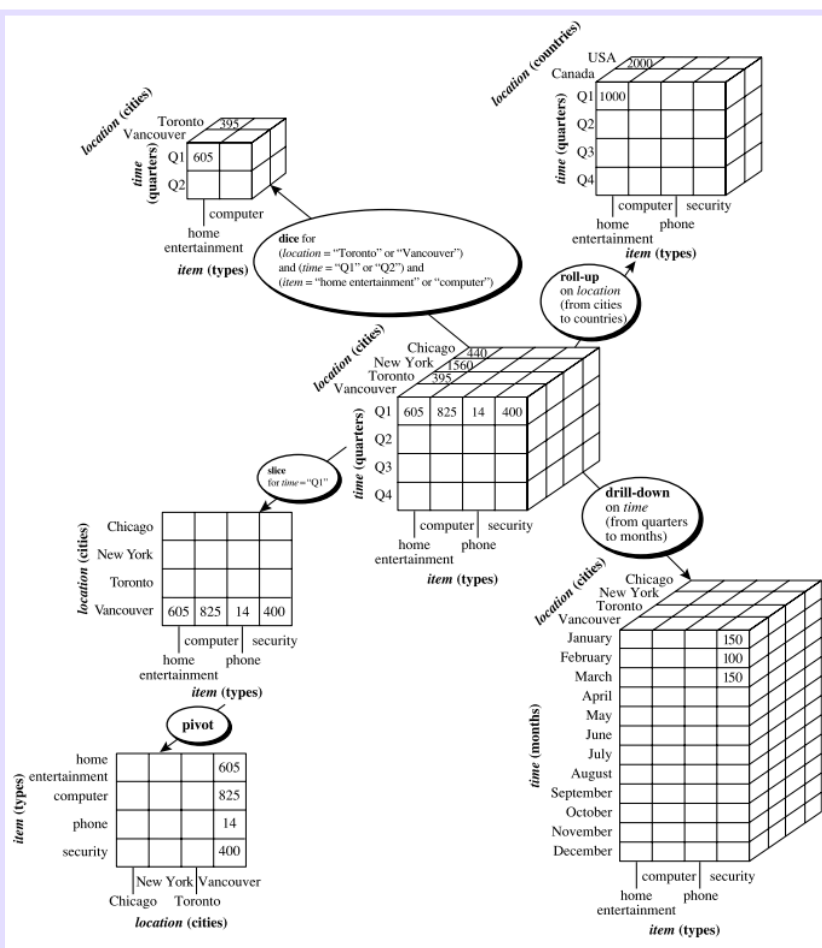
✿ Aggregation on Multidimensional Data

OLAP tools work on multidimensional data, characterized by dimension attributes and measure attributes.

- **Data Cube:** The data cube consists of multidimensional data summarized in different ways. Precomputing the data cube helps speed up queries on summaries of data. A data cube on an n-dimensional table can perform aggregation with grouping on each of the 2^n subsets of the n dimensions.



→ **OLAP Operations:** An online analytic processing (OLAP) system allows a data analyst to look at different **cross-tabs** on the same data by interactively selecting the attributes in the cross-tab.



◆ **Pivoting:** The operation of changing the dimensions used in a cross-tabulation display.

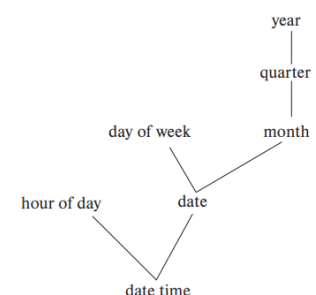
◆ **Slicing:** Viewing a specific slice of the data cube by fixing a single value for an attribute dimension.

◆ **Dicing:** Selecting a smaller subset of a data cube, particularly when values for multiple dimensions are fixed.

◆ **Rollup:** The operation of moving from finer-granularity data to a coarser granularity by means of aggregation.

◆ **Drill down:** The opposite operation of rollup, moving from coarser-granularity data to finer-granularity data.

The different levels of detail for an attribute can be organized into a **hierarchy**.



(a) time hierarchy

✿ Relational Representation of Cross-Tabs

A cross-tabulation display is different from relational tables usually stored in databases, since the number of columns in the cross-tab depends on the actual data. A change in data values may result in adding more columns, which is not desirable for data storage.

- A cross-tab with summary rows/columns can be represented in a relational table by introducing a special value **all** to represent subtotals. The SQL standard actually uses the null value in place of all, but the textbook uses all to avoid confusion with regular null values.
- Tuples with the value all for a dimension are obtained by performing an aggregation on the relation with a group by on the remaining columns.

item_name	color	clothes_size	quantity
skirt	dark	all	8
skirt	pastel	all	35
skirt	white	all	10
skirt	all	all	53
dress	dark	all	20
dress	pastel	all	10
dress	white	all	5
dress	all	all	35
shirt	dark	all	14
shirt	pastel	all	7
shirt	white	all	28
shirt	all	all	49
pants	dark	all	20
pants	pastel	all	2
pants	white	all	5
pants	all	all	27
all	dark	all	62
all	pastel	all	54
all	white	all	48
all	all	all	164

Figure 11.8 Relational representation of the data in Figure 11.4.

✿ OLAP in SQL

The SQL standard provides a variety of operators for data analysis, including cube, rollup, and pivot operations.

- **Pivot Clause:** Several SQL implementations support a pivot clause that allows the creation of cross-tabs directly from data. The for clause within the pivot clause specifies the pivot attribute, the specific values that should appear as attribute names, and the aggregate function used to compute the cells.
- **Cube Construct:** The cube construct generates groupings consisting of all subsets of the attributes listed. It produces a single relation where tuples contain null as the value for those attributes not present in a particular grouping.
- **Rollup Construct:** The order of the attributes in the rollup construct makes a difference. It specifies a sequence of drilling down or rolling up for hierarchies of frequent practical interest (e.g., Region, Country, State, City).
- **Grouping Function:** To substitute all for null, systems use the grouping() function. It returns 1 if its argument is a null value generated by a cube or rollup, and 0 otherwise, allowing case expressions to replace OLAP-generated nulls with the string 'all'.

✿ Reporting and Visualization Tools

Report generators are tools to generate human-readable summary reports from a database. They integrate querying the database with the creation of formatted text and summary charts (such as bar or pie charts).

Acme Supply Company, Inc. Quarterly Sales Report			
Period: Jan. 1 to March 31, 2009			
Region	Category	Sales	Subtotal
North	Computer Hardware	1,000,000	1,500,000
	Computer Software	500,000	
	All categories		
South	Computer Hardware	200,000	600,000
	Computer Software	400,000	
	All categories		
Total Sales			2,100,000

Figure 11.10 A formatted report.

Techniques for data visualization, that is, graphical representation of data, that go beyond the basic chart types, are very important for data analysis. Data-visualization systems help users to examine large volumes of data and to detect patterns visually.

Interaction is a key element of visualization. For example, a user can "drill down" into areas of interest, such as moving from an aggregate view showing the total sales across an entire year to the monthly sales figures for a particular year. Analysts may wish to interactively add selection conditions to visualize subsets of data using visualization tools.

Data Mining

Data mining is the process of semiautomatically analyzing large databases to find useful patterns. It differs from traditional machine learning and statistics in that it deals with large volumes of data stored primarily on disk.

Data-mining techniques form part of the process of knowledge discovery in databases (KDD). Some types of knowledge discovered from a database can be represented by a set of rules. Knowledge discovered by applying machine-learning techniques on past instances in a database is represented by a model.

✿ Types of Data-Mining Tasks

The most widely used applications of data mining are those that require some sort of prediction.

Examples include predicting if a credit card applicant is a good credit risk, predicting which customers may switch over to a competitor, predicting who is likely to respond to promotional mail, or predicting what phone-card usage is likely to be fraudulent.

Another class of applications looks for associations, for instance, books that tend to be bought together. Associations and clusters are an example of descriptive patterns.

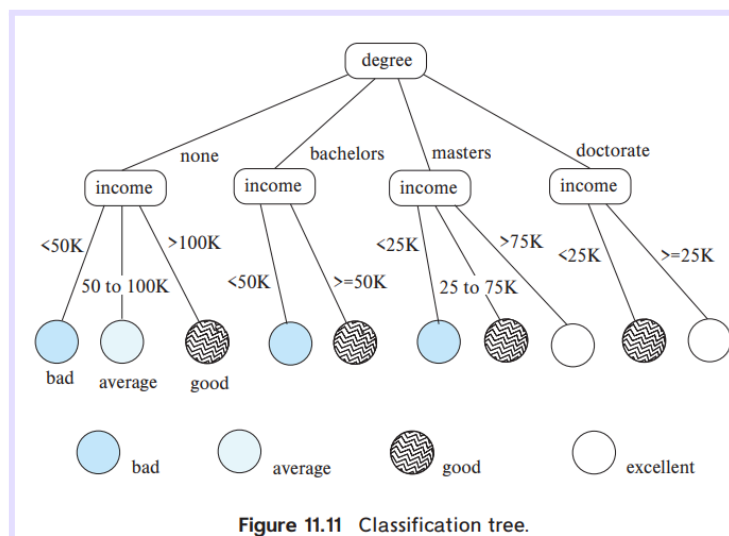
✿ Classification

Abstractly, the **classification** problem is this: Given that items belong to one of several classes, and given past instances (called **training instances**) of items along with the classes to which they belong, the problem is to **predict the class to which a new item belongs**.

Classification deals with predicting the class of **test instances** by using attributes of the test instances, based on attributes of **training instances**, and the actual class of training instances.

✿ Decision-Tree Classifiers

As the name suggests, decision-tree classifiers use a **tree structure where each leaf node has an associated class, and each internal node has a predicate (or function) associated with it**.



✿ Bayesian Classifiers

Bayesian classifiers find the **distribution of attribute values for each class in the training data**.

When given a new instance d , they use the distribution information to estimate, for each class c_j , the **probability that instance d belongs to class c_j** , denoted by $p(c_j | d)$.

Bayes' theorem:

$$p(c_j | d) = \frac{p(d | c_j)p(c_j)}{p(d)}$$

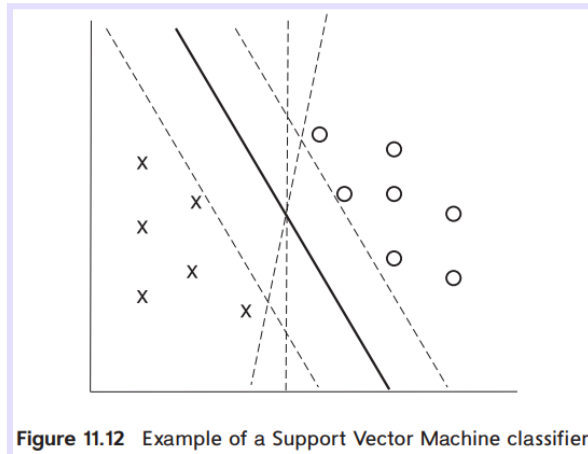
The class with maximum probability becomes the predicted class for instance d . **Naive Bayesian classifiers** work under the assumption that the attributes are **probabilistically independent of each other given the class**.

$$p(d | c_j) = p(d_1 | c_j) * p(d_2 | c_j) * \dots * p(d_n | c_j)$$

✿ Support Vector Machine Classifiers

The **Support Vector Machine (SVM)** is a type of classifier that can best be understood geometrically.

In the simplest case, given a training set of points in a two-dimensional plane belonging to two distinct classes, the SVM builds a classifier by identifying a **linear boundary that maximizes the margin separating the groups of points**.



✿ Neural Network Classifiers

Neural network classifiers utilize interconnected nodes layered to compute **continuous weights based on training inputs** to classify distinct complex datasets.

Deep neural networks, which are neural networks with a large number of layers, have proven very successful at such tasks, if given a very large number of training instances. The term **deep learning** refers to the machine-learning techniques that create such deep neural networks, and train them on very large numbers of training instances.

✿ Regression

While classification assigns an instance to a distinct categorical class, **regression** deals with **predicting a continuous or numerical output value** based on the attributes of input features.

One way is to infer coefficients $a_0, a_1, a_2, \dots, a_n$ such that:

$$Y = a_0 + a_1 * X_1 + a_2 * X_2 + \dots + a_n * X_n$$

Finding such a linear polynomial is called **linear regression**. In general, we wish to find a **curve that fits the data**; the process is also called **curve fitting**.

✿ Association Rules

Association rules identify **items that co-occur frequently**, for instance, items that tend to be bought by the same customer. A rule takes a form like: bread $\rightarrow$ milk.

An association rule must have an associated **population**: The population consists of a **set of instances**. In the grocery-store example, the population may consist of all grocery-store purchases.

Rules have an associated support, as well as an associated confidence. These are defined in the context of the population:

- **Support**: Measures what fraction of the population satisfies both the antecedent and the consequent of the rule (percentage of purchases that include bread and milk).
- **Confidence**: Measures the conditional probability that an item occurs given that another item is already in the transaction. *“how often the consequent is true when the antecedent is true.”*
- **Correlations** look for deviations from expected levels of association to isolate meaningful rules from simple accidental co-occurrences.

✿ Clustering

Intuitively, clustering refers to the problem of finding clusters of points in the given data. It groups similar points together into a single set without using any predefined categories.

One way to formalize it from distance metrics is to group points into k sets (for a given k) so that the average distance of points from the centroid of their assigned cluster is minimized. The centroid of a set of points is defined as a point whose coordinate on each dimension is the average of the coordinates of all the points of that set on that dimension.

✿ Text Mining

Text mining applies data-mining techniques to textual documents.

- **Sentiment Analysis**: Analyzing blocks of text, such as product reviews on e-commerce platforms, to automatically categorize user reaction scores as positive, negative, or neutral based on keyword correlations.
- **Information Extraction**: Discovering structures, facts, and context records embedded directly inside vast text repositories. Creates structured information from unstructured textual descriptions. Entity recognition, that is, the task of identifying mentions of entities in text and disambiguating them.
- **Disambiguation**: Process of figuring out which of these two is being referred to in a particular article, and it can be done based on the article context
- **Knowledge Graphs**: Extracted text information from web resources can be represented in a graph, where entities and values are nodes, and relationships are edges. Web search engines use these knowledge graphs to generate more meaningful answers to user queries.

Capitulo 12 - Database System Concepts

Review Terms

- Physical storage media
 - Cache
 - Main memory
 - Flash memory
 - Magnetic disk
 - Optical storage
 - Tape storage
- Volatile storage
- Non-volatile storage
- Sequential-access
- Direct-access
- Storage interfaces
 - Serial ATA (SATA)
 - Serial Attached SCSI (SAS)
 - Non-Volatile Memory Express (NVMe)
 - Storage area network (SAN)
 - Network attached storage (NAS)
- Magnetic disk
 - Platter
 - Hard disks
 - Tracks
 - Sectors
 - Read-write head
 - Disk arm
 - Cylinder
 - Disk controller
 - Checksums
 - Remapping of bad sectors
 - Level 6 (block striping, P + Q redundancy)
- Rebuild performance
- Software RAID
- Hardware RAID
- Hot swapping
- Optimization of disk-block access
- Disk block
- Performance measures of disks
 - Access time
 - Seek time
 - Latency time
 - I/O operations per second (IOPS)
 - Rotational latency
 - Data-transfer rate
 - Mean time to failure (MTTF)
- Flash Storage
 - Erase Block
 - Wear leveling
 - Flash translation table
 - Flash Translation Layer
- Storage class memory
 - 3D-XPoint
- Redundant arrays of independent disks (RAID)
 - Mirroring
 - Data striping
 - Bit-level striping
 - Block-level striping
- RAID levels
 - Level 0 (block striping, no redundancy)
 - Level 1 (block striping, mirroring)
 - Level 5 (block striping, distributed parity)
 - Disk-arm scheduling
 - Elevator algorithm
 - File organization
 - Defragmenting
 - Non-volatile write buffers
 - Log disk

Physical storage systems

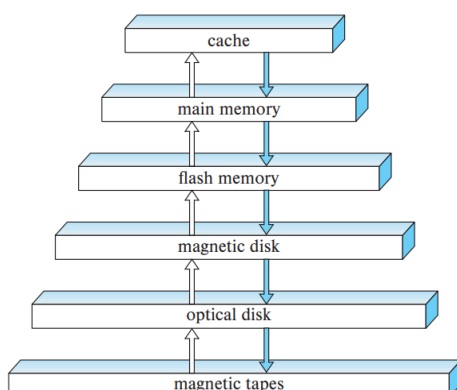
The goal of a database system is to simplify and facilitate access to data; users of the system should not be burdened unnecessarily with the physical details of the implementation of the system.

Overview of physical storage media

These storage media are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability.

- **Cache:** The cache is the fastest and most costly form of storage. Cache memory is relatively small; its use is managed by the computer system hardware
- **Main memory:** The storage medium used for data that are available to be operated on is the main memory. It is generally too small (or too expensive) for storing the entire database for very large databases, but many enterprise databases can fit in main memory. However, the contents of main memory are lost in the event of a power failure or system crash; main memory is therefore said to be **volatile**.
- **Flash memory:** Flash memory differs from main memory in that stored data are retained even if power is turned off (or fails)—that is, it is **non-volatile**. Flash memory has a lower cost per byte than main memory, but a higher cost per byte than magnetic disks. A **solid-state drive (SSD)** uses flash memory internally to store data but provides an interface similar to a magnetic disk, allowing data to be stored or retrieved in units of a block.
- **Optical storage:** The digital video disk (DVD) is an optical storage medium, with data written and read back using a laser light source.
- **Tape storage:** Tape storage is used primarily for backup and archival data. Archival data refers to data that must be stored safely for a long period of time.

However, access to data is much slower because the tape must be accessed sequentially from the beginning of the tape; tape storage is referred to as **sequential-access** storage. The various storage media can be organized in a **hierarchy** (Figure 12.1) according to their speed and their cost. The higher levels are expensive, but fast. As we move down the hierarchy, the cost per bit decreases, whereas the access time increases.



The fastest storage media—for example, cache and main memory—are referred to as **primary storage**. The media in the next level in the hierarchy—for example, flash memory and magnetic disks—are referred to as **secondary storage**, or online storage. The media in the lowest level in the hierarchy—for example, magnetic tape and optical disk jukeboxes—are referred to as **tertiary storage**, or offline storage.

Storage interfaces

Disks typically support either the **Serial ATA (SATA)** interface, or the **Serial Attached SCSI (SAS)** interface.

The **NonVolatile Memory Express (NVMe)** interface is a logical interface standard developed to better support SSDs and is typically used with the PCIe interface.

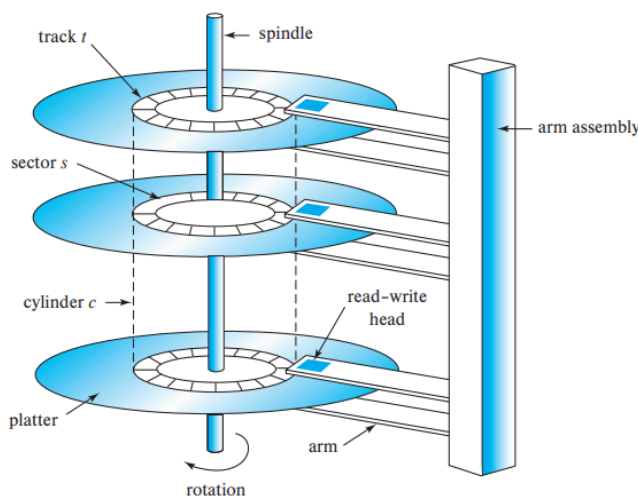
In the **storage area network (SAN)** architecture, large numbers of disks are connected by a high-speed network to a number of server computers. **Network attached storage (NAS)** is an alternative to SAN. NAS is much like SAN, except that instead of the networked storage appearing to be a large disk, it provides a file system interface using networked file system protocols such as NFS or CIFS

Magnetic disks

Magnetic disks provide the **bulk of secondary storage** for modern computer systems.

✿ Physical characteristics of disks

Each disk platter has a flat, circular shape. Its two surfaces are covered with a magnetic material, and information is recorded on the surfaces. Platters are made from rigid metal or glass.



The disk surface is logically divided into **tracks**, which are subdivided into **sectors**. A **sector** is the **smallest unit of information** that can be read from or written to the disk.

The **read-write head** stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material.

A disk typically contains many platters, and the read-write heads of all the tracks are mounted on a single assembly called a **disk arm** and move together. The disk platters mounted on a spindle and the **heads mounted on a disk arm** are together known as **head-disk assemblies**.

The ith tracks of all the platters together are called the ith **cylinder**. The read-write heads are kept as close as possible to the disk surface to **increase the recording density**.

Head crashes can be a problem. If the head contacts the disk surface, the head can scrape the recording medium off the disk, destroying the data that had been there; in older-generation disks, a head crash could thus result in **failure of the entire disk**. Current-generation disk drives **use a thin film of magnetic metal as recording medium**. They are much less susceptible to failure of the entire disk, but are susceptible to **failure of individual sectors**.

A **disk controller** interfaces between the computer system and the actual hardware of the disk drive;

A disk controller accepts **high-level commands to read or write a sector**, and initiates actions, such as moving the disk arm to the right track and actually reading or writing the data. Disk controllers also attach **checksums** to each sector that is written (el checksum sirve para comprobar si los datos fueron corrompidos).

Another interesting task that disk controllers perform is **remapping of bad sectors**. If the controller detects that a sector is damaged when the disk is initially formatted, or when an attempt is made to write the sector, **it can logically map the sector to a different physical location**

✿ **Performance measures of disks**

The main measures of the qualities of a disk are capacity, access time, data-transfer rate, and reliability.

Access time: Time from when a read or write request is issued to when data transfer begins.

The time for repositioning the arm is called the **seek time**, and it increases with the distance that the arm must move. The **average seek time** ($\frac{1}{3}$ of the worst seek time) is the average of the seek times, measured over a **sequence of random requests**.

Once the head has reached the desired track, the **time spent waiting for the sector to be accessed to appear under the head** is called the **rotational latency time**. The **average latency time** of the disk is **one-half the time for a full rotation of the disk**.

Once the first sector of the data to be accessed has come under the head, data transfer begins. The **data-transfer rate** is the **rate at which data can be retrieved from or stored to the disk**.

A **disk block** is a logical unit of storage allocation and retrieval, and block sizes today typically range from 4 to 16 kb. **Data are transferred between disk and main memory in units of blocks**. The term **page** is often used to refer to blocks

A sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. In a **sequential access pattern**, successive requests are for successive block numbers, which are on the same track.

In contrast, in a **random access pattern**, successive requests are for blocks that are randomly located on disk. Each request would require a seek.

The number of **I/O operations per second (IOPS)**, that is, the number of random block accesses that can be satisfied by a disk in a second, depends on the access time, and the block size, and the data transfer rate of the disk.

The final commonly used measure of a disk is the **mean time to failure (MTTF)**, which is a measure of the reliability of the disk. The mean time to failure of a disk (or of any other system) is the amount of time that, on average, we can expect the system to run continuously without any failure. According to vendors' claims, the mean time to failure of disks today ranges from 500,000 to 1,200,000 hours—about 57 to 136 years.

Flash memory

There are two types of flash memory:

1. NOR
2. **NAND**: predominantly used for data storage. Reading from NAND flash requires an entire page of data to be fetched from NAND into main memory (similar to sectors in magnetic disks).

Solid-state disks (SSDs) are built using NAND flash and provide the same block-oriented interface as disk storage. SSDs can provide **much faster random access**.

The **data transfer** rate of SSDs is higher than that of magnetic disks and is usually limited by the interconnect technology.

The **power consumption** of SSDs is also significantly lower than that of magnetic disks.

Writes to flash memory are a little more complicated. However, once written, a page of flash memory **cannot be directly overwritten**; it has to be erased and rewritten. The erase operation must be performed on a group of pages, called an **erase block**.

Flash memory systems limit the impact of both the slow erase speed and the update limits by **mapping logical page numbers to physical page numbers**. When a logical page is updated, it can be remapped to any already erased physical page, and the original location can be erased later. The logical-to-physical page mapping is replicated in an **in-memory translation table** for quick access.

Cold y hot data para las páginas que fueron borradas/actualizadas muchas o pocas veces. This principle of **evenly distributing erase operations across physical blocks** is called **wear leveling** and is usually performed transparently by flash-memory controllers. All the above

actions are carried out by a layer of software called the **flash translation layer**; above this layer, flash storage looks identical to magnetic disk storage; but flash is much faster.

✿ Performance

- **Number of random block reads per second:** Typical values in 2018 are about 10,000 random reads per second. Unlike magnetic disks, SSDs can support multiple random requests in parallel, with 32 parallel requests being commonly supported.
- **The data transfer rate for sequential reads and sequential writes:** 400 to 500 megabytes per second.
- **The number of random block writes per second:** Typical values in 2018 are about 40,000 random 4-kilobyte writes per second.

Hybrid disk drives are hard-disk systems that combine magnetic storage with a smaller amount of flash memory, which is used as a cache for frequently accessed data.

RAID

Having a large number of disks in a system presents opportunities for improving the rate at which data can be read or written, if the disks are operated in parallel. Potential for improving the reliability of data storage, because redundant information can be stored on multiple disks. Thus, failure of one disk does not lead to loss of data.

A variety of disk-organization techniques, collectively called **redundant arrays of independent disks (RAID)**, have been proposed to achieve improved performance and reliability. RAID systems are used for their higher reliability and higher performance rate, rather than for economic reasons. Another key justification for RAID use is easier management and operations.

✿ Improvement of reliability via redundancy

The chance that at least one disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail. If we store only one copy of the data, then each disk failure will result in loss of a significant amount of data.

The solution to the problem of reliability is to introduce **redundancy**; that is, we store extra information that is not needed normally but that can be used in the event of failure of a disk to rebuild the lost information.

The simplest (but most expensive) approach to introducing redundancy is to duplicate every disk. This technique is called **mirroring**. Data will be lost only if the second disk fails before the first failed disk is repaired.

The **mean time to failure** (where failure is the loss of data) of a mirrored disk depends on the mean time to failure of the individual disks, as well as on the **mean time to repair**, which is the time it takes (on an average) to replace a failed disk and to restore the data on it.

Then, if the mean time to failure of a single disk is 100,000 hours, and the mean time to repair is 10 hours, the **mean time to data loss** of a mirrored disk system is approx 57.000 years.

The assumption of **independence** (no connection between the failure of one disk and the failure of the other) of disk failures is not valid. As disks age, the probability of failure increases. Power failures are a particular source of concern, since they occur far more frequently than do natural disasters. **Power failures** are not a concern if there is no data transfer to disk in progress when they occur.

✿ Improvement in performance via parallelism

With disk mirroring, the rate at which read requests can be handled is doubled, since read requests can be sent to either disk. The transfer rate of each read is the same as in a single-disk system, but the **number of reads per unit time has doubled**.

With multiple disks, we can improve the transfer rate as well (or instead) by **striping data** across multiple disks. In its simplest form, data striping consists of splitting the bits of each byte across multiple disks; such striping is called **bit-level striping**.

Block-level striping stripes blocks across multiple disks. With an array of n disks, block-level striping assigns logical block i of the disk array to disk $(i \bmod n) + 1$.

In summary, there are **two main goals** of parallelism in a disk system:

1. **Load-balance multiple small accesses** (block accesses), so that the throughput of such accesses increases.
2. **Parallelize large accesses** so that the **response time of large accesses is reduced**.

✿ RAID levels

Mirroring provides high reliability, but it is expensive. Striping provides high data-transfer rates, but does not improve reliability.

Blocks in a RAID system are partitioned into sets. For a given set of blocks, a **parity block** can be computed and stored on disk.

The i th bit of the parity block is computed as the “exclusive or” (XOR) of the i th bits of the all blocks in the set. If the contents of any one of the blocks in a set is lost due to a failure, the **block contents can be recovered by computing the bitwise-XOR** of the remaining blocks in the set, along with the parity block. Whenever a block is written, the **parity block for its set must be recomputed** and written to disk.

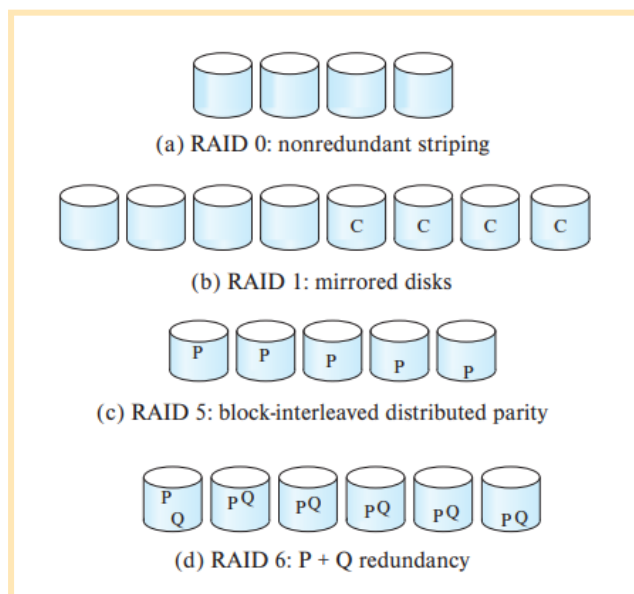
¿Cómo se calcula la paridad?

Los bloques de datos de los discos se agrupan en conjuntos. Para cada conjunto, el sistema calcula un bloque especial llamado "bloque de paridad". Este cálculo se realiza bit por bit aplicando la operación lógica **XOR (O exclusivo)** entre todos los bloques de datos del conjunto.

Recuperación ante fallos

Si un disco falla y se pierde un bloque de datos, ¡no se pierde la información! El sistema puede reconstruir exactamente el contenido del bloque perdido aplicando de nuevo la operación **XOR** entre los bloques que sobrevivieron y el bloque de paridad.

The schemes are classified into **RAID levels**. **P** indicates error-correcting bits, and **C** indicates a second copy of the data.



level 0: Disk arrays with striping at the level of blocks, but without any redundancy.

level 1: Disk mirroring with block striping. En la imagen vemos los discos con datos y sus respectivas copias

level 5: Block-interleaved distributed parity. P es el bloque de paridad

level 6: Is much like RAID level 5, but it stores extra redundant information to guard against multiple disk failures.

❁ Hardware issues

RAID can be implemented with no change at the hardware level, using only **software modification**. Such RAID implementations are called **software RAID**. Systems with special hardware support are called hardware **RAID systems**.

Hardware RAID implementations can use **non-volatile RAM** to record writes before they are performed. In case of power failure, when the system comes back up, it retrieves information about any incomplete writes from non-volatile RAM and then completes the writes. In contrast, with **software RAID** extra work needs to be done to detect blocks that may have been partially written before power failure.

Even if all writes are completed properly, there is a **small chance of a sector in a disk becoming unreadable at some point**, even though it was successfully written earlier. Reasons for loss of data on individual sectors could range from manufacturing defects to data corruption on a track when an adjacent track is written repeatedly. When such a failure

happens, if it is detected early the data can be recovered from the remaining disks in the RAID organization.

To minimize the chance of such data loss, good RAID controllers perform **scrubbing**; that is, during periods when disks are idle, every sector of every disk is read, and if any sector is found to be unreadable, the data are recovered from the remaining disks in the RAID organization, and the sector is written back.

Server hardware is often designed to permit **hot swapping**; that is, faulty disks can be removed and replaced by new ones without turning power off. The RAID controller can detect that a disk was replaced by a new one and can immediately proceed to reconstruct the data that was on the old disk, and write it to the new disk.

Many RAID implementations assign a **spare disk** for each array (or for a set of disk arrays). If a disk fails, the spare disk is immediately used as a replacement.

Good RAID implementations have **multiple redundant power supplies**. Such RAID systems have **multiple disk interfaces** and **multiple interconnections** to connect the RAID system to the computer system (or to a network of computer systems). Thus, failure of any single component will not stop the functioning of the RAID system.

✿ Choice of RAID level

The factors to be taken into account in choosing a RAID level are:

- **Monetary cost** of extra disk-storage requirements.
- Performance requirements in terms of **number of I/O operations per second**.
- Performance when a **disk has failed**.
- Performance **during rebuild** (i.e., while the data in a failed disk are being rebuilt on a new disk).

The **rebuild performance** of a RAID system may be an important factor if continuous availability of data is required, as it is in high-performance database systems.

- Rebuilding is easiest for **RAID level 1**, since data can be copied from another disk;
- **RAID level 0** is used in a few high-performance applications where data safety is not critical, but not anywhere else.
- **RAID level 1** is popular for applications such as **storage of log files in a database system**, since it offers the best write performance.
- **RAID level 5** has a **lower storage overhead** than level 1, but it has a higher time overhead for writes.
- For applications where **data are read frequently, and written rarely**, **level 5** is the preferred choice.
- **RAID level 5** has a significant **overhead for random writes**, since a single random block write requires 2 block reads (to get the old values of the block and parity block) and 2 block writes to write these blocks back.
- **RAID level 1** is therefore the RAID level of choice for many applications with moderate storage requirements and high random I/O requirements.

→ RAID level 6 offers better reliability than level 1 or 5. Higher storage cost than RAID level 5.

Armar una tabla con la característica especial de cada uno, y lugar de implementación/uso. También una tabla para comparar todos los medios de almacenamiento

Disk-block access

.As we saw earlier, a sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. A number of techniques have been developed for improving the speed of access to blocks, by minimizing the number of accesses, and in particular minimizing the number of random accesses.

Buffering: Blocks that are read from disk are stored temporarily in an in-memory buffer, to satisfy future requests.

Read-ahead: When a disk block is accessed, consecutive blocks from the same track are read into an in-memory buffer even if there is no pending request for the blocks. In the case of sequential access, such read-ahead ensures that many blocks are already in memory when they are requested, and it minimizes the time wasted in disk seeks and rotational latency per block read.

Scheduling: If several blocks from a cylinder need to be transferred from disk to main memory, we may be able to save access time by requesting the blocks in the order in which they will pass under the heads. Disk-arm-scheduling algorithms attempt to order accesses to tracks in a fashion that increases the number of accesses that can be processed. A commonly used algorithm is the elevator algorithm (lee de adentro hacia afuera, detecta si hay requests en algún track y sigue hacia afuera hasta que termina el escaneo, vuelve a ir para el centro, hasta que no ve ninguna request en los tracks que van al centro y empieza a ir hacia afuera nuevamente).

File organization: To reduce block-access time, we can organize blocks on disk in a way that corresponds closely to the way we expect data to be accessed. For example, if we expect a file to be accessed sequentially, then we should ideally keep all the blocks of the file sequentially on adjacent cylinders. Over time, a sequential file that has multiple small appends may become fragmented; that is, its blocks become scattered all over the disk. To reduce fragmentation, the system can make a backup copy of the data on disk and restore the entire disk.

scheduling = pedirlos como están ordenados; FO = Ordenarlos como los van a pedir

Non-volatile write buffers: Since the contents of main memory are lost in a power failure, information about database updates has to be recorded on disk to survive possible system crashes. The idea is that, when the database system (or the operating system) requests that a block be written to disk, the disk controller writes the block to a non-volatile write buffer and immediately notifies the operating system that the write completed successfully. The controller can subsequently write the data to their destination on disk in a way that minimizes disk arm movement.

Capitulo 13 - Database System Concepts

Data storage structures

In this chapter, we focus on the organization of data stored on the underlying storage media, and how data are accessed.

Database storage architecture

Persistent data are stored on non-volatile storage. Magnetic disks as well as SSDs are block structured devices, that is, data are read or written in units of a block. In contrast, databases deal with records, which are usually much smaller than a block.

Given a set of records, the next decision lies in how to organize them in the file structure; for example, they may be stored in sorted order, in the order they are created, or in an arbitrary order.

For a CPU to access data, it must be in main memory, whereas persistent data must be resident on non-volatile storage such as magnetic disks or SSDs.

Some applications need very fast access to data and have small enough data sizes that the entire database can fit into the main memory of a database server machine. In such cases, we can keep a copy of the entire database in memory. Databases that store the entire database in memory and optimize in-memory data structures as well as query processing and other algorithms used by the database to exploit the memory residency of data are called main-memory databases.

File organization

1. A database is mapped into a number of different files that are maintained by the underlying operating system.
2. These files reside permanently on disks.
3. A file is organized logically as a sequence of records.
4. These records are mapped onto disk blocks.
5. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying file system.

We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called blocks, which are the units of both storage allocation and data transfer. Many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications.

A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that no record is larger than a block. Each record is entirely contained in a single block;

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records.

✿ Fixed-length records

```
type instructor = record
    ID varchar (5);
    name varchar(20);
    dept_name varchar (20);
    salary numeric (8,2);
end
```

En este caso tendríamos 53 bytes por tupla

However, there are two problems with this simple approach:

1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries (two block accesses).
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block.

When a record is deleted, we could move the record that comes after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (It might be easier to move the final record of the file into the space occupied by the deleted record).

Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a file header to store the address of the first record whose contents are deleted (and other info). We can think of these stored addresses as pointers, since they point to the location of a record.

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

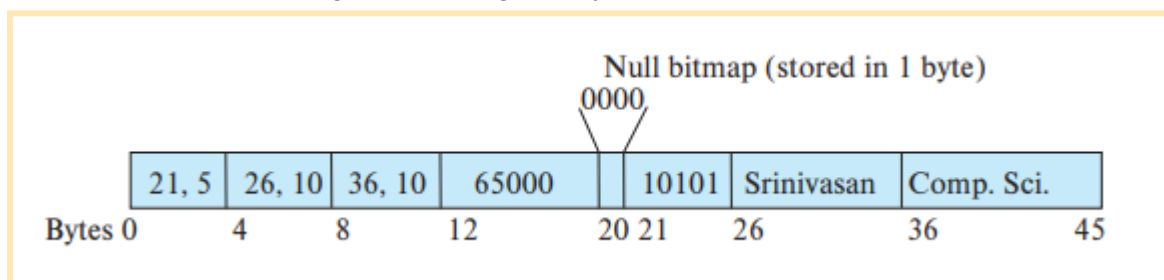
✿ Variable-length records

Variable-length records arise in database systems due to several reasons. The most common reason is the presence of variable length fields, such as strings. Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

1. How to represent a single record in such a way that individual attributes can be extracted easily, even if they are of variable length
2. How to store variable-length records within a block, such that records in a block can be extracted easily

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length information, whose structure is the same for all records of the same relation, followed by the contents of variable-length attributes.

Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (offset, length), where offset denotes where the data for that attribute begins within the record, and length is the length in bytes of the variable-sized attribute.

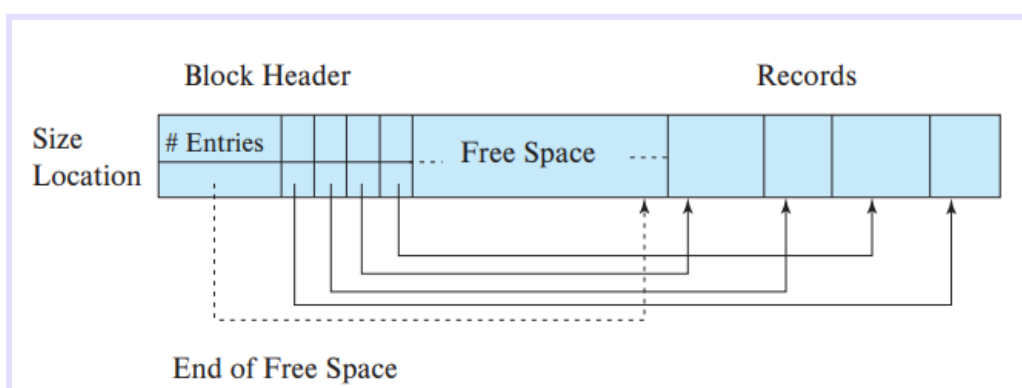


Instructor record whose first three attributes *ID*, *name*, and *dept_name* are variable-length strings, and whose fourth attribute *salary* is a fixed-sized number.

The figure also illustrates the use of a null bitmap, which indicates which attributes of the record have a null value. In this particular record, if the salary were null, the fourth bit of the bitmap would be set to 1, and the salary value stored in bytes 12 through 19 would be ignored.

The slotted-page structure is commonly used for organizing records within a block. There is a header at the beginning of each block, containing the following information:

- The number of record entries in the header
- The end of free space in the block
- An array whose entries contain the location and size of each record



The free space in the block is contiguous between the final entry in the header array and the first record. If a record is **deleted**, the space that it occupies is freed, and **its entry is set to deleted**.

The slotted-page structure **requires that there be no pointers that point directly to records**. Instead, pointers **must point to the entry in the header** that contains the actual location of the record.

✿ Storing large objects

Databases often store data that can be much larger than a disk block. Many databases internally restrict the size of a record to be no larger than the size of a block.

Large objects may be **stored either as files** in a file system area managed by the database, or as **file structures** stored in and managed by the database.

There are some **performance issues** with storing very large objects in databases. The **efficiency of accessing large objects via database interfaces** is one concern. A second concern is the **size of database backups**. Storing large objects in the database can result in a large **increase in the size of the database dumps**.

Storing data in files outside the database can result in **file names in the database pointing to files that do not exist**, perhaps because they have been deleted, which results in a form of **foreign-key constraint violation**. Some databases support **file system integration with the database**, to ensure that constraints are satisfied, and to ensure that access authorizations are enforced.

Organization of records in files

Several of the possible ways of organizing records in files are:

- **Heap file organization**: Any record can be placed anywhere in the file where there is space for the record. There is **no ordering of records**.
- **Sequential file organization**: Records are stored in **sequential order**, according to the value of a “search key” of each record.
- **Multitable clustering file organization**: Generally, a separate file or set of files is used to store the records of each relation. However, in a multitable clustering file organization, **records of several different relations are stored in the same file, and in fact in the same block within a file**, to reduce the cost of certain join operations.
- **B+-tree file organization**: The B+-tree file organization is related to the B+-tree index structure described in that chapter and can provide **efficient ordered access to records** even if there are a large number of insert, delete, or update operations. Further, it **supports very efficient access to specific records, based on the search key**.

- **Hashing file organization:** A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed.

✿ Heap file organization

Once placed in a particular location, the record is not usually moved. (se pone donde haya espacio)

When a record is inserted in a file, one option for choosing the location is to always add it at the end of the file. However, if records get deleted, it makes sense to use the space thus freed up to store new records. It is important for a database system to be able to efficiently find blocks that have free space, without having to sequentially search through all the blocks of the file.

Most databases use a space-efficient data structure called a **free-space map** to track which blocks have free space to store records. The free-space map is commonly represented by an array containing 1 entry for each block in the relation. Each entry represents a fraction f such that at least a fraction f of the space in the block is free.

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

A. Estructura de un Array (Arreglo)

El mapa de espacio libre se representa comúnmente como un gran arreglo en el que cada posición (índice) corresponde exactamente a un bloque del archivo de datos. Si tu archivo tiene 16 bloques, el mapa tendrá 16 entradas. PDF + 1

B. Almacenamiento en Fracciones (f)

Cada entrada en este mapa no guarda el número exacto de bytes libres, sino una fracción (f) que representa el espacio disponible estimado. PDF

- **Ejemplo teórico del libro:** Imagina que el sistema utiliza 3 bits para almacenar esta fracción en cada entrada. Al usar 3 bits, los números posibles van del 0 al 7, y para saber la fracción real de espacio libre, el valor de la entrada se divide por 8. PDF + 1
 - Si una entrada tiene el valor 7, significa que al menos $\frac{7}{8}$ del bloque está libre. PDF
 - Si tiene un 4, significa que al menos $\frac{4}{8}$ (la mitad) del bloque está libre.

The free-space map is written periodically; as a result, the free-space map on disk may be outdated, and when a database starts up, it may get outdated data about available free space.

✿ Sequential file organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order.

To minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. We can manage deletion by using pointer chains

Two rules for managing insertion:

1. Locate the record in the file that comes before the record to be inserted in searchkey order.
2. If there is a free record within the same block as this record, insert the new record there. Otherwise, insert the new record in an overflow block.

❁ Multitable Clustering File Organization

Most relational database systems store each relation in a separate file, or a separate set of files. Thus, each file, and as a result, each block, stores records of only one relation, in such a design. However, in some cases it can be useful to store records of more than one relation in a single block.

This structure allows for efficient processing of the join (between two relations).

A multitable clustering file organization is a file organization that stores related records of two or more relations in each block. The cluster key is the attribute that defines which records are stored together; in our preceding example, the cluster key is dept_name.

dept_name	building	budget
Comp. Sci.	Taylor	100000
Physics	Watson	70000

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

✿ Partitioning

Many databases allow the records in a relation to be partitioned into smaller relations that are stored separately. Such table partitioning is typically done on the basis of an attribute value.

The cost of some operations, such as finding free space for a record, increases with relation size; by reducing the size of each relation, partitioning helps reduce such overheads.

Data-dictionary storage

In general, such “data about data” are referred to as metadata. Relational schemas and other metadata about relations are stored in a structure called the data dictionary or system catalog. We store:

- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of views defined on the database, and definitions of those views
- Integrity constraints
- Names of users, the default schemas of the users, and passwords or other information to authenticate users
- Information about authorizations for each user DATA ON USERS OF THE SYSTEM

The data dictionary may also note the storage organization of relations.

También almacena donde están almacenadas las relaciones, información sobre los índices de cada relación.

All this metadata information constitutes a miniature database. By using database relations to store system metadata, we simplify the overall structure of the system and harness the full power of the database for fast access to system data. The exact choice of how to represent system metadata by relations must be made by the system designers.

Database buffer

Since data access from disk is much slower than in-memory data access, a major goal of the database system is to minimize the number of block transfers between the disk and memory.

One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The buffer is that part of main memory available for storage of copies of disk blocks. The subsystem responsible for the allocation of buffer space is called the buffer manager.

✿ Buffer manager

Programs in a database system make requests on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block.

Replacement strategy

When there is no room left in the buffer, a block must be evicted, that is, removed, from the buffer before a new one can be read in. Most operating systems use a least recently used (LRU) scheme.

Pinned blocks

The process executes a pin operation on the block; the buffer manager never evicts a pinned block. When it has finished reading data, the process should execute an unpin operation, allowing the block to be evicted when required.

The block cannot be evicted until all processes that have executed a pin have then executed an unpin operation. A simple way to ensure this property is to keep a pin count for each buffer block.

Shared and exclusive locks on buffer

The locking system provided by the buffer manager allows a database process to lock a buffer block either in shared mode or in exclusive mode before accessing the block, and to release the lock later, after the access is completed.

Output of blocks

It makes sense to not wait until the buffer space is needed, but to rather write out (output) updated blocks ahead of such a need. Then, when space is required in the buffer, a block that has already been written out can be evicted.

Forced output of blocks

There are situations in which it is necessary to write a block to disk, to ensure that certain data on disk are in a consistent state. Such a write is called a forced output of a block.

✿ Buffer replacement strategies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. It is not possible to predict accurately which blocks will be referenced. If a block must be replaced, the least recently referenced block is replaced. This approach is called the least recently used (LRU) block-replacement scheme.

```
select *  
from instructor natural join department;
```

The buffer manager should be instructed to free the space occupied by an instructor block as soon as the final tuple has been processed. This buffer-management strategy is called the **loss-immediate** strategy. For some systems, the optimal strategy for block replacement for the above procedure is the **most recently used (MRU)** strategy.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again.

Reordering of writes and recovery

Database buffers allow writes to be performed in-memory and output to disk at a later time, possibly in an **order different from the order in which the writes were performed**. Such reordering **can lead to inconsistent data on disk in the event of a system crash**.

However, most disks do not come with a non-volatile write buffer; instead, **modern file systems assign a disk for storing a log of the writes in the order that they are performed**. Such a disk is called a **log disk**.

For each write, the log contains the block number to be written to, and the data to be written, in the order in which the writes were performed. **File systems that support log disks as above are called journaling file systems**.

Column oriented storage

In contrast, in **column-oriented storage**, also called a **columnar storage**, each attribute of a relation is stored separately, with values of the attribute from successive tuples stored at successive positions in the file.

In the simplest form of column-oriented storage, each attribute is stored in a separate file.

Column-oriented storage thus has the **drawback** that **fetching multiple attributes of a single tuple requires multiple I/O operations**.

However, column-oriented storage is **well suited for data analysis queries**, which process many rows of a relation, but often only access some of the attributes. The reasons are as follows:

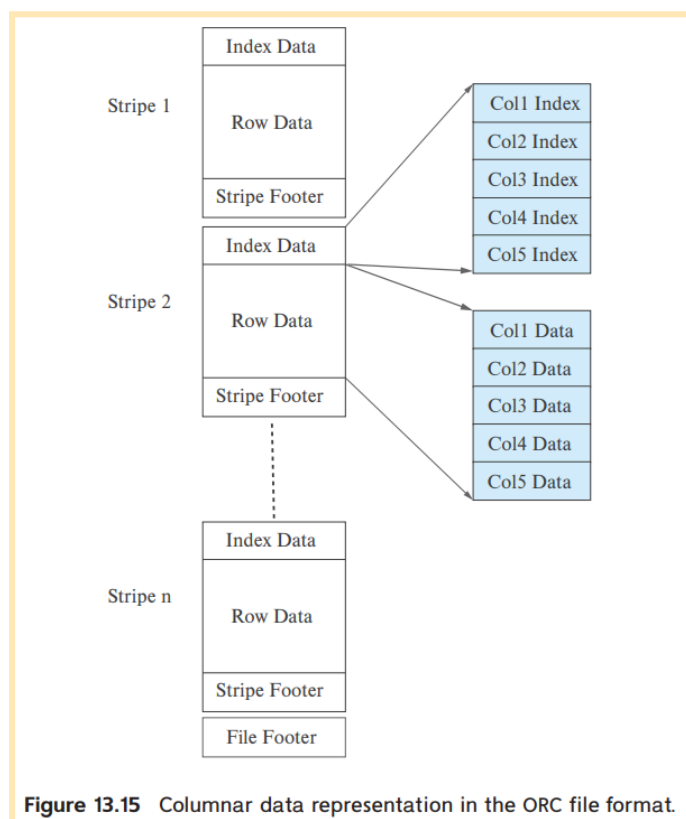
- **Reduced I/O**: When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory.
- **Improved CPU cache performance**: When the query processor fetches the contents of a particular attribute, with modern CPU architectures **multiple consecutive bytes, called a cache line**, are fetched from memory to CPU cache.

- **Improved compression:** Storing values of the same type together significantly increases the effectiveness of compression, when compared to compressing data stored in row format.
- **Vector processing:** Many modern CPU architectures support vector processing, which allows a CPU operation to be applied in parallel on a number of elements of an array.

Databases that use column-oriented storage are referred to as **column stores**, while databases that use row-oriented storage are referred to as **row stores**.

Drawbacks: Cost of tuple reconstruction, Cost of tuple deletion and update, Cost of decompression.

A sequence of tuples occupying several hundred megabytes is broken up into a columnar representation called a **stripe**.



Some column-store systems allow groups of columns that are often accessed together to be **stored together**, instead of breaking up each column into a different file.

The choice of which attributes to store together depends on the **query workload**.

Using column-oriented storage within the block provides the benefits of more **efficient memory access** and **cache usage**, as well as the **potential for using vector processing** on the data.

Storage organization in main-memory databases

Today, main-memory sizes are large enough, and main memory is cheap enough, that **many organizational databases fit entirely in memory**.

Such large main memories can be used by allocating a **large amount of memory to the database buffer**, which will allow the entire database to be loaded into buffer, **avoiding disk I/O operations for reading data**; **updated blocks still have to be written back to disk for persistence**.

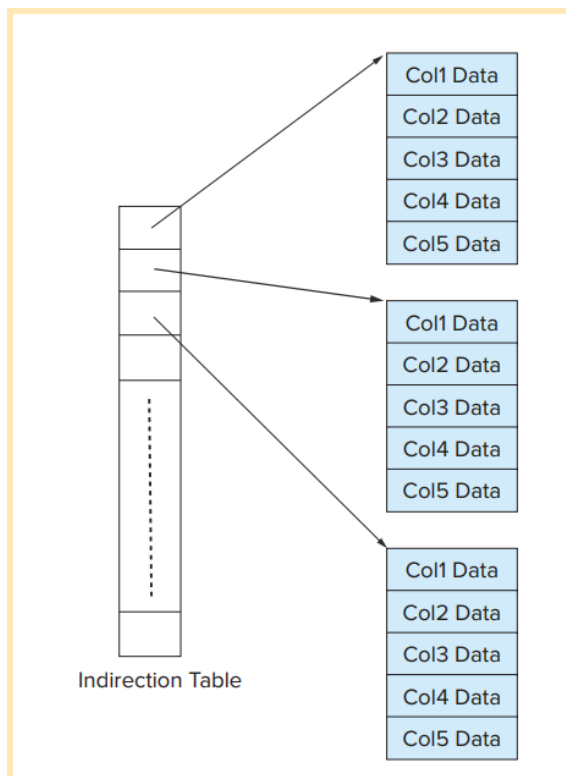
A **main-memory database** is a database where all data reside in memory.

In contrast, in a main-memory database, it is possible to keep direct pointers to records in memory, and accessing a record is just an in-memory pointer traversal, which is a very efficient operation. This is possible as long as records are not moved around.

Many main-memory databases do not use a slotted-page structure for allocating records. Instead they directly allocate records in main memory, and ensure that records never get moved due to updates to other records.

However, a problem with direct allocation of records is that memory may get fragmented if variable sized records are repeatedly inserted and deleted. The database must ensure that main memory does not get fragmented over time.

For a column-oriented storage scheme the logical array for a column may be divided into multiple physical arrays. An indirection table stores pointers to all the physical arrays.



Capitulo 14 - Database System Concepts

Review Terms

- Index type
 - Ordered indices
 - Hash indices
- Evaluation factors
 - Access types
 - Access time
 - Insertion time
 - Deletion time
 - Space overhead
- Search key
- Ordered indices
 - Ordered index
 - Clustering index
- Nonleaf nodes
- Internal nodes
- Range queries
- Node split
- Node coalesce
- Redistribute of pointers
- Uniquifier
- B⁺-tree extensions
 - Prefix compression
 - Bulk loading
 - Bottom-up B⁺-tree construction
- B-tree indices
- Hash file organization
 - Hash function
 - Bucket
 - Overflow chaining
 - Closed addressing
 - Closed hashing
 - Bucket overflow
 - Skew
 - Static hashing
 - Primary indices;
 - Nonclustering indices
 - Secondary indices
 - Index-sequential files
- Index entry
- Index record
- Dense index
- Sparse index
- Multilevel indices
- Nonunique search key
- Composite search key
- B⁺-tree index files
 - Balanced tree
 - Leaf nodes
- Dynamic hashing
- Multiple-key access
- Covering indices
- Write-optimized index structure
 - Log-structured merge (LSM) tree
 - Stepped-merge index
 - Buffer tree
- Bitmap index
- Bitmap intersection
- Indexing of spatial data
 - Range queries
 - Nearest neighbor queries
 - k-d tree
 - k-d-B tree
 - Quadtrees
 - R-tree
 - Bounding box
- Temporal indices
- Time interval
- Closed interval
- Open interval

Indexing

It is inefficient for the system to read every tuple in the instructor relation to check if the *dept_name* value matches our query. Ideally, the system should be able to locate these records directly.

Basic concepts

An index for a file in a database system works in much the same way as the index in this textbook. The words in the index are in sorted order, making it easy to find the word we want.

Without indices, every query would end up reading the entire contents of every relation that it uses; doing so would be unreasonably expensive for queries that only fetch a few records.

There are two basic kinds of indices:

- **Ordered indices:** Based on a sorted ordering of the values.
- **Hash indices:** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a hash function.

Each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:

- **Access types:** The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

An attribute or set of attributes used to look up records in a file is called a search key.

Ordered indices

An ordered index stores the values of the search keys in sorted order and associates with each search key the records that contain it.

If the file containing the records is sequentially ordered, a **clustering index** is an index whose search key also defines the sequential order of the file. Clustering indices are also called **primary indices**; the term primary index may appear to denote an index on a primary key.

Indices whose search key specifies an order different from the sequential order of the file are called **nonclustering indices**, or **secondary indices**.

Files that are ordered sequentially, and have a clustering index on the search key, are called **index-sequential files**.

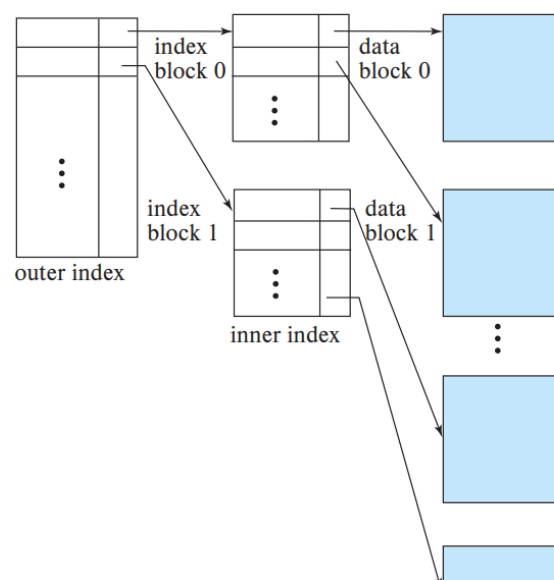
✿ **Dense and sparse indices:** An **index entry**, or **index record**, consists of a search-key value and pointers to one or more records with that value as their search-key value.

- **Dense index:** An index entry appears for every search-key value in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the first data record with that search-key value. The rest of the records with the same search-key value would be stored sequentially after the first record, since, because the index is a clustering one, records are sorted on the same search key. In a dense nonclustering index, the index must store a list of pointers to all records with the same search-key value.
- **Sparse index:** An index entry appears for only some of the searchkey values. Sparse indices can be used only if the relation is stored in sorted order of the search key; that is, if the index is a clustering index. As is true in dense indices, each index entry contains a search-key value and a pointer to the first data record with that search-key value. To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry and follow the pointers in the file until we find the desired record.

BUSCAR DIAGRAMAS EN EL LIBRO (ejemplo de search key = carrera para buscar estudiantes) (pag 657 reader)

✿ **Multilevel indices:** A sequential search requires b sequential block reads, which may take even longer (although in some cases the lower cost of sequential block reads may result in sequential search being faster than a binary search). Thus, the process of searching a large index may be costly. To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a sparse outer index on the original index, which we now call the inner index.

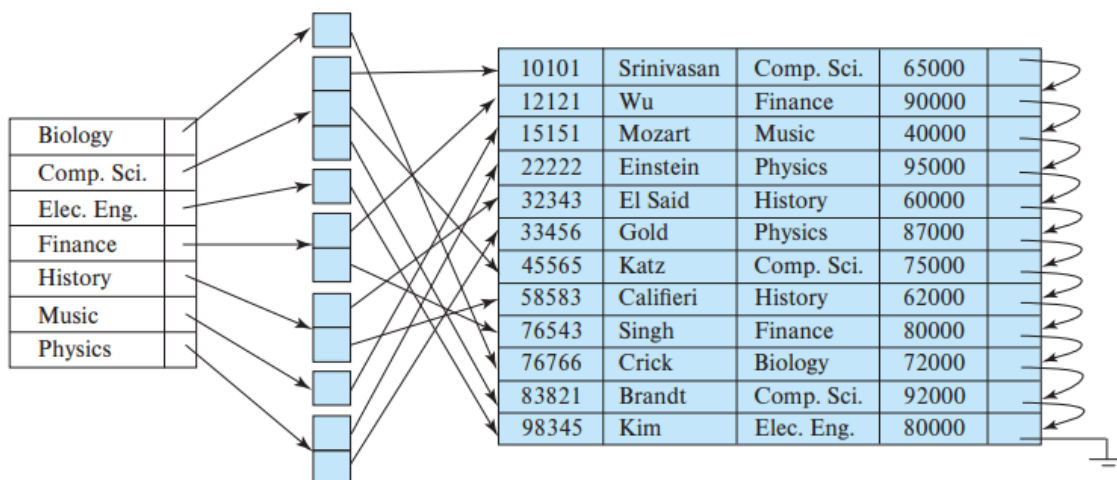
Indices with two or more levels are called **multilevel indices**.



✿ **Index update:** Every index must be updated whenever a record is either inserted into or deleted from the file. We only need to consider insertion and deletion on an index. BUSCAR INSERT Y DELETE PARA CASOS DE DENSE Y SPARSE

✿ **Secondary indices:** Secondary indices must be dense, with an index entry for every search-key value, and a pointer to every record in the file. In general, secondary indices may have a different structure from clustering indices.

If a relation can have more than one record containing the same search key value, the search key is said to be a nonunique search key. Secondary indices improve the performance of queries that use keys other than the search key of the clustering index.

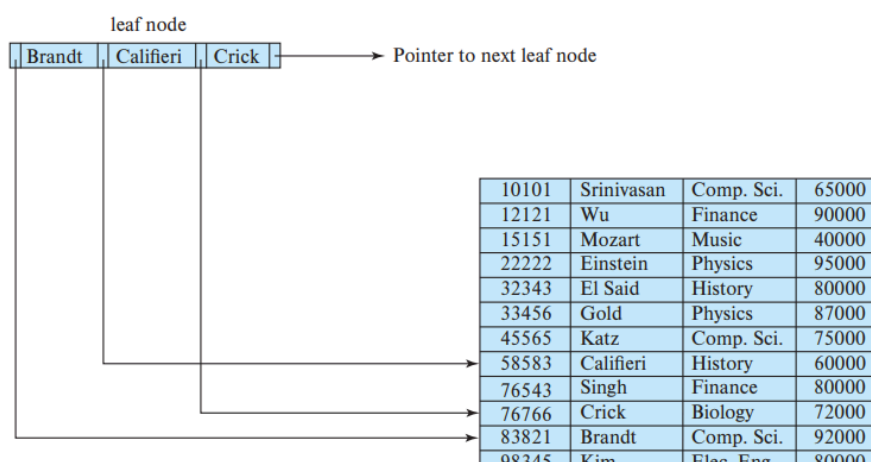


✿ **Indices on multiple keys:** A search key containing more than one attribute is referred to as a composite search key. The structure of the index is a list of attributes (represented as a tuple of values, of the form $(a_1, \dots, a_n)$). The ordering of search-key values is the lexicographic ordering

B+ Tree index files

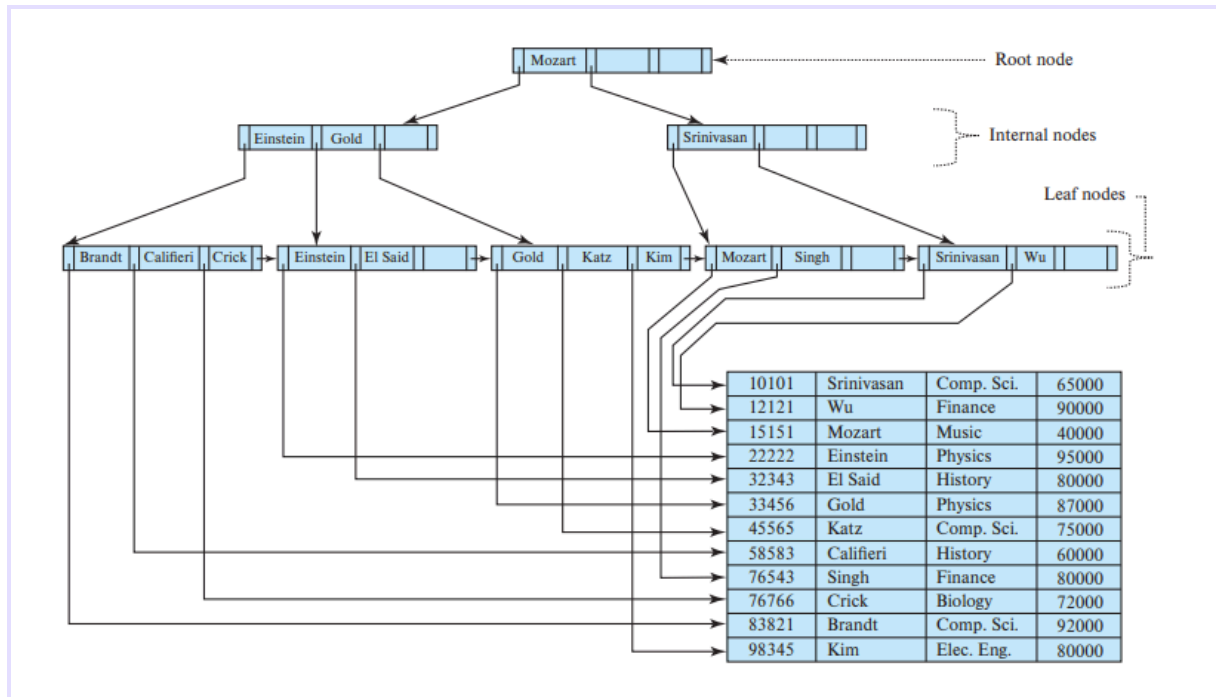
A B+tree index takes the form of a balanced tree in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree (other than the root) has between $\lceil n/2 \rceil$ and n children (n is fixed for a particular tree).

✿ **Structure of a B+tree:** A B+-tree index is a multilevel index. It contains up to $n - 1$ search-key values $K_1, K_2, \dots, K_{n-1}$, and n pointers $P_1, P_2, \dots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.



We consider first the structure of the **leaf nodes** → pointer P_i points to a file record with search-key value K_i

The **nonleaf nodes** of the B+-tree form a multilevel (sparse) index on the leaf nodes.



One way to handle the **case of nonunique search keys** is to modify the tree structure to store each search key at a leaf node as many times as it appears in records. The condition that $K_i < K_j$ if $i < j$ will need to be modified to $K_i \leq K_j$. This approach can **result in duplicate search key values at internal nodes**, making the insertion and deletion procedures more complicated and expensive. This approach is more complicated and can result in inefficient access, especially if the number of record pointers for a particular key is very large.

Most database implementations instead make search keys unique as follows: the unique composite search key (a_i, A_p) is used instead of a_i when building the index.

✿ **Queries on B+-trees:** The function starts at the root of the tree and traverses the tree down until it reaches a leaf node that would contain the specified value if it exists in the tree.

For **range queries**: first traverses to a leaf in a manner similar to $\text{find}(l_b)$; the leaf may or may not actually contain value l_b . It then steps through records in that and subsequent leaf nodes collecting pointers to all records with key values $C.K_i$ s.t. $l_b \leq C.K_i \leq u_b$ into a set resultSet .

Cost of querying on a B+-tree index: In processing a query, we traverse a path in the tree from the root to some leaf node. If there are N records in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(N) \rceil$.

✿ **Updates, insertions and deletions on B+-trees:** Estas nos las jugamos al truco pa

✿ **Complexity on B+trees:** It is the speed of operation on B+-trees that makes them a frequently used index structure in database implementations. Although B+-trees only guarantee that nodes will be at least half full, if entries are inserted in random order, nodes can be expected to be more than two-thirds full on average. If entries are inserted in sorted order, on the other hand, nodes will be only half full.

✿ **Nonunique search keys:** *"make search keys unique by creating a composite search key containing the original search key and extra attributes".* The extra attribute is called a **uniquifier attribute**.

One alternative is to store each key value only once in the tree, and to keep a bucket (or list) of record pointers with a search-key value. Another option is to store the search key value once per record.

A major problem with both these approaches, as compared to the unique searchkey approach, lies in the **efficiency of record deletion**.

B+ Tree extensions

In a **B+-tree file organization**, the leaf nodes of the tree store records, instead of storing pointers to records.

When we use a B+-tree for file organization, space utilization is particularly important, since the space occupied by the records is likely to be much more than the space occupied by keys and pointers.

✿ **Secondary Indices and Record Relocation:** Some file organizations, may change the location of records even when the records have not been updated. As an example, when a **leaf node is split** in a B+-tree file organization, **a number of records are moved to a new node**. In such cases, **all secondary indices** that store pointers to the relocated records **would have to be updated**.

✿ **Indexing strings:** With variable-length search keys, different nodes can have different fanouts even if they are full. The fanout of nodes can be increased by using a technique called **prefix compression**(we do not store the entire search key value at nonleaf nodes).

✿ **Bulk loading of indices:** **Insertion of a large number of entries at a time into an index** is referred to as bulk loading of the index. An efficient way to perform bulk loading of an index is as follows: Create a temporary file containing index entries for the relation, then sort the file on the search key of the index being constructed, and finally scan the sorted file and insert the entries into the index.

When the entries are inserted in sorted order, **all entries that go to a particular leaf node will appear consecutively, and the leaf needs to be written out only once**; nodes will never have to be read from disk during bulk load

✿ **B-tree index files:** B-tree indices are similar to B+-tree indices. The primary distinction between the two approaches is that a **B-tree eliminates the redundant storage of search-key**

values. A B-tree allows search-key values to appear only once. Every search-key value appears in some leaf node.

B-tree has a smaller fanout and therefore may have depth greater than that of the corresponding B+-tree. Deletion in a B-tree is more complicated. The space advantages of B-trees are marginal for large indices and usually do not outweigh the disadvantages that we have noted. Pretty much all database-system implementations use the B+-tree data structure.

✿ Indexing on flash storage:

- SSDs provide much faster random I/O operations than magnetic disks (lookups run much faster with data on SSDs, compared to data on magnetic disks.)
- The performance of write operations is more complicated with flash storage.
- An important difference between flash storage and magnetic disks is that flash storage does not permit in-place updates to data at the physical level. Every update turns into a copy+write of an entire flash-storage page.
- The optimum B+-tree node size for flash storage is smaller than that with magnetic disk, since flash pages are smaller than disk blocks (it makes sense for tree-node sizes to match to flash pages).
- Bulk loading provides significant performance benefits.

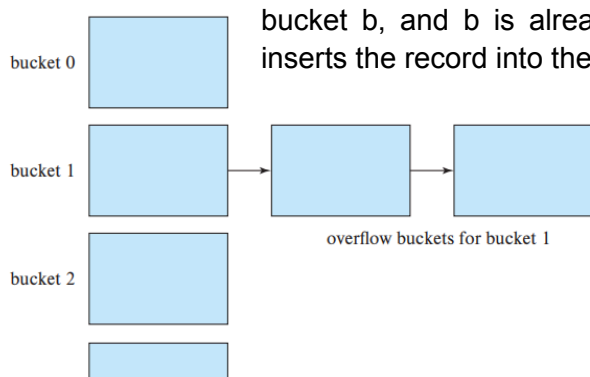
✿ Indexing on main memory:

- Since memory is costlier than disk space, internal data structures in main memory databases have to be designed to reduce space requirements.
- Data structures that require traversal of multiple pointers are acceptable for in-memory data. Thus, tree structures in main memory databases can be relatively deep, unlike B+-trees.
- Instead of scanning data linearly or using binary search within a node, the tree-structure within the large B+-tree node is used to access the data with a minimal number of cache misses.

Hash indices

We shall use the term bucket to denote a unit of storage that can store one or more records.

- In a hash file organization, instead of record pointers, buckets store the actual records; such structures only make sense with disk-resident data.
- No usamos direccionamiento abierto porque las operaciones de borrado no son eficientes.
- If the bucket does not have enough space, a bucket overflow is said to occur. We handle bucket overflow by using overflow buckets. If a record must be inserted into a bucket b, and b is already full, the system provides an overflow bucket for b and inserts the record into the overflow bucket.



Hash indexing as described above, where the number of buckets is fixed when the index is created, is called static hashing.

Several schemes have been proposed that allow the number of buckets to be increased in a more incremental fashion. Such schemes are called dynamic hashing techniques;

Multiple-key access

For certain types of queries, it is advantageous to use multiple indices if they exist, or to use an index built on a multiattribute search key.

✿ **Using Multiple Single-Key Indices:** Consider the following query: “Find all instructors in the Finance department with salary equal to \$80,000.”

There are three strategies possible for processing this query:

1. Use the index on *dept_name* to find all records pertaining to the Finance department. Examine each such record to see whether *salary* = 80000.
2. Use the index on *salary* to find all records pertaining to instructors with *salary* of \$80,000. Examine each such record to see whether the *dept_name* is “Finance”.
3. Use the index on *dept_name* to find pointers to all records pertaining to the Finance department. Also, use the index on *salary* to find pointers to all records pertaining to instructors with a salary of \$80,000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to instructors of the Finance department and with *salary* of \$80,000.

La última estrategia no es tan buena si ambos atributos tienen muchos o pocos registros.

✿ **Indices on multiple keys:** An alternative strategy for this case is to create and use an index on a composite search key (*dept name*, *salary*).

✿ **Covering Indices:** Covering indices are indices that store the values of some attributes (other than the search-key attributes) along with the pointers to the record.

Storing extra attribute values is useful with secondary indices, since they allow us to answer some queries using just the index, without even looking up the actual records.

Creation of indices

Indices can be created using the following syntax, which is supported by most databases:

create index <index-name> **on** <relation-name> (<attribute-list>);
and **drop index** <index-name>; to drop it

The attribute-list is the list of attributes of the relations that form the search key for the index. To declare that an attribute or list of attributes is a candidate key, we can use the syntax create unique index in place of create index above.

However, indices do have a cost, since they have to be updated whenever there is an update to the underlying relation. Creating too many indices would slow down update processing, since each update would have to also update all affected indices.

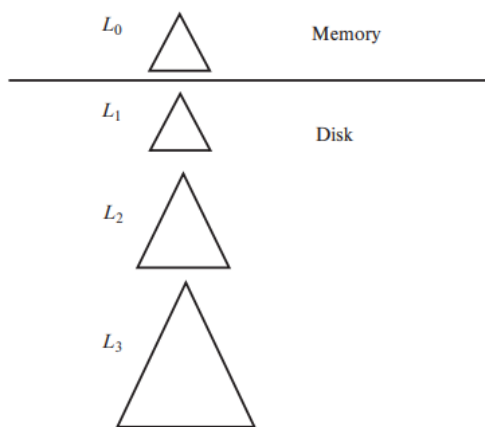
It is important when building an application to figure out which indices are important for performance and to create them before the application goes live. Although most database systems do not automatically create them, it is often a good idea to create indices on foreign-key attributes, too.

Write-optimized index structures

One of the drawbacks of the B+-tree index structure is that performance can be quite poor with random writes.

Now suppose writes or inserts are done in an order that does not match the sort order of the index. Then, each write/insert is likely to touch a different leaf node; if the number of leaf nodes is significantly larger than the buffer size, most of these leaf accesses would require a random read operation, as well as a subsequent write operation to write the updated leaf page back to disk.

The log-structured merge tree or LSM tree and its variants are write-optimized index structures that have seen very significant adoption.



✿ **LSM trees:** An LSM tree consists of several B+-trees, starting with an in-memory tree, called L_0 , and on-disk trees L_1 , L_2 , ..., L_k (where k is called the level). When a record is first inserted into an LSM tree, it is inserted into the in-memory B+-tree structure L_0 . A fairly large amount of memory space is allocated for this tree.

The tree grows as more inserts are processed, until it fills the memory allocated to it. At this point, we need to move data from the in-memory structure to a B+-tree on disk.

(pág 696 si querés saber más xd)

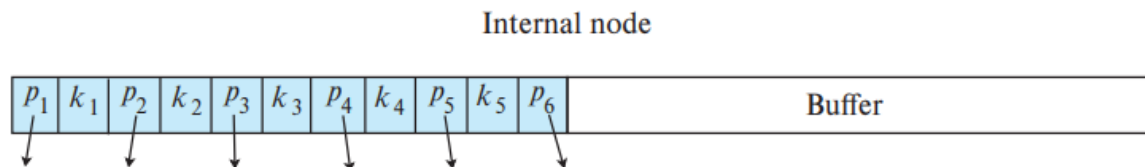
- Each level (other than L_0) can have up to some number b of trees, instead of just 1 tree. When an L_0 tree is written to disk, a new L_1 tree is created instead of merging it with an existing L_1 tree. When there are b such L_1 trees, they are merged into a single new L_2 tree. Similarly, when there are b trees at level L_i they are merged into a new L_{i+1} tree.

This variant of the LSM tree is called a **stepped-merge index**. The stepped-merge index decreases the insert cost significantly compared to having only one tree per level, but it can result in an increase in query cost, since multiple trees may need to be searched. Bitmap-based structures called *Bloom filters*, described in Section 24.1, are used to reduce the number of lookups by efficiently detecting that a search key is not present in a particular tree. Bloom filters occupy very little space, but they are quite effective at reducing query cost.

These systems are built on top of distributed file systems that allow appends to files but do not support updates to existing data. The fact that LSM trees do not perform in-place update made LSM trees a very good fit for these systems. These systems are built on top of distributed file systems that allow appends to files but do not support updates to existing

data. Subsequently, a large number of BigData storage systems such as Apache Cassandra, Apache AsterixDB, and MongoDB added support for LSM trees, with most implementing versions with multiple trees in each layer.

✿ **Buffer tree:** The key idea behind the buffer tree is to associate a buffer with each internal node of a B+-tree, including the root node.



When an index record is inserted into the buffer tree, instead of traversing the tree to the leaf, the index record is inserted into the buffer of the root. If the buffer becomes full, each index record in the buffer is pushed one level down the tree to the appropriate child node.

child node = internal node → index record added to the child node's buffer

child node = leaf node → index records are inserted into the leaf as usual

Buffer trees provide better worst-case complexity bounds on the number of I/O operations than do LSM tree variants. In terms of read cost, buffer trees are significantly faster than LSM trees. However, write operations on buffer trees involve random I/O, 670 Chapter 14 Indexing requiring more seeks, in contrast to sequential I/O operations with LSM tree variants

Buffer trees have been implemented as part of the **Generalized Search Tree (GiST) index structure** in PostgreSQL. The GiST index allows user-defined code to be executed to implement search, update, and split operations on nodes and has been used to implement R-trees and other spatial index structures.

Bitmap indices

Bitmap indices are a **specialized type of index designed for easy querying on multiple keys**, although each bitmap index is built on a single key.

For bitmap indices to be used, **records in a relation must be numbered sequentially**, starting from, say, 0.

A bitmap is simply an array of bits. In its simplest form, a bitmap index on the attribute A of relation r consists of **one bitmap for each value that A can take**. Each bitmap has as many bits as the number of records in the relation.

In fact, bitmap indices are useful for selections mainly when there are **selections on multiple keys**.

record number	ID	gender	income_level	Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
0	76766	m	L1	m	10010	L1	10100
1	22222	f	L2	f	01101	L2	01000
2	12121	f	L1			L3	00001
3	15151	m	L4			L4	00010
4	58583	f	L3			L5	00000

Indexing of spatial and temporal data

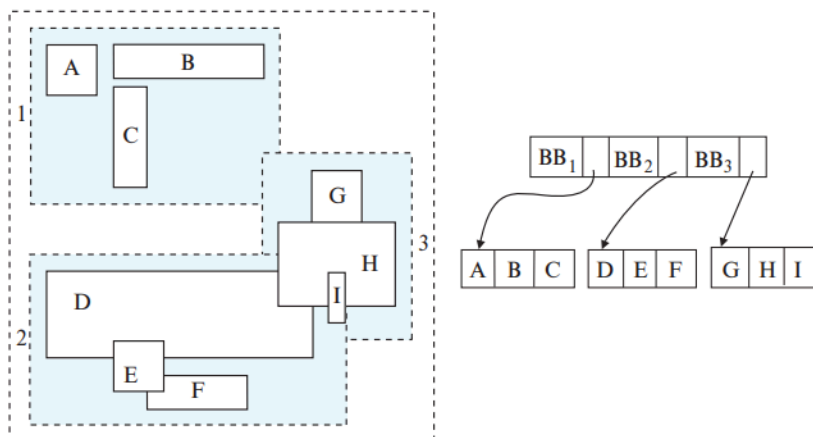
When tuples have temporal intervals associated with them, and queries may specify time points or time intervals, the traditional index structures may result in poor performance.

✿ **Indexing of spatial data** ¶: Spatial data refers to data referring to a point or a region in two or higher dimensional space (coordenadas latitud y longitud por ej.). The goal of spatial indexing is to support different forms of spatial queries, with range and nearest neighbor queries being of particular interest, since they are widely used.

A tree structure called a **k-d tree** was one of the early structures used for indexing in multiple dimensions. Each level of a k-d tree partitions the space into two. The partitioning is done along one dimension at the node at the top level of the tree, along another dimension in nodes at the next level, and so on, cycling through the dimensions. Partitioning stops when a node has less than a given maximum number of points.

A storage structure called an **R-tree** is useful for indexing of objects spanning regions of space, such as line segments, rectangles, and other polygons, in addition to points. Instead of a range of values, a rectangular **bounding box** is associated with each tree node.

Each internal node stores the bounding boxes of the child nodes along with the pointers to the child nodes. Each leaf node stores the indexed objects.



R-tree is at the right side

✿ **Indexing temporal data:** Temporal data refers to data that has an associated time period. The time period associated with a tuple indicates the period of time for which the tuple is valid. A time interval has a start time and an end time. Any valid period can be represented by multiple intervals; thus, a tuple with any valid period can be represented by multiple tuples, each of which has a valid period that is a single time interval.

- A better solution is to use a spatial index such as an R-tree, with the indexed tuple treated as having two dimensions; one issue that complicates the use of a spatial index such as an R-tree is that the end time interval may be infinity.
- Specialized indices, such as the interval B+-tree, are designed to index intervals in a single dimension, and provide better complexity guarantees than R-tree indices.

- Most database implementations find it simpler to use R-tree indices instead of implementing yet another type of index for time intervals.
- Recall that with temporal data, more than one tuple may have the same value for a primary key, as long as the tuples with the same primary-key value have non-overlapping time intervals.

Capitulo 15 - Database System Concepts

Review Terms

- Query processing
- Evaluation primitive
- Query-execution plan
- Query-evaluation plan
- Query-execution engine
- Measures of query cost
- Sequential I/O
- Random I/O
- File scan
- Linear search
- Selections using indices
- Access paths
- Index scans
- Conjunctive selection
- Hash-join
 - Build
 - Probe
 - Build input
 - Probe input
 - Recursive partitioning
 - Hash-table overflow
 - Skew
 - Fudge factor
 - Overflow resolution
 - Overflow avoidance
- Hybrid hash-join
- Disjunctive selection
- Composite index
- Intersection of identifiers
- External sorting
- External sort-merge
- Runs
- *N*-way merge
- Equi-join
- Nested-loop join
- Block nested-loop join
- Indexed nested-loop join
- Merge join
- Sort-merge join
- Hybrid merge join
- Spatial join
- Operator tree
- Materialized evaluation
- Double buffering
- Pipelined evaluation
 - Demand-driven pipeline (lazy, pulling)
 - Producer-driven pipeline (eager, pushing)
 - Iterator
 - Pipeline stages
- Double-pipelined join
- Continuous query evaluation

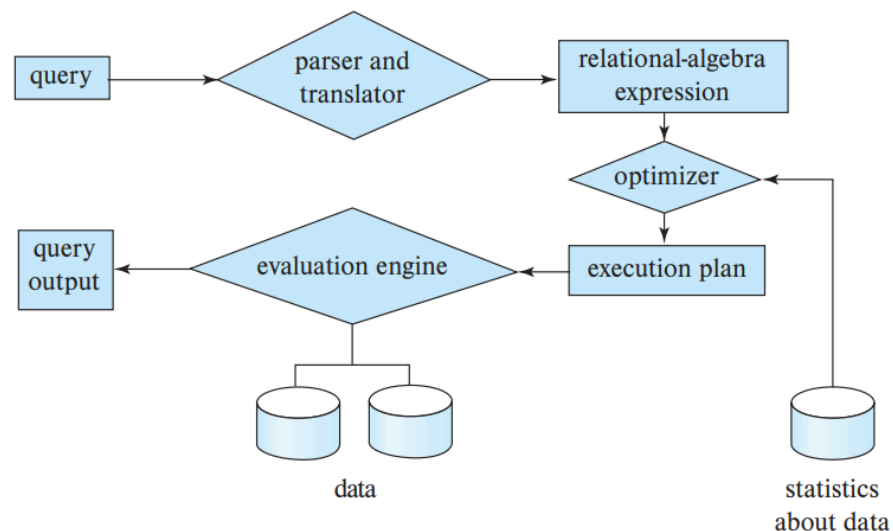
Query processing

Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

The basic steps involved in processing are:

1. Parsing and translation
2. Optimization
3. Evaluation

Before query processing can begin, the system must translate the query into a usable form. Thus, the first action the system must take in query processing is to translate a given query into its internal form. This translation process is similar to the work performed by the parser of a compiler.



To specify fully how to evaluate a query, we need not only to provide the relational algebra expression, but also to annotate it with instructions specifying how to evaluate each operation. Annotations may state the algorithm to be used for a specific operation or the particular index or indices to use.

A relational-algebra operation annotated with instructions on how to evaluate it is called an evaluation primitive. A sequence of primitive operations that can be used to evaluate a query is a query-execution plan or query-evaluation plan.

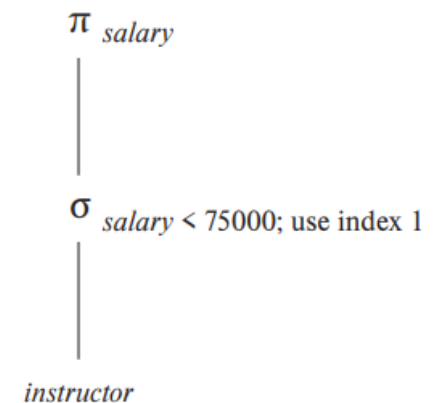
The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query. It is the responsibility of the system to construct a query-evaluation plan that minimizes the cost of query evaluation; this task is called query optimization.

In order to optimize a query, a query optimizer must know the cost of each operation.

Several operations may be grouped together into a **pipeline**, in which each of the operations starts working on its input tuples even as they are being generated by another operation.

Measures of query cost

The cost of query evaluation can be measured in terms of a number of different resources, including disk accesses, CPU time to execute a query, and, in parallel and distributed database systems, the cost of communication.



The database has default values for each of these costs (CPU cost per tuple, for processing each index entry and per operator/function), which are multiplied by the number of tuples processed, the number of index entries processed, and the number of operators and functions executed, respectively. The defaults can be changed as a configuration parameter.

We use the number of blocks transferred from storage and the number of random I/O accesses, each of which will require a disk seek on magnetic storage, as two important factors in estimating the cost of a query-evaluation plan.

If the disk subsystem takes an average of tT seconds to transfer a block of data and has an average block-access time (disk seek time plus rotational latency) of tS seconds, then an operation that transfers b blocks and performs S random I/O accesses would take:

$$b * tT + S * tS \text{ seconds.}$$

The values of tT and tS must be calibrated for the disk system used.

Acá el libro hace una banda de cálculos, pero vos cerrá los ojos y confía

Given the wide diversity of speeds of different storage devices, database systems must ideally perform test seeks and block transfers to estimate tS and tT for specific systems/storage devices, as part of the software installation process. Databases that do not automatically infer these numbers often allow users to specify the numbers as part of configuration files.

We can refine our cost estimates further by distinguishing block reads from block writes. Block writes are typically about twice as expensive as reads on magnetic disks. On PCIe flash, write throughput may be about 50 percent less than read throughput, but the difference is masked by the limited speed of SATA interfaces, leading to write throughput matching read throughput.

The cost estimates we give do not include the cost of writing the final result of an operation back to disk. These are taken into account separately where required.

The costs of all the algorithms that we consider depend on the **size of the buffer in main memory**.

Best case → Data fits in the buffer and can be read into it (menos acceso a disco)

Worst case → Buffer can only hold a few blocks of data

However, with large main memories available today, such worst-case assumptions are overly pessimistic.

The **response time** for a query-evaluation plan (that is, the wall-clock time required to execute the plan), would account for all these costs, and could be used as a measure of the cost of the plan. Unfortunately, the response time of a plan is very hard to estimate without actually executing the plan, for the following two reasons:

1. The response time depends on the contents of the buffer when the query begins execution; this information is not available when the query is optimized.
2. In a system with multiple disks, the response time depends on how accesses are distributed among disks

As a result, instead of trying to minimize the response time, **optimizers generally try to minimize the total resource consumption of a query plan**.

Selection operation

In query processing, the **file scan** is the lowest-level operator to access data. File scans are search algorithms that locate and retrieve records that fulfill a selection condition. A file scan allows an entire relation to be read in those cases where the relation is stored in a single, dedicated file.

✿ **Selections using file scans and indices:** The most straightforward way of performing a selection is using a linear search.

A1 (linear search). In a linear search, the **system scans each file block and tests all records to see whether they satisfy the selection condition**. An initial seek is required to access the first block of the file. In case blocks of the file are not stored contiguously, extra seeks may be required, but we ignore this effect for simplicity. The linear-search algorithm can be applied to any file

- Most real-life optimizers assume that the internal nodes of the tree are present in the in-memory buffer since they are frequently accessed
- Index structures are referred to as **access paths**, since they provide a path through which data can be located and accessed.
- Search algorithms that use an index are referred to as **index scans**. We use the selection predicate to guide us in the choice of the index to use in processing the query.

Esta tabla por toda la cara. Por favor que la sergiada no abarque esto

We use h_i to represent the height of the B+-tree, and assume a random I/O operation is required for each node in the path from the root to a leaf.

	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.
A1	Linear Search, Equality on Key	Average case $t_S + (b_r/2) * t_T$	Since at most one record satisfies the condition, scan can be terminated as soon as the required record is found. In the worst case, b_r block transfers are still required.
A2	Clustering B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
A3	Clustering B ⁺ -tree Index, Equality on Non-key	$h_i * (t_T + t_S) + t_S + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don't require additional seeks.
A4	Secondary B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	This case is similar to clustering index.
A4	Secondary B ⁺ -tree Index, Equality on Non-key	$(h_i + n) * (t_T + t_S)$	(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.
A5	Clustering B ⁺ -tree Index, Comparison	$h_i * (t_T + t_S) + t_S + b * t_T$	Identical to the case of A3, equality on non-key.
A6	Secondary B ⁺ -tree Index, Comparison	$(h_i + n) * (t_T + t_S)$	Identical to the case of A4, equality on non-key.

A2 (clustering index, equality on key). For an equality comparison on a key attribute with a clustering index, we can use the index to retrieve a single record that satisfies the corresponding equality condition.

A3 (clustering index, equality on non-key). We can retrieve multiple records by using a clustering index when the selection condition specifies an equality comparison on a non-key attribute.

A4 (secondary index, equality). Selections specifying an equality condition can use a secondary index. This strategy can retrieve a single record if the equality condition is on a key; multiple records may be retrieved if the indexing field is not a key.

In the first case, only one record is retrieved. Cost (in this case) = A2. In the second case, each record may be resident on a different block, which may result in one I/O operation per retrieved record, with each I/O operation requiring a seek and a block transfer.

It is possible to construct an estimate of the average or expected cost of the selection by taking into account the probability of the block containing the record already being in the buffer.

“secondary indices store the values of the attributes used as the search key in a B+-tree file organization”. Accessing a record through such a secondary index is then more expensive: First the secondary index is searched to find the B+-tree file organization search-key values, then the B+-tree file organization is looked up to find the records. The cost formulae described for secondary indices have to be modified appropriately if such indices are used.

✿ **Selections involving comparisons:** Consider a selection of the form($\sigma_{A \leq v}(r)$). We can implement the selection either by using linear search or by using indices in one of the following ways:

A5 (clustering index, comparison). A clustering ordered index (for example, a clustering B+-tree index) can be used when the selection condition is a comparison. For comparison conditions of the form $A > v$ or $A \geq v$, a clustering index on A can be used to direct the retrieval of tuples

A6 (secondary index, comparison). We can use a secondary ordered index to guide retrieval for comparison conditions involving $<$, $\leq$, $\geq$, or $>$. The lowest-level index blocks are scanned, either from the smallest value up to v, or from v up to the maximum value (for $>$ and $\geq$).

As long as the number of matching tuples is known ahead of time, a query optimizer can choose between using a secondary index or using a linear scan based on the cost estimates. However, if the number of matching tuples is not known accurately at compilation time, either choice may lead to bad performance, depending on the actual number of matching tuples.

To deal with the above situation, PostgreSQL uses a hybrid algorithm that it calls a **bitmap index scan**, when a secondary index is available, but the number of matching records is not known precisely. Algorithm:

1. First creates a bitmap with as many bits as the number of blocks in the relation.
2. Then uses the secondary index to find index entries for matching tuples

3. As each index entry is found, the algorithm gets the block number from the index entry, and sets the corresponding bit in the bitmap to 1.
4. Once all index entries have been processed, the bitmap is scanned to find all blocks whose bit is set to 1. These are exactly the blocks containing matching records.

In the worst case, this algorithm is only slightly more expensive than linear scan, but in the best case it is much cheaper than linear scan.

✿ Implementation of complex selections:

- **Conjunction:** A conjunctive selection is a selection of the form:

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$$

- **Disjunction:** A disjunctive selection is a selection of the form:

$$\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$$

A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions θ_i .

- **Negation:** The result of a selection $\sigma_{\neg\theta}(r)$ is the set of tuples of r for which the condition θ evaluates to false. In the absence of nulls, this set is simply the set of tuples in r that are not in $\sigma_{\theta}(r)$.

We can implement a selection operation involving either a conjunction or a disjunction of simple conditions by using one of the following algorithms:

A7 (conjunctive selection using one index). We first determine whether an access path is available for an attribute in one of the simple conditions. If one is, one of the selection algorithms A2 through A6 can retrieve records satisfying that condition.

A8 (conjunctive selection using composite index). An appropriate composite index (index on multiple attributes) may be available for some conjunctive selections. If the selection specifies an equality condition on two or more attributes, and a composite index exists on these combined attribute fields, then the index can be searched directly.

A9 (conjunctive selection by intersection of identifiers). This algorithm requires indices with record pointers, on the fields involved in the individual conditions. The algorithm scans each index for pointers to tuples that satisfy an individual condition. The intersection of all the retrieved pointers is the set of pointers to tuples that satisfy the conjunctive condition. The algorithm then uses the pointers to retrieve the actual records.

A10 (disjunctive selection by union of identifiers). If access paths are available on all the conditions of a disjunctive selection, each index is scanned for pointers to tuples that satisfy the individual condition. The union of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition. We then use the pointers to retrieve the actual records.

However, if even one of the conditions does not have an access path, we have to perform a linear scan of the relation to find tuples that satisfy the condition.

Sorting

Sorting of data plays an important role in database systems for two reasons. First, SQL queries can specify that the output be sorted. Second, and equally important for query processing, several of the relational operations, such as joins, can be implemented efficiently if the input relations are first sorted.

We can sort a relation by building an index on the sort key and then using that index to read the relation in sorted order. This process orders the relation only logically rather than physically. Hence, the reading of tuples in the sorted order may lead to a disk access for each record, which can be very expensive, since the number of records can be much larger than the number of blocks.

Here, we are going to discuss the case where we have to sort relations bigger than memory.

✿ **External sort-merge algorithm:** Sorting of relations that do not fit in memory is called external sorting. The most commonly used technique for external sorting is the external sort-merge algorithm.

Let M denote the number of blocks in the main memory buffer available for sorting (number of disk blocks whose contents can be buffered in available main memory).

1. A number of sorted runs are created; each run is sorted but contains only some of the records of the relation.
2. In the second stage, the runs are merged. Suppose, for now, that the total number of runs, N , is less than M , so that we can allocate one block to each run and have space left to hold one block of output. The merge stage operates as follows: Read one block of each of the N files R_i into a buffer block in memory;

repeat

 choose the first tuple (in sort order) among all buffer blocks;
 write the tuple to the output, and delete it from the buffer block;
 if the buffer block of any run R_i is empty **and not** end-of-file(R_i)
 then read the next block of R_i into the buffer block;

until all input buffer blocks are empty

The output of the merge stage is the sorted relation. The preceding merge operation is a generalization of the two-way merge used by the standard in-memory sort-merge algorithm; it merges N runs, so it is called an N -way merge.

If the relation is much larger than memory, there may be M or more runs generated in the first stage. Each pass reduces the number of runs by a factor of $M - 1$. The passes repeat as many times as required, until the number of runs is less than M ; a final pass then generates the sorted output.

✿ **Cost analysis of external sort-merge:** Let br denote the number of blocks containing records of relation r . Let M denote the number of blocks in the main memory buffer available for sorting. Let bb denote the size of the block buffer (SEGUN LA IA)

→ The total number of merge passes required is

$$\lceil \log_{LM / bb} (br / M) \rceil$$

Finally:

$$2[br / M] + [br / bb](2[\log_{LM / bb} (br / M)] - 1)$$

Applying this equation to the example in Figure 15.4, we get a total of $8 + 12 * (2 * 2 - 1) = 44$ disk seeks if we set the number of buffer blocks per run bb to 1

Join operation

We use the term **equi-join** to refer to a **join natural**.

✿ **Nested-loop join:** This algorithm is called the **nested-loop join**

```

for each tuple  $t_r$  in  $r$  do begin
  for each tuple  $t_s$  in  $s$  do begin
    test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$ 
    if they do, add  $t_r \cdot t_s$  to the result;
  end
end

```

Relation r is called the **outer relation** and relation s the **inner relation** of the join.

- The nested-loop join algorithm **requires no indices**, and it can be used regardless of what the join condition is. The only change **required is an extra step of deleting repeated attributes** from the tuple $t_r \cdot t_s$, before adding it to the result.
- The nested-loop join algorithm is **expensive**, since it examines every pair of tuples in the two relations
- If one of the relations fits entirely in main memory, it is beneficial to use that relation as the inner relation, since the inner relation would then be read only once.

✿ **Block nested-loop join:** **Variant of the nested-loop join** where **every block of the inner relation is paired with every block of the outer relation**. Within each pair of blocks, every tuple in one block is paired with every tuple in the other block, to generate all pairs of tuples.

In the worst case, **each block in the inner relation s is read only once for each block in the outer relation, instead of once for each tuple in the outer relation**. Thus, in the worst case, there will be a lot of block transfers.

- If the join attributes in a natural join or an equi-join form a key on the inner relation, then for each outer relation tuple the inner loop can terminate as soon as the first match is found.
- In the block nested-loop algorithm we can use the biggest size that can fit in memory, while leaving enough space for the buffers of the inner relation and the output.

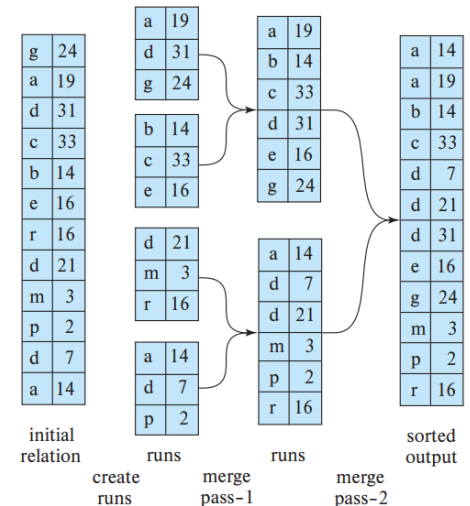


Figure 15.4 External sorting using sort-merge.

→ We can scan the inner loop alternately forward and backward. This scanning method orders the requests for disk blocks so that the data remaining in the buffer from the previous scan can be reused (reducing the number of disk accesses needed).

✿ **Index nested-loop join:** If an index is available on the inner loop's join attribute, index lookups can replace file scans: **indexed nested-loop join**

The cost of an indexed nested-loop join can be computed as follows:

$br(tT + tS) + nr * c$, where nr is the number of records in relation r , and c is the cost of a single selection on s using the join condition. (tT y tS son tuplas de t y s respectivamente y br es la cantidad de bloques que tienen registros de r).

The cost formula indicates that, if indices are available on both relations r and s , it is generally most efficient to use the one with fewer tuples as the outer relation.

✿ **Merge join:** It can be used to compute natural joins and equi-joins.

pr →

$a1$	$a2$
a	3
b	1
d	8
d	13
f	7
m	5
q	6

r

ps →

$a1$	$a3$
a	A
b	G
c	L
d	N
m	B

s

Algorithm

The merge-join algorithm associates one pointer with each relation. These pointers point initially to the first tuple of the respective relations. As the algorithm proceeds, the pointers move through the relation. A group of tuples of one relation with the same value on the join attributes is read into S . Then, the corresponding tuples (if any) of the other relation are read in and are processed as they are read.

Cost analysis

Each tuple in the sorted order needs to be read only once, and, as a result, each block is also read only once. Since it makes only a single pass through both files (assuming all sets S_s fit in memory), the merge-join method is efficient; the number of block transfers is equal to the sum of the number of blocks in both files, $br + bs$.

Assuming that bb buffer blocks are allocated to each relation, the number of disk seeks required would be $\lceil br / bb \rceil + \lceil bs / bb \rceil$ disk seeks. Since seeks are much more expensive than data transfer, it makes sense to allocate multiple buffer blocks to each relation, provided extra memory is available.

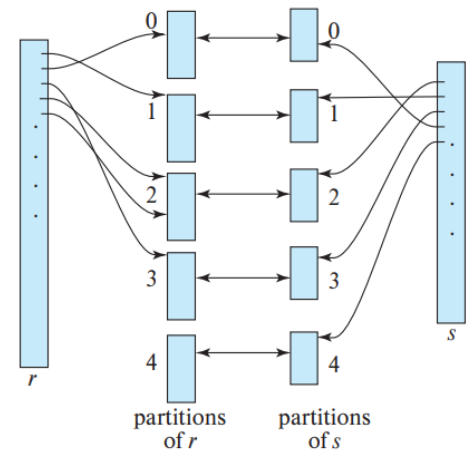
Hybrid merge join

Suppose that one of the relations is sorted; the other is unsorted, but has a secondary B+-tree index on the join attributes. The **hybrid merge-join** algorithm merges the sorted relation with the leaf entries of the secondary B+-tree index.

✿ **Hash join:** The hash-join algorithm can be used to implement natural joins and equi-joins. A hash function h is used to partition tuples of both relations. The basic idea is to partition the tuples of each of the relations into sets that have the same hash value on the join attributes.

Suppose that an r tuple and an s tuple satisfy the join condition; then, they have the same value for the join attributes.

...To do so, it first builds a hash index on each s_i , and then probes (that is, looks up s_i) with tuples from r_i . The relation s is the build input, and r is the probe input.



- The hash index on s_i is built in memory, so there is no need to access the disk to retrieve the tuples.
- The build and probe phases require only a single pass through both the build and probe inputs.

If the size of the build relation is bs blocks, then, for each of the nh partitions to be of size less than or equal to M , nh must be at least $\lceil bs / M \rceil$.

Recursive partitioning: Partitioning has to be done in repeated passes. In one pass, the input can be split into at most as many partitions as there are blocks available for use as output buffers. The hash function used in a pass is different from the one used in the previous pass. The system repeats this splitting of the input until each partition of the build input fits in memory.

Hash table overflow: Occurs in partition i of the build relation s if the hash index on s_i is larger than main memory. Some of the partitions will have more tuples than the average, whereas others will have fewer; partitioning is then said to be skewed (sesgada, desvirtuada).

We can handle a small amount of skew by increasing the number of partitions: that small increased value is called **fudge factor**.

Overflow resolution: Performed during the build phase if a hash-index overflow is detected. Overflow resolution proceeds in this way: If any partition is found to be too large, it is further partitioned into smaller partitions by using a different hash function.

Overflow avoidance: performs the partitioning carefully, so that overflows never occur during the build phase.

A hash join is estimated to require:

$3(br + bs) + 4nh \rightarrow$ BLOCK TRANSFERS

$2(br + bs)\lceil \log \frac{LM}{bb} \rceil - 1(bs/M) + br + bs \rightarrow$ " WITH RECURSIVE PARTITIONING

$2(\lceil br / bb \rceil + \lceil bs / bb \rceil) + 2nh \rightarrow$ SEEKS.

$2(\lceil br / bb \rceil + \lceil bs / bb \rceil)\lceil \log \frac{LM}{bb} \rceil - 1(bs/M) \rightarrow$ " WITH RECURSIVE PARTITIONING

The hash join can be improved if the main memory size is large

The hybrid hash-join algorithm performs another optimization; To reduce the impact of seeks, a larger number of blocks would be used as a buffer.

✿ **Complex joins:** Nested-loop and block nested-loop joins can be used regardless of the join conditions. We can implement joins with complex join conditions, such as conjunctions and disjunctions, by using the efficient join techniques.

✿ **Joins over spatial data:** The presented join algorithms assume the use of standard comparison operations such as equality, less than, or greater than, where the values are linearly ordered.

Selection and join conditions on spatial data involve comparison operators that check if one region contains or overlaps another, or whether a region contains a particular point; and the regions may be multi-dimensional.

Other operations

✿ **Duplicate elimination:** We can implement duplicate elimination easily by sorting. Identical tuples will appear adjacent to each other as a result of sorting, and all but one copy can be removed. The worst-case cost estimate for duplicate elimination is the same as the worst-case cost estimate for sorting of the relation. (external sort-merge and hashing)

✿ **Projection:** We can implement projection easily by performing projection on each tuple, which gives a relation that could have duplicate records, and then removing duplicate records.

✿ **Set operations:** We can implement the union, intersection, and set-difference operations by first sorting both relations, and then scanning once through each of the sorted relations to produce the result.

• $r \cup s$

1. Build an in-memory hash index on r_i .
2. Add the tuples in s_i to the hash index only if they are not already present.
3. Add the tuples in the hash index to the result.

• $r \cap s$

1. Build an in-memory hash index on r_i .
2. For each tuple in s_i , probe the hash index and output the tuple to the result only if it is already present in the hash index.

• $r - s$

1. Build an in-memory hash index on r_i .
2. For each tuple in s_i , probe the hash index, and, if the tuple is present in the hash index, delete it from the hash index.
3. Add the tuples remaining in the hash index to the result.

✿ **Outer join:** We can implement the outer-join operations by using one of two strategies.

1_ Compute the corresponding **join**, and then **add further tuples** to the join result to get the outer-join result

2_ It is easy to **extend the nested-loop** join algorithms to compute the left outer join: Tuples in the outer relation that do not match any tuple in the inner relation are written to the output after being padded with null values. However, it is hard to extend the nested-loop to compute the full outer join.

Natural outer joins and outer joins with an equi-join condition can be computed by extensions of the merge-join and hash-join algorithms.

✿ **Aggregation:** The aggregation operation can be implemented in the same way as duplicate elimination. We use either sorting or hashing, just as we did for duplicate elimination, but based on the grouping attributes (dept_name in the preceding example).

Instead of eliminating tuples with the same value for the grouping attribute, we gather them into groups and apply the aggregation operations on each group to get the result.

Instead of gathering all the tuples in a group and then applying the aggregation operations, we can implement the aggregation operations sum, min, max, count, and avg on the fly as the groups are being constructed.

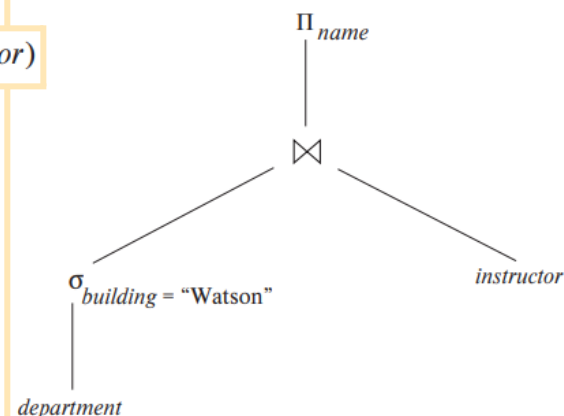
Evaluation of expressions

Evaluating an expression at a time (in appropriate order): the result of each evaluation is **materialized** in a temporary relation for subsequent use. The temporary relations **must be written to disk**.

Evaluating several operations simultaneously: We use a **pipeline**, with the results of **one operation passed on to the next**, without the need to store a temporary relation.

✿ **Materialization:** It is easiest to understand intuitively how to evaluate an expression by looking at a pictorial representation of the expression in an **operator tree**.

$\Pi_{name}(\sigma_{building = \text{"Watson"}}(department) \bowtie instructor)$



Evaluation as just described is called **materialized evaluation**, since the results of each intermediate operation are created (materialized) and then are used for evaluation of the next-level operations.

Double buffering (using two buffers, with one continuing execution of the algorithm while the other is being written out) allows the algorithm to execute more quickly by performing CPU activity in parallel with I/O activity.

✿ **Pipelining:** Reduction of number of produced temporary files by combining several relational operations into a pipeline of operations, in which the results of one operation are passed along to the next operation in the pipeline (**pipelined evaluation**).

Benefits:

1. It eliminates the cost of reading and writing temporary relations, reducing the cost of query evaluation.
2. It can start generating query results quickly, if the root operator of a query evaluation plan is combined in a pipeline with its inputs.

As a result of pipelining, the inputs to the operations are not available all at once for processing.

Ways to execute:

1. **Demand-driven pipeline:** The system makes repeated requests for tuples from the operation at the top of the pipeline. Each time that an operation receives a request for tuples, it computes the next tuple to be returned and then returns that tuple. Implemented as an iterator that provides the following functions: open(), next(), and close(). Tuples are generated eagerly.
 2. **Producer-driven pipeline:** Operations do not wait for requests to produce tuples, but instead generate the tuples eagerly. Implemented in a way that, for each pair of adjacent operations, the system creates a buffer to hold tuples being passed from one operation to the next. Tuples are generated lazily.
- Using producer-driven pipelining can be thought of as pushing data up an operation tree from below, whereas using demand-driven pipelining can be thought of as pulling data up an operation tree from the top.
- Query plans can be annotated to mark edges that are pipelined; such edges are called **pipelined edges**. In contrast, non-pipelined edges are referred to as **blocking edges** or **materialized edges**. The two operators connected by a pipelined edge must be executed concurrently.
- A query plan can be divided into subtrees such that each subtree has only pipelined edges, and the edges between the subtrees are nonpipelined. Each such subtree is called a **pipeline stage**.
- Some operations, such as sorting, are inherently **blocking operations**, that is, they may not be able to output any results until all tuples from their inputs have been examined.

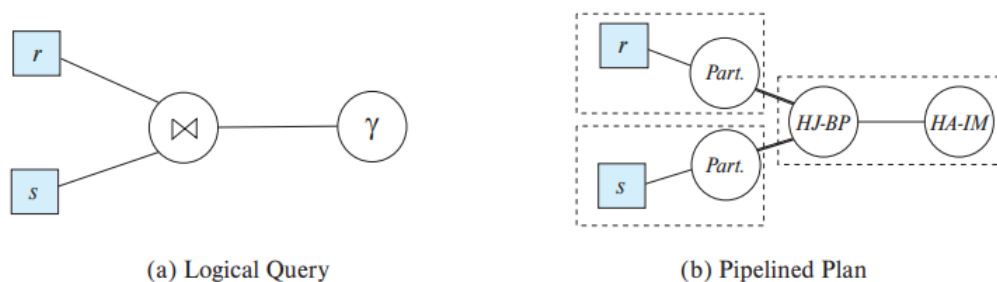


Figure 15.12 Query plan with pipelining.

Double-pipelined join: The algorithm assumes that the input tuples for both input relations, r and s , are pipelined.

✿ **Pipelines for continuous stream data:** Queries may be written over stream data in order to respond to data as they arrive. Such queries are called **continuous queries**. The operations in a continuous query should be implemented using pipelined algorithms, so that results from the pipeline can be output without blocking.

Many such queries perform aggregation with windowing; tumbling windows which divide time into fixed size intervals.

Query processing in memory

In this section, we discuss extensions to the query processing techniques that help minimize memory access costs by using cache-conscious query processing algorithms and query compilation.

✿ **Cache conscious algorithms:** The speed difference between cache memory and main memory, results in a situation where the relationship between cache and main memory is not dissimilar to the relationship between main memory and disk.

While the contents of the main memory buffers disk-based data are controlled by the database system, CPU cache is controlled by the algorithms built into the computer hardware. Thus, the database system cannot directly control what is kept in cache.

Ways we can design a query processing algorithm in a way that makes the best use of cache (pág 732 del libro / 761 pdf reader)

Primero habla de cómo ordenar relaciones usando external merge-sort y de el algoritmo de hash-join. Está enfocado en los tamaños de las relaciones en relación a la RAM (memory).

...

Attributes in a tuple can be arranged such that attributes that tend to be accessed together are laid out consecutively.

✿ **Query compilation:** To avoid overhead due to interpretation, modern main-memory databases compile query plans into machine code or intermediate level byte-code. The

compiler can also combine the code for multiple functions in a way that minimizes function calls.

✿ **Column oriented storage:** *“only a few attributes of a large schema may be needed, and that in such cases, storing a relation by column instead of by row may be advantageous”.*

Selection operations on a single attribute have significantly lower cost in a column store since only the relevant attributes need to be accessed. However, since accessing each attribute requires its own data access, the cost of retrieving many attributes is higher and may incur additional seeks if data are stored on disk.

Capitulo 16 - Database System Concepts

Review Terms

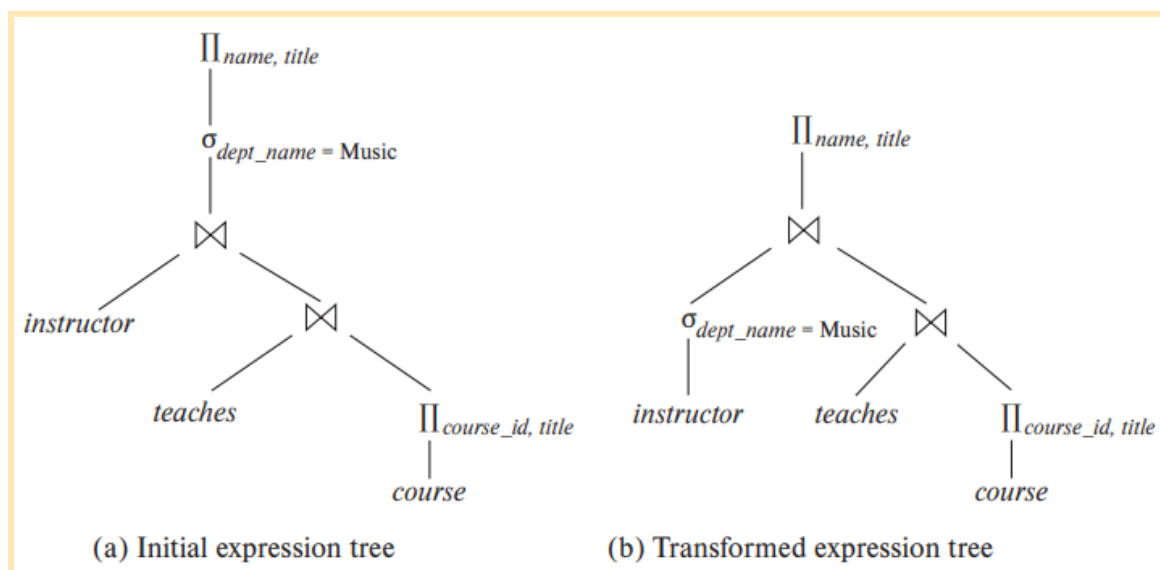
- Query optimization
- Transformation of expressions
- Equivalence of expressions
- Equivalence rules
 - Join commutativity
 - Join associativity
- Minimal set of equivalence rules
- Enumeration of equivalent expressions
- Statistics estimation
- Catalog information
- Size estimation
 - Selection
 - Selectivity
 - Join
- Correlated evaluation
- Decorrelation
- Semijoin
- Anti-semijoin
- Materialized views
- Materialized view maintenance
 - Recomputation
 - Incremental maintenance
 - Insertion
- Histograms
- Distinct value estimation
- Random sample
- Choice of evaluation plans
- Interaction of evaluation techniques
- Cost-based optimization
- Join-order optimization
 - Dynamic-programming algorithm
 - Left-deep join order
 - Interesting sort order
- Heuristic optimization
- Plan caching
- Access-plan selection
 - Deletion
 - Updates
- Query optimization with materialized views
- Index selection
- Materialized view selection
- Top- K optimization
- Join minimization
- Halloween problem
- Multiquery optimization

Query optimization

Query optimization is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex. We expect the system to construct a query-evaluation plan that minimizes the cost of query evaluation.

One aspect of optimization occurs at the relational-algebra level; another aspect is selecting a detailed strategy for processing the query, such as choosing the algorithm to use for executing an operation, choosing the specific indices to use, and so on.

It is worthwhile for the system to spend a substantial amount of time on the selection of a good strategy for processing a query.



An evaluation plan defines exactly what algorithm should be used for each operation and how the execution of the operations should be coordinated.

Given a relational-algebra expression, it is the job of the query optimizer to come up with a query-evaluation plan that computes the same result as the given expression, and is the least costly way of generating the result.

Generation of query-evaluation plans involves three steps:

1. Generating expressions that are logically equivalent to the given expression.
2. Annotating the resultant expressions in alternative ways to generate alternative query-evaluation plans
3. Estimating the cost of each evaluation plan, and choosing the one whose estimated cost is the least.

As evaluation plans are generated, their costs are estimated by using statistical information about the relations, such as relation sizes and index depths.

Since the cost is an estimate, the selected plan is not necessarily the least costly plan; however, as long as the estimates are good, the plan is likely to be the least costly one.

Transformation of relational expressions

Two relational-algebra expressions are said to be **equivalent** if, on every legal database instance, the two expressions generate the same set of tuples.

✿ **Equivalence rules:** An **equivalence rule** says that expressions of two forms are equivalent.

We can **replace** an expression of the first form with an expression of the second form, or vice versa since the two expressions generate the same result on any valid database.

Now, let θ denote predicates, L denote lists of attributes and E denote relational-algebra expressions:

1. Conjunctive selection operations can be **deconstructed** into a sequence of individual selections. This transformation is referred to as a cascade of σ .

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are **commutative**.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the final operations in a sequence of projection operations are needed; the others can be omitted. This transformation can also be referred to as a cascade of Π .

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$.

4. Selections can be **combined** with Cartesian products and theta joins.

$$a. \sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$$

This expression is just the definition of the theta join.

$$b. \sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

5. Theta-join operations are **commutative**.

$$E_1 \bowtie_{\theta} E_2 \equiv E_2 \bowtie_{\theta} E_1$$

The order of attributes **differs between the left-hand side and right-hand** side of the commutativity rule, so the equivalence does not hold if the order of attributes is taken into account.

6. a. Natural-join operations are associative.

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

- b. Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

7. The selection operation distributes over the theta-join operation under the following two conditions:

a. Selection distributes over the theta-join operation when all the attributes in selection condition θ_1 involve only the attributes of one of the expressions (say, E_1) being joined.

$$\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$$

b. Selection distributes over the theta-join operation when selection condition θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

8. The projection operation distributes over the theta-join operation under the following conditions.

a. Let L_1 and L_2 be attributes of E_1 and E_2 , respectively. Suppose that the join condition θ involves only attributes in $L_1 \cup L_2$. Then,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

b. Consider a join $E_1 \bowtie_{\theta} E_2$. Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively. Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in L_1 and let L_4 be attributes of E_2 that are involved in join condition θ , but are not in L_2 . Then,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

9. The set operations union and intersection are commutative.

$$a. E_1 \cup E_2 \equiv E_2 \cup E_1$$

$$b. E_1 \cap E_2 \equiv E_2 \cap E_1$$

Set difference is not commutative.

10. Set union and intersection are associative.

$$a. (E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$

$$b. (E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over the union, intersection, and set difference operations.

$$a. \sigma_{\theta}(E_1 \cup E_2) \equiv \sigma_{\theta}(E_1) \cup \sigma_{\theta}(E_2)$$

$$b. \sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2)$$

$$c. \sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

$$d. \sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap E_2$$

$$e. \sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - E_2$$

The preceding equivalence does not hold if $-$ is replaced by $\cup$.

12. The projection operation distributes over the union operation

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

provided E_1 and E_2 have the same schema.

13. Selection distributes over aggregation under the following conditions. Let G be a set of group by attributes, and A a set of aggregate expressions. When θ only involves attributes in G , the following equivalence holds.

$$\sigma_{\theta}(G \gamma A(E)) \equiv G \gamma A(\sigma_{\theta}(E))$$

- 14.

a. Full outer join is commutative.

$$E_1 \bowtie_{\text{full}} E_2 \equiv E_2 \bowtie_{\text{full}} E_1$$

b. Left and right outer join are not commutative. However, left outer join and right outer join can be exchanged as follows.

$$E_1 \bowtie_{\text{left}} E_2 \equiv E_2 \bowtie_{\text{right}} E_1$$

15. Selection distributes over left and right outer join under some conditions. Specifically, when the selection condition θ_1 involves only the attributes of one of the expressions being joined, say E_1 , the following equivalences hold.

$$\text{a. } \sigma_{\theta_1} (E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1} (E_1)) \bowtie_{\theta} E_2$$

$$\text{b. } \sigma_{\theta_1} (E_2 \bowtie_{\theta} E_1) \equiv (E_2 \bowtie_{\theta} (\sigma_{\theta_1} (E_1)))$$

16. Outer joins can be replaced by inner joins under some conditions. Specifically, if θ_1 has the property that it evaluates to false or unknown whenever the attributes of E_2 are null, then the following equivalences hold.

$$\text{a. } \sigma_{\theta_1} (E_1 \bowtie_{\theta} E_2) \equiv \sigma_{\theta_1} (E_1 \bowtie_{\theta} E_2)$$

$$\text{b. } \sigma_{\theta_1} (E_2 \bowtie_{\theta} E_1) \equiv \sigma_{\theta_1} (E_2 \bowtie_{\theta} E_2)$$

Some equivalences that hold for joins do not hold for outer joins.

✿ **Examples of transformations:** pagina 753 del libro/781 pdf reader

✿ **Join ordering:** A good ordering of join operations is important for reducing the size of temporary results; hence, most query optimizers pay a lot of attention to the join order.

$$(r_1 \bowtie r_2) \bowtie r_3 \equiv r_1 \bowtie (r_2 \bowtie r_3)$$

✿ **Enumeration of equivalent expressions:** Query optimizers can use equivalence rules to systematically generate expressions equivalent to the given query expression.

The process of enumeration proceeds as follows:

1. Given a query expression E , the set of equivalent expressions EQ initially contains only E
2. Each expression in EQ is matched with each equivalence rule
3. If a subexpression e_j of any expression $E_i \in EQ$ matches one side of an equivalence rule, the optimizer generates a copy E_k of E_i , in which e_j is transformed to match the other side of the rule, and adds E_k to EQ .

This process continues until no more new expressions can be generated. With a properly chosen set of equivalence rules, the set of equivalent expressions is finite, and the process can be guaranteed to terminate. Eventually applying any equivalence rule will only generate expressions that were already generated earlier.

The preceding process is extremely costly both in space and in time, but optimizers can greatly reduce both the space and time cost, using two key ideas:

1. If we generate an expression E' from an expression E_1 by using an equivalence rule on subexpression e_i , then E' and E_1 have identical subexpressions except for e_i and its transformation.
2. It is not always necessary to generate every expression that can be generated with the equivalence rules.

With these and other techniques to reduce the optimization time, equivalence rules can be used to enumerate alternative plans, whose costs can be computed; the lowest-cost plan

amongst the alternatives is then chosen. Although this approach can be implemented to execute quite fast, there is no guarantee that it will find the optimal plan.

Estimating statistics of expression results

The cost of an operation depends on the size and other statistics of its inputs.

We first list some statistics about database relations that are stored in database-system catalogs, and then show how to use the stored statistics to estimate statistics on the results of various relational operations.

Given a query expression, we consider it as a tree; we can start from the bottomlevel operations, and estimate their statistics, and continue the process on higher-level operations, till we reach the root of the tree. The size estimates that we compute as part of these statistics can be used to compute the cost of algorithms for individual operations in the tree, and these costs can be added up to find the cost of an entire query plan.

Estimates are not very accurate, since they are based on assumptions that may not hold exactly.

The plans with the lowest estimated costs usually have actual execution costs that are either the lowest actual execution costs or are close to the lowest actual execution costs.

✳ **Catalog information:** The database-system catalog stores the following statistical information about database relations:

- n_r , the number of tuples in the relation r .
- b_r , the number of blocks containing tuples of relation r .
- l_r , the size of a tuple of relation r in bytes.
- f_r , the blocking factor of relation r —that is, the number of tuples of relation r that fit into one block.
- $V(A, r)$, the number of distinct values that appear in the relation r for attribute A . This value is the same as the size of $\Pi_A(r)$. If A is a key for relation r , $V(A, r)$ is n_r .

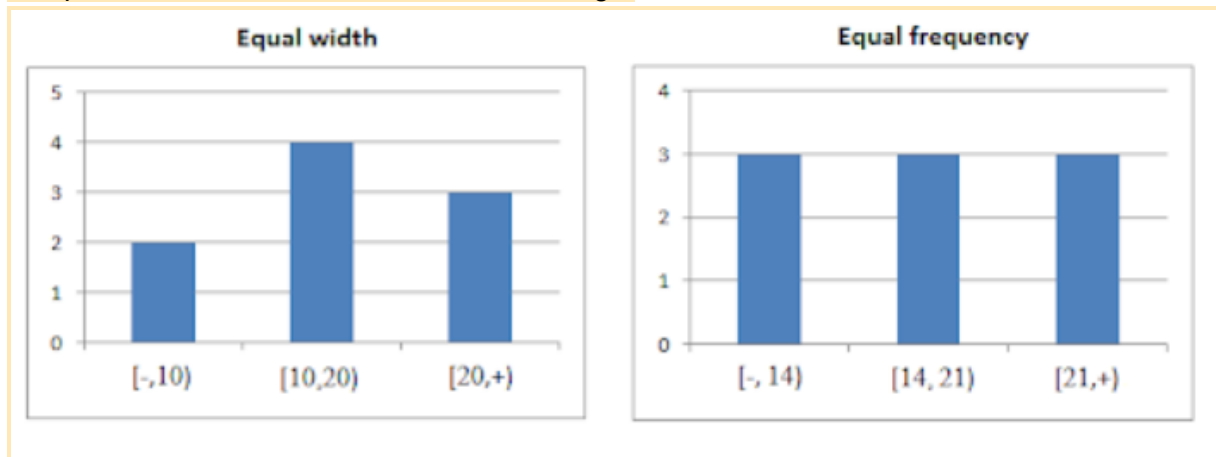
If we assume that the tuples of relation r are stored together physically in a file, the following equation holds:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

If we wish to maintain accurate statistics, then every time a relation is modified, we must also update the statistics. This update incurs a substantial amount of overhead.

Because of this, most systems update the statistics during periods of light system load.

Real-world optimizers often maintain further statistical information to improve the accuracy of their cost estimates of evaluation plans. For instance, most databases store the distribution of values for each attribute as a **histogram**: in a histogram, the values for the attribute are divided into a number of ranges, and with each range the histogram associates the number of tuples whose attribute value lies in that range.



An **equi-width histogram** divides the range of values into equal sized ranges

An **equi-depth histogram** adjusts the boundaries of the ranges such that each range has the same number of values

Histograms used in database systems can also record the number of distinct values in each range, in addition to the number of tuples with attribute values in that range.

In many database applications, some values are very frequent, compared to other values. To get better estimates for queries that specify these values, many databases store a list of **n most frequent values** for some n (say 5 or 10), along with the number of times each value appears. The histogram then only stores statistics for values other than these on the list, since now we have exact counts for these values.

A histogram takes up only a little space, so histograms on several different attributes can be stored in the system catalog.

✿ **Selection size estimation:** The size estimate of the result of a selection operation depends on the selection predicate.

→ $\sigma_{A=a}(r)$: If 'a' is a frequently occurring value for which the occurrence count is available, we can use that value directly as the size estimate for the selection. Otherwise if there is no histogram available, we assume uniform distribution of values. Despite the fact that the uniform-distribution assumption is often not correct, it is a reasonable approximation of reality in many cases

→ $\sigma_{A \leq v}(r)$: Assuming that values are uniformly distributed, we can estimate the number of records that will satisfy the condition $A \leq v$ as:

- 0 if $v < \min(A, r)$
- n_r if $v \geq \max(A, r)$, and,
- $n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$, otherwise.

→ Complex selections: conjunction, disjunction, negation

✿ **Join size estimation:** Estimating the size of a natural join is somewhat more complicated than estimating the size of a selection or of a Cartesian product. Let $r(R)$ and $s(S)$ be relations.

- If $R \cap S = \emptyset$, then $r \bowtie s$ is the same as $r \times s$
- If $R \cap S$ is a key for R , then we know that a tuple of s will join with at most one tuple from r . Therefore, the number of tuples in $r \bowtie s$ is no greater than the number of tuples in s .
- The most difficult case is when $R \cap S$ is a key for neither R nor S . In this case, we assume, as we did for selections, that each value appears with equal probability.

✿ **Size estimation for other operations:**

- **Projection**($\Pi_A(r)$): $V(A, r)$, since projection eliminates duplicates.
- **Aggregation**: $V(G, r)$, since there is one tuple in the ag for each distinct value of G .
- **Set operations**: If the two inputs to a set operation are selections on the same relation, we can rewrite the set operation as disjunctions, conjunctions, or negations. Union or intersection (sizes of r and s and minimum of the sizes of r and s respectively)
- **Outer join**: The estimated size is $r \bowtie s$ plus the size of the outer relation.

✿ **Estimation of number of distinct values:**

For selections, $V(A, \sigma_\theta(r))$ can be estimated in these ways:

- If the selection condition θ forces A to take on a specified value: 1
- If θ forces A to take on one of a specified set of values: Set to the number of specified values
- If the selection condition θ is of the form $A \text{ op } v$, where op is a comparison operator: $V(A, r) * s$ (where s is the selectivity of the selection)
- In all other cases, we assume: $\min(V(A, r), n_{\sigma_\theta(r)})$

For joins, $V(A, r \bowtie s)$ can be estimated in these ways:

- If all attributes in A are from r/s , $V(A, r \bowtie s)$ is estimated as $\min(V(A, r/s), n_{r \bowtie s})$
- If A contains attributes $A1$ from r and $A2$ from s , then $V(A, r \bowtie s)$ is estimated as: $\min(V(A1, r) * V(A2 - A1, s), V(A1 - A2, r) * V(A2, s), n_{r \bowtie s})$

Choice of evaluation plans

A cost-based optimizer explores the space of all query-evaluation plans that are equivalent to the given query, and chooses the one with the least estimated cost.

Cost-based optimization with arbitrary equivalence rules is fairly complicated.

Exploring the space of all possible plans may be too expensive for complex queries.

✿ **Cost-based join-order selection:** In this section we consider the problem of choosing the optimal join order for a query with a couple joins and selections specified in the where clause.

Consider the expression: $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$. With $n = 3$, there are 12 different join orderings:

$r_1 \bowtie (r_2 \bowtie r_3)$	$r_1 \bowtie (r_3 \bowtie r_2)$	$(r_2 \bowtie r_3) \bowtie r_1$	$(r_3 \bowtie r_2) \bowtie r_1$
$r_2 \bowtie (r_1 \bowtie r_3)$	$r_2 \bowtie (r_3 \bowtie r_1)$	$(r_1 \bowtie r_3) \bowtie r_2$	$(r_3 \bowtie r_1) \bowtie r_2$
$r_3 \bowtie (r_1 \bowtie r_2)$	$r_3 \bowtie (r_2 \bowtie r_1)$	$(r_1 \bowtie r_2) \bowtie r_3$	$(r_2 \bowtie r_1) \bowtie r_3$

with n relations, there are $(2(n-1))! / (n-1)!$ different join orders.

Dynamic-programming algorithms store results of computations and reuse them, a procedure that can reduce execution time greatly.

We now consider how to find the optimal join order for a set of n relations $S = \{r_1, r_2, \dots, r_n\}$, where each relation may have selection conditions, and a set of join conditions between the relations r_i is provided. We assume that relations have unique names.

The procedure applies selections on individual relations at the earliest possible point, that is, when the relations are accessed.

The procedure stores the evaluation plans it computes in an associative array **bestplan**, which is indexed by sets of relations. Each element of the associative array contains two components: the cost of the best plan of S , and the plan itself. The value of **bestplan**[S].cost is assumed to be initialized to ∞ if **bestplan**[S] has not yet been computed.

- Checks if the best plan for computing the join of the given set of relations S has been computed already; if so, it returns the already computed plan.
- If S contains only one relation, the best way of accessing S is recorded in **bestplan**.
- If S contains more than one relation, the procedure tries every way of dividing S into two disjoint subsets. For each division, the procedure recursively finds the best plans for each of the two subsets. It then considers all possible algorithms for joining the results of the two subsets.

The cost of each alternative is considered, and the least cost option chosen.

The order in which tuples are generated by the join of a set of relations is important for finding the best overall join order, since it can affect the cost of further joins.

A particular sort order of the tuples is said to be an interesting sort order if it could be useful for a later operation. Hence, it is not sufficient to find the best join order for each subset of the set of n given relations. Instead, we have to find the best join order for each subset, for each interesting sort order of the join result for that subset.

✿ **Cost-based optimization with equivalence rules:** In this section we outline how to create a general-purpose cost-based optimizer based on equivalence rules.

The procedure for generating equivalent expressions can be modified to generate all possible evaluation plans as follows: A new class of equivalence rules, called physical equivalence rules, is added that allows a logical operation, such as a join, to be transformed to a physical operation, such as a hash join, or a nested-loops join.

But this is very expensive; to make the approach work efficiently requires the following:

1. A space-efficient representation of expressions that avoids making multiple copies of the same subexpressions when equivalence rules are applied.
2. Efficient techniques for detecting duplicate derivations of the same expression.
3. A form of dynamic programming based on memoization, which stores the optimal query evaluation plan for a subexpression when it is optimized for the first time; subsequent requests to optimize the same subexpression are handled by returning the already memorized plan.
4. Techniques that avoid generating all possible equivalent plans by keeping track of the cheapest plan generated for any subexpression up to any point of time, and pruning away any plan that is more expensive than the cheapest plan found so far for that subexpression.

✿ **Heuristics in optimization:** Optimizers use heuristics to reduce the cost of optimization. Here is an example of an heuristics rule:

“Perform selection operations as early as possible”

We say that the preceding rule is a heuristic because it usually, but not always, helps to reduce the cost.

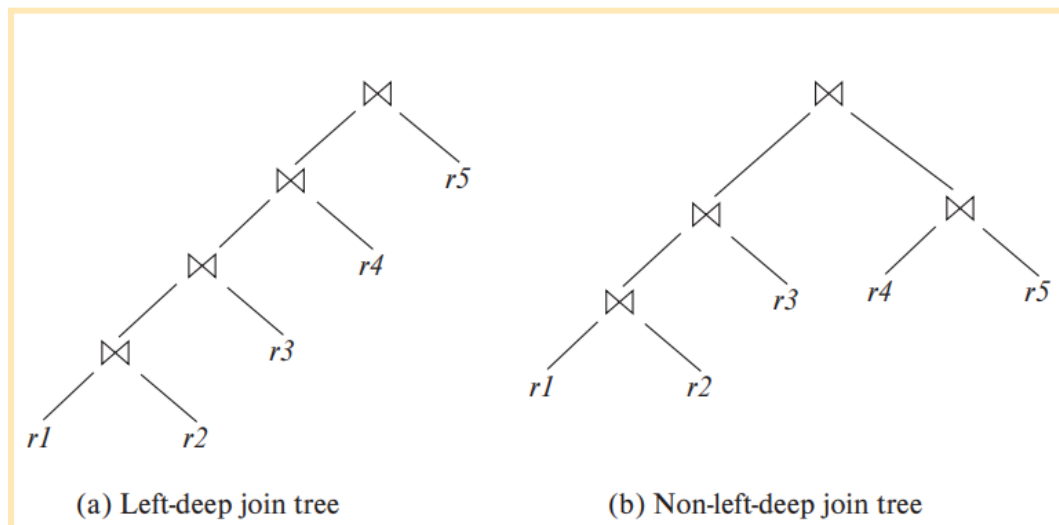
Whenever we need to generate a temporary relation, it is advantageous to apply immediately any projections that are possible (because projection reduces the size of relations).

“Perform projections early”

Optimizers based on join-order enumeration typically use heuristic transformations to handle constructs other than joins, and applying the cost-based join-order selection algorithm to subexpressions involving only joins and selections.

Most practical query optimizers have further heuristics to reduce the cost of optimization. For example, many query optimizers, such as the System R optimizer, do not consider all join orders. The System R optimizer considers only those join orders where the right operand of

each join is one of the initial relations $r_1, \dots, r_n$. Such join orders are called **left-deep join orders**.



The time it takes to consider all left-deep join orders is $O(n!)$, which is much less than the time to consider all join orders.

With the use of dynamic-programming optimizations, the System R optimizer can find the best join order in time $O(n^2)$. Contrast this cost with the $O(3^n)$ time required to find the best overall join order.

Un enfoque heurístico para reducir el costo del join-order selection, funciona algo así:

1. Para un n -way join, considera n planes de evaluación. Cada plan utiliza un left-deep join order, comenzando con una de las n relaciones diferentes.
2. La heurística construye el orden de unión para cada uno de los n planes de evaluación seleccionando repetidamente la "mejor" relación para unir a continuación, sobre la base de una clasificación de las rutas de acceso disponibles.
3. Se elige la unión de nested-loop o la de sort-merge para cada una de las uniones, dependiendo de las rutas de acceso disponibles.
4. Finalmente, la heurística elige uno de los n planes de evaluación de manera heurística, sobre la base de minimizar el número de uniones de bucles anidados que no tienen un índice disponible en la relación interna y en el número de uniones de ordenación-fusión.

Los enfoques de optimización de queries que han adoptado elecciones heurísticas de plan ya se han adoptado en varios sistemas.

Most optimizers allow a cost budget to be specified for query optimization. The search for the optimal plan is terminated when the optimization cost budget is exceeded, and the best plan found up to that point is returned. The budget itself may be set dynamically.

If the best plan found so far is expensive, it makes sense to invest more time in optimization, which could result in a significant reduction in execution time; optimizers usually first apply cheap heuristics to find a plan and then start full cost-based optimization with a budget based on the heuristically chosen plan.

Caching and reuse of query plans is referred to as plan caching.

Even with the use of heuristics, cost-based query optimization imposes a substantial overhead on query processing. However, the added cost of cost-based query optimization is usually more than offset by the saving at query-execution time, which is dominated by slow disk accesses.

The difference in execution time between a good plan and a bad one may be huge, making query optimization essential.

The achieved saving is magnified in those applications that run on a regular basis. Therefore, most commercial systems include relatively sophisticated optimizers.

✿ **Optimizing nested subqueries:** SQL conceptually treats nested subqueries in the **where** clause as functions that take parameters and return either a single value or a set of values.

The parameters are the variables from an outer-level query that are used in the nested subquery (these variables are called correlation variables).

```
select name
from instructor
where exists (select *
              from teaches
              where instructor.ID = teaches.ID
              and teaches.year = 2019);
```

Conceptually, the subquery can be viewed as a function that takes a parameter (here, *instructor.ID*) and returns the set of all courses taught in 2019 by instructors (with the same *ID*).

The technique of evaluating a nested subquery by invoking it as a function is called correlated evaluation. Correlated evaluation is not very efficient, since the subquery is separately evaluated for each tuple in the outer level query. A large number of random disk I/O operations may result. (SQL optimizers therefore attempt to transform nested subqueries into joins).

En esta parte habla de semijoin y anti semijoin pero yo voy a fingir demencia (por favor que no entre)(pág 777 del libro/806 pdf reader)

The process of replacing a nested query by a query with a join, semijoin, or antisemijoin is called decorrelation.

Decorrelation is clearly more complicated when the nested subquery uses aggregation, or when the nested subquery is used as a scalar subquery. In fact, decorrelation is not possible for certain cases of subqueries.

Optimization of complex nested subqueries is a complicated task, as you can infer from the preceding discussion, and many optimizers do only a limited amount of decorrelation. It is better to avoid using complex nested subqueries, where possible, since we cannot be sure that the query optimizer will succeed in converting them to a form that can be evaluated efficiently.

Materialized views

A **materialized view** is a view whose contents are **computed and stored**. It is much **cheaper** in many cases to read the contents of a materialized view than to compute the contents of the view by executing the query defining the view.

✿ **View maintenance:** Materialized views must be kept up-to-date when the data used in the view definition changes. The task of **keeping a materialized view up-to-date with the underlying data** is known as **view maintenance**.

- **Manually written code:** Every piece of code that updates salary **must also update the total salary amount for the corresponding department**, but this approach is error prone, since it is **easy to miss some places** where the salary is updated.
- **Define triggers:** On insert, delete, and update of each relation in the view definition. The **triggers must modify the contents of the materialized view**, to take into account the change that caused the trigger to fire.
- **Incremental view maintenance:** **Modifying only the affected parts of the materialized view**
- **Immediate view maintenance:** Incremental view maintenance is **performed as soon as an update occurs**
- **Deferred view maintenance:** View maintenance is **deferred to a later time**

In modern database systems, once a view is declared to be materialized, the DBS computes the contents of the view and incrementally updates the contents when the underlying data change.

✿ **Incremental view maintenance:** To understand how to maintain materialized views incrementally, we start off by considering individual operations, and then we see how to handle a complete expression.

The **changes** (inserts and deletes) **to a relation or expression** are referred to as its **differential**.

- 1) **Join:** Supongamos que tenemos una vista $v = r \bowtie s$, pero hacemos una inserción de tuplas i en r , por lo tanto v ahora es: $v^{new} = v^{old} \cup (i_r \bowtie s)$. Si en su lugar hubiéramos borrado las tuplas d en r , v ahora sería: $v^{new} = v^{old} - (d_r \bowtie s)$

- 2) **Selection:** Nuestra vista $v = \sigma_{\theta}(r)$. Si hacemos una inserción de un conjunto de tuplas i : $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$; Si borramos un conjunto de tuplas d : $v^{new} = v^{old} - \sigma_{\theta}(d_r)$.
- 3) **Projection:** For each tuple in a projection such as $\Pi_A(r)$, we will keep a count of how many times it was derived. Si queremos borrar un conjunto de tuplas d en r , para cada tupla en d tenemos que:
 - a) Encontrar $t.A$ (proyección de t en el atributo A) en la vista materializada y decrementar el contador de “almacenamiento”(count stored) en 1.
 - b) Si la cuenta se hace 0, borramos la tupla
 - c) Para insertar: Si la tupla a insertar ya está en la vista materializada, aumentamos el contador de “almacenamiento” en 1, sino, la agregamos a la vista con el contador en 1.
- 4) **Agregación:** count, sum, avg, min y max tienen procedimientos parecidos a la proyección. Para avg nos basamos en sum y count
- 5) **Intersección, unión y diferencia:** En la inserción, vemos si la agregamos si se cumple la condición (según la operación). Podemos borrar directamente.

✿ **Handling expressions:** To handle an entire expression, we can derive expressions for computing the incremental change to the result of each subexpression, starting from the smallest subexpressions

✿ **Query optimization and materialized views:** Query optimization can be performed by treating materialized views just like regular relations. However, materialized views offer further opportunities for optimization:

Rewriting queries to use materialized views: Using $v \bowtie t$ instead of $r \bowtie s \bowtie t$ (where $v = r \bowtie s$)
 Replacing a use of a materialized view with the view definition: Case of $\sigma_{A=10}(v)$ (where $v = r \bowtie s$) but r has an index on A and s has an index on B . The best plan for this query may be to replace v with $r \bowtie s$.

✿ **Materialized views and index selection:** Another related optimization problem is that of **materialized view selection**, namely, “What is the best set of views to materialize?”. This decision must be made on the basis of the system **workload**, which is a sequence of queries and updates that reflects the **typical load on the system**.

Indices are just like materialized views, in that they too are derived data, can speed up queries, and may slow down updates.

Advanced topics in query optimization

✿ **Top-K optimization:** Many queries fetch results sorted on some attributes, and require only the top K results for some K (sometimes the bound K is specified explicitly).

One approach is to use pipelined plans that can generate the results in sorted order. Another approach is to estimate what is the highest value on the sorted attributes that will appear in the top-K output

✿ **Join minimization:** When queries are generated through views, sometimes more relations are joined than are needed for computation of the query. Dropping a relation from a join as above is an example of join minimization (el ejemplo habla de usar solo los atributos de una relación en un join, por lo que podríamos no hacer el join e ignorar una de las relaciones)

✿ **Update optimization:** Update queries often involve subqueries in the set and where clauses, which must also be taken into account in optimizing the update. If the update is done while the selection is being evaluated by an index scan, an updated tuple may be reinserted in the index ahead of the scan and seen again by the scan.

The problem of an update affecting the execution of a query associated with the update is known as the **Halloween problem**. The problem can be avoided by executing the queries defining the update first, creating a list of affected tuples, and updating the tuples and indices as the last step.

Update plans can be optimized by checking if the Halloween problem can occur, and if it cannot occur, updates can be performed while the query is being processed, reducing the update overheads.

✿ **Multiquery optimization and shared scans:** When a batch of queries are submitted together, a query optimizer can potentially exploit common subexpressions between the different queries, evaluating them once and reusing them where required. Such optimization is known as **multiquery optimization**.

Common subexpression elimination optimizes subexpressions shared by different expressions in a program by computing and storing the result and reusing it wherever the subexpression occurs.

Sharing of relation scans between queries is another limited form of multiquery optimization that is implemented in some databases. The **shared-scan optimization** works as follows: Instead of reading the relation repeatedly from disk, data are read once from disk and pipelined to each of the queries.

✿ **Parametric query optimization**

Reuse of plans by plan caching is reasonable if the optimal query plan is not significantly affected by the exact value of the constants in the query. However, if the plan is affected by the value of the constants, parametric query optimization is an alternative.

In **parametric query optimization**, a query is optimized **without** being provided specific values for its parameters. The optimizer then outputs several plans, each optimal for a different parameter value. A plan would be output by the optimizer only if it is optimal for some

possible value of the parameters. The set of alternative plans output by the optimizer are stored.

⌘ **Adaptive query processing**

The optimizer may choose a plan that turns out to perform very badly; **adaptive operators** that choose the specific operator at execution time provide a partial solution to this problem.

For example, SQL Server supports an adaptive join algorithm that checks the size of its outer input, and chooses either nested loops join, or hash join depending on the size of the outer input. Many systems also include the ability to monitor the behavior of a plan during query execution, and adapt the plan accordingly.

Capitulo 17 - Database System Concepts

Review Terms

- Transaction
- ACID properties
 - Atomicity
 - Consistency
 - Isolation
 - Durability
- Inconsistent state
- Storage types
 - Volatile storage
 - Non-volatile storage
 - Stable storage
- Concurrency-control system
- Recovery system
- Transaction state
 - Active
 - Partially committed
 - Failed
 - Aborted
 - Committed
 - Terminated
- compensating transaction
- Transaction
 - Restart
 - Kill
- Observable external writes
- Concurrent executions
- Serial execution
- Schedules
- Conflict of operations
- Conflict equivalence
- Conflict serializability
- Serializability testing
- Precedence graph
- Serializability order
- Recoverable schedules
- Cascading rollback
- Cascadeless schedules
- Isolation levels
 - Serializable
 - Repeatable read
 - Read committed
 - Read uncommitted
- Dirty writes
- Automatic commit
- Concurrency control
- Locking
- Timestamp ordering
- Snapshot isolation
- Phantom phenomenon
- Predicate locking

Transactions

It is essential that all these operations occur, or that, in case of a failure, none occur. It would be unacceptable if the checking account were debited but the savings account not credited.

Transaction concept

A transaction is a unit of program execution that accesses and possibly updates various data items. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language.

ACID Properties

- **Atomicity**: Either all operations of the transaction are reflected properly in the database, or none are (all-or-none).
- **Consistency**: Execution of a transaction in isolation (i.e., with no other transaction executing concurrently) preserves the consistency of the database. (good data-integrity constraints)
- **Isolation**: Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started or T_j started execution after T_i finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
- **Durability**: After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

A simple transaction model

We begin our study of transactions with a simple database language that focuses on when data are moved from disk to main memory and from main memory to disk.

Transactions access data using two operations:

- **Read(X)**: transfers the data item X from the database to a variable, also called X, in a buffer in main memory belonging to the transaction that executed the read operation.
- **Write(X)**: transfers the value in the variable X in the main-memory buffer of the transaction that executed the write to the data item X in the database. The write operation may be temporarily stored elsewhere and executed on the disk later.

Let us now consider each of the ACID properties for:

```
 $T_i$ : read(A);  
      A := A - 50;  
      write(A);  
      read(B);  
      B := B + 50;  
      write(B).
```

- **Atomicity:** Suppose that the failure happened after the write(A) operation but before the write(B) operation. In this case, the values of accounts A and B reflected in the database are \$950 and \$2000. Because of the failure, the state of the system no longer reflects a real state of the world that the database is supposed to capture. We term such a state an **inconsistent state**. We must ensure that such inconsistencies are not visible in a database system.

The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write. This information is written to a file called the log.

"If the transaction does not complete its execution, the database system restores the old values from the log to make it appear as though the transaction never executed."

- **Consistency:** The consistency requirement here is that the sum of A and B be unchanged by the execution of the transaction. Without the consistency requirement, money could be created or destroyed by the transaction.
- **Isolation:** A way to avoid the problem of concurrently executing transactions is to execute transactions serially. However, concurrent execution of transactions provides significant performance benefits. Ensuring the isolation property is the responsibility of a component of the database system called the **concurrency-control system**.
- **Durability:** Once the execution of the transaction completes successfully, it must be the case that no system failure can result in a loss of data corresponding to this transfer of funds. Once a transaction completes successfully, all the updates that it carried out on the database persist. We can guarantee durability by ensuring that either:
- ◆ Updates carried out by the transaction have been written to disk before the transaction completes.
 - ◆ Information about the updates carried out by the transaction is written to disk, and such information is sufficient to enable the database to reconstruct the updates when the database system is restarted after the failure.

Storage structure

To understand how to ensure the atomicity and durability properties of a transaction, we must gain a better understanding of how the various data items in the database may be stored and accessed.

In Chapter 12, we saw that storage media can be distinguished by their relative speed, capacity, and resilience to failure, and classified as volatile storage or non-volatile storage; we are going to introduce another class of storage called, **stable storage**.

- **Volatile storage:** Information does not usually survive system crashes. Examples of such storage are main memory and cache memory. Access to volatile storage is extremely fast, both because of the speed of the memory access itself and because it is possible to access any data item in volatile storage directly.
- **Non-volatile storage:** Information survives system crashes. Examples of non-volatile storage include secondary storage devices such as magnetic disk and flash storage, used for online storage, and tertiary storage devices such as optical media and magnetic tapes, used for archival storage. Non-volatile storage is slower than volatile storage, particularly for random access. Both secondary and tertiary storage devices, however, are susceptible to failures that may result in loss of information.
- **Stable storage:** Information is never lost (it is possible, although extremely unlikely). Although stable storage is theoretically impossible to obtain, it can be closely approximated by techniques that make data loss extremely unlikely. To implement stable storage, we replicate the information in several non-volatile storage media (usually disk) with independent failure modes. Updates must be done with care to ensure that a failure during an update to stable storage does not cause a loss of information.

For a transaction to be durable, its changes need to be written to stable storage.

For a transaction to be atomic, log records need to be written to stable storage before any changes are made to the database on disk.

The degree to which a system ensures durability and atomicity depends on how stable its implementation of stable storage really is.

Transaction atomicity and durability

A transaction that does not complete its execution successfully is termed **aborted**.

An aborted transaction must have no effect on the state of the database. Once the changes caused by an aborted transaction have been undone, we say that the transaction has been **rolled back**. It is part of the responsibility of the recovery scheme to manage transaction **aborts** (typically using logs).

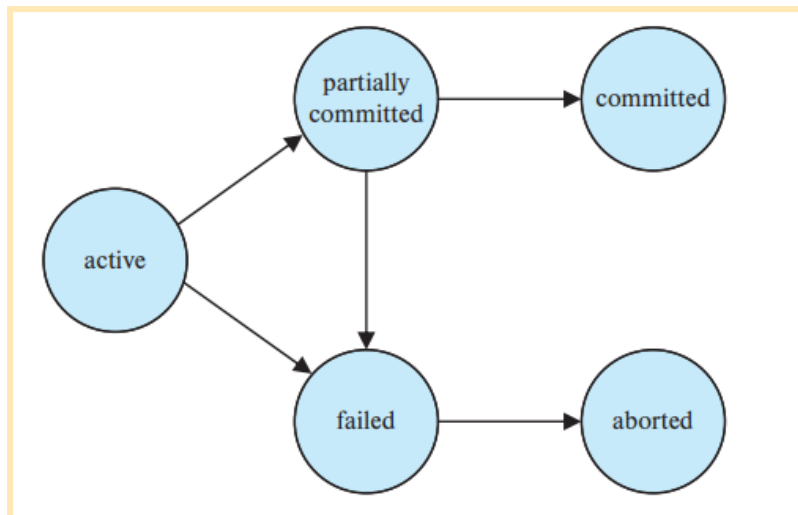
A transaction that completes its execution successfully is said to be **committed**. A committed transaction that has performed updates transforms the database into a new consistent state, which must persist even if there is a system failure. The only way to undo the effects of a committed transaction is to execute a **compensating transaction**.

However, it is not always possible to create such a compensating transaction. Therefore, the responsibility of writing and executing a compensating transaction is left to the user and is not handled by the database system.

A transaction must be in one of the following states (abstract transaction model):

- **Active**: The initial state; the transaction stays in this state while it is executing.
- **Partially committed**: After the final statement has been executed.
- **Failed**: After the discovery that normal execution can no longer proceed.
- **Aborted**: After the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.
- **Committed**: After successful completion.

A transaction is said to have **terminated** if it has either committed or aborted.



A transaction enters the **failed** state after the system determines that the transaction can no longer proceed with its normal execution. Such a transaction must be rolled back. Then, it enters the **aborted** state. At this point, the system has two options:

1. **restart (new) the transaction**, but only if the transaction was aborted as a result of some hardware or software error that was not created through the internal logic of the transaction.
2. **kill the transaction**. It usually does so because of some internal logical error that can be corrected only by rewriting the application program

Once an **observable external write** has occurred, it cannot be erased, since it may have been seen external to the database system. Most systems allow such writes to take place only after the transaction has entered the committed state.

For certain applications, it may be desirable to allow active transactions to display data to users, particularly for long-duration transactions that run for minutes or hours. Unfortunately, we cannot allow such output of observable data unless we are willing to compromise transaction atomicity.

Transaction Isolation

Allowing multiple transactions to update data concurrently causes several complications with consistency of the data, as we saw earlier. Ensuring consistency in spite of concurrent execution of transactions requires extra work; it is far easier to insist that transactions run serially.

There are two good reasons for allowing concurrency:

- **Improved throughput and resource utilization:** The parallelism of the CPU and the I/O system can be exploited to run multiple transactions in parallel. (Ejemplo: Un disco puede estar haciendo lectura/escritura de una transacción en un disco, otra transacción puede estar corriendo en la CPU y otro disco puede estar ejecutando una tercera transacción). All of this increases the throughput (rendimiento) of the system; the processor and disk utilization also increase.
- **Reduced waiting time:** If transactions run serially, a short transaction may have to wait for a preceding long transaction to complete, which can lead to unpredictable delays in running a transaction. Concurrent execution reduces the unpredictable delays in running transactions. It also reduces the average response time: the average time for a transaction to be completed after it has been submitted.

The motivation for using concurrent execution in a database is essentially the same as the motivation for using multiprogramming in an operating system.

The database system must control the interaction among the concurrent transactions to prevent them from destroying the consistency of the database. It does so through a variety of mechanisms called concurrency-control schemes.

The execution sequences just described are called schedules. They represent the chronological order in which instructions are executed in the system. A schedule for a set of transactions must consist of all instructions of those transactions and they must preserve the order in which the instructions appear in each individual transaction. We include in our schedules the commit operation to indicate that the transaction has entered the committed state.

These schedules are **serial**: Each serial schedule consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule.

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B) commit	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B) commit

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

When the database system executes several transactions concurrently, the corresponding schedule no longer needs to be serial. Several execution sequences are possible, since the various instructions from both transactions may now be interleaved. In general, it is not possible to predict exactly how many instructions of a transaction will be executed before the CPU switches to another transaction.

It is the job of the database system to ensure that any schedule that is executed will leave the database in a consistent state. The concurrency-control component of the database system carries out this task.

That is, the schedule should, in some sense, be equivalent to a serial schedule. Such schedules are called serializable schedules.

Serializability

Before we can consider how the concurrency-control component of the database system can ensure serializability, we consider how to determine when a schedule is serializable.

Certainly, serial schedules are serializable, but if steps of multiple transactions are interleaved, it is harder to determine whether a schedule is serializable. It is difficult to determine exactly what operations a transaction performs and how operations of various transactions interact.

We discuss different forms of schedule equivalence but focus on a particular form called conflict serializability.

Analizamos el caso donde, en un schedule S, tenemos dos operaciones I y J consecutivas en las transacciones T_i y T_j respectivamente ($i \neq j$). Si I y J se refieren al mismo data item Q, su orden nos va a importar.

Since we are dealing with only read and write instructions, there are four cases that we need to consider:

1. $I = \text{read}(Q)$, $J = \text{read}(Q)$. The order of I and J does not matter, since the same value of Q is read by T_i and T_j , regardless of the order.
2. $I = \text{read}(Q)$, $J = \text{write}(Q)$. If I comes before J, then T_i does not read the value of Q that is written by T_j in instruction J. If J comes before I, then T_i reads the value of Q that is written by T_j . Thus, the order of I and J matters.
3. $I = \text{write}(Q)$, $J = \text{read}(Q)$. The order of I and J matters for reasons similar to those of the previous case.
4. $I = \text{write}(Q)$, $J = \text{write}(Q)$. Since both instructions are write operations, the order of these instructions does not affect either T_i or T_j . However, the value obtained by the

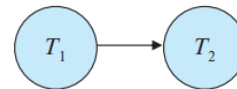
next $\text{read}(Q)$ instruction of S is affected, since the result of only the latter of the two write instructions is preserved in the database. If there is no other $\text{write}(Q)$ instruction after I and J in S , then the order of I and J directly affects the final value of Q in the database state that results from schedule S .

We say that I and J **conflict** if they are operations by different transactions on the same data item, and at least one of these instructions is a write operation.

If a schedule S can be transformed into a schedule S' by a series of swaps of nonconflicting instructions, we say that S and S' are **conflict equivalent**. We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.

We now present a simple and efficient method for determining the conflict serializability of a schedule. Consider a schedule S . We construct a **directed graph**, called a precedence graph, from S . This graph consists of a pair $G = (V, E)$, where V is a set of transactions and E is the set of edges $T_i \rightarrow T_j$ for which one of three conditions hold:

1. T_i executes $\text{write}(Q)$ before T_j executes $\text{read}(Q)$
2. T_i executes $\text{read}(Q)$ before T_j executes $\text{write}(Q)$.
3. T_i executes $\text{write}(Q)$ before T_j executes $\text{write}(Q)$.



If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then, in any serial schedule S' equivalent to S , T_i must appear before T_j . If the graph contains no cycles, then the schedule S is conflict serializable.

A **serializability order** of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph. This process is called **topological sorting**. Thus, to test for conflict serializability, we need to construct the precedence graph and to invoke a cycle-detection algorithm.

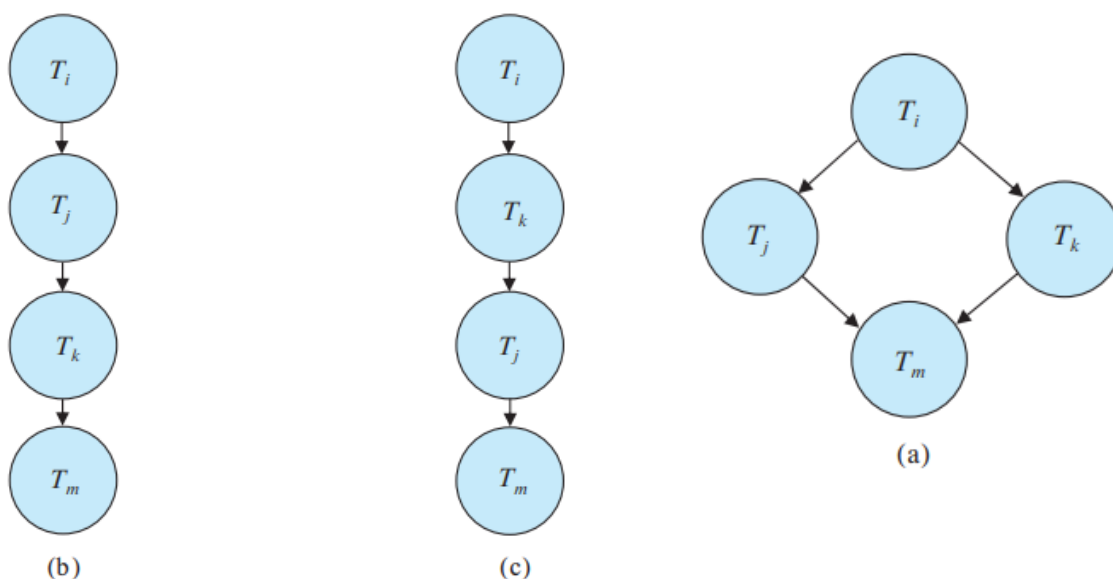


Figure 17.12 Illustration of topological sorting.

Transaction isolation and atomicity

So far, we have studied schedules while assuming implicitly that there are no transaction failures. We now address the **effect of transaction failures** during concurrent execution.

In a system that allows concurrent execution, the atomicity property requires that any transaction T_j that is dependent on T_i is also aborted. To achieve this, we need to place **restrictions on the types of schedules permitted** in the system.

✿ Recoverable schedules

T_6	T_7
read(A) write(A)	
	read(A) commit
read(B)	

Consider this schedule, in which T_7 is a transaction that performs only one instruction: read(A). We call this a **partial schedule** because we have not included a commit or abort operation for T_6 .

Now suppose that T_6 fails before it commits. T_7 has read the value of data item A written by T_6 . Therefore, we say that T_7 is **dependent** on T_6 . Because of this, we must abort T_7 to ensure atomicity

The schedule from the example is said to be a **nonrecoverable schedule**. A **recoverable schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the commit operation of T_j .

✿ Cascadeless schedules

Even if a schedule is recoverable, to recover correctly from the failure of a transaction T_i , we may have to **roll back several transactions**.

T_8	T_9	T_{10}
read(A) read(B) write(A)		
	read(A) write(A)	
		read(A)
abort		

Transaction T_8 writes a value of A that is read by transaction T_9 . Transaction T_9 writes a value of A that is read by transaction T_{10} . Suppose that, at this point, T_8 **fails**. T_8 must be rolled back. Therefore, T_9 and T_{10} must be rolled back too.

Cascading rollback is undesirable, since it leads to the **undoing of a significant amount of work**.

A **cascadeless schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .

Transaction isolation levels

If every transaction has the property that it maintains database consistency if executed alone, then serializability ensures that concurrent executions maintain consistency. However, the protocols required to ensure serializability may allow too little concurrency for certain applications. In these cases, weaker levels of consistency are used.

The isolation levels specified by the SQL standard are as follows:

- **Serializable:** Usually ensures serializable execution.
- **Repeatable read:** Allows only committed data to be read and further requires that, between two reads of a data item by a transaction, no other transaction is allowed to update it.
- **Read committed:** Allows only committed data to be read, but does not require repeatable reads. Between two reads of a data item by the transaction, another transaction may have updated the data item and committed.
- **Read uncommitted:** Allows uncommitted data to be read.

All the isolation levels above additionally disallow dirty writes, that is, they disallow writes to a data item that has already been written by another transaction that has not yet committed or aborted.

Many database systems run, by default, at the read-committed isolation level.

set transaction isolation level serializable

By default, most databases commit individual statements as soon as they are executed. Such automatic commit of individual statements must be turned off to allow multiple statements to run as a single transaction.

Implementation of isolation levels

There are various concurrency-control policies that we can use to ensure that, even when multiple transactions are executed concurrently, only acceptable schedules are generated.

As a trivial example of a concurrency-control policy, consider this: A transaction acquires a lock on the entire database before it starts and releases the lock after it has committed. While a transaction holds a lock, no other transaction is allowed to acquire the lock, and all must therefore wait for the lock to be released. As a result of the locking policy, only one transaction can execute at a time (serializable).

Here we provide an overview of how some of the most important concurrency-control mechanisms work.

✿ **Locking:** Instead of locking the entire database, a transaction could instead lock only those data items that it accesses. Under such a policy, the transaction must hold locks long enough to ensure serializability, but for a period short enough not to harm performance

excessively. Further improvements to locking result if we have two kinds of locks: **shared** (for data that T reads) and **exclusive** (for data that T writes).

✿ **Timestamps:** For each data item, the system keeps two timestamps.

- The **read timestamp** of a data item holds the largest timestamp of those transactions that read the data item.
- The **write timestamp** of a data item holds the timestamp of the transaction that wrote the current value of the data item.

Timestamps are used to ensure that transactions access each data item in order of the transactions' timestamps if their accesses conflict.

✿ **Multiple versions and snapshot isolation:** By maintaining more than one version of a data item, it is possible to allow a transaction to read an old version of a data item rather than a newer version written by an uncommitted transaction or by a transaction that should come later in the serialization order.

There are a variety of multiversion concurrency-control techniques. One in particular, called **snapshot isolation**: Where each transaction is given its own version, or snapshot, of the database when it begins (isolated from the updates made by other transactions). If the transaction updates the database, that update appears only in its own version; this updates are applied to the "real" database if the transaction commits.

Snapshot isolation ensures that attempts to read data never need to wait. Read-only transactions cannot be aborted; only those that modify data run a slight risk of aborting. It provides too much isolation.

Transactions as SQL statements

In a concurrent execution of these transactions (Al mismo tiempo que hacemos un select, otra query inserta un dato que cumple con los criterios), it is intuitively clear that they conflict, but this is a conflict that may not be captured by our simple model. This situation is referred to as the **phantom phenomenon** because a conflict may exist on "phantom" data.

So the same transaction, if run more than once, might reference different data items each time it is run if the values in the database change between runs.

It is not sufficient for concurrency control to consider only the tuples that are accessed by a transaction; the information used to find the tuples that are accessed by the transaction must also be considered for the purpose of concurrency control.

Capitulo 18 - Database System Concepts

Review Terms

- Concurrency control
- Lock types
 - Shared-mode (S) lock
 - Exclusive-mode (X) lock
- Lock
 - Compatibility
 - Request
 - Wait
 - Grant
- Deadlock
- Starvation
- Locking protocol
- Legal schedule
- Two-phase locking protocol
 - Growing phase
 - Shrinking phase
 - Lock point
- Timeout-based schemes
- Deadlock detection
 - Wait-for graph
- Deadlock recovery
 - Total rollback
 - Partial rollback
- Multiple granularity
 - Explicit locks
 - Implicit locks
 - Intention locks
- Intention lock modes
 - Intention-shared (IS)
 - Intention-exclusive (IX)
 - Shared and intention-exclusive (SIX)
- Multiple-granularity locking protocol
- Timestamp
 - System clock
 - Logical counter
 - W-timestamp(Q)
 - R-timestamp(Q)
- Timestamp-ordering protocol
 - Thomas' write rule
- Validation-based protocols
 - Read phase
 - Validation phase
- Strict two-phase locking
- Rigorous two-phase locking
- Lock conversion
 - Upgrade
 - Downgrade
- Graph-based protocols
 - Tree protocol
 - Commit dependency
- Deadlock handling
 - Prevention
 - Detection
 - Recovery
- Deadlock prevention
 - Ordered locking
 - Preemption of locks
 - Wait-die scheme
- Validation test
- Multiversion timestamp ordering
- Multiversion two-phase locking
 - Read-only transactions
 - Update transactions
- Snapshot isolation
 - Lost update
 - First committer wins
 - First updater wins
 - Write skew
 - Select for update
- Insert and delete operations
- Phantom phenomenon
- Index-locking protocol
- Predicate locking
- Weak levels of consistency
 - Degree-two consistency
 - Cursor stability
- Optimistic concurrency control without read validation
- Conversations
- Concurrency in indices
 - Crabbing protocol
 - B-link trees
 - B-link-tree locking protocol
 - Next-key locking
- Latch-free data structures
- Compare-and-swap (CAS) instruction

Concurrency control

When several transactions execute concurrently in the database, however, the isolation property may no longer be preserved. To ensure that it is, the system must control the interaction among the concurrent transactions; this control is achieved through one of a variety of mechanisms called concurrency-control schemes.

Lock-based protocols

“While one transaction is accessing a data item, no other transaction can modify that data item.” The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a lock on that item.

✿ Locks

We introduce two modes in which a data item may be locked:

1. **Shared:** If a transaction T_i has obtained a shared-mode lock (denoted by S) on item Q, then T_i can read, but cannot write, Q.
2. **Exclusive:** If a transaction T_i has obtained an exclusive-mode lock (denoted by X) on item Q, then T_i can both read and write Q.

We require that every transaction request a lock in an appropriate mode on data item Q, depending on the types of operations that it will perform on Q. The transaction makes the request to the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager grants the lock to the transaction.

To state this more generally, given a set of lock modes, we can define a compatibility function on them as follows:

1. Let A and B represent arbitrary lock modes.
2. Ahora habla de que una transacción 1 pide un lock de modo A en Q, en donde la transacción 2 tiene un lock de modo B.
3. Si 1 puede acceder a Q, decimos que el modo A es compatible con el modo B

Note that shared mode is compatible with shared mode.

A transaction requests a shared lock on data item Q by executing the `lock-S(Q)` instruction. Similarly, a transaction requests an exclusive lock through the `lock-X(Q)` instruction. A transaction can unlock a data item Q by the `unlock(Q)` instruction.

Unfortunately, locking can lead to an undesirable situation. For example:

Since T_3 is holding an exclusive-mode lock on B and T_4 is requesting a shared-mode lock on B, T_4 is waiting for T_3 to unlock B. Similarly, since T_4 is holding a shared-mode lock on A and T_3 is requesting an exclusive mode lock on A, T_3 is waiting for T_4 to unlock A.

We have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called **deadlock**.

T_3	T_4
$\text{lock-X}(B)$ $\text{read}(B)$ $B := B - 50$ $\text{write}(B)$ $\text{lock-X}(A)$	 $\text{lock-S}(A)$ $\text{read}(A)$ $\text{lock-S}(B)$

When deadlock occurs, the system must roll back one of the two transactions.

If we do not use locking, or if we unlock data items too soon after reading or writing them, we may get inconsistent states. On the other hand, if we do not unlock a data item before requesting a lock on another data item, deadlocks may occur.

Deadlocks are definitely preferable to inconsistent states, since they can be handled by rolling back transactions, whereas inconsistent states may lead to real-world problems that cannot be handled by the database system.

We shall require that each transaction in the system follow a set of rules, called a **locking protocol**. Locking protocols restrict the number of possible schedules. We shall present several locking protocols that allow only conflict-serializable schedules, and thereby ensure isolation. Before doing so, we introduce some terminology:

1. We say that T_i **precedes** T_j in S, written $T_i \rightarrow T_j$, if there exists a data item Q such that T_i has held lock mode A on Q, and T_j has held lock mode B on Q later, and $\text{comp}(A,B) = \text{false}$.
2. We say that a schedule S is **legal** under a given locking protocol if S is a possible schedule for a set of transactions that follows the rules of the locking protocol.
3. We say that a locking protocol **ensures** conflict serializability if and only if all legal schedules are conflict serializable

✿ Granting of locks

When a transaction requests a lock on a data item in a particular mode, and no other transaction has a lock on the same data item in a conflicting mode, the lock can be granted.

Acá entra un ejemplo en el que una transacción T hace un request de lock exclusivo, pero varios nodos hacen request de lock compartido, por lo que T tiene mucho tiempo de espera. The transaction T may never make progress, and is said to be **starved**.

We can avoid starvation of transactions by granting locks in the following manner:

When a transaction T_i requests a lock on a data item Q in a particular mode M , the concurrency-control manager grants the lock provided that:

- There is no other transaction holding a lock on Q in a mode that conflicts with M .
- There is no other transaction that is waiting for a lock on Q and that made its lock request before T_i .

✿ The two-phase locking protocol

The two-phase locking protocol ensures serializability and requires that each transaction issue lock and unlock requests in two phases:

1. **Growing phase:** A transaction may obtain locks, but may not release any lock.
2. **Shrinking phase:** A transaction may release locks, but may not obtain any new locks.

Consider any transaction. The point in the schedule where the transaction has obtained its final lock (the end of its growing phase) is called the **lock point** of the transaction. Now, transactions can be ordered according to their lock points.

Cascading rollbacks can be avoided by a modification of two-phase locking called the **strict two-phase locking protocol**. This protocol requires that all exclusive-mode locks taken by a transaction be held until that transaction commits.

Another variant of two-phase locking is the **rigorous two-phase locking protocol**, which requires that all locks be held until the transaction commits.

T_8	T_9
lock-S(a_1)	
lock-S(a_2)	lock-S(a_1)
lock-S(a_3)	lock-S(a_2)
lock-S(a_4)	
	unlock(a_1)
lock-S(a_n)	unlock(a_2)
upgrade(a_1)	

Figure 18.9 Incomplete schedule with a lock conversion.

This observation leads us to a refinement of the basic two-phase locking protocol, in which **lock conversions** are allowed. We shall provide a mechanism for upgrading a shared lock to an exclusive lock, and downgrading an exclusive lock to a shared lock.

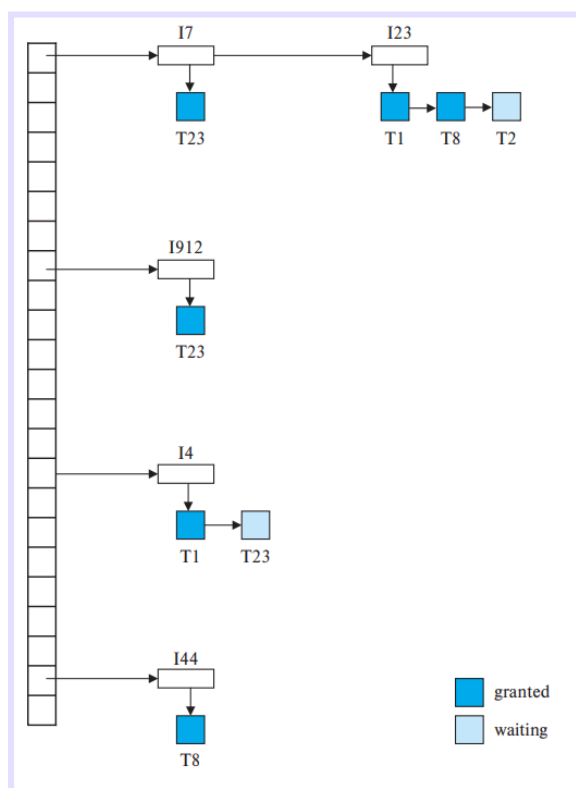
We denote conversion from shared to exclusive modes by **upgrade**, and from exclusive to shared by **downgrade**. Lock conversion cannot be allowed arbitrarily. Rather, upgrading can take place in only the growing phase, whereas downgrading can take place in only the shrinking phase.

Note that a transaction attempting to upgrade a lock on an item Q may be forced to wait. This enforced wait occurs if Q is currently locked by another transaction in shared mode.

✿ Implementation of locking

A lock manager can be implemented as a process that receives messages from transactions and sends messages in reply. The lock-manager process replies to lock-request messages with lock-grant messages, or with messages requesting rollback of the transaction. Unlock messages require only an acknowledgment in response, but may result in a grant message to another waiting transaction.

The lock manager uses this data structure: For each data item that is currently locked, it maintains a linked list of records, one for each request, in the order in which the requests arrived. It uses a hash table, indexed on the name of a data item, to find the linked list for a data item; this table is called the **lock table**. There is also a list of transactions that have been granted locks, or are waiting for locks, for each of the data items.



The table contains locks for five different data items, I4, I7, I23, I44, and I912.

Granted locks are the rectangles filled in a darker shade, while waiting requests are the rectangles filled in a lighter shade.

The lock manager processes requests this way:

1. When a lock request message arrives, it adds a record to the end of the linked list for the data item, if the linked list is present. Otherwise it creates a new linked list, containing only the record for the request.
2. When the lock manager receives an unlock message from a transaction, it deletes the record for that data item in the linked list corresponding to that transaction.
3. If a transaction aborts, the lock manager deletes any waiting request made by the transaction.

This algorithm guarantees freedom from starvation for lock requests, since a request can never be granted while a request received earlier is waiting to be granted.

✿ Graph-based protocols

If we wish to develop protocols that are not two phase, we need additional information on how each transaction will access the database.

The simplest model requires that we have prior knowledge about the order in which the database items will be accessed. Given such information, it is possible to construct locking protocols that are not two phase, but that, nevertheless, ensure conflict serializability.

To acquire such prior knowledge, we impose a partial ordering $\rightarrow$ on the set $D = \{d_1, d_2, \dots, d_h\}$ of all data items. If $d_i \rightarrow d_j$, then any transaction accessing both d_i and d_j must access d_i before accessing d_j .

The partial ordering implies that the set D may now be viewed as a directed acyclic graph, called a **database graph**.

In the **tree protocol**, the only lock instruction allowed is lock-X. Each transaction T_i can lock a data item at most once, and must observe the following rules:

1. The first lock by T_i may be on any data item.
2. Subsequently, a data item Q can be locked by T_i only if the parent of Q is currently locked by T_i .
3. Data items may be unlocked at any time.
4. A data item that has been locked and unlocked by T_i cannot subsequently be relocked by T_i .

All schedules that are legal under the tree protocol are conflict serializable.

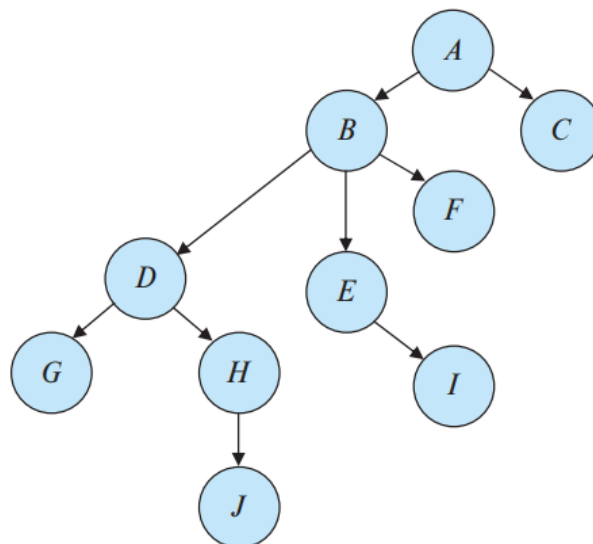


Figure 18.11 Tree-structured database graph.

T_{10} : lock-X(B); lock-X(E); lock-X(D); unlock(B); unlock(E); lock-X(G);
unlock(D); unlock(G).
 T_{11} : lock-X(D); lock-X(H); unlock(D); unlock(H).
 T_{12} : lock-X(B); lock-X(E); unlock(E); unlock(B).
 T_{13} : lock-X(D); lock-X(H); unlock(D); unlock(H).

It can be shown not only that the tree protocol ensures conflict serializability, but also that this protocol ensures freedom from deadlock.

The tree protocol does not ensure recoverability and cascadelessness. To ensure recoverability and cascadelessness, the protocol can be modified to not permit release of exclusive locks until the end of the transaction. Holding exclusive locks until the end of the transaction reduces concurrency.

Here is an alternative that improves concurrency, but ensures only recoverability:

1. For each data item with an uncommitted write, we record which transaction performed the last write to the data item.
2. Whenever a transaction T_i performs a read of an uncommitted data item, we record a commit dependency of T_i on the transaction that performed the last write to the data item.
3. Transaction T_i is then not permitted to commit until the commit of all transactions on which it has a commit dependency. If any of these transactions aborts, T_i must also be aborted.

On the tree-locking protocol, no rollbacks are required (because it is deadlock free). Earlier unlocking may lead to shorter waiting times and to an increase in concurrency.

In some cases, a transaction may have to lock data items that it does not access. For example, a transaction that needs to access data items A and J in the database graph of must lock not only A and J, but also data items B, D, and H. This additional locking results in:

- Increased locking overhead
- Possibility of additional waiting time
- Potential decrease in concurrency.

Further, without prior knowledge of what data items will need to be locked, transactions will have to lock the root of the tree, and that can reduce concurrency greatly.

Deadlock handling

A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.

The only remedy to this undesirable situation is for the system to invoke some drastic action, such as rolling back some of the transactions involved in the deadlock. Rollback of a transaction may be partial: That is, a transaction may be rolled back to the point where it obtained a lock whose release resolves the deadlock.

There are two principal methods for dealing with the deadlock problem:

deadlock prevention protocol: This would ensure that the system will never enter a deadlock state.

deadlock detection and deadlock recovery scheme: we allow the system to enter a deadlock state, and then try to recover.

Both methods may result in transaction rollback. Prevention is commonly used if the probability that the system would enter a deadlock state is relatively high.

✿ Deadlock prevention

There are two approaches:

1. One approach ensures that no cyclic waits can occur by ordering the requests for locks, or requiring all locks to be acquired together.
2. The other approach is closer to deadlock recovery, and it performs transaction rollback instead of waiting for a lock whenever the wait could potentially result in a deadlock.

The simplest scheme under the first approach requires that each transaction locks all its data items before it begins execution.

Disadvantages (1): Often hard to predict what data items need to be locked before the transaction begins and data-item utilization may be very low.

Another approach is to impose an ordering of all data items and to require that a transaction lock data items only in a sequence consistent with the ordering (we have seen one such scheme in the tree protocol, which uses a partial ordering of data items). A variation of this approach is to use a total order of data items, in conjunction with two-phase locking.

The second approach for preventing deadlocks is to use preemption and transaction rollbacks. In preemption, when a transaction T_j requests a lock that transaction T_i holds, the lock granted to T_i may be preempted (adelantada) by rolling back of T_i , and granting of the lock to T_j .

To control the preemption, we assign a unique timestamp, based on a counter or on the system clock, to each transaction when it begins. The system uses these timestamps only to decide whether a transaction should wait or roll back. If a transaction is rolled back, it retains its old timestamp when restarted. Two different deadlock-prevention schemes using timestamps have been proposed:

1. The wait-die scheme is a nonpreemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a smaller timestamp than that of T_j . Otherwise, T_i is rolled back (dies).
2. The wound-wait scheme is a preemptive technique. It is a counterpart to the wait-die scheme. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a larger timestamp than that of T_j (i.e., T_i is younger than T_j). Otherwise, T_j is rolled back.

The major problem with both of these schemes is that unnecessary rollbacks may occur.

Another simple approach to deadlock prevention is based on **lock timeouts**.

In this approach, a transaction that has requested a lock **waits for at most a specified amount of time**. If the lock has **not been granted within that time**, the transaction is said to **time out**, and it **rolls itself back and restarts**. If there was in fact a **deadlock**, **one or more transactions involved in the deadlock will time out and roll back**. This scheme **falls somewhere between** deadlock prevention and deadlock detection and recovery.

The timeout scheme is particularly **easy to implement**, and it works well if **transactions are short** and **if long waits are likely to be due to deadlocks**. However, in general it is hard to decide **how long a transaction must wait** before timing out.

Starvation is also a possibility with this scheme. Hence, the timeout-based scheme has **limited applicability**.

❁ **Deadlock detection and recovery**

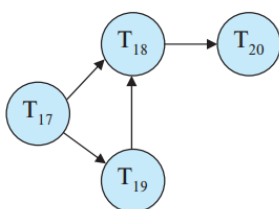
An algorithm that examines the state of the system is **invoked periodically to determine whether a deadlock has occurred**. If one has, then the **system must attempt to recover from the deadlock**. **To do so**, the system must:

- Maintain information about the **current allocation of data items to transactions**, as well as any outstanding data item requests.
- Provide an algorithm that uses this information to **determine whether the system has entered a deadlock state**.
- **Recover from the deadlock** when the detection algorithm determines that a deadlock exists.

❁ Deadlock detection

Deadlocks can be described precisely in terms of a **directed graph** called a **wait-for graph**. Graph $G = (V, E)$ where **transactions are vertices** and the ordered pairs $T_i \rightarrow T_j$ are edges.

When transaction T_i **requests** a data item currently being held by transaction T_j , then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This **edge is removed** only when transaction T_j is **no longer holding a data item needed by transaction T_i** .



A deadlock exists in the system if and only if the wait-for graph contains a cycle.

Each transaction involved in the cycle is said to be **deadlocked**. To detect deadlocks, the system **needs to maintain the wait-for graph**, and **periodically invoke an algorithm that searches for a cycle in the graph**.

Since the **graph has no cycle**, the system is not in a deadlock state.

When should we invoke the detection algorithm? The answer depends on two factors:

1. How often does a deadlock occur?

If deadlocks occur frequently, then the detection algorithm should be invoked more frequently. Data items allocated to deadlocked transactions will be unavailable to other transactions until the deadlock can be broken. In the worst case, we would invoke the detection algorithm every time a request for allocation could not be granted immediately.

2. How many transactions will be affected by the deadlock?

✿ Recovery from deadlock

The most common solution is to roll back one or more transactions to break the deadlock. Three actions need to be taken:

1. **Selection of a victim:** Given a set of deadlocked transactions, we must determine which transaction/s to roll back to break the deadlock. We should roll back those transactions that will incur the minimum cost. But many factors may determine the cost of a roll back:
 - a. How long the transaction has computed
 - b. How much longer the transaction will compute
 - c. How many data items the transaction has used.
 - d. How many more data items the transaction needs for it to complete.
 - e. How many transactions will be involved in the rollback.
2. **Rollback:** We must determine how far the selected transaction should be rolled back. The simplest solution is a **total rollback**: Abort the transaction and then restart it. **Partial rollback** requires the system to maintain additional information about the state of all the running transactions. The sequence of lock requests/grants and updates performed by the transaction needs to be recorded. The transactions must be capable of resuming execution after a partial rollback.
3. **Starvation:** It may happen that the same transaction is always picked as a victim. As a result, this transaction never completes its designated task, thus there is starvation. We must ensure that a transaction can be picked as a victim only a (small) finite number of times.

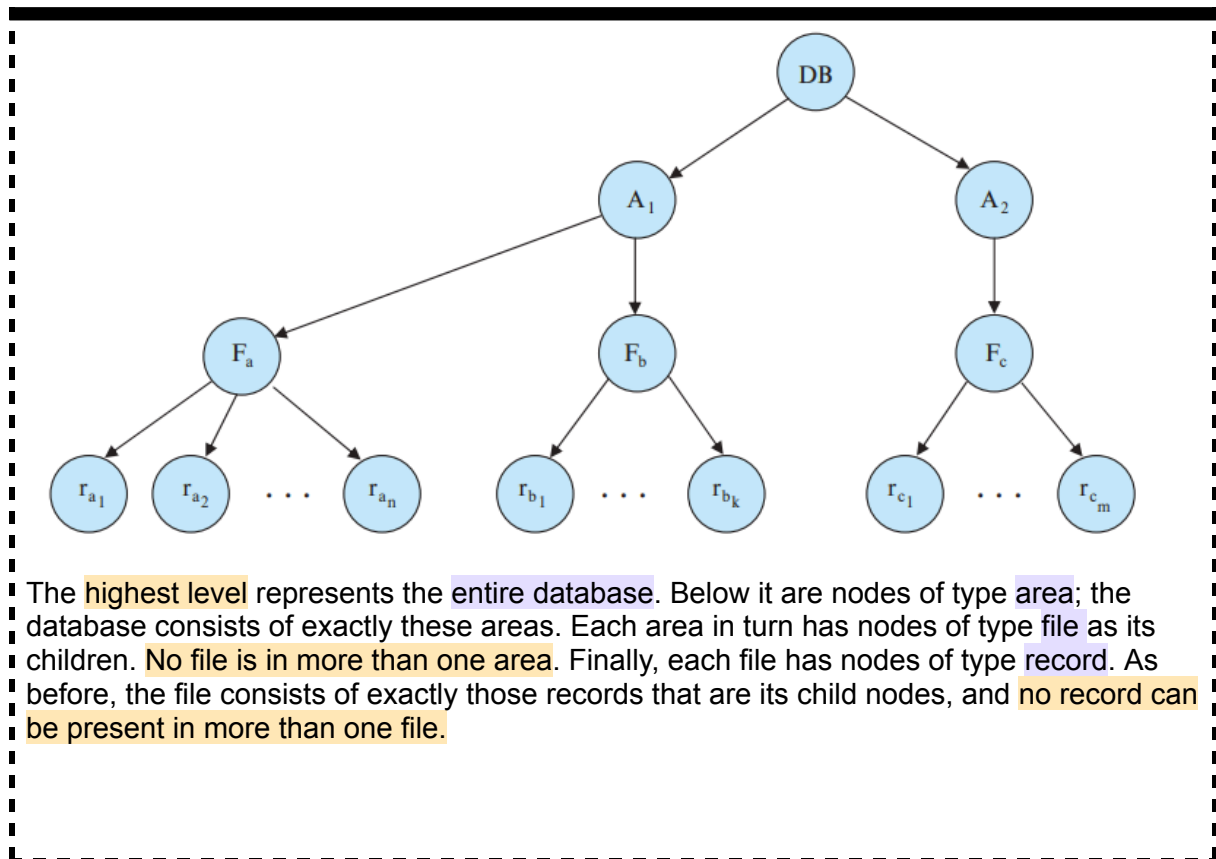
Multiple granularity

We have used each individual data item as the unit on which synchronization is performed, however, there are circumstances where it would be advantageous to group several data items, and to treat them as one individual synchronization unit.

What is needed is a mechanism to allow the system to define multiple levels of granularity. This is done by allowing data items to be of various sizes and defining a hierarchy of data

granularities, where the small granularities are nested within larger ones. Such a hierarchy can be represented graphically as a tree.

A nonleaf node of the multiple-granularity tree represents the data associated with its descendants. In the tree protocol, each node is an independent data item.



The highest level represents the entire database. Below it are nodes of type area; the database consists of exactly these areas. Each area in turn has nodes of type file as its children. No file is in more than one area. Finally, each file has nodes of type record. As before, the file consists of exactly those records that are its child nodes, and no record can be present in more than one file.

Each node in the tree can be locked individually. When a transaction locks a node, in either shared or exclusive mode, the transaction also has implicitly locked all the descendants of that node in the same lock mode.

How does the system determine if the root node can be locked?

One possibility is for it to search the entire tree. This solution, however, defeats the whole purpose of the multiple-granularity locking scheme.

A more efficient way to gain this knowledge is to introduce a new class of lock modes, called intention lock modes. Intention locks are put on all the ancestors of a node before that node is locked explicitly.

There is an intention mode associated with shared mode, and there is one with exclusive mode.

- If a node is locked in intention-shared (IS) mode, explicit locking is being done at a lower level of the tree, but with only shared-mode locks.
- If a node is locked in intention-exclusive (IX) mode, then explicit locking is being done at a lower level, with exclusive-mode or shared-mode locks.

→ If a node is locked in **shared and intention-exclusive (SIX) mode**, the **subtree rooted by that node is locked explicitly in shared mode**, and that explicit locking is being done at a lower level with exclusive-mode locks.

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

Figure 18.16 Compatibility matrix.

The **multiple-granularity locking protocol** uses these lock modes to **ensure serializability**. It requires that a transaction T_i that attempts to lock a node Q must follow these rules:

- Transaction T_i must observe the lock-compatibility function of Figure 18.16.
- Transaction T_i must lock the root of the tree first and can lock it in any mode.
- Transaction T_i can lock a node Q in S or IS mode only if T_i currently has the parent of Q locked in either IX or IS mode.
- Transaction T_i can lock a node Q in X, SIX, or IX mode only if T_i currently has the parent of Q locked in either IX or SIX mode.
- Transaction T_i can lock a node only if T_i has not previously unlocked any node (i.e., T_i is two phase).
- Transaction T_i can unlock a node Q only if T_i currently has none of the children of Q locked.

The multiple-granularity protocol requires that locks be **acquired in topdown (root-to-leaf) order**, whereas locks must be **released in bottom-up (leaf-to-root) order**. **Deadlock is possible** in the multiple-granularity protocol.

It is particularly **useful in applications that include a mix of**:

- Short transactions that access only a few data items
- Long transactions that produce reports from an entire file or set of files.

The number of locks that an SQL query may need to acquire can usually be **estimated based on the relation scan operations** performed by a query.

In case a transaction acquires a large number of tuple locks, the **lock table may become overfull**. To deal with this situation, the lock manager **may perform lock escalation**, replacing many lower level locks by a single higher level lock

Insert and delete operations and predicate reads

To examine how such transactions affect concurrency control, we introduce these additional operations: insert(Q), delete(Q)

✿ Deletion

We must decide when a delete instruction conflicts with another instruction. . Let $I_i = \text{delete}(Q)$. We consider several instructions I_j .

- $I_j = \text{read}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the read operation successfully.
- $I_j = \text{write}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the write operation successfully.
- $I_j = \text{delete}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_i will have a logical error.
- $I_j = \text{insert}(Q)$. I_i and I_j conflict. Suppose that data item Q did not exist prior to the execution of I_i and I_j . Then, if I_i comes before I_j , a logical error results for T_i .

Under the two-phase locking protocol, an exclusive lock is required on a data item before that item can be deleted.

Under the timestamp-ordering protocol, a test similar to that for a write must be performed.

✿ Insertion

Since an insert(Q) assigns a value to data item Q, an insert is treated similarly to a write for concurrency-control purposes:

- Under the two-phase locking protocol: if T_i performs an insert(Q) operation, T_i is given an exclusive lock on the newly created data item Q.
- Under the timestamp-ordering protocol: if T_i performs an insert(Q) operation, the values R-timestamp(Q) and W-timestamp(Q) are set to $TS(T_i)$.

??

✿ Predicate reads and the phantom phenomenon

Supongamos que tenemos 2 transacciones: T_{30} que quiere hacer select count(*) en la relación de instructores y T_{31} que inserta un valor que coincide con esa búsqueda.

Si un schedule adopta esas 2 transacciones, nos podemos esperar que ocurra alguna de las siguientes cosas:

- T_{30} cuenta la tupla que acaba de insertar T_{31} , pero en un serial schedule, T_{31} debe venir antes que el T_{30} .
- Si T_{30} no encuentra la tupla que se acaba de insertar, entonces en un serial schedule, T_{30} tiene que venir antes que T_{31} .

The second of these two cases is curious. T_{30} and T_{31} do not access any tuple in common, yet they conflict with each other. If concurrency control is performed at the tuple granularity, this conflict would go undetected. As a result, the system could fail to prevent a nonserializable schedule. This problem is an instance of the phantom phenomenon.

Phantom phenomena can occur not just with inserts, but also with updates.

T_{30} is an example of a transaction that reads information about what tuples are in a relation, and T_{31} is an example of a transaction that updates that information. Clearly, it is not sufficient merely to lock the tuples that are accessed; the information used to find the tuples that are accessed by the transaction must also be locked.

Do not confuse the locking of an entire relation, as in multiple-granularity locking, with the locking of the data item corresponding to the relation. By locking the data item, a transaction only prevents other transactions from updating information about what tuples are in the relation.

The major disadvantage of locking a data item corresponding to the relation, or locking an entire index, is the low degree of concurrency.

A better solution is an index-locking technique that avoids locking the whole index. Any transaction that inserts a tuple into a relation must insert information into every index maintained on the relation. We eliminate the phantom phenomenon by imposing a locking protocol for indices.

The index-locking protocol takes advantage of the availability of indices on a relation, by turning instances of the phantom phenomenon into conflicts on locks on index leaf nodes. The protocol operates as follows:

1. Every relation must have at least one index.
2. A transaction T_i can access tuples of a relation only after first finding them through one or more of the indices on the relation. A relation scan is treated as a scan through all the leaves of one of the indices.
3. A transaction T_i that performs a lookup must acquire a shared lock on all the index leaf nodes that it accesses.
4. A transaction T_i may not insert, delete, or update a tuple t_i in a relation r without updating all indices on r .
5. Locks are obtained on tuples as usual.
6. The rules of the two-phase locking protocol must be observed.
7. Ver libro en pag 889 pdf reader

Locking an index leaf node prevents any update to the node, even if the update did not actually conflict with the predicate.

Resumen: Bloqueo de Predicados (Predicate Locking)

El texto explica un enfoque alternativo para gestionar conflictos entre transacciones en bases de datos:

1. **El Concepto:** En lugar de bloquear registros individuales, el sistema adquiere bloqueos compartidos sobre **predicados** (condiciones lógicas) de una consulta. Por ejemplo, se bloquea la condición "**salario > 90000**".
2. **Funcionamiento:**
 - Cualquier intento de **insertar o eliminar** datos debe verificarse contra ese predicado. Si los datos cumplen la condición, se genera un conflicto y la operación debe esperar.
 - En las **actualizaciones**, se deben revisar tanto el valor original como el nuevo para asegurar que no interfieran con el conjunto de tuplas seleccionado por la consulta original.
3. **Objetivo:** Evitar que transacciones concurrentes modifiquen el conjunto de resultados de una consulta activa que posee el bloqueo del predicado.
4. **Uso en la práctica:** Este protocolo **no se utiliza habitualmente** debido a que su implementación es mucho más costosa en cuanto a recursos que el bloqueo de índices (index-locking) y **no ofrece beneficios adicionales significativos**.

Timestamp-based protocols

Another method for determining the serializability order is to select an ordering among transactions in advance. The most common method for doing so is to use a **timestamp-ordering scheme**.

✿ Timestamps

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by $TS(T_i)$. There are two simple methods for implementing this scheme:

1. Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

To implement this scheme, we associate with each data item Q two timestamp values:

1. **W-timestamp**(Q) denotes the largest timestamp of any transaction that executed **write**(Q) successfully.
2. **R-timestamp**(Q) denotes the largest timestamp of any transaction that executed **read**(Q) successfully

✿ The timestamp ordering protocol

The timestamp-ordering protocol ensures that any conflicting read and write operations are executed in timestamp order. This protocol operates as follows:

- Suppose that transaction T_i issues $\text{read}(Q)$.
 - If $\text{TS}(T_i) < \text{W-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten. Hence, the read operation is rejected, and T_i is rolled back.
 - If $\text{TS}(T_i) \geq \text{W-timestamp}(Q)$, then the read operation is executed, and $\text{R-timestamp}(Q)$ is set to the maximum of $\text{R-timestamp}(Q)$ and $\text{TS}(T_i)$.
- Suppose that transaction T_i issues $\text{write}(Q)$.
 - If $\text{TS}(T_i) < \text{R-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced. Hence, the system rejects the write operation and rolls T_i back.
 - If $\text{TS}(T_i) < \text{W-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, the system rejects this write operation and rolls T_i back.
 - Otherwise, the system executes the write operation and sets $\text{W-timestamp}(Q)$ to $\text{TS}(T_i)$.

If a transaction T_i is rolled back by the concurrency-control scheme as result of issuance of either a read or write operation, the system assigns it a new timestamp and restarts it.

The timestamp-ordering protocol ensures conflict serializability. This is because conflicting operations are processed in timestamp order. The protocol ensures freedom from deadlock, since no transaction ever waits. However, there is a possibility of starvation of long transactions if a sequence of conflicting short transactions causes repeated restarting of the long transaction.

The protocol can generate schedules that are not recoverable. However, it can be extended to make the schedules recoverable, in one of several ways:

- Recoverability and cascadelessness can be ensured by performing all writes together at the end of the transaction.
- Recoverability and cascadelessness can also be guaranteed by using a limited form of locking, whereby reads of uncommitted items are postponed until the transaction that updated the item commits
- Recoverability alone can be ensured by tracking uncommitted writes and allowing a transaction T_i to commit only after the commit of any transaction that wrote a value that T_i read.

If the timestamp-ordering protocol is applied only to tuples, the protocol would be vulnerable to the phantom problems that we saw previously. To avoid this problem, the timestamp-ordering protocol could be applied to all data that is read by a transaction, including relation metadata and index data.

✿ Thomas' write rule

This modification to the TSOP allows greater potential concurrency.

T_{27}	T_{28}
read(Q)	write(Q)
write(Q)	

Since T_{27} starts before T_{28} , we shall assume that $TS(T_{27}) < TS(T_{28})$. The read(Q) operation of T_{27} succeeds, as does the write(Q) operation of T_{28} . When T_{27} attempts its write(Q) operation, we find that $TS(T_{27}) < W\text{-timestamp}(Q)$, since $W\text{-timestamp}(Q) = TS(T_{28})$. Thus, the write(Q) by T_{27} is rejected and transaction T_{27} must be rolled back.

Any transaction T_i with $TS(T_i) < TS(T_{28})$ that attempts a read(Q) will be rolled back, since $TS(T_i) < W\text{-timestamp}(Q)$. Any transaction T_j with $TS(T_j) > TS(T_{28})$ must read the value of Q written by T_{28} , rather than the value that T_{27} is attempting to write.

This observation leads to a **modified version of the timestamp-ordering protocol** in which **obsolete write operations can be ignored** under certain circumstances (we change some of the rules for write operations):

2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, **this write operation can be ignored**.

Antes decía que había que hacer un rollback, ahora solo ignoramos el write. By ignoring the write, Thomas' write rule **allows schedules that are not conflict serializable** but are nevertheless correct.

Validation based protocols

The validation protocol requires that **each transaction T_i executes in two or three different phases in its lifetime**, depending on whether it is a read-only or an update transaction. The phases are, in order:

1. **Read phase:** During this phase, the system executes transaction T_i . It **reads the values of the various data items and stores them in variables local to T_i** . It performs all write operations on temporary local variables, without updates of the actual database.
2. **Validation phase:** The **validation test is applied to transaction T_i** . This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.
3. **Write phase:** If the validation test succeeds for transaction T_i , the temporary local variables that hold the results of any **write operations performed by T_i** are copied to the database. Read-only transactions omit this phase.

Each transaction must go through the phases in the order shown. To perform the validation test, we need to know when the various phases of transactions took place (now we have startTS, validationTS, finishTS).

We determine the serializability order by the timestamp-ordering technique, using the value of the timestamp ValidationTS(T_i).

The validation test for transaction T_i requires that, for all transactions T_k with $TS(T_k) < TS(T_i)$, one of the following two conditions must hold:

1. $FinishTS(T_k) < StartTS(T_i)$. Since T_k completes its execution before T_i started, the serializability order is indeed maintained.
2. The set of data items written by T_k does not intersect with the set of data items read by T_i , and T_k completes its write phase before T_i starts its validation phase ($StartTS(T_i) < FinishTS(T_k) < ValidationTS(T_i)$).

The validation scheme automatically guards against cascading rollbacks, since the actual writes take place only after the transaction issuing the write has committed. However, there is a possibility of starvation of long transactions, due to a sequence of conflicting short transactions that cause repeated restarts of the long transaction.

This validation scheme is called the optimistic concurrency-control scheme since transactions execute optimistically, assuming they will be able to finish execution and validate at the end. In contrast, locking and timestamp ordering are pessimistic in that they force a wait or a rollback whenever a conflict is detected, even though there is a chance that the schedule may be conflict serializable.

Multiversion schemes

In multiversion concurrency-control schemes, each write(Q) operation creates a new version of Q. When a transaction issues a read(Q) operation, the concurrency control manager selects one of the versions of Q to be read. The concurrency-control scheme must ensure that the version to be read is selected in a manner that ensures serializability

✿ Multiversion timestamp ordering

The database system assigns this timestamp before the transaction starts execution, as described in timestamp-based protocols. With each data item Q, a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$ is associated. Each version Q_k contains three data fields:

1. **Content**: Value of the version Q_k
2. **W-timestamp(Q_k)**: timestamp of the transaction that created the version Q_k .
3. **R-timestamp(Q_k)**: largest timestamp of any transaction that successfully read version Q_k .

The multiversion timestamp-ordering scheme presented next ensures serializability. The scheme operates as follows (Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than or equal to $TS(T_i)$):

1. If transaction T_i issues a $read(Q)$, then the value returned is the content of version Q_k .
2. If transaction T_i issues $write(Q)$, and if $TS(T_i) < R\text{-timestamp}(Q_k)$, then the system rolls back transaction T_i . On the other hand, if $TS(T_i) = W\text{-timestamp}(Q_k)$, the system overwrites the contents of Q_k ; otherwise (if $TS(T_i) > R\text{-timestamp}(Q_k)$), it creates a new version of Q .

The justification for rule 1 is clear. A transaction reads the most recent version that comes before it in time.

The valid interval of a version Q_i of Q with W -timestamp t is defined as follows: if Q_i is the latest version of Q , the interval is $[t, \infty]$; otherwise let the next version of Q have timestamp s ; then the valid interval is $[t, s)$.

Versions that are no longer needed are removed (se borra la versión mas vieja). The multiversion timestamp-ordering scheme has the desirable property that a read request never fails and is never made to wait.

The scheme, however, suffers from two undesirable properties. First, the reading of a data item also requires the updating of the R -timestamp field, resulting in two potential disk accesses, rather than one. Second, the conflicts between transactions are resolved through rollbacks, rather than through waits. This alternative may be expensive.

✿ Multiversion two-phase locking

Attempts to combine the advantages of multiversion concurrency control with the advantages of two-phase locking. This protocol differentiates between read-only transactions and update transactions.

Update transactions perform rigorous two-phase locking; that is, they hold all locks up to the end of the transaction. The timestamp in this case is not a real clock-based timestamp, but rather is a counter.

When an update transaction wants to write an item, it first gets an exclusive lock on the item and then creates a new version of the data item. The write is performed on the new version, and the timestamp of the new version is initially set to a value ∞ , a value greater than that of any possible timestamp.

Read-only transactions see the old value of ts -counter until T_i has successfully committed. As a result, read-only transactions that start after T_i commits will see the values updated by T_i , whereas those that start before T_i commits will see the value before the updates by T_i . In either case, read-only transactions never need to wait for locks.

Snapshot isolation

Snapshot isolation involves giving a transaction a “snapshot” of the database at the time when it begins its execution. It then operates on that snapshot in complete isolation from concurrent transactions. This isolation is ideal for read-only transactions since they never wait and are never aborted by the concurrency manager.

Updates performed by a transaction must be validated before the transaction is allowed to commit.

When a transaction T is allowed to commit, the transition of T to the committed state and the writing of all of the updates made by T to the database must be conceptually done as an atomic action so that any snapshot created for another transaction either includes all updates by transaction T or none of them.

✿ Multiversioning in snapshot isolation

Transactions are given two timestamps: StartTS and CommitTS (when the T requested validation).

Snapshot isolation is based on multiversioning, and each transaction that updates a data item creates a version of the data item. Versions have only one timestamp, which is the write timestamp, indicating when the version was created. The timestamp of a version created by transaction T_i is set to $\text{CommitTS}(T_i)$.

When a transaction T_i reads a data item, T_i does see the updates of all transactions that commit before it started (snapshot).

✿ Validation steps for update transactions

Deciding whether or not to allow an update transaction to commit requires some care. Potentially, two transactions running concurrently might both update the same data item.

There are two variants of snapshot isolation, both of which prevent lost updates. They are called first committer wins and first updater wins. Both approaches are based on testing the transaction against concurrent transactions.

Formally, T_j is concurrent with T_i if either $\text{StartTS}(T_j) \leq \text{StartTS}(T_i) \leq \text{CommitTS}(T_j)$, or $\text{StartTS}(T_i) \leq \text{StartTS}(T_j) \leq \text{CommitTS}(T_i)$.

First committer wins

When a transaction T_i starts validation:

- A test is made to see if any transaction that was concurrent with T_i has already written an update to the database for some data item that T_i intends to write.
- If any such data item is found, then T_i aborts.

- If no such data item is found, then T_i commits and its updates are written to the database.

First updater wins

(locking mechanism) When a transaction T_i attempts to update a data item, it requests a write lock on that data item. If the lock is not held by a concurrent transaction, the following steps are taken after the lock is acquired:

- If the item has been updated by any concurrent transaction, then T_i aborts.
- Otherwise T_i may proceed with its execution, including possibly committing.

If, however, some other concurrent transaction T_j already holds a write lock on that data item, then T_i cannot proceed, and the following rules are followed:

- T_i waits until T_j aborts or commits; if T_j commits, T_i must abort and vice versa

Locks are released when the transaction commits or aborts.

✿ **Serializability issues and solutions**

It is worth noting that integrity constraints that are enforced by the database, such as primary-key and foreign-key constraints, cannot be checked on a snapshot; this problem is handled by checking these constraints on the current state of the database, rather than on the snapshot.

Even with the above fix, there is still a serious problem with the snapshot isolation scheme as we have presented it and as it is implemented in practice: snapshot isolation does not ensure serializability!

T_i	T_j
read(A)	
read(B)	
	read(A)
	read(B)
$A=B$	
	$B=A$
write(A)	
	write(B)

Figure 18.20 Nonserializable schedule under snapshot isolation.

However, the execution is not serializable, since it results in swapping of the values of A and B, whereas any serializable schedule would set both A and B to the same value: either the initial value of A or the initial value of B, depending on the order of T_i and T_j .

????????????????????

This situation, where each of a pair of transactions has read a data item that is written by the other, but the set of data items written by the two transactions do not have any data item in common, is referred to as write skew.

Ver ejemplo en pag 905 pdf reader

The problems listed above seem to indicate that the snapshot isolation technique is vulnerable to many serializability problems and should never be used. However, serializability problems are relatively rare for two reasons:

1. The fact that the database must check integrity constraints at the time of commit, and not on a snapshot, helps avoid inconsistencies in many situations.
2. In many applications that are vulnerable to serializability problems, such as skew writes on some data items, the transactions conflict on other data items, ensuring such transactions cannot execute concurrently; ??????

Nonserializable may nevertheless occur with snapshot isolation. Several solutions:

→ Using serializable snapshot isolation

Suppose we track all conflicts between transactions and construct a graph with that information; one way to ensure serializability is to look for cycles in the transaction precedence graph and roll back transactions if a cycle is found.

→ Allowing different transactions to run under different isolation levels

Running long read-only transactions under the snapshot isolation level while running update transactions under the serializable isolation level ensures that the read-only transaction does not block updaters, while also ensuring that some anomalies cannot occur.

→ Provide a way for SQL programmers to create artificial conflicts

Adding the for update clause causes the system to treat data that are read as if they had been updated for purposes of concurrency control.

Weak levels of consistency in practice

We first briefly outline some older terminology relating to consistency levels weaker than serializability and relate it to the SQL standard levels. We then discuss the issue of concurrency control for transactions that involve user interaction.

✿ **Degree-two consistency** (ver ejemplo 910 pdf reader / 881 libro)

The purpose of degree-two consistency is to avoid cascading aborts without necessarily ensuring serializability. The locking protocol for degree-two consistency uses the same two lock modes that we used for the two-phase locking protocol, but two-phase behavior is not required.

In contrast to the situation in two-phase locking, S-locks may be released at any time, and locks may be acquired at any time. Exclusive locks, however, cannot be released until the transaction either commits or aborts.

- Serializability is not ensured by this protocol.
- A transaction may read the same data item twice and obtain different results.
- Reads are not repeatable, but since exclusive locks are held until transaction commit, no transaction can read an uncommitted value.
- A transaction that is scanning an index may potentially see two versions of a record that was updated while the scan was in progress and may also potentially see neither version

✿ **Cursor stability**

Form of degree-two consistency designed for programs that iterate over tuples of a relation by using cursors. Instead of locking the entire relation, cursor stability ensures that:

- The tuple that is currently being processed by the iteration is locked in shared mode. Once the tuple is processed, the lock on the tuple can be released.

→ Any modified tuples are locked in exclusive mode until the transaction commits.

These rules ensure that degree-two consistency is obtained. But locking is not done in a two-phase manner, and serializability is not guaranteed. Cursor stability is used in practice on heavily accessed relations as a means of increasing concurrency and improving system performance.

✿ Concurrency control across user interactions

Concurrency-control protocols usually consider transactions that do not involve user interaction.

If two-phase locking is used, the entire set of seats on a flight would be locked in shared mode until the user has completed the seat selection, and no other transaction would be able to update the seat allocation information in this period.

Timestamp protocols or validation could be used instead, which avoid the problem of locking, but both these protocols would abort the transaction for a user A if any other user B has updated the seat allocation information.

Snapshot isolation is a good option in this situation, since it would not abort the transaction of user A as long as B did not select the same seat as A. But this method requires the database to remember information about updates performed by a transaction even after it has committed, which can be problematic for long-duration transactions.

Another option is to split a transaction that involves user interaction into two or more transactions, such that no transaction spans a user interaction. The above idea has been generalized in an alternative concurrency control scheme, which uses version numbers stored in tuples to avoid lost updates.

When a transaction reads (for the first time) a tuple that it intends to update, it remembers the version number of that tuple. Updates are done locally and copied to the database as part of commit processing. El update se comitea si la versión del dato es la misma que la guardada, caso contrario se hace un rollback.

Unlike snapshot isolation, the reads performed by a transaction may not correspond to a snapshot of the database; and unlike the validation-based protocol, reads performed by the transaction are not validated.

We refer to the above scheme as optimistic concurrency control without read validation. This provides a weak level of serializability, and it does not ensure serializability.

The validation and update steps performed as part of commit processing are then executed as a single transaction in the database, using the concurrency-control scheme of the database to ensure atomicity for commit processing.

PEDIR EXPLICACION A LA IA

Advanced topics in concurrency control

Instead of using two-phase locking, special-purpose concurrency control techniques can be used for index structures, resulting in improved concurrency.

Concurrency control actions often become bottlenecks in main-memory databases. It is possible to consider operations, such as increment of a counter, as basic operations, and perform concurrency control on the basis of conflicts between operations.

✿ Online index creation

When we are dealing with large volumes of data (ranging in the terabytes), operations such as creating an index can take a long time.

The index contents must be consistent with the contents of the relation, and all further updates to the relation must maintain the index. One way to ensure this is to block all updates to the relation while the index is created. After the index is created, and the relation metadata are updated, locks can be released.

The above approach would make the system unavailable for updates to the relation for a very long time. Instead, most database systems support online index creation, which allows relation updates to occur even as the index is being created.

1. Index creation gets a snapshot of the relation and uses it to create the index; meanwhile, the system logs all updates to the relation that happen after the snapshot is created.
2. When the index on the snapshot data is complete, the log of updates to the relation is used to update the index. But while the index update is being carried out, further updates may be happening on the relation.
3. The index update then obtains a shared lock on the relation to prevent further updates and applies all remaining updates to the index. The relation metadata are then updated to indicate the existence of the new index.

Creation of materialized views that are maintained immediately, can also benefit from online construction techniques that are similar to online index construction. (Se ejecuta el query con un snapshot, se agregan los logs, se crea un shared lock y se actualiza la vista)

Schema changes such as adding or deleting attributes or constraints can also have a significant impact if relations are locked while the schema change is implemented on all tuples. (Podríamos usar un número de versión para cada tupla)(Para agregar restricciones, tenemos que verificar que los datos existentes las cumplan)

✿ Concurrency in index structures

It is possible to treat access to index structures like any other database structure and to apply the concurrency-control techniques discussed earlier. However, since indices are

accessed frequently, they would become a point of great lock contention, leading to a low degree of concurrency.

Luckily, indices do not have to be treated like other database structures; it is desirable to release index locks early, in a non-two-phase manner, to maximize concurrency.

Operation serializability for index operations is defined as follows: A concurrent execution of index operations on an index is said to be serializable if there is a serialization order of the operations that is consistent with the results that each index operation in the concurrent execution sees.

We outline two techniques for managing concurrent access to B+-trees as well as an index-concurrency control technique to prevent the phantom phenomenon.

1. **Crabbing protocol**: When searching for a key value, the crabbing protocol first locks the root node in shared mode. Then it acquires a shared lock on the child and releases the lock on the parent node. It repeats this process until it reaches a leaf. When inserting or deleting: Para hacer cambios requiere usar locks exclusivos (si hay que hacer un split en el parent, vuelve hacia arriba y luego sigue su camino). The system can easily handle such deadlocks by restarting the search operation from the root, after releasing the locks held by the operation.

Locks that are held for a short duration, instead of being held in a two-phase manner, are often referred to as latches. Latches are used internally in databases to achieve mutual exclusion on shared data structures.

2. **B-link-tree locking protocol**: (B-link trees require that every node maintain a pointer to its right sibling) Holds locks on only one internal node at a time. The protocol releases the lock on the current internal node before requesting a lock on a child or parent node. The protocol ensures operation serializability while avoiding deadlocks between operations and increasing concurrency compared to the crabbing protocol. }

Instead of locking an entire index leaf node, some index concurrency-control schemes use key-value locking on individual key values, allowing other key values to be inserted or deleted from the same leaf.

To prevent the phantom phenomenon, the next-key locking technique is used: every index lookup must lock not only the keys found within the range (or the single key, in case of a point lookup) but also the next-key value.

✿ Concurrency control in main memory databases

When disk I/O is the bottleneck cost in a system, there is little benefit from optimizing other smaller costs, such as the cost of concurrency control.

Thus, it may be acceptable to perform locking at a coarse granularity: for example, the entire index could be locked using a single latch, the operation performed, and the latch released.

The reduced overhead of locking has been found to make up for the slightly reduced concurrency, and to improve overall performance.

There is another way to improve performance with in-memory indices, using atomic instructions to carry out index updates without acquiring any latches at all. Data structures implementations that support concurrent operations without requiring latches are called latch-free data structure implementations.

It is best to use latch-free data structure implementations (more often referred to as lock-free data structure implementations) that are provided by standard libraries.

✿ Long-Duration Transactions

Serious problems arise when this concept is applied to database systems that involve human interaction. Such transactions have these key properties:

Long duration: Once a human interacts with an active transaction, that transaction becomes a long-duration transaction from the perspective of the computer, since human response time is slow relative to computer speed.

Exposure of uncommitted data: Data generated and displayed to a user by a long duration transaction are uncommitted, since the transaction may abort.

Subtasks: An interactive transaction may consist of a set of subtasks initiated by the user.

Recoverability: It is unacceptable to abort a long-duration interactive transaction because of a system crash. The active transaction must be recovered to a state that existed shortly before the crash so that relatively little human work is lost.

Performance: Good performance in an interactive transaction system is defined as fast response time. Systems with high throughput make efficient use of system resources.

✿ Concurrency control with operations

Two important points about the concept of correctness without serializability:

1. Correctness depends on the specific consistency constraints for the database.
2. Correctness depends on the properties of operations performed by each transaction.

Concurrency can be increased by treating some operations besides read and write as fundamental low-level operations and to extend concurrency control to deal with them.

The increment operation does not lock the variable in a two-phase manner; however, individual operations should be executed serially on the variable. Thus, if two increment operations are initiated concurrently on the same variable, one must finish before the other is allowed to start.

Compensating operation: If one of the transactions rolls back, the increment(v, n) operation must be rolled back by executing an operation increment($v, -n$), which adds a negative of the original value

We can define a **locking protocol** to handle the preceding situation by defining an increment **lock**. The increment lock is compatible with itself but is not compatible with shared and exclusive locks.

Una banda de ejemplos que no quiero leer...

✿ **Real time transaction systems**

In certain applications, the constraints include **deadlines** by which a task must be completed. Deadlines are characterized as follows:

Hard deadline: Serious problems may occur if a task is not completed by its deadline.

Firm deadline: The task has zero value if it is completed after the deadline.

Soft deadlines: The task has **diminishing value** if it is completed after the deadline, with the value approaching zero as the degree of lateness increases.

Systems with deadlines are called **real-time systems**.

Due to the unpredictable nature of delays when reading data from disk, **main memory databases** are often used if **real-time constraints have to be met**. However, even if data are resident in main memory, variances in execution time arise from lock waits, transaction aborts, and so on.

Capitulo 19 - Database System Concepts

Recovery System

A computer system, like any other device, is subject to failure from a variety of causes. The database system must take actions in advance to ensure that the atomicity and durability properties of transactions are preserved. An integral part of a database system is a recovery scheme that can restore the database to the consistent state that existed before the failure.

The recovery scheme must also support high availability (the database should be usable for a very high percentage of time). To support high availability in the face of machine failure, the recovery scheme must support the ability to keep a backup copy of the database synchronized with the current contents of the primary copy of the database.

Failure classification

We'll consider only the following types of failure:

- **Transaction failure:** There are two types of errors that may cause a transaction to fail:
 - ◆ **Logical error:** The transaction can no longer continue with its normal execution because of some internal condition, such as bad input, data not found, overflow, or resource limit exceeded.
 - ◆ **System error:** The system has entered an undesirable state (e.g., deadlock), as a result of which a transaction cannot continue with its normal execution. The transaction, however, can be re-executed at a later time.
- **System crash:** There is a hardware malfunction, or a bug in the database software or the operating system, that causes the loss of the content of volatile storage and brings transaction processing to a halt. The content of non-volatile storage remains intact and is not corrupted. The assumption that hardware errors and bugs in the software bring the system to a halt, but do not corrupt the non-volatile storage contents, is known as the fail-stop assumption. Well-designed systems have numerous internal checks, at the hardware and the software level, that bring the system to a halt when there is an error. Hence, the fail-stop assumption is a reasonable one.
- **Disk failure:** A disk block loses its content as a result of either a head crash or failure during a data-transfer operation. Copies of the data on other disks, or archival backups on tertiary media, such as DVD or tapes, are used to recover from the failure.

To determine how the system should recover from failures, we need to:

1. Identify modes of those devices used for storing data.
2. Consider how these failure modes affect the contents of the database.
3. Propose algorithms to ensure database consistency and transaction atomicity despite failures.

The algorithms consist of the actions taken during normal transaction processing (to ensure that enough information exists to allow recovery from failures) and actions taken after a failure to recover the db.

Storage

We identified three categories of storage:

1. Volatile storage
2. Non-Volatile storage
3. Stable storage

✿ Stable storage implementation

To implement stable storage, we need to replicate the needed information in several non-volatile storage media (usually disk) with independent failure modes and to update the information in a controlled manner to ensure that failure during data transfer does not damage the needed information.

RAID systems guarantee that the failure of a single disk will not result in loss of data. The simplest and fastest form of RAID is the mirrored disk, which keeps two copies of each block on separate disks. Other forms of RAID offer lower costs, but at the expense of lower performance.

Block transfer between memory and disk storage can result in:

- **Successful completion:** The transferred information arrived safely at its destination
- **Partial failure:** A failure occurred in the midst of transfer, and the destination block has incorrect information.
- **Total failure:** The failure occurred sufficiently early during the transfer that the destination block remains intact.

We require that, if a data-transfer failure occurs, the system detects it and invokes a recovery procedure to restore the block to a consistent state. To do so, the system must maintain two physical blocks for each logical database block. An output operation is executed as follows:

1. Write the information onto the first physical block.
2. When the first write completes successfully, write the same information onto the second physical block.
3. The output is completed only after the second write completes successfully.

If the system fails while blocks are being written, it is possible that the two copies of a block could be inconsistent with each other. During recovery, for each block, the system would need to examine two copies of the blocks.

The requirement of comparing every corresponding pair of blocks during recovery is expensive to meet. We can reduce the cost greatly by keeping track of block writes that are

in progress, using a small amount of non-volatile RAM. On recovery, only blocks for which writes were in progress need to be compared.

✿ Data access

The database is partitioned into fixed-length storage units called blocks. Blocks are the units of data transfer to and from disk and may contain several data items. We shall assume that no data item spans two or more blocks. This assumption is realistic for most data-processing applications, such as a bank or a university

The blocks residing on the disk are referred to as physical blocks; the blocks residing temporarily in main memory are referred to as buffer blocks. The area of memory where blocks reside temporarily is called the disk buffer.

Block movements between disk and main memory are initiated through input (physical block to memory) and output (buffer block to the disk and replaces physical block)

Each transaction has a private work area; the system creates this work area when the transaction is initiated, and the system removes the area when the transaction either commits or aborts. Transaction T_i interacts with the database system by transferring data to and from its work area to the system buffer.

1. $\text{read}(X)$ assigns the value of data item X to the local variable x_i . It executes this operation as follows:
 - a. If block B_X on which X resides is not in main memory, it issues $\text{input}(B_X)$.
 - b. It assigns to x_i the value of X from the buffer block.
2. $\text{write}(X)$ assigns the value of local variable x_i to data item X in the buffer block. It executes this operation as follows:
 - a. If block B_X on which X resides is not in main memory, it issues $\text{input}(B_X)$.
 - b. It assigns the value of x_i to X in buffer B_X .

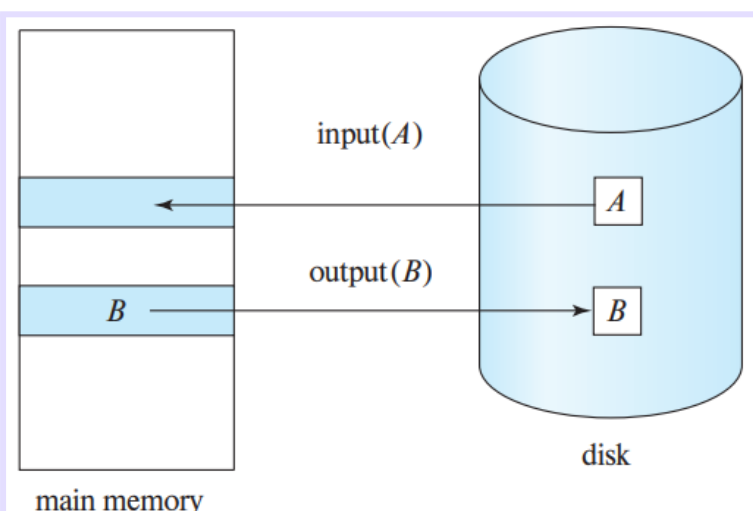


Figure 19.1 Block storage operations.

If the system crashes after the $\text{write}(X)$ operation was executed but before $\text{output}(B_X)$ was executed, the new value of X is never written to disk and, thus, is lost.

Recovery and atomicity

Our goal is to perform either all or no database modifications made by T_i . However, if T_i performed multiple database modifications, several output operations may be required, and a failure may occur after some of these modifications have been made, but before all of them are made.

To achieve our goal of atomicity, we must first output to stable storage information describing the modifications, without modifying the database itself. As we shall see, this information can help us ensure that all modifications performed by committed transactions are reflected in the database. We also need to store information about the old value of any item updated by a modification in case the transaction performing the modification fails (aborts).

The most commonly used technique for recovery is based on log records, and we study log-based recovery in detail in this chapter.

✿ Log records

The log is a sequence of log records, recording all the update activities in the database. There are several types of log records. An update log record describes a single database write. It has these fields:

- Transaction identifier: which is the unique identifier of the transaction that performed the write operation.
- Data-item identifier: which is the unique identifier of the data item written. Typically, it is the location on disk of the data item, consisting of the block identifier of the block on which the data item resides and an offset within the block.
- Old value: which is the value of the data item prior to the write.
- New value: which is the value that the data item will have after the write.

Among the types of log records are:

- <T_i start> Transaction T_i has started.
- <T_i commit> Transaction T_i has committed.
- <T_i abort> Transaction T_i has aborted.

Once a log record exists, we can output the modification to the database if that is desirable. For log records to be useful for recovery from system and disk failures, the log must reside in stable storage.

✿ Database modification

A transaction creates a log record prior to modifying the database.

In order for us to understand the role of these log records in recovery, we need to consider the steps a transaction takes in modifying a data item:

1. The transaction performs some computations in its own private part of main memory.
2. The transaction modifies the data block in the disk buffer in main memory holding the data item.

3. The database system executes the output operation that writes the data block to disk.

We say a transaction modifies the database if it performs an update on a disk buffer, or on the disk itself. If a transaction does not modify the database until it has committed, it is said to use the **deferred-modification** technique. If database modifications occur while the transaction is still active, the transaction is said to use the **immediate-modification** technique.

The recovery algorithms we describe in this chapter support immediate modification. As described, they work correctly even with deferred modification, but they can be optimized to reduce overhead when used with deferred modification;

A recovery algorithm must take into account a variety of factors, including:

- The possibility that a transaction may have committed although some of its database modifications exist only in the disk buffer in main memory and not in the database on disk.
- The possibility that a transaction may have modified the database while in the active state and, as a result of a subsequent failure, may need to abort.

Because all database modifications must be preceded by the creation of a log record, the system has available both the old value prior to the modification of the data item and the new value that is to be written for the data item. This allows the system to perform undo and redo operations as appropriate.

✿ Concurrency control and recovery

Recovery algorithms usually require that if a data item has been modified by a transaction, no other transaction can modify the data item until the first transaction commits or aborts.

This requirement can be ensured by acquiring an exclusive lock on any updated data item and holding the lock until the transaction commits; in other words, by using **strict two-phase locking**. When either **snapshot isolation or validation** is used for concurrency control, database updates of a transaction are (conceptually) deferred until the transaction is partially committed.

✿ Transaction commit

We say that a transaction has committed when its commit log record has been output to stable storage; If a system crash occurs before a log record <Ti commit> is output to stable storage, transaction Ti will be rolled back.

✿ Using the log to redo and undo transactions

Consider our simplified banking system. Let T_0 be a transaction that transfers \$50 from account A to account B :

```
 $T_0$ : read( $A$ );
       $A := A - 50$ ;
      write( $A$ );
      read( $B$ );
       $B := B + 50$ ;
      write( $B$ ).
```

Let T_1 be a transaction that withdraws \$100 from account C :

```
 $T_1$ : read( $C$ );
       $C := C - 100$ ;
      write( $C$ ).
```

Log	Database
$\langle T_0 \text{ start} \rangle$	
$\langle T_0, A, 1000, 950 \rangle$	
$\langle T_0, B, 2000, 2050 \rangle$	
	$A = 950$
	$B = 2050$
$\langle T_0 \text{ commit} \rangle$	
$\langle T_1 \text{ start} \rangle$	
$\langle T_1, C, 700, 600 \rangle$	
	$C = 600$
$\langle T_1 \text{ commit} \rangle$	

Using the log, the system can handle any failure that does not result in the loss of information in non-volatile storage. Two recovery procedures:

- **redo(T_i)**: The procedure sets the value of all data items updated by transaction T_i to the new values.
- **undo(T_i)**: The procedure restores the value of all data items updated by transaction T_i to the old values. The undo operation not only restores the data items to their old value, but also writes log records to record the updates performed as part of the undo process. These log records are special redo-only log records. When the undo operation for transaction T_i completes, it writes a log record $\langle T_i \text{ abort} \rangle$, indicating that the undo has completed.

After a system crash has occurred, the system consults the log to determine which transactions need to be redone and which need to be undone so as to ensure atomicity.

Undone if log contains the record T_i start but does not contain either T_i commit or abort

Redo if log contains T_i start and commit or abort.

(ejemplo del banco: pag 919 / 948 pdf reader)

✿ Checkpoints

When a system crash occurs, we must consult the log to determine those transactions that need to be redone and those that need to be undone. In principle, we need to search the entire log to determine this information.

To reduce these types of overhead (time-consuming search process and a lot of redones), we introduce checkpoints.

A checkpoint is performed as follows:

1. Output onto stable storage all log records currently residing in main memory.
2. Output to the disk all modified buffer blocks.
3. Output onto stable storage a log record of the form <checkpoint L>, where L is a list of transactions active at the time of the checkpoint.

The redo or undo operations need to be applied only to transactions in L, and to all transactions that started execution after the record was written to the log.

The requirement that transactions must not perform any updates to buffer blocks or to the log during checkpointing can be bothersome, since transaction processing has to halt while a checkpoint is in progress. A fuzzy checkpoint is a checkpoint where transactions are allowed to perform updates even while buffer blocks are being written out.

Recovery algorithm

We are now ready to present the full recovery algorithm using log records for recovery from transaction failure and a combination of the most recent checkpoint and log records to recover from a system crash.

The recovery algorithm described in this section requires that a data item that has been updated by an uncommitted transaction cannot be modified by any other transaction, until the first transaction has either committed or aborted. (strict two-phase locking)

✿ Recovery after a system crash

1. In the redo phase, the system replays updates of all transactions by scanning the log forward from the last checkpoint. The log records that are replayed include log records for transactions that were rolled back before system crash, and those that had not committed when the system crash occurred. This phase also determines all transactions that were incomplete at the time of the crash, and must therefore be rolled back.

At the end of the redo phase, undo-list contains the list of all transactions that are incomplete, that is, they neither committed nor completed rollback before the crash.

2. In the undo phase, the system rolls back all transactions in the undo-list. It performs rollback by scanning the log backward from the end.

After the undo phase of recovery terminates, normal transaction processing can resume.

REVISAR LAS 3 ETAPAS DENTRO DE CADA ETAPA (pag 923 / 953 pdf reader) Y EJEMPLOS EN LA SIG PAGINA

The actions are repeated in the same order in which they were originally carried out; hence, this process is called **repeating history**. Although it may appear wasteful, repeating history even for failed transactions simplifies recovery schemes.

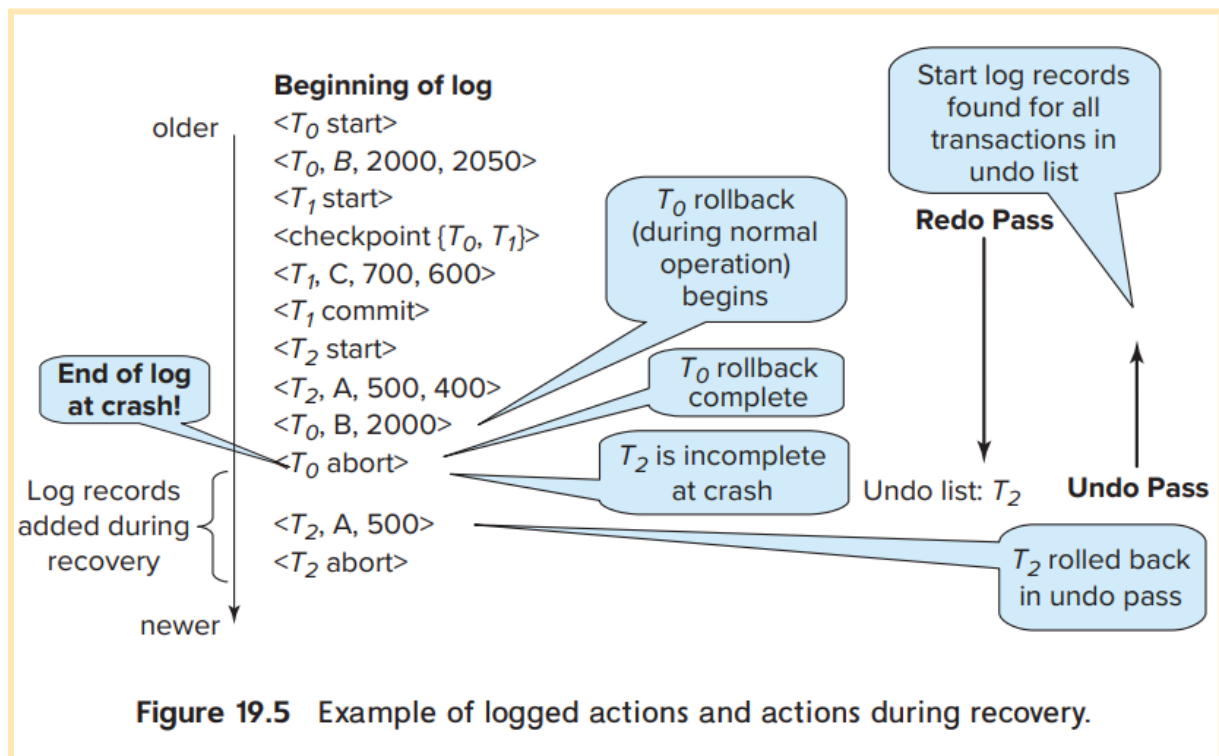


Figure 19.5 Example of logged actions and actions during recovery.

✿ Optimizing commit processing

Committing a transaction requires that its log records have been forced to disk. If a separate log flush is done for each transaction, each commit incurs a significant log write overhead.

The rate of transaction commit can be increased using the **group-commit** technique; Instead of attempting to force the log as soon as a transaction completes, the system waits until several transactions have completed, or a certain period of time has passed since a transaction completed execution. It then commits the group of transactions that are waiting, together. The system can ensure that blocks are full when they are written to stable storage without making transactions wait excessively.

Although group commit reduces the overhead imposed by logging, it results in a slight delay in commit of transactions that perform updates. When the rate of commits is low, the delay may not be worth the benefit, but with high rates of transaction commit, the overall delay in commit is actually reduced by using group commit.

Buffer management

✿ Log-record buffering

So far, we have assumed that every log record is output to stable storage at the time it is created. This assumption imposes a high overhead on system execution.

The cost of outputting a block to stable storage is sufficiently high that it is desirable to output multiple log records at once. To do so, we write log records to a log buffer in main memory, where they stay temporarily until they are output to stable storage. Multiple log records can be gathered in the log buffer and output to stable storage in a single output operation. The order of log records in the stable storage must be exactly the same as the order in which they were written to the log buffer.

Con esta táctica los logs solo permanecen en memoria hasta que son mandados a disco, por lo tanto, si el sistema falla, los logs se pierden.

Additional requirements on the recovery techniques to ensure transaction atomicity:

- Transaction T_i enters the commit state after the $\langle T_i \text{ commit} \rangle$ log record has been output to stable storage.
- Before the $\langle T_i \text{ commit} \rangle$ log record can be output to stable storage, all log records pertaining to transaction T_i must have been output to stable storage.
- Before a block of data in main memory can be output to the database (in non-volatile storage), all log records pertaining to data in that block must have been output to stable storage.

This rule is called the write-ahead logging (WAL) rule. There is no problem resulting from the output of log records earlier than necessary. Writing the buffered log to disk is sometimes referred to as a log force.

✿ Database buffering

Since main memory is typically much smaller than the entire database, it may be necessary to overwrite a block B1 in main memory when another block B2 needs to be brought into memory. If B1 has been modified, B1 must be output prior to the input of B2.

One might expect that transactions would force-output all modified blocks to disk when they commit. Such a policy is called the force policy. The alternative, the no-force policy, allows a transaction to commit even if it has modified some blocks that have not yet been written back to disk.

Similarly, one might expect that blocks modified by a transaction that is still active should not be written to disk. This policy is called the no-steal policy. The alternative, the steal policy, allows the system to write modified blocks to disk even if the transactions that made those modifications have not all committed.

When a block B1 is to be output to disk, all log records pertaining to data in B1 must be output to stable storage before B1 is output. It is important that no writes to the block B1 be in progress while the block is being output, since such a write could violate the write-ahead logging rule. We can ensure that there are no writes in progress by using a special means of locking:

- The following sequence of actions is taken when a block is to be output:
 - Obtain an exclusive lock on the block, to ensure that no transaction is performing a write on the block.
 - Output log records to stable storage until all log records pertaining to block B_1 have been output.
 - Output block B_1 to disk.
 - Release the lock once the block output has completed.

Database systems usually have a process that continually cycles through the buffer blocks, outputting modified buffer blocks back to disk. The above locking protocol must of course be followed when the blocks are output. As a result of continuous output of modified blocks, the number of dirty blocks in the buffer, that is, blocks that have been modified in the buffer but have not been subsequently output, is minimized.

✿ Operating system role in buffer management

We can manage the database buffer by using one of two approaches:

1. The database system reserves part of main memory to serve as a buffer that it, rather than the operating system, manages. The database system manages datablock transfer in accordance with the requirements in section 19.5.2. This approach has the drawback of limiting flexibility in the use of main memory..
2. The database system implements its buffer within the virtual memory provided by the operating system. Since the operating system knows about the memory requirements of all processes in the system, ideally it should be in charge of deciding what buffer blocks must be force-output to disk, and when.

Unfortunately, almost all current-generation operating systems retain complete control of virtual memory. The operating system reserves space on disk for storing virtual-memory pages that are not currently in main memory; this space is called swap space.

✿ Fuzzy checkpointing

The checkpointing technique described in Section 19.3.6 requires that all updates to the database be temporarily suspended while the checkpoint is in progress. If the number of pages in the buffer is large, a checkpoint may take a long time to finish, which can result in an unacceptable interruption in processing of transactions.

To avoid such interruptions, the checkpointing technique can be modified to permit updates to start once the checkpoint record has been written, but before the modified buffer blocks are written to disk. The checkpoint thus generated is a fuzzy checkpoint.

Since pages are output to disk only after the checkpoint record has been written, it is possible that the system could crash before all pages are written.

El problema: Los checkpoints (puntos de control) en el disco pueden quedar incompletos si el sistema falla durante su escritura.

La solución: La ubicación del **último checkpoint completo** se guarda en una posición fija del disco llamada **last-checkpoint**.

El proceso:

1. Antes de escribir el checkpoint, el sistema crea una lista de todos los bloques de memoria (buffers) modificados.
2. El registro del checkpoint en el disco **no** se actualiza inmediatamente.
3. La información de **last-checkpoint** se actualiza **solo después** de que todos los bloques modificados de la lista se hayan guardado por completo en el disco.

Failure with loss of non-volatile storage

The basic scheme is to dump the entire contents of the database to stable storage periodically—say, once per day. If a failure occurs that results in the loss of physical database blocks, the system uses the most recent dump in restoring the database to a previous consistent state.

One approach to database dumping requires that no transaction may be active during the dump procedure, and it uses a procedure similar to checkpointing:

1. Output all log records currently residing in main memory onto stable storage.
2. Output all buffer blocks onto the disk.
3. Copy the contents of the database to stable storage.
4. Output a log record onto the stable storage.

A dump of the database contents is also referred to as an **archival dump**, since we can archive the dumps and use them later to examine old states of the database.

Such dumps are useful when migrating data to a different instance of the database, or to a different version of the database software, since the physical locations and layout may be different in the other database instance or database software version.

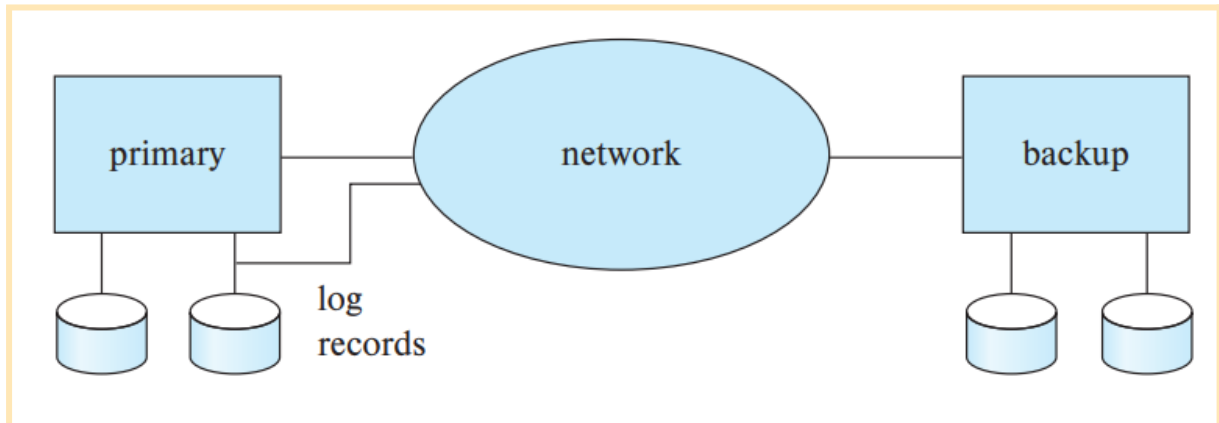
Fuzzy dump schemes have been developed that allow transactions to be active while the dump is in progress.

High availability using remote backup systems

Today's applications need transaction-processing systems that can function in spite of system failures or environmental disasters. Such systems must provide **high availability**; that is, the time for which the system is unusable must be extremely short.

We can achieve high availability by performing transaction processing at one site, called the **primary site**, and having a **remote backup** site where all the data from the primary site are replicated. The **remote backup site** is sometimes also called the **secondary site**. The remote

site must be kept synchronized with the primary site as updates are performed at the primary. We achieve synchronization by sending all log records from the primary site to the remote backup site.



El backup debe estar en un lugar físico diferente que la unidad de procesamiento para tener datos de respaldo frente a inundaciones u otros factores.

When the primary site fails, the remote backup site takes over processing. First, however, it performs recovery, using its (perhaps outdated) copy of the data from the primary and the log records received from the primary. In effect, the remote backup site is performing recovery actions that would have been performed at the primary site when the latter recovered.

Standard recovery algorithms, with minor modifications, can be used for recovery at the remote backup site. Several issues must be addressed in designing a remote backup system:

- **Detection of failure:** It is important for the remote backup system to detect when the primary has failed. Failure of communication lines can fool the remote backup into believing that the primary has failed. To avoid this problem, we maintain several communication links with independent modes of failure between the primary and the remote backup.
- **Transfer of control:** When the primary fails, the backup site takes over processing and becomes the new primary. The decision to transfer control can be done manually or can be automated using software provided by database system vendors. Queries must now be sent to the new primary. When the original primary site recovers, it can either play the role of remote backup or it can take over the role of primary site again. In either case, the old primary must receive a log of updates carried out by the backup site while the old primary was down.
- **Time to recover:** If the log at the remote backup grows large, recovery will take a long time. A **hot-spare configuration** can make takeover by the backup site almost instantaneous. In this configuration, the remote backup site continually processes redo log records as they arrive, applying the updates locally.

→ **Time to commit:** To ensure that the updates of a committed transaction are durable, a transaction must not be declared committed until its log records have reached the backup site. This delay can result in a longer wait to commit a transaction, and some systems therefore permit lower degrees of durability. The degrees of durability can be classified as follows:

- ◆ **One-safe:** A transaction commits as soon as its commit log record is written to stable storage at the primary site.
- ◆ **Two-very-safe:** A transaction commits as soon as its commit log record is written to stable storage at the primary and the backup site
- ◆ **Two-safe:** This scheme is the same as two-very-safe if both primary and backup sites are active.

Some systems instead use a **high availability proxy** machine. Application clients do not connect to the database directly, but connect through the proxy. The proxy transparently routes application requests to the current primary.

Remote backups are kept synchronized with the primary by ensuring that all block writes performed at the primary are also replicated at the backup

If the database forces a block to disk and then performs some other update actions at the primary, the block must also be forced to disk at the backup, before subsequent updates are performed at the backup system.

Capítulo 1 - Data Mining

Introduction

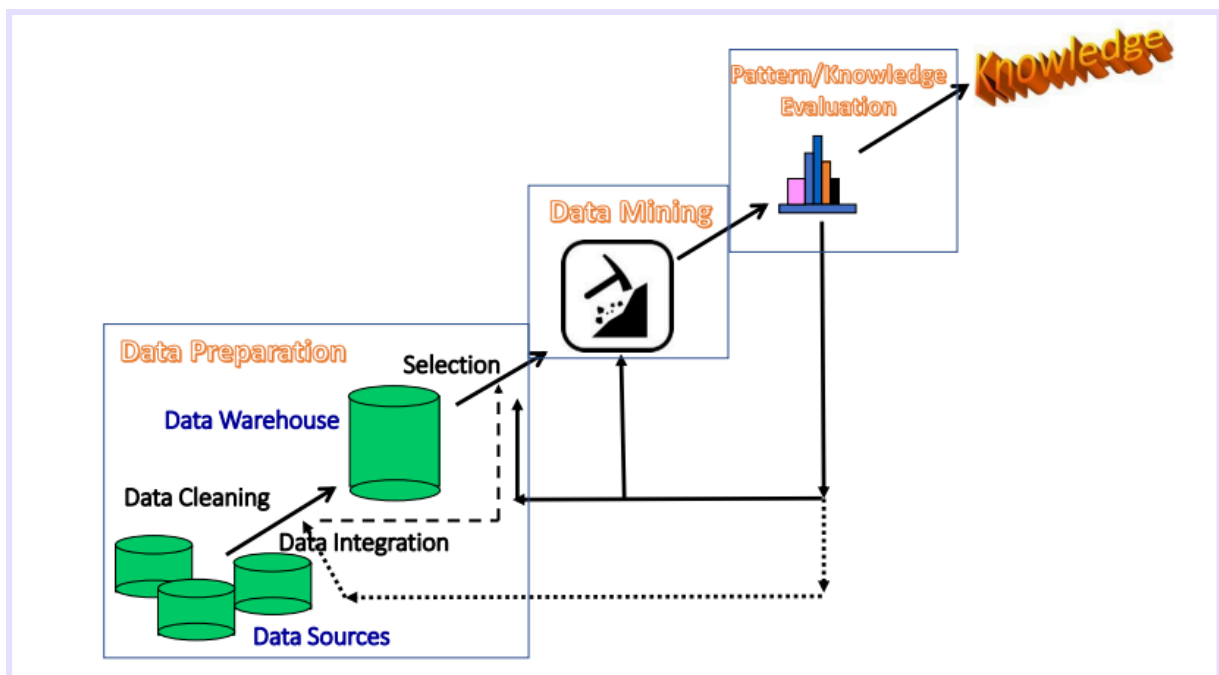
What is data mining?

Data mining is the process of discovering interesting patterns, models, and other kinds of knowledge in large data sets.

An essential step in knowledge discovery

Many people treat data mining as a synonym for another popularly used term, **knowledge discovery from data, or KDD**, whereas others view data mining as merely an essential step in the overall process of knowledge discovery. The overall knowledge discovery process is shown as an iterative sequence of the following steps:

1. **Data preparation**
 - a. Data cleaning (to remove noise and inconsistent data)
 - b. Data integration (where multiple data sources may be combined)
 - c. Data transformation (where data are transformed and consolidated into forms appropriate for mining by performing summary or aggregation operations)
 - d. Data selection (where data relevant to the analysis task are retrieved from the database)
2. **Data mining** (an essential process where intelligent methods are applied to extract patterns or construct models). This step may interact with a user or a knowledge base.
3. **Pattern/model evaluation** (to identify the truly interesting patterns or models representing knowledge based on interestingness measures)
4. **Knowledge presentation** (where visualization and knowledge representation techniques are used to present mined knowledge to users)



Diversity of data types for data mining

Different kinds of data may need rather different data mining methodologies, from simple to rather sophisticated, making data mining a rich and diverse field.

✿ Structured vs unstructured data

Data stored in relational databases, data cubes, data matrices, and many data warehouses have uniform, record- or table-like structures, defined by their data dictionaries, with a fixed set of attributes (or fields, columns), each with a fixed set of value ranges and semantic meaning. These data sets are typical examples of highly structured data.

A more sophisticated type of semistructured data set is graph or network data, where a set of nodes are connected by a set of edges

Beyond such structured or semistructured data, there are also large amounts of unstructured data, such as text data and multimedia (e.g., audio, image, video) data.

Data mining methods are often developed for mining some particular type of data, and their results can be integrated and coordinated to serve the overall goal.

✿ Data associated with different applications

Different applications may generate or need to handle very different data sets and require rather different data analysis methods. Thus when categorizing data sets for data mining, we should take specific applications into consideration.

Even for the same data set, finding different kinds of patterns or knowledge may require different data mining methods.

✿ Stored vs streaming data

Usually, data mining handles finite, stored data sets, such as those stored in various kinds of large data repositories. However, in some applications such as video surveillance or remote sensing, data may stream in dynamically and constantly, as infinite data streams. (We use different methods for these)

Mining various kinds of knowledge

In general, data mining tasks can be put into two categories:

1. **Descriptive data mining:** Characterizes properties of the interested set of data.
2. **Predictive data mining:** Performs induction on the data set in order to make predictions.

✿ **Multidimensional data summarization** (pag 37 pdf reader)

It is desirable to automatically summarize an interested set of data and compare it with the contrasting sets at some high levels. Such summaritive description of an interested set of data is called data summarization. Data summarization can often be conducted in a multidimensional space.

The output of such multidimensional summarization can be presented in various forms, such as pie charts, bar charts, curves, multidimensional data cubes, and multidimensional tables, including crosstabs.

✿ **Mining frequent patterns, associations, and correlations** (pag 37 pdf reader)

Frequent patterns: patterns that occur frequently in data. (frequent itemsets (ejemplo del super), subsequences (como comprarse un celular y luego una funda), substructures).

Acá hay un ejemplo de análisis de asociación

✿ **Classification and regression for predictive analysis** (pag 38)

Classification: process of finding a model that describes and distinguishes data classes or concepts. The model is derived based on the analysis of a set of training data.

Según el modelo de clasificación, un modelo derivado puede estar en forma de: reglas de clasificación, árboles de decisión, fórmula matemática o una red neuronal.

Introduce el concepto de la regresión...

✿ **Cluster analysis** (pag 40)

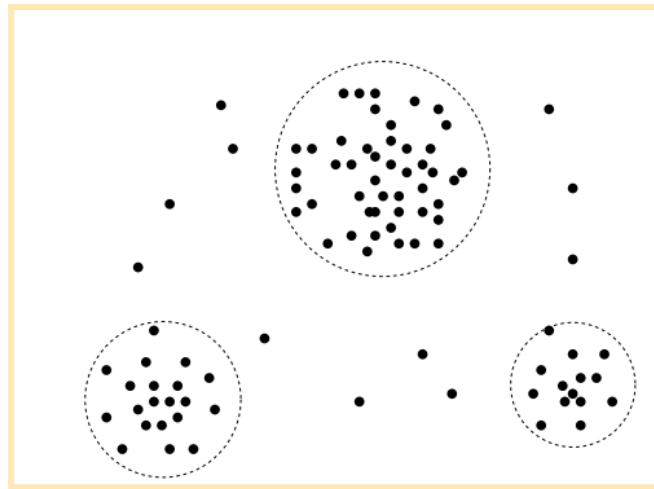
Cluster analysis groups data objects without consulting class labels. In many cases, class-labeled data may simply not exist at the beginning. Clustering can be used to generate class labels for a group of data.

Each cluster so formed can be viewed as a class of objects, from which rules can be derived.

Clustering can also facilitate taxonomy formation, that is, the organization of observations into a hierarchy of classes that group similar events together.

✿ **Deep learning** (pag 40)

Deep learning techniques provide an automatic way for feature engineering, which is capable of generating semantically meaningful features (e.g., weekly positive rate) from the initial input features. The generated features often significantly improve the mining performance.



Deep learning is based on neural networks. A neural network is a set of connected input-output units where each connection has a weight associated with it.

The core algorithm to learn such weights is called backpropagation, which searches for a set of weights and bias values that can model the data to minimize the loss function between the network's prediction and the actual target output of data tuples.

✿ Outlier analysis (pag 41)

A data set may contain objects that do not comply with the general behavior or model of the data. These data objects are outliers. The analysis of outlier data is referred to as outlier analysis or anomaly mining.

✿ Are all mining results interesting? (pag 41)

To answer the first question, a pattern is interesting if it is (1) easily understood by humans, (2) valid on new or test data with some degree of certainty, (3) potentially useful, and (4) novel. A pattern is also interesting if it validates a hypothesis that the user sought to confirm.

Several objective measures of pattern interestingness exist. These are based on the structure of discovered patterns and the statistics underlying them. In general, each interestingness measure is associated with a threshold, which may be controlled by the user.

"Can a data mining system generate all of the interesting patterns?" — refers to the completeness of a data mining algorithm. It is often unrealistic and inefficient for a pattern mining system to generate all possible patterns since there could be a very large number of them.

Confluence of multiple disciplines

As a discipline that studies efficient and effective methods for uncovering patterns and knowledge from various kinds of massive data sets for many applications, data mining naturally serves a confluence of multiple disciplines.

✿ Statistics (pag 43)

A statistical model is a set of mathematical functions that describe the behavior of the objects in a target class in terms of random variables and their associated probability distributions.

✿ Machine learning (pag 44)

In general, machine learning addresses two classical problems:

1. **Supervised learning:** The supervision in the learning comes from the labeled examples in the training data set. Ej: classification
2. **Unsupervised learning:** The learning process is unsupervised since the input examples are not class-labeled. Ej: clustering.

Data mining differs from machine learning in several major aspects.

1. Data mining often works on very large data sets, scalability can be an important concern, and many efficient and highly scalable data mining algorithms or stream mining algorithms have to be developed to accomplish such tasks.
2. In many data mining problems, the data sets are usually large, but the training data can still be rather small since it is expensive for experts to provide quality labels for many examples.
3. Machine learning methods may not be able to handle many kinds of knowledge discovery problems on big data.

✿ Database technology (pag 46)

Database technology is well known for its scalability in processing very large, relatively structured data sets.

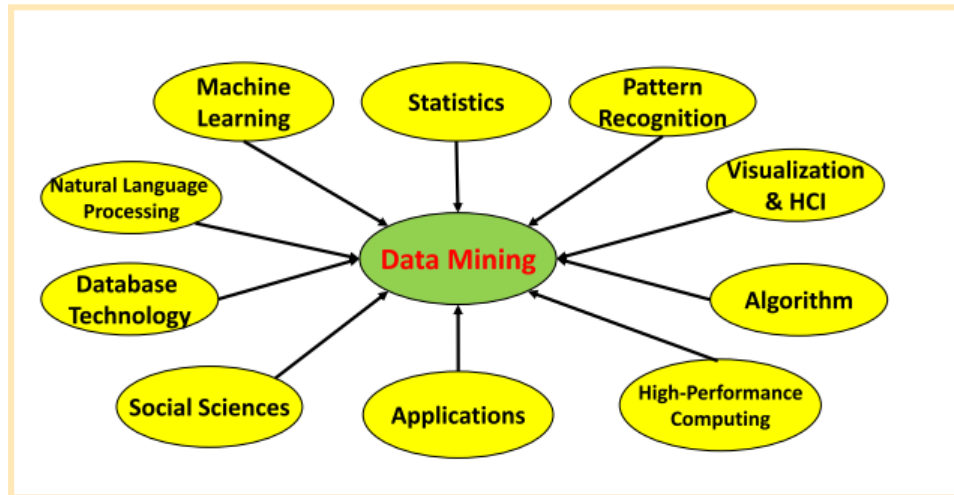
Many data mining tasks need to handle large data sets or even real-time, fast streaming data. Data mining can make good use of scalable database technologies to achieve high efficiency and scalability on large data sets. Moreover, data mining tasks can be used to extend the capability of existing database systems to satisfy users' sophisticated data analysis requirements.

A data warehouse integrates data originated from multiple sources and various timeframes.

✿ Data science (pag 46)

Big data generally refers to huge amounts of structured and unstructured data of various forms, and **data science** is an interdisciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from massive data of various forms.

For most people, data science is a concept that unifies statistics, machine learning, data mining, and their related methods in order to understand and analyze massive data.



Applications

- **Business intelligence:** For commercial context (historical, current and predictive views of business operations)
- **Web search engines:** Huge amount of data, online data.
- **Social media and social networks:** Social event detection, public health monitoring and surveillance, sentiment analysis in social media, recommendation, etc.
- **Biology, medical science and health care:** mining massive genomic and proteomic sequence data, frequent subgraph patterns for classifying biological data, predictive analysis of medical images)

Society

Data mining will help scientific discovery, business management, economy recovery, and security protection. However, it also poses the risk of unintentionally disclosing some confidential business or government information and disclosing an individual's personal information. Studies on data security in data mining and privacy-preserving data publishing and data mining are important, ongoing research theme.

Capitulo 2 - Data mining

Data, Measurements, and Data Preprocessing

To conduct successful data mining, the first important thing is to get familiar with your data. Gaining insights into attribute types, value distributions, and data similarity helps with subsequent analysis. Real-world data are typically noisy, enormous in volume, and may originate from a hodgepodge of heterogeneous sources. Low-quality data lead to low-quality mining results, requiring huge efforts in data preprocessing.

Data Types

Data sets are made up of data objects. A data object represents an entity (e.g., customers, store items, and sales in a sales database; patients in a medical database; or students, professors, and courses in a university database). Data objects can also be referred to as samples, examples, instances, data points, or objects. If they are stored in a database, they are data tuples, where the rows correspond to data objects and columns correspond to attributes.

✿ What is an attribute?

An attribute is a data field representing a characteristic or feature of a data object. The nouns *attribute*, *dimension*, *feature*, and *variable* are often used interchangeably in the literature.

- Dimension is commonly used in data warehousing.
- Feature is typically used in machine learning literature.
- Variable is preferred by statisticians.
- Attribute is commonly used by data mining and database professionals.

✿ Nominal attributes

The values of a nominal attribute are symbols or names of things. Each value represents some kind of category, code, or state, and so nominal attributes are also referred to as categorical. The values do not have any meaningful order.

It makes no sense to find the mean (average) value or median (middle) value for such an attribute, however, one thing that is of interest is the attribute's most commonly occurring value.

✿ Binary attributes

A binary attribute is a nominal attribute with only two categories or states: 0 or 1, where 0 typically means that the attribute is absent, and 1 means that it is present.

A binary attribute is symmetric if both of its states are equally valuable and carry the same weight (i.e: gender). A binary attribute is asymmetric if the outcomes of the states are not equally important, (i.e: positive and negative outcomes of a medical test for HIV).

✿ Ordinal attributes

An ordinal attribute is an attribute with possible values that have a **meaningful order or ranking among them**, but the magnitude between successive values is not known (i.e: small, medium, large sizes). Useful for **registering subjective assessments of qualities that cannot be measured objectively**.

The central tendency of an ordinal attribute can be represented by its **mode** and its **median** (the middle value in an ordered sequence), but the **mean cannot be defined**.

Note that nominal, binary, and ordinal attributes are **qualitative**. That is, they **describe a feature of an object without giving an actual size or quantity**.

✿ Numeric attributes

A numeric attribute is **quantitative**; that is, it is a **measurable quantity**, represented in integer or real values.

- **Interval-based attributes**: Measured on a **scale of equal-size units**. Because interval-scaled attributes are numeric, we can compute their mean value, in addition to the median and mode measures of central tendency. (i.e: temperature C° and F°)
- **Ratio-scaled attributes**: Numeric attribute with an inherent zero-point. We can speak of a **value as being a multiple (or ratio) of another value**. (i.e: temperature Kelvin)

✿ Discrete vs. continuous attributes

- **Discrete**: **Finite or countably infinite set of values**, which may or may not be represented as integers (hair_color, smoker, medical_test, and drink_size each have a finite number of values, and so are discrete)
- **Continuous**: typically represented as **floating-point variables**.

Observed values for a given attribute are known as **observations**. A set of attributes used to describe a given object is called an attribute vector (or feature vector).

1. **Univariate distribution**: The distribution of data involving one attribute or variable.
2. **Bivariate distribution**: A distribution involving two attributes.

Statistics of Data

Basic statistical descriptions can be used to learn more about each attribute's values. This section covers three main areas: measures of central tendency, data dispersion measures, and graphic displays.

✿ Measures of Central Tendency

These measure the **location of the middle or center of a data distribution**—intuitively, where most of its values fall. They make it easier to fill in missing values, smooth noisy values, and spot outliers during data preprocessing. Plotting these measures shows if the data are **symmetric or skewed**.

- **Mean**: The **average** value of an attribute.
- **Median**: The middle value of a distribution. For grouped data, the median can be approximated using the following formula:

$$median \approx L_1 + \left(\frac{N/2 - (\sum freq)_l}{freq_{median}} \right) \times width$$

Where L_1 is the lower boundary of the median interval, N is the number of values in the entire data set, $\sum freq$ is the sum of the frequencies of all intervals lower than the median interval, $freq\{median\}$ is the frequency of the median interval, and width is the width of the median interval.

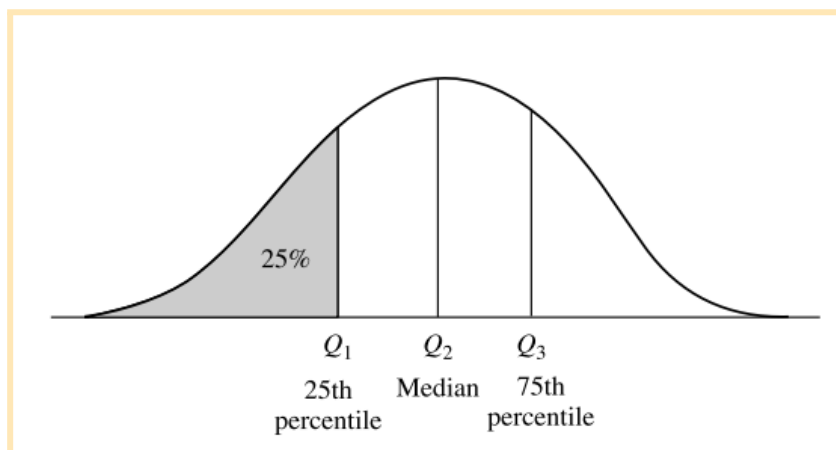
Divide a la distribución exactamente por la mitad, dejando el 50% de los valores por debajo y el 50% por encima. Es muy útil para evitar sesgos cuando existen valores extremos.

- **Mode**: The **value that occurs most frequently compared to all neighboring values in the set**. It can be determined for both qualitative and quantitative attributes.
 - ◆ Unimodal: A data set with one mode.
 - ◆ Bimodal: A data set with two modes.
 - ◆ Trimodal: A data set with three modes.
 - ◆ Multimodal: A data set with two or more modes.
- **Midrange**: Briefly noted as another central tendency measure alongside the mean, median, and mode.

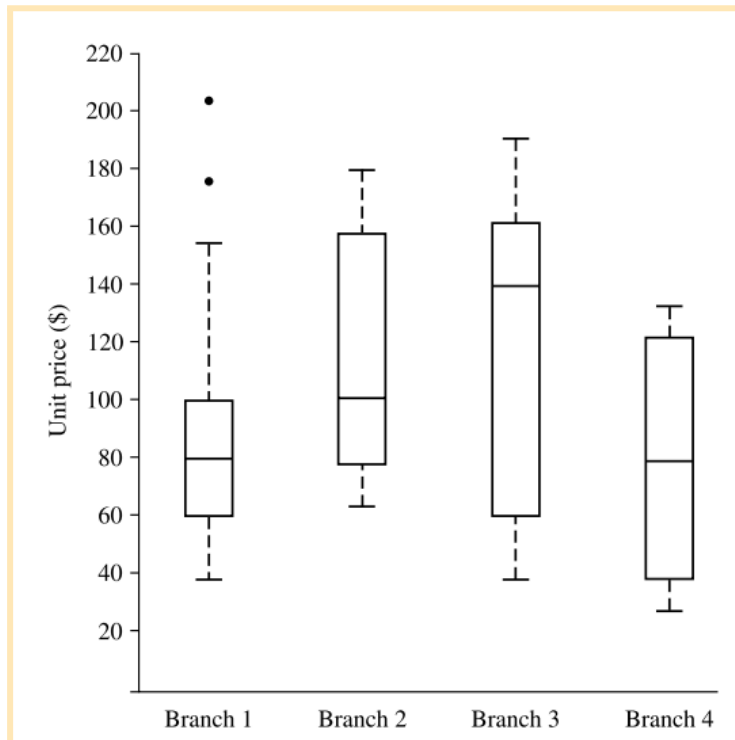
✿ Data Dispersion Measures

These give an idea of how the data are spread out and are useful for identifying outliers.

- **Range**: The difference between the largest ($\max()$) and smallest ($\min()$) values
- **Quartiles**: Data points taken at **regular intervals** of a data distribution, dividing it into essentially equal-size consecutive sets.



- **Interquartile Range (IQR)**: The distance between the quartiles.
- **Five-number summary and Boxplots**: Graphical representations and summaries used to assess data dispersion. (en este caso usaría: mínimo, Q1, median/mediana, Q3, máximo). Usamos boxplots para graficar esto:



- **Variance and Standard Deviation**: Standard statistical measures of data spread. They indicate how spread out a data distribution is. **Variance** is also described as a special case of covariance where the two attributes are identical (i.e., the covariance of an attribute with itself).

The **variance** of N observations, $x_1, x_2, \dots, x_N$ (when N is large), for a numeric attribute X is

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 = \left(\frac{1}{N} \sum_{i=1}^N x_i^2 \right) - \bar{x}^2,$$

- σ measures spread about the mean and should be considered only when the mean is chosen as the measure of center.
- $\sigma = 0$ only when there is no spread, that is, when all observations have the same value. Otherwise, $\sigma > 0$.

✿ Covariance and correlation analysis

Correlation and covariance are two similar measures for assessing how much two attributes change together.

Correlation coefficient for numeric data: For numeric attributes, we can evaluate the correlation between two attributes, A and B, by computing the correlation coefficient, also known as Pearson's product moment coefficient.

$$\frac{\sum_{i=1}^n (a_i b_i) - n \bar{A} \bar{B}}{n \sigma_A \sigma_B}$$

Where n is the number of tuples, a_i and b_i are the respective values of A and B in tuple i , A^- and B^- are the respective mean values of A and B , σA and σB are the respective standard deviations of A and B . The higher the value, the stronger the correlation

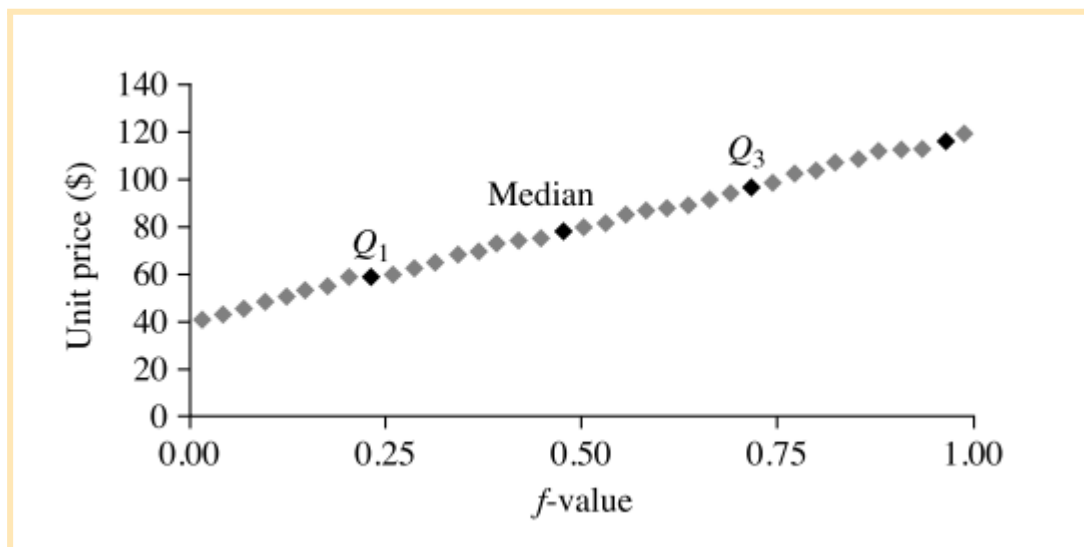
- **Covariance**: The text presents an example with **positive covariance**, indicating that the values for both variables **rise together**.
- **Contingency Tables**: Used for **evaluating relationships** (such as gender and preferred reading habits) to determine if two attributes are independent or strongly correlated.

Basic Contingency Table Example				
Favorite Flavor	Boys		Girls	
Vanilla	8	32%	9	26%
Chocolate	10	40%	6	17%
Strawberry	5	20%	14	40%
Mint Chip	2	8%	6	17%
Total	25	100%	35	100%

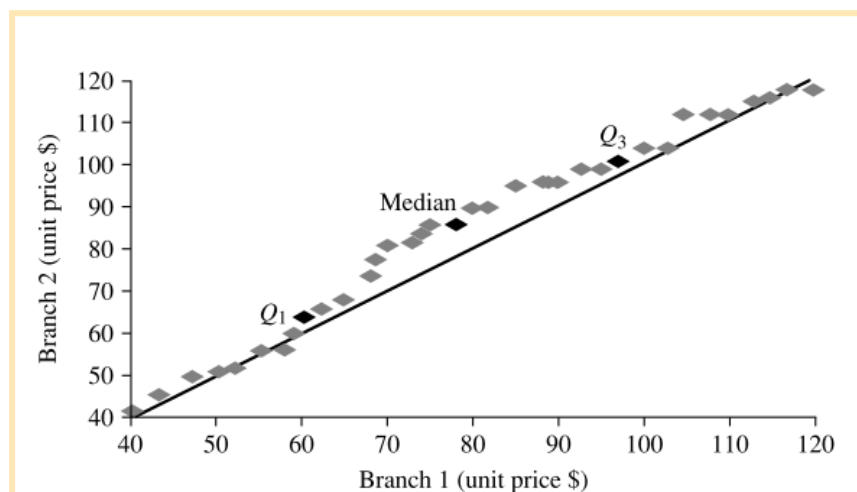
✿ Graphic Displays of Basic Statistics of Data

These graphs are helpful for the visual inspection of data, providing insights into areas for mining and data preprocessing.

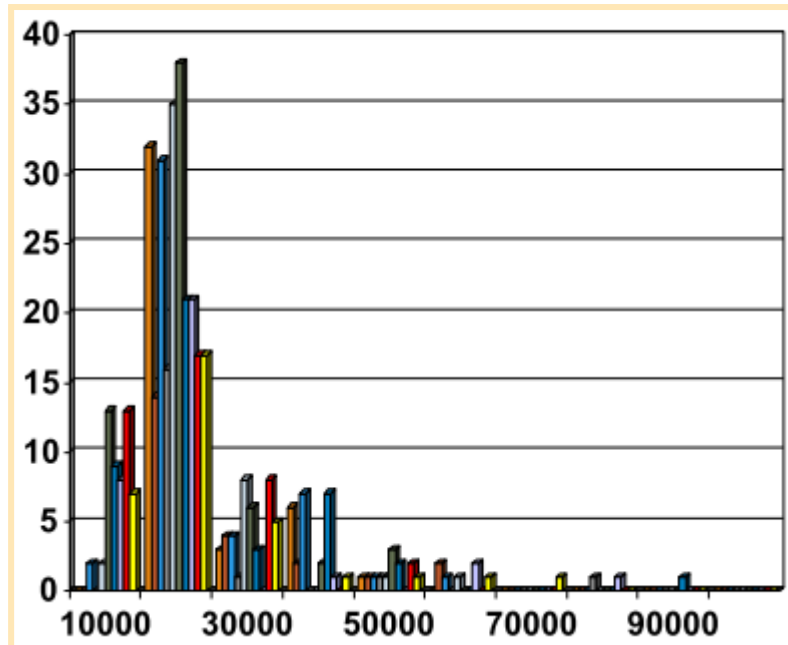
- **Quantile plots**: First, it displays all of the data for the given attribute. Second, it plots quantile information.



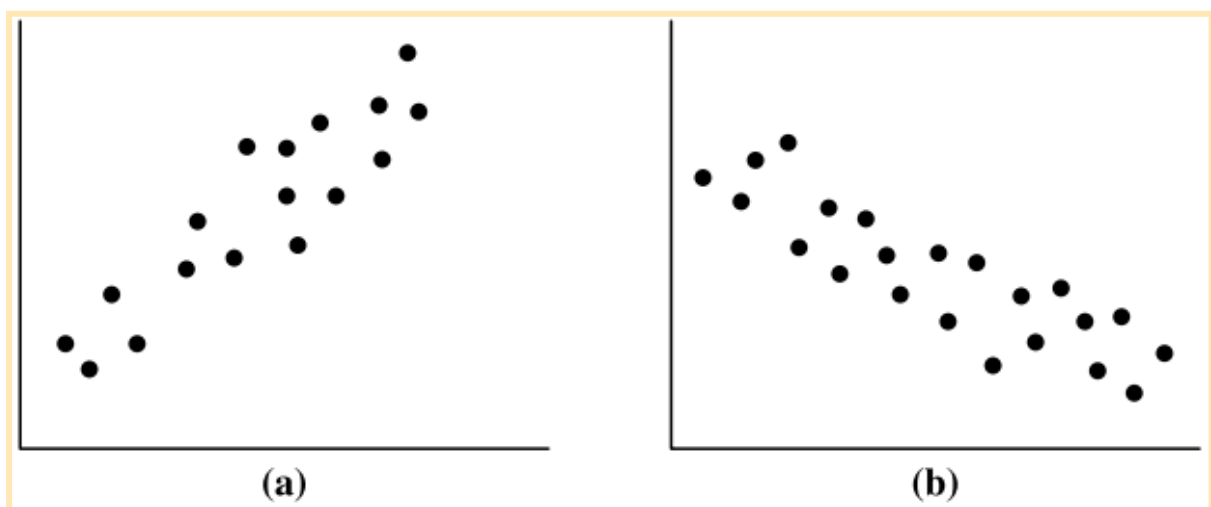
- **Quantile-quantile (q-q) plots**: Graphs the quantiles of one univariate distribution against the corresponding quantiles of another.



- **Histograms:** Also known as frequency histograms. A graphical method for summarizing the distribution of a given attribute, X . The range of values for X is partitioned into a set of disjoint consecutive subranges called **buckets or bins**. The range of a bucket is its **width** (typically of equal width). A bar is drawn for each subrange with a height representing the total count of items observed.



- **Scatter plots:** Each pair of values is treated as a **pair of coordinates** in an algebraic sense and plotted as points in the plane. . They are used to find positive or negative correlations.
- ◆ **Positive correlation:** The plotted points pattern slopes from lower left to upper right; values of X increase as values of Y increase (**creciente**).
 - ◆ **Negative correlation:** The pattern slopes from upper left to lower right; values of X increase as values of Y decrease (**decreciente**).
 - ◆ **Line of best fit:** Can be drawn on a scatter plot to study the correlation between variables.



Similarity and Distance Measures

Assessing how alike or unlike objects are is fundamental for data mining applications like clustering, outlier analysis, and nearest-neighbor classification.

- **Cluster:** A collection of data objects such that the objects within a cluster are similar to one another and dissimilar to the objects in other clusters.
- **Outlier analysis:** Employs clustering-based techniques to identify potential outliers as objects highly dissimilar to others.
- **Nearest-neighbor classification:** Schemes where a given object is assigned a class label based on its similarity toward other objects in the model.

✿ Data Structures

Clustering and nearest-neighbor algorithms typically operate on two main data structures:

1. **Data matrix (or object-by-attribute structure):** Stores n data objects in an n -by- p matrix (n objects $\times$ p attributes), where each row corresponds to an object and f indexes through the attributes.

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}$$

2. **Dissimilarity matrix (or object-by-object structure):** Stores a collection of proximities or dissimilarity values for pairs of objects.

$$\begin{bmatrix} 0 & & & & \\ d(2, 1) & 0 & & & \\ d(3, 1) & d(3, 2) & 0 & & \\ \vdots & \vdots & \vdots & \ddots & \\ d(n, 1) & d(n, 2) & \cdots & \cdots & 0 \end{bmatrix}$$

✿ Proximity measures for nominal attributes

A nominal attribute can take on two or more states (denoted by M), which can be letters, symbols, or integers used purely for data handling without representing any specific ordering. Dissimilarity can be computed as:

$$d(i, j) = \frac{p - m}{p}$$

Similarity can be computed as:

$$\text{sim}(i, j) = 1 - d(i, j) = \frac{m}{p}.$$

Where m represents matching states and p is the total number of attributes. Nominal attributes can also be encoded using **asymmetric binary attributes** by creating a new binary attribute for each of the M states (setting it to 1 if present, and 0 otherwise).

✿ Proximity Measures for Binary Attributes

A binary attribute has only two states, 0 and 1, where 0 means absent and 1 means present. Treating binary attributes as standard numeric fields can be misleading.

q: number of attributes that equal 1 for both objects i and j
r: number of attributes that equal 1 for object i but equal 0 for object j
s: number of attributes that equal 0 for object i but equal 1 for object j
t: number of attributes that equal 0 for both objects i and j .

→ **Symmetric binary dissimilarity**: Dissimilarity that is based on symmetric binary attributes:

$$d(i, j) = \frac{r + s}{q + r + s + t}.$$

For **asymmetric binary attributes**, the two states are not equally important

→ **Asymmetric binary dissimilarity**: The number of negative matches (t) is considered unimportant and ignored:

$$d(i, j) = \frac{r + s}{q + r + s}.$$

→ **Asymmetric binary similarity (Jaccard coefficient)**: Difference between two binary attributes based on the notion of similarity instead of dissimilarity.

$$\text{sim}(i, j) = \frac{q}{q + r + s} = 1 - d(i, j).$$

✿ Dissimilarity of Numeric Data: Minkowski Distance

Common distance measures used for computing the dissimilarity of numeric attributes include the Euclidean distance, Manhattan distance, and Minkowski distance.

Minkowski distance is a generalization of the Euclidean and Manhattan distances. It is defined as:

$$d(i, j) = \sqrt[h]{|x_{i1} - x_{j1}|^h + |x_{i2} - x_{j2}|^h + \cdots + |x_{ip} - x_{jp}|^h},$$

where h is a real number such that $h \geq 1$.

✿ Proximity Measures for Ordinal Attributes

An ordinal attribute has ordered states that define a ranking $1 \dots M$. After:

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}.$$

, where z_{if} to represent the f value for the i th object, Their treatment is quite similar to that of numeric attributes when computing dissimilarity.

✿ Proximity Measures for Mixed Attribute Types

In real applications, data sets often contain multiple attribute types. Processing them together in a single analysis is preferable to separate independent analyses per type. One technique combines different attributes into a single dissimilarity matrix, bringing all meaningful attributes onto a common scale of the interval $[0.0, 1.0]$.

✿ Cosine similarity

Measures the similarity between two vectors of an inner product space. It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in roughly the same direction (often used in text analysis).

A document can be represented by thousands of attributes, each recording the frequency of a particular word. Term-frequency vectors are typically very long and sparse.

Cosine similarity is a measure of similarity that can be used to compare documents or, say, give a ranking of documents with respect to a given vector of query words.

Table 2.7 Document vector or term-frequency vector.

Document	Team	Coach	Hockey	Baseball	Soccer	Penalty	Score	Win	Loss	Season
Document1	5	0	3	0	2	0	0	2	0	0
Document2	3	0	2	0	1	1	0	1	0	1
Document3	0	7	0	2	1	0	0	3	0	0
Document4	0	1	0	0	1	2	2	0	3	0

✿ Comparing Probability Distributions (Kullback-Leibler Divergence)

The **Kullback-Leibler divergence** (or KL divergence) measures the difference between two probability distributions $p(x)$ and $q(x)$ over the same variable x . Originating from probability and information theory, it is closely related to relative entropy, information divergence, and information for discrimination. It is a **nonsymmetric measure of information loss when $q(x)$ is used to approximate $p(x)$** .

$$D_{KL}(p(x)||q(x)) = \sum_{x \in X} p(x) \ln \frac{p(x)}{q(x)}$$

Although the KL divergence measures the “distance” between two distributions, it is not a distance measure. This is because that the **KL divergence is not a metric measure**.

✿ Capturing Hidden Semantics in Similarity Measures

Advanced similarity notions include **distributive representation and representation learning**, where text embedding and deep learning are used to compute complex object similarities.

Data Quality, Data Cleaning, and Data Integration

✿ Data Quality Measures

Data have quality if they satisfy the requirements of the intended use. The primary factors defining data quality are:

- **Accuracy**: Ensuring data contains **correct values rather than errors or noise**. Inaccurate data can stem from faulty collection instruments, entry errors, or disguised missing data (when users intentionally submit incorrect mandatory fields, like a default birthday of "January 1", to protect privacy).
- **Completeness**: Making sure the **data does not lack attribute values** or entire attributes of interest.
- **Consistency**: Eliminating discrepancies, such as inconsistent formatting for fields like dates, or mismatched department codes.
- **Other factors**: Timeliness, believability (how much the data is trusted by users), and interpretability.

✿ Data Cleaning

Data cleaning (or data cleansing) routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies.

1. **Handling Missing Values**:
 - a. Ignore the tuple,
 - b. Fill in the missing value (manually or using a global constant, use a measure of central tendency for filling, use the mean or median, use the most probable value).
 - c. Attributes should ideally have rules regarding the null condition. Good database design helps minimize missing values from the start.
2. **Data Smoothing**: Used to smooth out noise (Random error or variance in a measured variable).

- a. **Binning**: smooth a sorted data value by consulting its “neighborhood” (we can smooth by bin means, bin medians and bin boundaries).

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin boundaries:

Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

- b. **Regression**: Technique that conforms data values to a function. **Linear regression** involves finding the “best” line to fit two attributes (or variables) so that one attribute can be used to predict the other.
- c. **Outlier analysis**: Outliers may be detected by clustering.

Many data smoothing methods are also used for data discretization.

3. **Data Cleaning as a Process:**

- a. **Discrepancy detection**: poorly designed data entry forms, human error in data entry, deliberate errors, inconsistent data representations and inconsistent use of codes. For this we use: Data scrubbing tools, Data auditing tools.
- b. **Data transformation**: May be corrected manually using external references. For this we use: Data migration tools, ETL (extraction/transformation/loading) tools.

The two-step process of discrepancy detection and data transformation (to correct discrepancies) **iterates**. This process, however, is **error-prone and time-consuming**.

✿ **Data Integration**

Data integration is the **merging of data from multiple data stores into a coherent store** like a data warehouse. It **reduces redundancies and inconsistencies**, improving the accuracy and speed of subsequent mining tasks. Challenges include structural and semantic heterogeneity, such as:

- **Entity Identification Problem:** Matching schemas and objects from different sources. When matching attributes from one database to another during integration, special attention must be paid to the structure of the data. This is to ensure that any attribute functional dependencies and referential constraints in the source system match those in the target system.
- **Redundancy and correlation Tests:** An attribute may be redundant if it can be "derived" from another attribute or set of attributes. Some redundancies can be detected by correlation analysis. Given two attributes, such analysis can measure how strongly one attribute implies the other
- **Tuple Duplication & Value Conflicts:** Detecting duplicate records and resolving differences in data representation or abstraction levels (e.g., total sales referring to a single branch vs. an entire region). Inconsistencies often arise between various duplicates, due to inaccurate data entry or updating some but not all data occurrences.
- **Data value conflict detection and resolution:**

Data Transformation

Data transformation transforms or consolidates data into forms appropriate for mining, making the process more efficient and the found patterns easier to understand.

✿ Normalization

The choice of measurement units can skew analysis (e.g., meters vs. inches), as smaller units yield a larger range and give an attribute greater "weight". Normalization **scales attribute data to fall within a smaller or common range**, such as [0.0, 1.0] or [-1.0, 1.0]. The terms *standardize* and *normalize* are used interchangeably in data preprocessing.

i.e: min-max, z-score, normalization by decimal scaling.

✿ Discretization

Data discretization **replaces the raw values of a numeric attribute with interval labels** (e.g., 0–10) or conceptual labels (e.g., youth, adult, senior). These labels can be organized into a **concept hierarchy**.

- **Supervised** discretization: Uses class information during the process.
- **Unsupervised** discretization: Does not use class information.
- **Top-down vs. Bottom-up:** Top-down methods start by finding split points or cut points to partition the data.

Two basic discretization techniques:

1. **Binning:** Equal-width or equal-frequency binning and then replacing each bin value by the bin mean or median. Top-down and unsupervised
2. **Histogram analysis:** A histogram partitions the values of an attribute, A, into disjoint ranges called buckets or bins. If each bucket represents only a single attribute-value/frequency pair, the buckets are called singleton buckets. Singleton buckets are useful for storing high-frequency outliers. Unsupervised (equal width or frequency).

✿ Data Compression

Data compression applies transformations to obtain a reduced representation of the data.

- **Lossless compression:** The original data can be reconstructed from the compressed version without any information loss. String compression algorithms are often lossless but allow limited data manipulation.
- **Lossy compression:** Reconstructs only an approximation of the original data.

✿ Sampling

A data reduction technique that transforms input data to a reduced representation that is much smaller in volume, yet closely maintains the integrity of the original data.

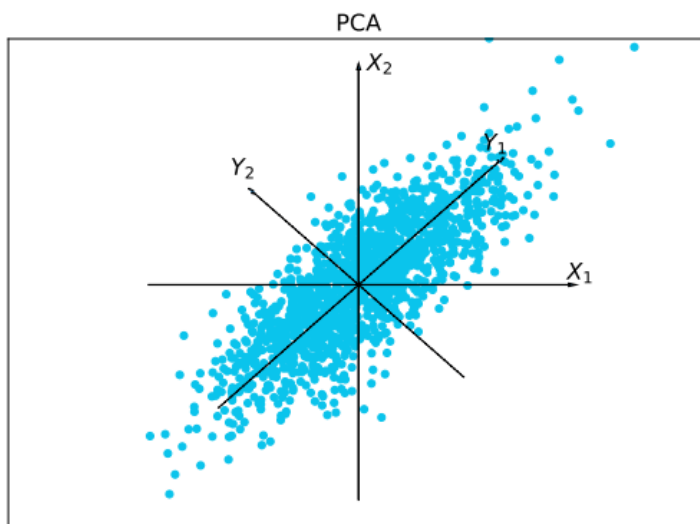
- **Simple random sample without replacement:** Recolectar muestras (samples) del dataset sin devolverlas a este cuando se termine de usar.
- **Simple random sample with replacement**
- **Cluster sample:** If the tuples in D are grouped into M mutually disjoint “clusters,” then a sample of s clusters can be obtained
- **Stratified sample:** If D is divided into mutually disjoint parts called strata, a stratified sample of D is generated by obtaining a sample at each stratum.

Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of random variables, attributes, or features under consideration. Keeping irrelevant or redundant attributes can confuse mining algorithms, slow down the process, and lead to poor-quality patterns.

✿ Principal Components Analysis (PCA)

PCA is a linear dimensionality reduction method that transforms or projects the original data onto a smaller space. The projected components are linear combinations of the attributes.



Two principal components, Y1 and Y2, for the given set of data originally mapped to the axes X1 and X2.

✿ Attribute Subset Selection

A method of dimensionality reduction where irrelevant, weakly relevant, or redundant dimensions are detected and removed. An example of a technique used here is **decision tree induction**.

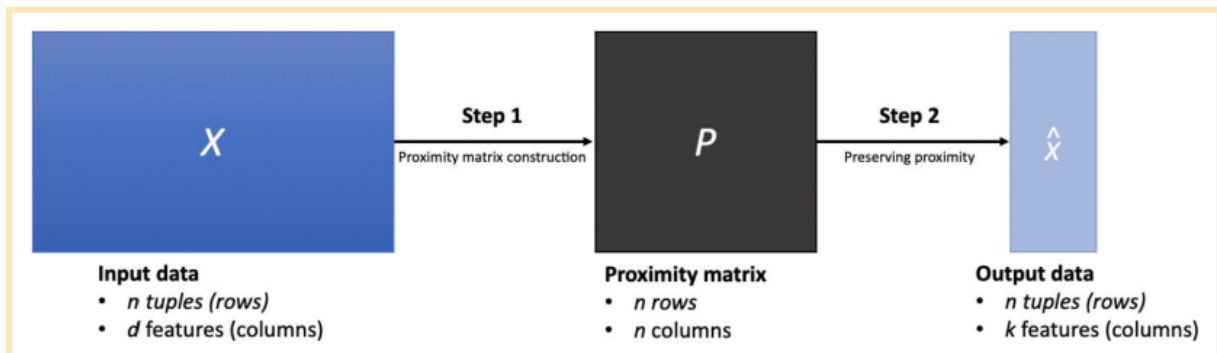
- 1. Stepwise forward selection:** The procedure starts with an empty set of attributes as the reduced set. The best of the original attributes is determined and added to the reduced set. At each subsequent iteration or step, the best of the remaining original attributes is added to the set.
- 2. Stepwise backward elimination:** The procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.
- 3. Combination of forward selection and backward elimination:** The stepwise forward selection and backward elimination methods can be combined so that, at each step, the procedure selects the best attribute and removes the worst from among the remaining attributes.

- A decision tree consists of **internal (nonleaf) nodes** denoting a test on an attribute, branches corresponding to outcomes, and **external (leaf) nodes** denoting a class prediction.
- When used for subset selection, attributes that do not appear in the constructed tree are assumed to be **irrelevant and are removed**, leaving the tree's attributes as the reduced subset.

✿ Nonlinear Methods for Dimensionality Reduction

Nonlinear methods generally preserve the proximity between data tuples from the same cluster much **better than linear methods**, leading to higher accuracy in complex datasets.

PCA is a linear method for dimensionality reduction in that each principal component is a linear combination of the original input attributes.



How can we reduce the dimensionality to k where $k < d$? In other words, we want to represent each of input data tuples by a k -dimensional attribute vector

→ **Kernel PCA (KPCA):** Employs kernel functions such as the radial basis function (RBF) kernel:

$$\kappa(x_i, x_j) = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$$

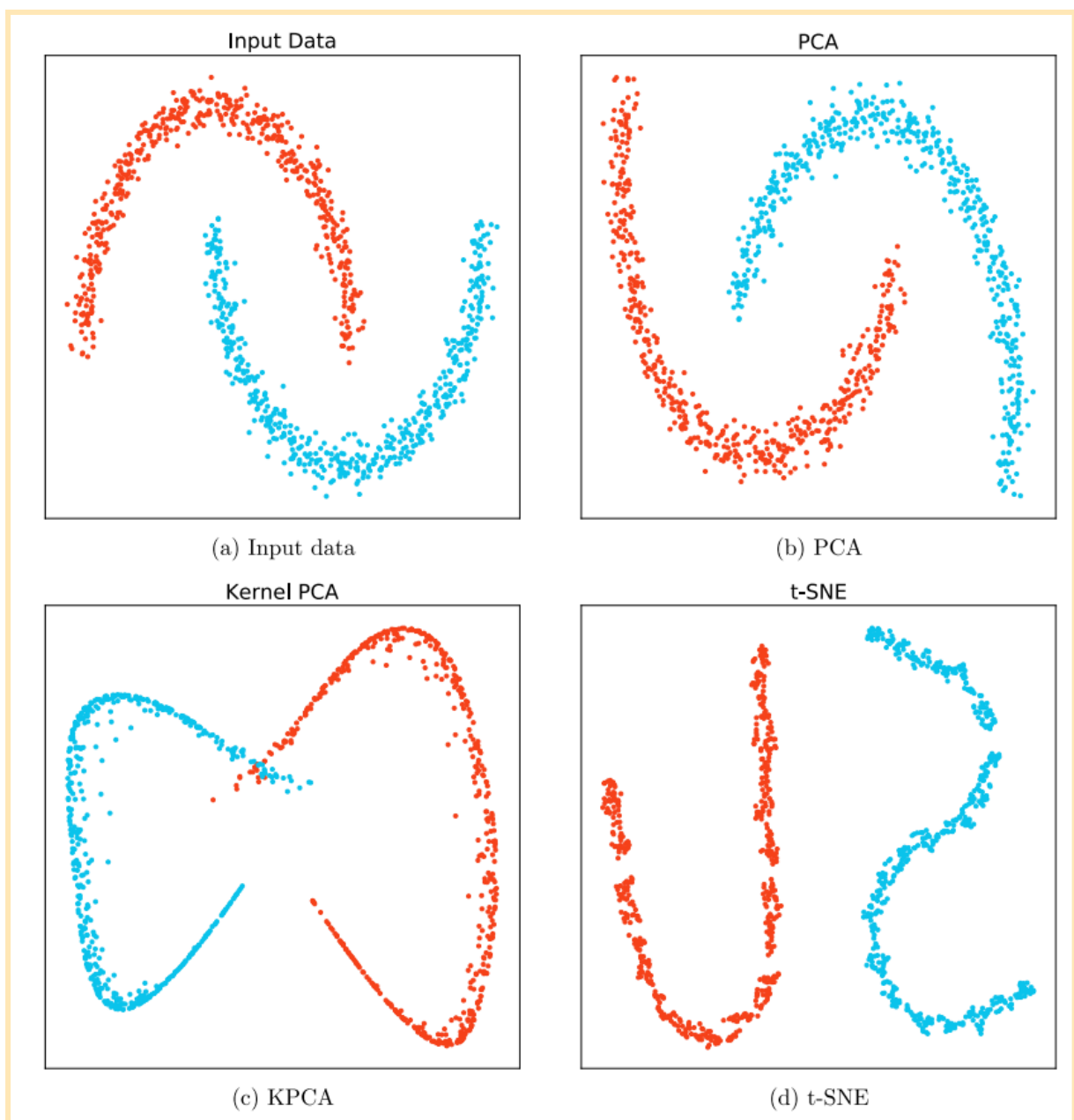
If a linear kernel $k(x_i, x_j) = x_i \cdot x_j$ is chosen, KPCA degenerates to standard PCA.

→ Stochastic Neighbor Embedding (SNE) / t-SNE: In SNE, a proximity matrix P is constructed where entry $P(i, j)$ represents a probability value derived from the distances between points:

$$P(i, j) = \frac{e^{-d_{ij}^2}}{\sum_{l=1, l \neq i}^n e^{-d_{il}^2}}$$

Where

$$d_{ij}^2 = \frac{\|x_i - x_j\|^2}{2\sigma^2}.$$



Capítulo 3 - Data Mining

Data Warehousing and Online Analytical Processing

Data analytics, also popularly called business intelligence, consists of the strategies and technologies that enable enterprises to extract deep and actionable insights out of complex business data. Data mining serves as the core computational framework within business intelligence.

Data Warehouse

Fundamentally, data warehouses generalize and consolidate enterprise data within a multidimensional space. The physical construction of a data warehouse acts as an important data preparation phase for subsequent data mining steps because it integrates activities such as data cleaning, data integration, and data transformation.

✿ Data Warehouse: What and Why?

In standard operational environments, enterprise information is typically recorded at the raw operational level. For instance, an e-commerce platform records minute details of individual consumer transactions in one table, customer profile data in another, and vendor logistics in a third.

The four keywords—subject-oriented, integrated, time-variant, and nonvolatile—distinguish data warehouses from other data repository systems, such as relational database systems, transaction processing systems, and file systems.

The construction of a data warehouse requires data cleaning, data integration, and data consolidation.

To distinguish the operational environments from analytical data platforms, the book outlines explicit operational contrasts across distinct categories:

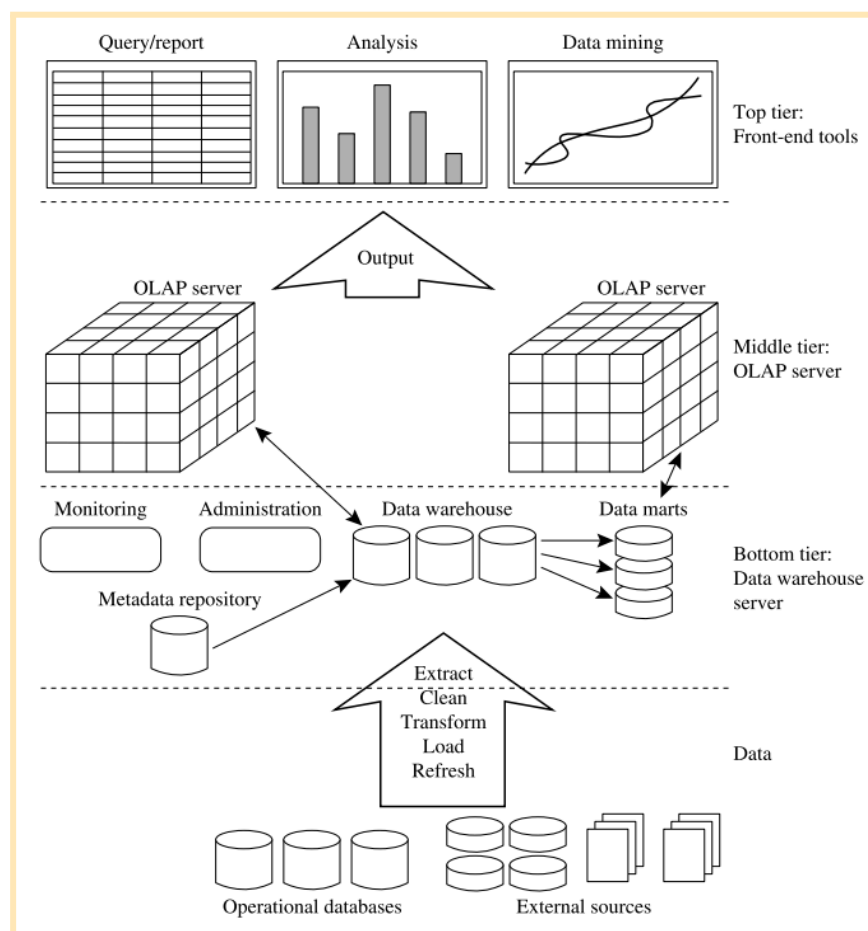
- **OLTP (Online Transaction Processing):** These systems manage day-to-day, real-time transactional updates. Their access patterns consist primarily of short, atomic database operations—such as executing an account-to-account funds transfer. Consequently, OLTP designs place a critical dependency on strong concurrency control and transaction recovery mechanisms.
 - ◆ Transaction oriented
 - ◆ Current data that are typically too detailed
 - ◆ ER model
- **OLAP (Online Analytical Processing):** These platforms are specialized for broad, multi-granular, and interactive data analysis. Access patterns are primarily read-only operations because historical data values are preserved rather than subjected to real-time micro-updates. Queries submitted to OLAP systems are frequently highly complex and require massive data scanning.
 - ◆ Business insight oriented
 - ◆ Historic data
 - ◆ Star or snowflake model
- **Schema Evolution & Heterogeneity:** OLTP databases focus on immediate operational states, whereas OLAP systems span multiple variations of an organization's

database schema that have mutated over a long historical timeline. OLAP environments also resolve semantic discrepancies across entirely separate external organizations by integrating highly heterogeneous information from autonomous data stores.

✿ Architecture of Data Warehouses: Enterprise Data Warehouses and Data Marts

Data warehouses typically conform to a structured three-tier architecture:

1. **Bottom Tier:** Comprises the raw warehouse database server. This tier is typically backed by standard relational database management systems (RDBMS) or specialized key-value distributed stores. Data is pushed into this layer via back-end utilities performing ETL (Extraction, Transformation, and Loading) operations. ETL scripts extract data from operational or external channels, clean out noise, resolve structural mismatches, and refresh the core repository. This tier also maintains the metadata repository, which acts as "data about data" by explicitly defining warehouse objects, mapping sources, recording cleaning timestamps, and capturing added default values.
2. **Middle Tier:** Consists of a specialized OLAP server that provides a multidimensional abstraction of the underlying data. This server is deployed using either a ROLAP (Relational OLAP) model—which maps multidimensional array operations into standard SQL queries over an extended relational engine—or a MOLAP (Multidimensional OLAP) model, which utilizes optimized multidimensional array data structures directly.
3. **Top Tier:** Represents the front-end client layer. It exposes graphic tools for end-user querying, dashboard reporting, multi-axis visualization, trend analysis, and comprehensive predictive data mining tasks.



✿ Data Warehouse Models

The **scope** of an analytical deployment determines its structural model:

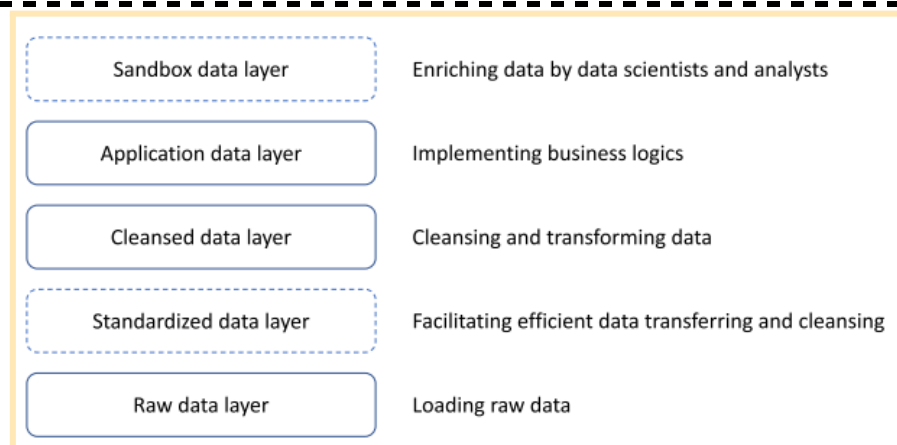
- **Enterprise Data Warehouse**: Gathers comprehensive information concerning the entire corporate entity, spanning multi-departmental business subjects.
- **Data Mart**: Contains a tightly focused subset of corporate data restricted to a single distinct department or user group. Data marts are classified as independent data marts if they are constructed directly from raw operational systems or external vendor data feeds without a centralized warehouse. Alternatively, they are defined as dependent data marts if they are populated as downstream branches sliced out from an existing enterprise data warehouse.

✿ **Data Lakes**

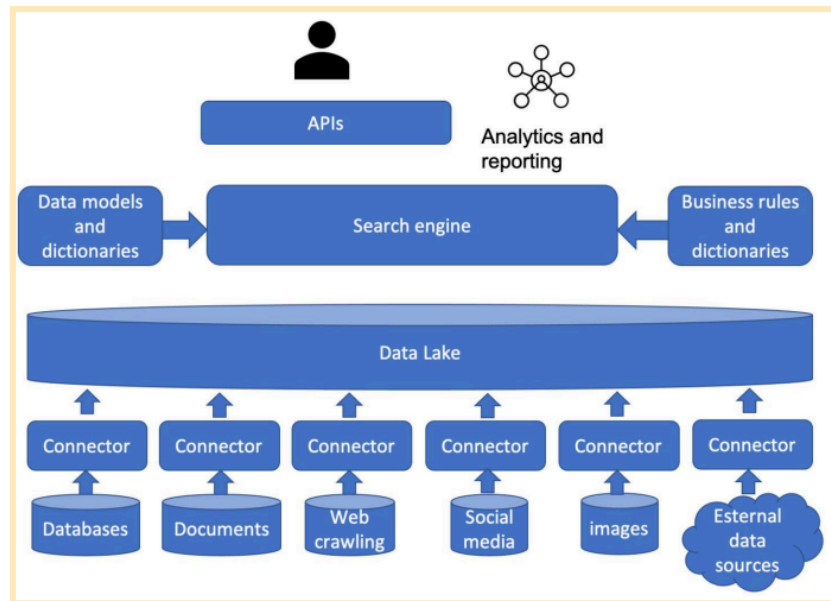
A data lake is an enterprise information infrastructure designed to **ingest and store** massive, diverse volumes of raw **data in its native format**. Unlike data warehouses, which enforce a strict schema-on-write restriction during data integration, data lakes **preserve raw, un-transformed information streams**. They unify heterogeneous data types alongside automated, deeply integrated metadata tags. This **flexible foundation** enables data scientists and analysts to conduct highly effective, open-ended **data exploration directly on source material**.

Especial considerations:

1. Centralized data repository for an entire enterprise, it has to be **scalable**.
2. The data storage layer has to have high durability (**robustness**).
3. Data lake storage has to support different types of data in various **formats**.
4. Data storage should be able to support various data **schemas** (DS independent from each schema).
5. Storage layer should be **decoupled with computation resources**



A data lake takes data from a wide spectrum of data repositories in an enterprise or an organization, such as the databases, the store of documents, data crawled from the web, social media, images (e.g., products), and possibly external data sources. Data from those data sources are loaded into the data lake through connectors in a continuous manner. Once data is ingested into a data lake, the data goes through the layers that we just discussed.



Data lakes serve as the centralized data repositories of enterprises and organizations = huge efficiency and advantage in data-driven business operation and decision making.

Data Warehouse Modeling: Schema and Measures

Data warehouse models organize data collections around core business subjects rather than individual corporate applications. These models represent data dimensions (such as time, item categories, or geographic branch domains) alongside numerical metrics known as measures. **Nonvolatile.**

✿ Data Cube: A Multidimensional Data Model

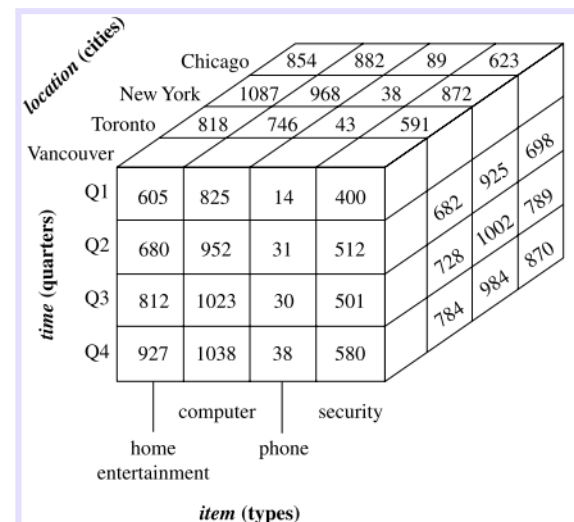
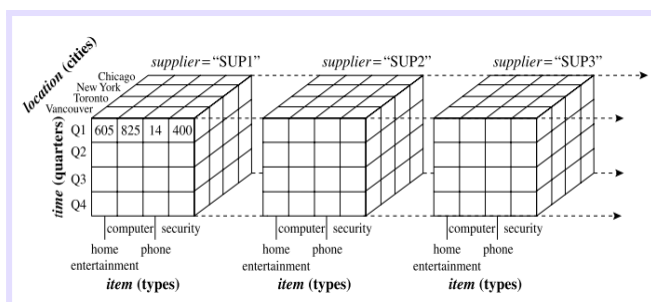
A multidimensional data model represents analytical information through an abstract, n-dimensional structure called a data cube. At its conceptual foundation, a data cube allows data to be examined simultaneously across many intersecting dimensions.

- **Dimensions:** The analytical perspectives, organizational entities, or view axes under which an enterprise maintains records (e.g., **item**, **time**, or **location**).
- **Facts:** Measurements on a subject (i.e: dollars_sold)
- **Fact table:** stores the names of the facts, or measures, as well as (foreign) keys referencing each of the related dimension tables.

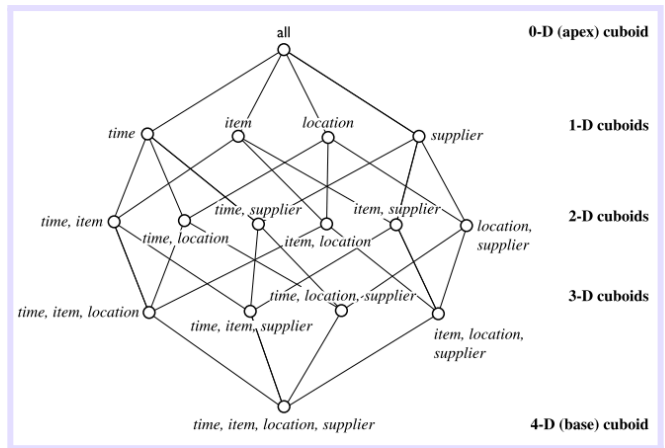
Table 3.2 3-D view of sales data according to time, item, and location.

time	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
	item				item				item				item			
	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is dollars_sold (in thousands).



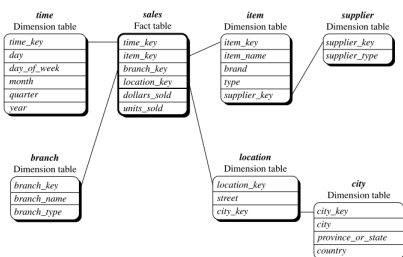
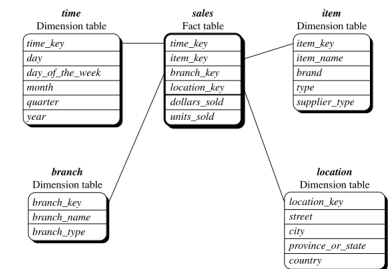
→ **Cuboid:** A data cube does not represent a singular, static block; rather, it is a complex lattice of multi-dimensional tables aggregated at distinct granularities. The lowest-level, non-aggregated multidimensional table is termed the base cuboid. The highest-level, fully summarized single-cell table is called the apex cuboid (often representing a grand total). Every distinct level of sub-aggregation mapping between the base and apex forms an intermediate cuboid.



❁ Schemas for Multidimensional Data Models: Stars, Snowflakes, and Fact Constellations

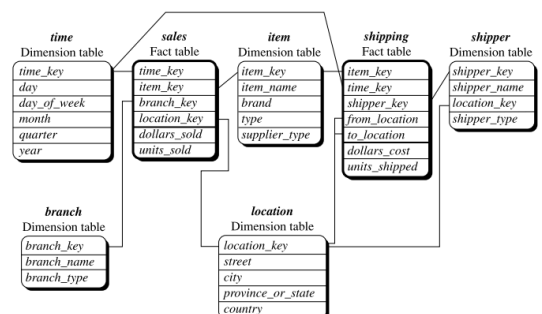
To implement multidimensional data cubes over relational engines, standard data warehouse designs adopt specific multi-table schema architectures:

- **Star Schema:** Features a massive, centralized table called the fact table, which contains the core numerical metrics (measures) and foreign keys linking out to surrounding dimension tables. In a pure star schema, dimension tables are deliberately left un-normalized, resulting in an explicit, single-hop radial structure.



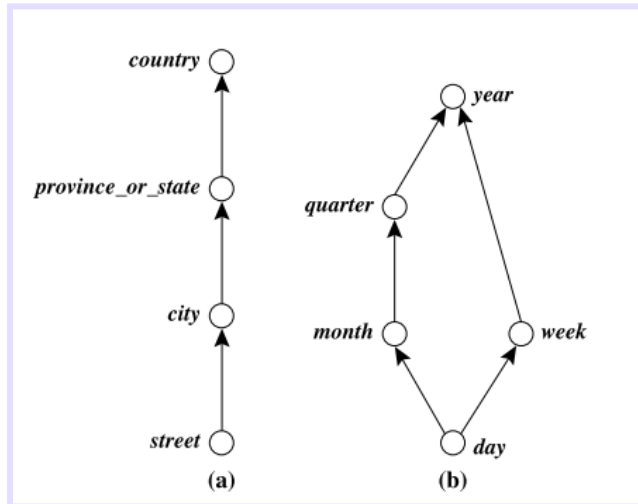
- **Snowflake Schema:** A direct variation of the star schema where certain dimension tables are systematically normalized into secondary, subordinate tables to eliminate data redundancy. For example, an **item** dimension table might break out its text values by linking its own attributes to a separate, normalized **supplier** table. This structural breakdown reduces storage overhead but introduces a multi-hop join path for analytical queries.

- **Fact Constellation Schema (Galaxy Schema):** Sophisticated enterprise applications often require multiple fact tables to operate across shared dimension tables. A fact constellation schema models this interconnected framework by mapping multiple central fact tables (such as **sales_fact** and **shipping_fact**) tied directly to a collective pool of shared, reusable dimensions.



❁ Concept Hierarchies

A concept hierarchy defines a sequence of systematic mappings that transform lower-level, granular data attributes into abstract, higher-level conceptual groupings. For instance, a raw geographic attribute path might follow the conceptual ladder: **street** → **city** → **province_or_state** → **country**. Concept hierarchies allow analytical tools to move fluidly between detailed transactions and corporate summary data.



❁ Measures: Categorization and Computation

A **measure** in a data cube is a **numeric function** that can be evaluated at each point in the data cube space. A measure value is computed for a given point by aggregating the data corresponding to the respective dimension–value pairs defining the given point.

Measures can be organized into three categories:

1. **Distributive Measures:** These metrics can be evaluated by **partitioning** a massive dataset into independent chunks, calculating the function over each local piece, and then directly combining those intermediate results to compute the global answer. Examples include **SUM()**, **MIN()**, **MAX()**, and **COUNT()**.
2. **Algebraic Measures:** These measures can be computed by applying a **deterministic algebraic function** to a small, fixed set of distributed distributive measures extracted from data segments. For example, **AVERAGE()** is algebraic because its global state can be computed by tracking **SUM() / COUNT()** across fragments. Other examples include standard deviation, variance, and min-max normalization metrics.
3. **Holistic Measures:** These metrics cannot be computed by **combining intermediate sub-aggregations** from independent data segments. To establish the exact value, the entire dataset must be analyzed as a single, cohesive unit; subset values are structurally insufficient. Examples include **MEDIAN()**, **MODE()**, and rank-tracking statistics.

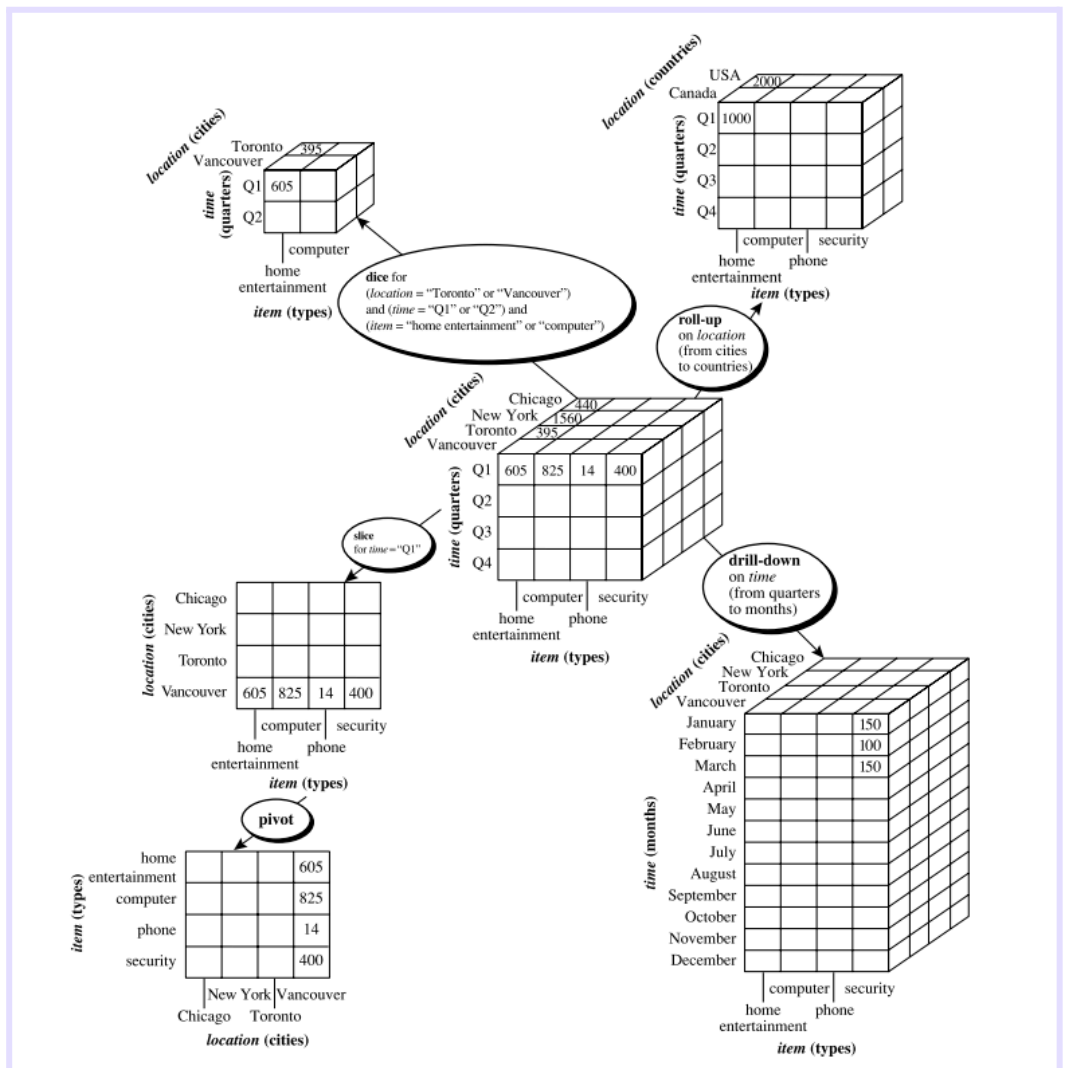
OLAP Operations

Online Analytical Processing (OLAP) provides an interactive environment that allows business analysts to manipulate multidimensional data views across multiple granularities.

✿ Typical OLAP Operations

The core structural manipulation of data cubes involves a set of standard relational adjustments. Hence, OLAP provides a user-friendly environment for interactive data analysis:

- **Roll-up:** Performs data summarization by either climbing up a concept hierarchy along a dimension axis or by dropping a dimension entirely from the active cuboid view.
- **Drill-down:** The reverse of a roll-up operation; it exposes deep detail by either stepping down a concept hierarchy or by introducing a new, clarifying dimension axis into the view.
- **Slice:** Selects a single, specific dimension value along one axis of a data cube, effectively extracting a lower-dimensional sub-cube (e.g., viewing data restricted exclusively to **time = "Q1"**).
- **Dice:** Extracts a smaller, localized sub-cube from the parent cube by defining selection criteria across multiple intersecting dimensions (e.g., restricting data to **time = "Q1" OR "Q2" AND item = "computer"**).
- **Pivot (Rotate):** Rotates the data axes on the visualization layout to provide a different geometric perspective of the data grid (e.g., swapping row dimensions with column dimensions).



✿ Indexing OLAP Data: Bitmap Index and Join Index

To achieve sub-second interactive speeds over fact tables containing billions of records, standard database index implementations are augmented with specialized OLAP indexing structures:

✿ Bitmap Index

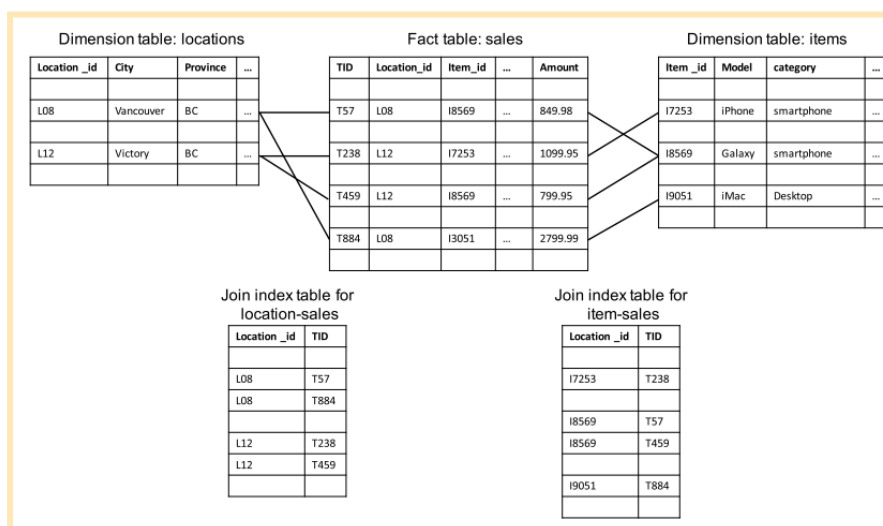
A bitmap index is an exceptionally compact indexing method designed for columns with low cardinality (i.e., columns containing a small number of distinct values, such as gender or marital_status).

- **Structure:** For an attribute with M unique states, the database engine constructs M distinct bit vectors (bitmaps). If the total number of records in the table is N, each individual bitmap consists of exactly N bits.
- **Operation:** The i-th bit in a specific bitmap is set to 1 if the i-th record in the database matches that bitmap's specific attribute state; otherwise, it is set to 0.
- **Advantage:** When an analytical query filters data across low-cardinality attributes, the database engine executes high-speed bitwise logical operations (AND, OR, NOT) directly inside memory, avoiding expensive row-by-row scanning.

Customer information				Bitmap index	A table counting 0s in bytes	
Name	...	Gender	...	Gender	Byte	Number of 0s
Ada	...	Female	...	0	00000000	8
Bob	...	Male	...	1	00000001	7
Cathy	...	Female	...	0	00000010	7
Dan	...	Male	...	1	00000011	6
Elsa	...	Female	...	0	...	...
Flora	...	Female	...	0	01010011	4
George	...	Male	...	1	...	...
Hogan	...	Male	...	1	...	...
...	...	...	...	...	...	...

✿ Join Index

Traditional database systems must perform costly multi-table joins at runtime to connect foreign keys in a fact table to the descriptive fields in dimension tables. A join index eliminates this operational bottleneck by precomputing and storing the join relationships directly inside a specialized index structure. It maps the descriptive attribute values located inside external dimension tables directly to the internal row identifiers (or primary keys) of matching tuples inside the central fact table.



✿ Storage Implementation: Column-Based Databases

To optimize the high-volume aggregation queries characteristic of OLAP systems, contemporary data warehouse storage layers frequently adopt a column-based architecture rather than a traditional row-based format.

- **Row-Based Databases:** Store all attributes of a single database record consecutively within physical disk blocks. This design is optimal for OLTP applications that frequently write or modify individual rows. However, an OLAP query computing the total sales across a single column must read all irrelevant attributes (e.g., names, descriptions, addresses) into memory, causing major I/O bottlenecks.
- **Column-Based Databases:** Store all values for a single attribute consecutively on disk. This column-oriented isolation provides substantial database benefits:
 - ◆ **I/O Efficiency:** An analytical aggregation query fetches only the precise data blocks corresponding to the requested attributes, drastically reducing disk I/O overhead.
 - ◆ **Advanced Compression:** Storing data of identical types consecutively (e.g., repeating sequences of product codes or dates) allows compression algorithms (like dictionary encoding or run-length encoding) to operate with exceptionally high compression ratios.

Data Cube Computation

The primary engineering challenge of data warehousing is the physical materialization and computational tracking of the multidimensional cuboid lattice.

✿ Terminology of Data Cube Computation

- **Cell:** A tuple in a cuboid, which represents a point in the data cube space. A cell in the base cuboid is a
- **Base cell:** Cell in the base cuboid.
- **Aggregate cell:** Cell from a nonbase cuboid.

✿ Data Cube Materialization: Ideas

Materialization: The physical precomputation and storage of data cube cuboids on disk to minimize query response latency.

When choosing a materialization strategy, database engineers evaluate a critical trade-off between storage constraints and query processing costs. Rather than performing full precomputation, systems use optimization algorithms (such as greedy algorithms) to identify the ideal subset of cuboids to materialize under a specified storage budget. If an unmaterialized cuboid is queried at runtime, the execution engine dynamically computes it by scanning the closest, most efficient materialized ancestor cuboid in the lattice.

- **Full Materialization:** Precomputes and physically stores every single cuboid within the multidimensional lattice. While this approach ensures instantaneous response times for all queries, it triggers an exponential explosion in storage requirements as the number of dimensions increases.

- **Partial Materialization**: Strategically precomputes only a carefully selected subset of cuboids. Cuboids chosen for partial materialization typically feature high query frequencies or offer substantial computational savings when deriving unmaterialized child views at runtime.
- **No materialization**: Do not precompute any of the “nonbase” cuboids. This leads to computing expensive multidimensional aggregates **on-the-fly**, which can be extremely **slow**.

✿ OLAP Server Architectures: ROLAP vs. MOLAP vs. HOLAP

Different database architectures handle data cube materialization using distinctive storage abstractions:

- **ROLAP (Relational OLAP)**: Uses standard relational database engines to manage the data cube lattice. Aggregations are stored as standard relational tables within a star or snowflake schema. ROLAP systems **scale exceptionally well** to large volumes of data and handle high-cardinality dimensions efficiently. However, they rely on **complex multi-table joins and recursive query optimization** to simulate multidimensional operations.
- **MOLAP (Multidimensional OLAP)**: Stores aggregated data cube cuboids directly within optimized, specialized multidimensional array data structures. By mapping cell locations to physical memory offsets, MOLAP eliminates table joins entirely, delivering fast query performance. However, MOLAP structures **struggle with data sparsity** (cubes where a large percentage of cells contain zero or null values), which can lead to **inefficient storage utilization**.
- **HOLAP (Hybrid OLAP)**: Combines the scalable storage capacity of ROLAP with the fast query performance of MOLAP. In a HOLAP architecture, the massive, highly detailed base transactions are retained in **relational tables** (ROLAP), while the highly summarized, frequently accessed upper-level cuboids are stored in high-performance **multidimensional arrays** (MOLAP).

Specialized SQL servers: Some database system vendors implement specialized SQL servers that provide advanced query language and query processing support for SQL queries over star and snowflake schemas in a readonly environment.

✿ General Strategies for Data Cube Computation

Although ROLAP and MOLAP may each explore different cube computation techniques, some optimization techniques are popularly used:

- **Sorting, hashing and grouping optimization**: Sorting, hashing, and grouping operations should be applied to the dimension attributes to reorder and cluster related tuples.
- **Simultaneous aggregation and caching of intermediate results**: It is efficient to compute higher-level aggregates from previously computed lower-level aggregates, rather than from the base fact table, since the number of tuples of higher-level aggregates is far less than the number of tuples at the base fact table
- **Aggregation from the smallest child when there exist multiple child cuboids**: It is usually more efficient to compute the desired parent (i.e., more generalized) cuboid from the smallest in size, previously computed child cuboid

- The downward antimonotonicity can be used to prune search space in iceberg cube computation ??????????????

Data cube computation methods

Data cube computation is an essential task in data warehouse implementation. The precomputation of all or part of a data cube can greatly reduce the response time and enhance the performance of online analytical processing. However, such computation is challenging because it may require substantial computational time and storage space.

✿ Multiway Array Aggregation for Full Cube Computation

Multiway Array Aggregation (MWA) is a bottom-up, array-based algorithm designed specifically for full cube computation within MOLAP systems. It operates directly on multidimensional array structures rather than relational tables. MultiWay constructs an array-based cube as follows:

1. **Partitioning via "Chunking"**: The full multidimensional space is partitioned into smaller, memory-manageable blocks called **chunks**. A chunk is a sub-array that fits completely within the system's main memory cache.
2. **The Array-Based Ordering Strategy**: To minimize the number of times a chunk must be read from disk into memory, the algorithm computes aggregates along multiple dimensions simultaneously. It visits chunks in a specific sequence that optimizes cache reuse, known as **multiway aggregation**.
3. **Memory Minimums**: By ordering the chunk visits appropriately, the algorithm minimizes the amount of memory required to hold intermediate aggregation results.

✿ BUC: Computing Iceberg Cubes from the Apex Cuboid Downward

BUC (Bottom-Up Computation) is an algorithm that, despite its name, actually computes a data cube using a top-down traversal strategy through the cuboid lattice, starting from the apex cuboid (the most aggregated layer) and moving downward toward the base cuboid (the most granular layer). BUC is highly optimized for computing iceberg cubes.

✿ Iceberg Cubes

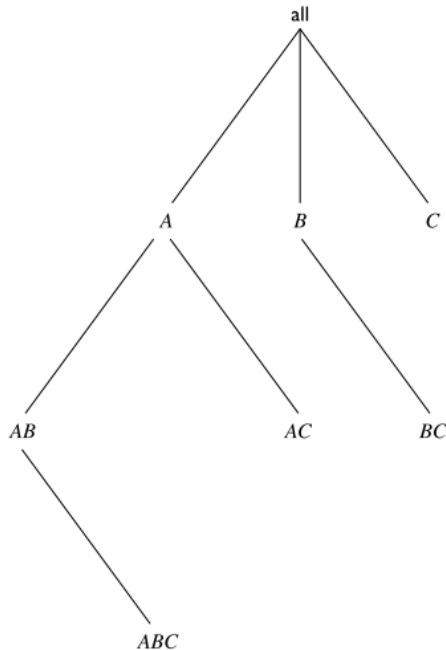
An iceberg cube is a partially materialized data cube that contains only those cells whose aggregate measure satisfies a user-specified threshold condition, known as a **minimum support threshold** or iceberg condition (e.g., $COUNT(*) \geq 100$).

✿ The BUC Algorithm Execution Path

- **Recursive Partitioning**: BUC processes data by sorting and partitioning the input relation along one dimension at a time. It counts the occurrences of each unique attribute value.
- **A Priori Property / Pruning**: BUC applies the fundamental anti-monotonic property of aggregation: if a particular cell at a higher level of abstraction (e.g., $item = "computer"$) fails to satisfy the minimum support threshold, then none of its

descendant cells at a deeper level of granularity (e.g., `item = "computer"` AND `location = "Mendoza"`) can satisfy the threshold.

- **Pruning the Search Space:** When a cell fails the threshold check, BUC immediately stops exploring that branch of the lattice. This sparsity pruning avoids computing massive amounts of low-frequency, uninformative detail, which drastically reduces processing time and disk usage.



BUC's exploration for a 3-D data cube computation. Note that the computation starts from the apex cuboid.

PEDIR EXPLICACIÓN A LA IA

3.5.3 Star-Cubing: An Integration of Top-Down and Bottom-Up Computation

✿ High-Dimensional OLAP: Computing Shell Fragments

When the number of dimensions in an enterprise dataset becomes exceptionally high (e.g., $n > 30$ or $n = 100$), computing a full data cube or even an iceberg cube becomes computationally impossible due to the exponential 2^n barrier, a phenomenon known as the curse of dimensionality. To resolve this limitation, the book introduces Shell Fragment Computation.

- **The Shell Fragment Core Concept:** Instead of attempting to build a massive, single n -dimensional cube structure, the high-dimensional space is systematically pre-partitioned into a set of small, manageable, and overlapping dimensional subsets called shell fragments (e.g., splitting a 30-dimensional space into ten distinct 3-dimensional shell fragments).
- **Partial Materialization of Fragments:** Each individual shell fragment cuboid group is fully computed and materialized. Because each fragment contains only a few dimensions (typically 2 or 3), the materialization costs remain low.
- **Dynamic Query Assembly:** When an analytical query requests an aggregation that spans multiple distinct shell fragments, the OLAP engine does not calculate a new cube from scratch. Instead, it utilizes high-speed inverted index structures and bitwise coordination arrays to dynamically join and aggregate the precomputed shell

fragments at runtime, delivering instantaneous response times even within high-dimensional spaces.

✿ **Efficient processing of OLAP queries using cuboids 2**

1. Determine which operations should be performed on the available cuboids:
2. Determine to which materialized cuboid(s) the relevant operations should be applied:

Capítulo 4 - Data Mining

Pattern Mining: Basic Concepts and Methods

Basic Concepts

The discovery of frequent patterns, associations, and correlation relationships among huge amounts of data is useful in selective marketing, decision analysis, and business management. Essentially, frequent patterns are **patterns that occur frequently in data**. Frequent pattern mining uncovers **recurring relationships in a given data set**.

✿ Market Basket Analysis: A Motivating Example

A typical example of frequent itemset mining is **market basket analysis**. This process analyzes customer buying habits by **finding associations between the different items that customers place in their "shopping baskets"**. The discovery of these associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers. For instance, if customers are buying milk, how likely are they to also buy bread on the same trip to the supermarket? This information can lead to increased sales, revenue, and customer acquisition by helping retailers perform selective marketing and plan shelf space strategically.

✿ Frequent Itemsets, Closed Itemsets, and Association Rules

Let $\mathcal{I} = \{I_1, I_2, \dots, I_m\}$ be an itemset. Let D , the task-relevant data, be a set of database transactions where each transaction T is a nonempty itemset such that $T \subseteq \mathcal{I}$. Each transaction is associated with an identifier, called a *TID*. Let A be a set of items. A transaction T is said to contain A if $A \subseteq T$. An association rule is an implication of the form $A \Rightarrow B$, where $A \subset \mathcal{I}$, $B \subset \mathcal{I}$, $A \neq \emptyset$, $B \neq \emptyset$, and $A \cap B = \emptyset$. The rule $A \Rightarrow B$ holds in the transaction set D with **support** s , where s is the percentage of transactions in D that contain $A \cup B$ (i.e., the *union* of sets A and B say, or, both A and B). This is taken to be the probability, $P(A \cup B)$.² The rule $A \Rightarrow B$ has **confidence** c in the transaction set D , where c is the percentage of transactions in D containing A that also contain B . This is taken to be the conditional probability, $P(B|A)$. That is,

$$\text{support}(A \Rightarrow B) = P(A \cup B) \quad (4.2)$$

$$\text{confidence}(A \Rightarrow B) = P(B|A). \quad (4.3)$$

La fórmula para **calcular $P(B|A)$** es :

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

Ejemplo de caso de uso:

3. Ejemplo Práctico Basado en tus Textos

Imagina un supermercado con 100 tickets totales donde:

- 40 personas compraron Café (Evento A).
- 10 personas compraron Café y Azúcar juntos (Intersección $A \cap B$).

Si queremos calcular $P(\text{Azúcar}|\text{Café})$ (la probabilidad de que compren Azúcar dado que ya sabemos que compraron Café):

$$P(\text{Azúcar}|\text{Café}) = \frac{10}{40} = 0.25 \Rightarrow 25\%$$

Significado: Tu universo se reduce únicamente a las 40 personas que compraron café. De ese grupo selecto, el 25% también llevó azúcar.

El **soporte** mide la **frecuencia global** con la que un conjunto de productos aparece unido dentro de toda la base de datos de transacciones. Responde a la pregunta: *¿Qué tan popular o común es este grupo de objetos en el total de los datos?*

La **confianza** mide la **fuerza predictiva** o **certeza** de la regla. Responde a la pregunta: *De todas las veces que un cliente compró el producto A, ¿qué porcentaje de ellos también terminó comprando el producto B?*

An itemset that contains k items is a **k -itemset**. The set {computer, antivirus_software} is a 2-itemset. The **occurrence frequency of an itemset** is the **number of transactions that contain the itemset**. Occurrence frequency is also referred to as the frequency, support count, or count of the itemset. The itemset support is sometimes referred to as **relative support**, whereas the occurrence frequency is called the **absolute support**.

If the relative support of an itemset I satisfies a prespecified minimum support threshold (or its absolute support satisfies the corresponding minimum support count threshold), then I is a **frequent itemset**. The set of frequent k -itemsets is commonly denoted by L_k . Rules that satisfy both minimum support and minimum confidence are called **strong association rules**.

An itemset X is **closed** in a data set D if there exists no proper superitemset Y such that Y has the same support count as X in D . An itemset X is a **maximal frequent pattern** (or max-pattern) if X is frequent and there exists no superitemset Y such that Y is frequent.

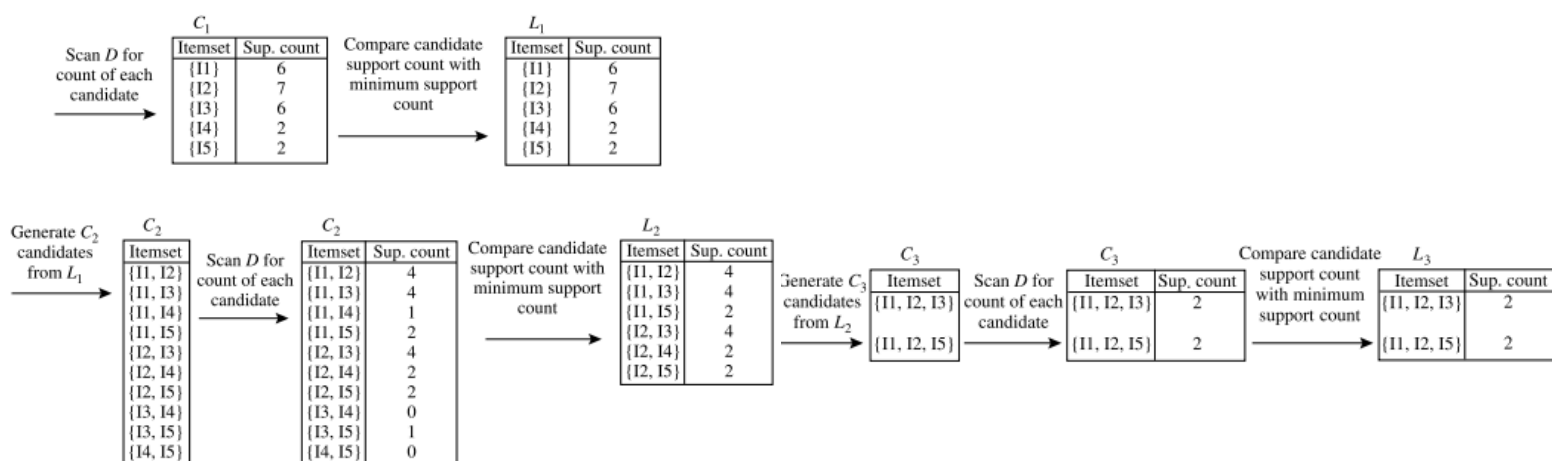
Frequent Itemset Mining Methods

Apriori Algorithm: Confined Candidate Generation

The **Apriori algorithm** is a **seminal algorithm** that finds frequent itemsets using an **iterative level-wise approach** based on candidate generation. **Uses prior knowledge of frequent itemset properties**.

It relies on the **Apriori property**: **All nonempty subsets of a frequent itemset must also be frequent**. The algorithm executes in a sequence of steps:

1. **Join Step**: To find L_k , a set of candidate k -itemsets C_k is generated by joining L_{k-1} with itself.
2. **Prune Step**: Any candidate in C_k that contains a $(k-1)$ -subset that is not in L_{k-1} is immediately pruned from C_k .



✿ Generating Association Rules from Frequent Itemsets

Once the frequent itemsets from a database D have been found, strong association rules can be generated directly from them. The procedure operates as follows:

- For each frequent itemset I , generate all nonempty subsets of I .
- For every nonempty subset s of I , output the rule " $s \rightarrow (I - s)$ " if:

$$\text{confidence}(A \Rightarrow B) = P(B|A) = \frac{\text{support_count}(A \cup B)}{\text{support_count}(A)}.$$

Because the rules are derived from frequent itemsets, they automatically satisfy the minimum support threshold. Frequent itemsets can be stored ahead of time in hash tables along with their counts so that they can be accessed quickly.

Generating association rules

Let's try an example based on the transactional data shown before in Table 4.1. The data contain frequent itemset $X = \{I1, I2, I5\}$. What are the association rules that can be generated from X ? The nonempty subsets of X are $\{I1, I2\}$, $\{I1, I5\}$, $\{I2, I5\}$, $\{I1\}$, $\{I2\}$, and $\{I5\}$. The resulting association rules are as shown below, each listed with its confidence:

$$\begin{aligned} \{I1, I2\} &\Rightarrow I5, & \text{confidence} &= 2/4 = 50\% \\ \{I1, I5\} &\Rightarrow I2, & \text{confidence} &= 2/2 = 100\% \\ \{I2, I5\} &\Rightarrow I1, & \text{confidence} &= 2/2 = 100\% \\ I1 &\Rightarrow \{I2, I5\}, & \text{confidence} &= 2/6 = 33\% \\ I2 &\Rightarrow \{I1, I5\}, & \text{confidence} &= 2/7 = 29\% \\ I5 &\Rightarrow \{I1, I2\}, & \text{confidence} &= 2/2 = 100\% \end{aligned}$$

✿ Improving the Efficiency of Apriori

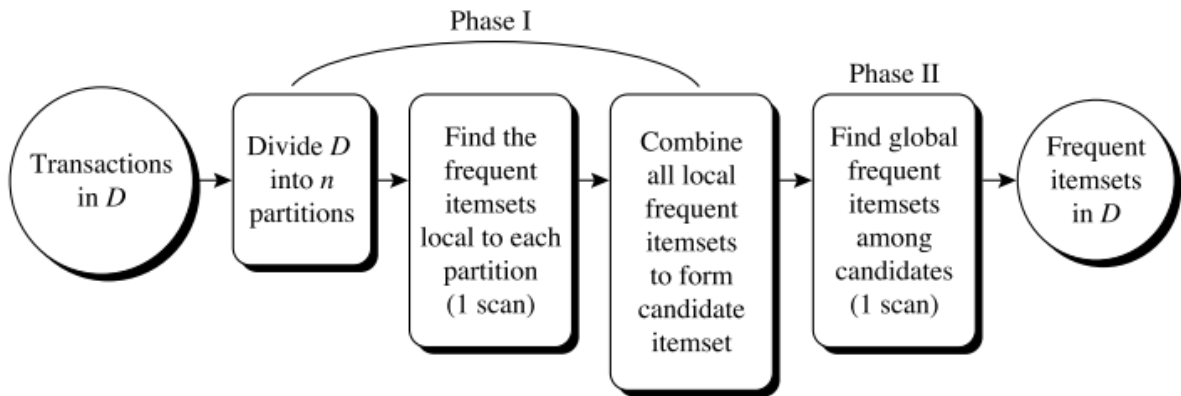
To overcome the overhead of scanning the entire database multiple times and generating vast numbers of candidates, several variations have been proposed:

- **Hash-based technique (Direct Hashing and Pruning - DHP):** Hashes itemsets into buckets during the first scan. Candidates whose corresponding bucket count is below the minimum support threshold can be pruned immediately.

Create hash table H_2 using hash function
 $h(x, y) = ((\text{order of } x) \times 10 + (\text{order of } y)) \bmod 7$

H_2							
bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5} {I1, I5}	{I2, I3} {I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3} {I1, I3}

- **Transaction reduction**: Eliminates transactions that do not contain any frequent itemsets from subsequent scans.
- **Partitioning**: A partitioning technique requires just two database scans to find the frequent itemsets. It divides the database into n distinct partitions. If an itemset is globally frequent, it must be locally frequent in at least one of the partitions.



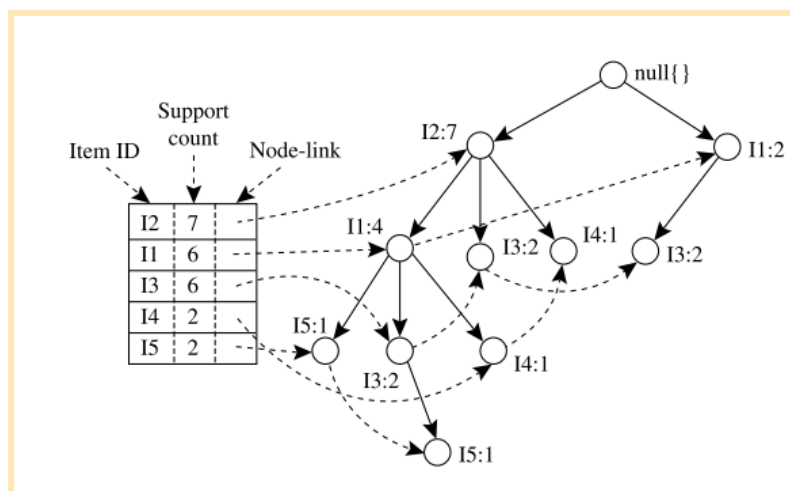
- **Sampling**: Computes frequent itemsets on a random sample of the database, using a lower support threshold to avoid missing patterns.
- **Dynamic itemset counting (DIC)**: Relaxes the step-by-step nature of Apriori, adding candidate itemsets at check-points during a single scan as soon as all of their subsets are estimated to be frequent.

✿ A Pattern-Growth Approach for Mining Frequent Itemsets

The **FP-growth** (Frequent Pattern Growth) algorithm adopts a candidate-free (divide and conquer) approach. It encodes the database into a highly compressed data structure called the **FP-tree** (Frequent Pattern Tree).

The layout preserves item co-occurrence semantics explicitly. Mining is performed via the following process:

1. Build the FP-tree by scanning the database twice: once to find frequent 1-items and sort them in descending support order, and a second time to map each transaction as a path in the tree.
2. For each frequent item, extract its conditional pattern base (prefix paths ending at the item).
3. Construct a conditional FP-tree from the conditional pattern base.
4. Recursively mine the conditional paths. If the tree contains only a single path, frequent patterns are generated by compiling all combinations of the items on that path.



In general, when considering the branch to be added for a transaction, the count of each node along a common prefix is incremented by 1, and nodes for the items following the prefix are created and linked accordingly.

✿ Mining Frequent Itemsets Using the Vertical Data Format

The Eclat (Equivalence Class Transformation) algorithm transforms the traditional horizontal transaction format (TID-itemset) into a vertical data format (item-TID_set).

The support count of any k-itemset is determined simply by intersecting the TID_sets of its (k-1) sub-components. To minimize memory footprint when dealing with long and dense patterns, an optimization called **diffset** can be used. Instead of recording the absolute intersecting TIDs, a diffset records only the differences or TIDs that belong to a prefix but drop out in the longer pattern. This reduces storage and speeds up intersections.

Table 4.3 The vertical data format of the transaction data set *D* of Table 4.1.

itemset	TID_set
I1	{T100, T400, T500, T700, T800, T900}
I2	{T100, T200, T300, T400, T600, T800, T900}
I3	{T300, T500, T600, T700, T800, T900}
I4	{T200, T400}
I5	{T100, T800}

✿ Mining Closed and Max Patterns

Instead of mining all frequent patterns, mining can target **closed patterns** or **max patterns** to achieve **orders-of-magnitude compression without losing support info**. Algorithms like **MaxMiner** perform look-ahead prunings to identify max-patterns early, while **CLOSET** uses a tail-closure property on FP-trees to efficiently identify closed frequent itemsets directly.

Acá cerrá los ojos y confiá

Which Patterns Are Interesting?—Pattern Evaluation Methods

The support–confidence framework can be deceiving and output rules that lack true predictive strength or correlation.

✿ Strong Rules Are Not Necessarily Interesting

Example 4.7: Suppose we analyze 10,000 transactions regarding computer games and videos. 6,000 transactions contain games, 7,500 contain videos, and 4,000 contain both. Under a minimum support of 30% and a minimum confidence of 60%, the rule **game→video**

passes because its support is 40% and its confidence is $4,000 / 6,000 = 66.7\%$. However, the background probability of buying a video is 75%. Knowing that a customer buys a game actually *decreases* the likelihood that they will buy a video (from 75% down to 66.7%). Thus, the rule is misleading and negatively correlated.

✿ From Association Analysis to Correlation Analysis

To remedy the limitations of the support–confidence framework, correlation measures are augmented to form correlation rules of the form $A \rightarrow B$ [support, confidence, correlation].

✿ Lift

Lift evaluates the ratio of the joint probability of A and B to their independent probabilities:

$$\text{lift}(A,B) = P(A \cup B) / P(A)P(B)$$

- If $\text{lift}(A,B) < 1$, A and B are negatively correlated.
- If $\text{lift}(A,B) > 1$, A and B are positively correlated.
- If $\text{lift}(A,B) = 1$, A and B are independent.

From the table, we can see that the probability of purchasing a computer game is $P(\{game\}) = 0.60$, the probability of purchasing a video is $P(\{video\}) = 0.75$, and the probability of purchasing both is $P(\{game, video\}) = 0.40$. By Eq. (4.8), the lift of Rule (4.6) is $P(\{game, video\}) / (P(\{game\}) \times P(\{video\})) = 0.40 / (0.60 \times 0.75) = 0.89$. Because this value is less than 1, there is a negative correlation between the occurrence of $\{game\}$ and $\{video\}$. The numerator is the likelihood of a customer purchasing both, whereas the denominator is what the likelihood would have been if the two purchases were completely independent. Such a negative correlation cannot be identified by a support–confidence framework. □

✿ Chi-Square (χ^2)

χ^2 tests the statistical independence between variables by comparing observed frequencies against expected frequencies across a contingency table. However, χ^2 is heavily influenced by the total transaction count-1 and can fluctuate wildly when large numbers of null-transactions are introduced.

Example 4.9. Correlation analysis using χ^2 . To compute the correlation using χ^2 analysis for nominal data, we need the observed value and expected value (displayed in parenthesis) for each slot of the contingency table, as shown in Table 4.7. From the table, we can compute the χ^2 value as follows:

$$\begin{aligned} \chi^2 = \sum \frac{(\text{observed} - \text{expected})^2}{\text{expected}} &= \frac{(4000 - 4500)^2}{4500} + \frac{(3500 - 3000)^2}{3000} \\ &+ \frac{(2000 - 1500)^2}{1500} + \frac{(500 - 1000)^2}{1000} = 555.6. \end{aligned}$$

Because the χ^2 value is greater than 1, and the observed value of the slot $(game, video) = 4000$, which is less than the expected value of 4500, *buying game* and *buying video* are *negatively correlated*. This is consistent with the conclusion derived from the analysis of the *lift* measure in Example 4.8. □

✿ A Comparison of Pattern Evaluation Measures

A measure is defined as **null-invariant** if its value is completely free from the influence of null-transactions (transactions that contain neither of the itemsets under analysis). In massive datasets (such as web store logs), the count of null-transactions easily dominates, making null-invariance a crucial property.

The textbook examines six prominent measures:

1. Lift: Not null-invariant.
2. χ^2 : Not null-invariant.
3. All_confidence: Null-invariant. Defined as:

Given two itemsets, A and B , the **all_confidence** measure of A and B is defined as

$$all_conf(A, B) = \frac{sup(A \cup B)}{\max\{sup(A), sup(B)\}} = \min\{P(A|B), P(B|A)\},$$

4. **Max_confidence**: Null-invariant. Measures the maximum confidence of the rules $A \rightarrow B$ and $B \rightarrow A$.

$$max_conf(A, B) = \max\{P(A|B), P(B|A)\}.$$

5. **Cosine**: Null-invariant. Calculated as:

$$\begin{aligned} cosine(A, B) &= \frac{P(A \cup B)}{\sqrt{P(A) \times P(B)}} = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}} \\ &= \sqrt{P(A|B) \times P(B|A)}. \end{aligned}$$

6. **Kulczynski (Kulc)**: Null-invariant. Calculated as the arithmetic mean of the two conditional probabilities:

$$Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A)).$$

The book strongly recommends using the **Kulczynski measure** in combination with the **Imbalance Ratio (IR)** to evaluate patterns explicitly. The **Imbalance Ratio** quantifies any asymmetry in support between two itemsets:

$$IR(A, B) = \frac{|sup(A) - sup(B)|}{sup(A) + sup(B) - sup(A \cup B)},$$

- $IR = 0$ represents a perfectly balanced case.
- Higher values approaching 1 signify highly skewed, imbalanced distributions where one item is far more common than the other. Kulc remains steady and neutral under severe imbalances, whereas IR reveals the implicit direction of the skewness.

Capítulo 6 - Data Mining

Classification: Basic Concepts and Methods

Basic Concepts

✿ What is Classification?

Classification is a foundational form of data analysis that extracts models describing critical data classes. A **classifier**, or classification model, is built to predict categorical class labels (which are discrete and unordered). This process structurally contrasts with numeric prediction, which models continuous-valued functions to forecast missing or future numerical data values rather than explicit categorical labels.

✿ General Approach to Classification

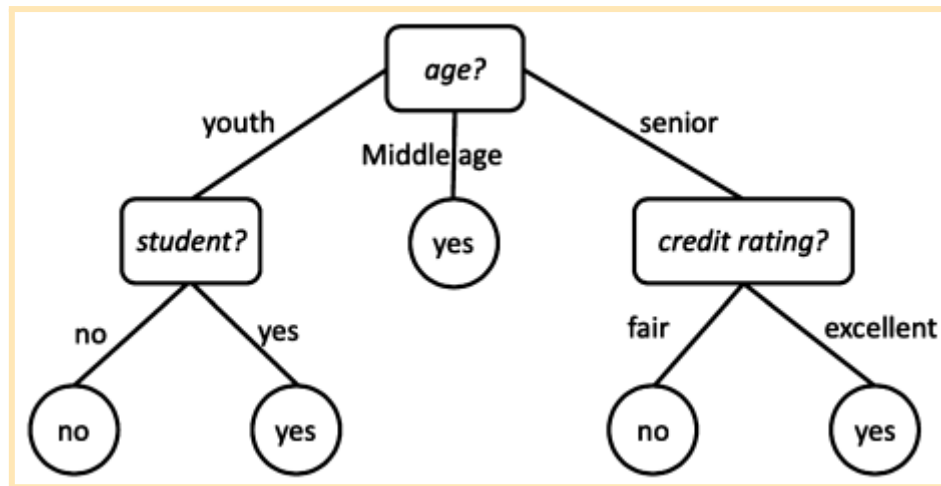
The data classification workflow is explicitly defined as a **two-step process**:

- **The Learning Step (Model Construction)**: A classifier is built describing a predetermined set of data classes or concepts.
 - ◆ A classification algorithm analyzes a designated training set consisting of database tuples that possess pre-assigned class labels. Each individual tuple, denoted as a multi-dimensional attribute vector, is assumed to belong to a predefined class dictated by a distinct **class label attribute**. Because the training tuples have known class labels, this mapping phase is classified as **supervised learning**. It contrasts directly with *unsupervised learning* (or clustering), where the labels of the training tuples are entirely unknown beforehand.
- **The Classification Step (Model Usage)**: The constructed model is evaluated and utilized to classify future or previously unseen data tuples.
 - ◆ **Classifier Accuracy Estimation**: Before a model is deployed for unseen instances, its predictive quality must be evaluated. The accuracy of a classifier on a given test set is defined as the **exact percentage of test tuples that are correctly mapped to their true class labels by the model**.
 - ◆ To ensure an unbiased estimation of performance, the test set must be entirely independent of the training data set; otherwise, the evaluation will exhibit over-optimism.

Decision Tree Induction

Decision tree induction is the automatic mapping and learning of flowchart-like decision tree structures from class-labeled training tuples. Within this structure:

- Each **internal node** (nonleaf node) explicitly denotes a test or conditional check on a specific database attribute.
- Each **branch** emanating from an internal node represents an **explicit operational outcome of that test**.
- Each **leaf node** (terminal node) holds a **final class label assignment**.
- The topmost initial node is the **root node**.



✿ Decision Tree Induction Algorithm

The canonical strategy for generating a decision tree is a top-down, greedy recursive divide-and-conquer algorithm. Standard frameworks like **ID3**, **C4.5**, and **CART** follow this backbone.

The core algorithm, typically named `Generate_decision_tree`, operates on a data partition D :

1. **Tree Initialization:** The algorithm creates a node N to represent the current partition D .
2. **Base Case Checks (Termination Criteria):**
 - If all tuples in D belong to exactly the same class C , node N is immediately turned into a leaf node and labeled with class C .
 - If the attribute list is empty (no remaining features to test), node N becomes a leaf node labeled with the majority class found among the tuples in D .
3. **Splitting Criterion Determination:** The algorithm invokes an attribute selection measure to determine the optimal splitting criterion. This criterion explicitly determines the single feature to test (the splitting attribute) and how to partition the data.
4. **Partitioning Rules:** Based on the attribute type, branching happens as follows:
 - **Discrete-valued Attribute:** Branches are grown for each individual known value of the attribute.
 - **Continuous-valued Attribute:** A threshold split point is calculated; two branches are grown matching the conditions $A \leq \text{Split_point}$ and $A > \text{Split_point}$.
 - **Discrete-valued (Binary Tree Restriction):** Formulated as a set inclusion test: " $A \in S_A?$ ", where S_A is the splitting subset for A , returned by `Attribute_selection_method` as part of the splitting criterion.
5. **Recursive Execution:** The algorithm partitions the data into child subsets and calls itself recursively for each child subset until the base cases are met.

✿ Attribute Selection Measures

An attribute selection measure is a heuristic rule utilized to select the attribute that "best" separates a given data partition D of training tuples into pure individual classes.

✿ Information Gain (Used in ID3)

Based on Claude Shannon's information theory, **Information Gain** minimizes the expected total number of bits required to classify an unseen tuple. It measures the expected reduction in **entropy** (impurity).

The attribute with the highest information gain is chosen as the splitting attribute for node N.

The **expected information** (entropy) needed to classify a tuple in partition D is calculated as:

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

Where p_i is the non-zero probability that an arbitrary tuple in D belongs to class C_i (estimated by $|C_i \cap D| / |D|$).

The expected information requirement after partitioning D into v distinct subsets by attribute A (conditional entropy) is defined as:

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Este criterio se fundamenta en la teoría de la información y utiliza el concepto de Entropía, que mide el grado de desorden, aleatoriedad o impureza de un conjunto de datos.

¿Cómo funciona? Primero calcula la entropía del nodo actual (cuánta mezcla de clases hay). Luego, calcula la entropía combinada si se dividieran los datos usando un atributo específico. La *Ganancia de Información* es simplemente la *reducción neta de la entropía*:

$$Gain(D, A) = Entropy(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} Entropy(D_v)$$

Information gain is **heavily biased** toward choosing attributes with a large number of distinct values

✿ Gain Ratio (Used in C4.5)

To correct the multi-value bias of Information Gain, C4.5 utilizes the **Gain Ratio**. It applies a **normalization technique** called **Split Information**, which measures the potential entropy generated by splitting the training data partition into v partitions:

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right).$$

The Gain Ratio scales the standard gain:

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}$$

Nuance & Edge Cases: The attribute that maximizes the Gain Ratio is chosen as the splitting feature. However, as $\text{SplitInfo}_A(D)$ approaches 0, the ratio can become unstable or blow up. To safeguard against this, a standard constraint requires that the Information Gain of the tested attribute must be at least as large as the average gain of all examined attributes.

¿Cómo funciona? Introduce una métrica llamada Split Information (Información de División), que mide el "costo potencial" de segmentar el conjunto basándose en la cantidad y tamaño de las ramas generadas:

$$\text{SplitInfo}_A(D) = - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \log_2 \left(\frac{|D_v|}{|D|} \right)$$

Luego, el *Gain Ratio* normaliza la ganancia dividiéndola por este costo:

$$\text{GainRatio}(A) = \frac{\text{Gain}(D, A)}{\text{SplitInfo}_A(D)}$$

Si un atributo divide los datos en miles de ramas idénticas (como el `ID_Cliente`), su *SplitInfo* será extremadamente alto, aplastando el valor del ratio y previniendo que sea seleccionado.

✿ Gini Index (Used in CART)

The Gini Index measures data impurity by analyzing the squared class probabilities:

$$\text{Gini}(D) = 1 - \sum_{i=1}^m p_i^2,$$

where p_i is the probability that a tuple in D belongs to class C_i and is estimated by $|C_{i,D}|/|D|$. The sum is computed over m classes.

If attribute A is used to split partition D into two parts, D_1 and D_2 (enforcing a binary tree structure), the Gini index of that split is:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2).$$

The reduction in impurity is measured by

$$\Delta \text{Gini}(A) = \text{Gini}(D) - \text{Gini}_A(D).$$

The attribute and specific binary split that minimizes $\text{Gini}_A(D)$ is chosen as the splitting criterion.

These measures are not without their biases. Information gain, as we saw, is biased toward multivalued attributes. Although the gain ratio adjusts for this bias, it tends to prefer unbalanced splits in which one partition is much smaller than the others. The Gini impurity is biased toward multivalued attributes and has difficulty when the number of classes is large.

A diferencia de los dos anteriores, el índice Gini no se apoya en conceptos logarítmicos o de la teoría de la información, sino que mide la probabilidad de clasificar incorrectamente un elemento elegido al azar si se le asignara una etiqueta basándose en la distribución de clases del conjunto.

- ¿Cómo funciona? Evalúa la impureza de un conjunto mediante la ecuación:

$$\text{Gini}(D) = 1 - \sum_{i=1}^m (p_i)^2$$

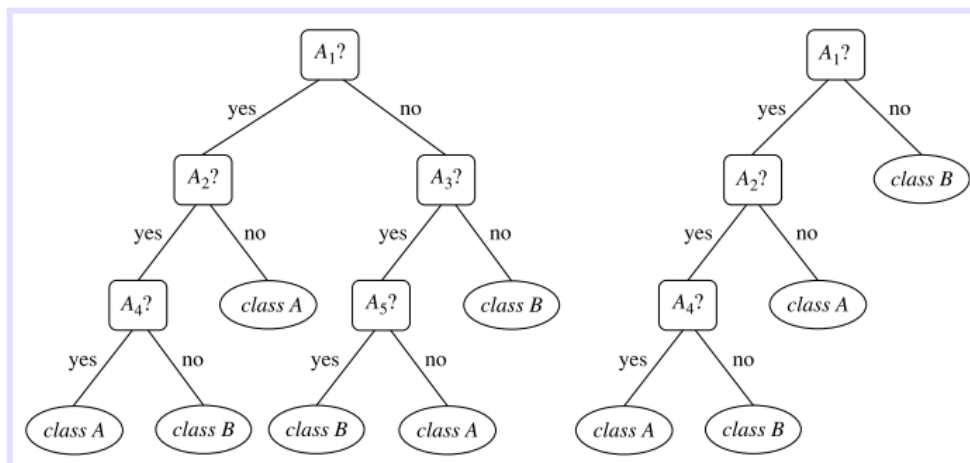
Donde p_i es la probabilidad relativa de que un registro pertenezca a la clase i . Cuando un nodo se divide mediante un atributo A , se calcula el Gini ponderado de los subconjuntos resultantes, buscando aquel atributo que maximice la reducción del índice Gini (es decir, el que deje los nodos hijos con el menor valor de Gini posible, siendo 0 la pureza absoluta).

✿ Tree Pruning

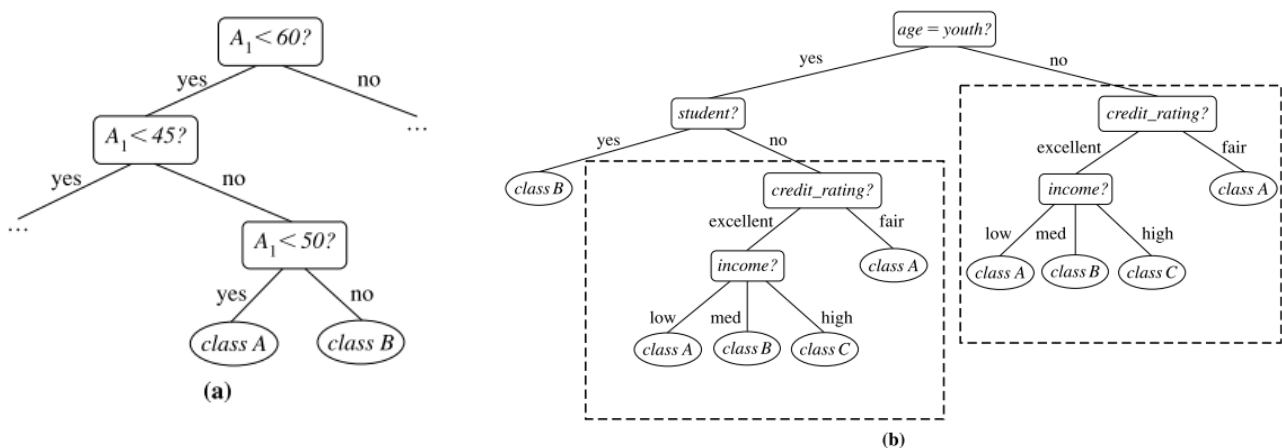
When a decision tree is fully grown, it often suffers from overfitting. This happens when the algorithm fits anomalies, outliers, or background noise present in the training partition, leading to poor predictive generalization on unseen test data. Tree pruning removes lower branches that reflect noise or data anomalies to optimize test performance.

✿ Pruning Strategies

1. **Prepruning (Early Halting):** The algorithm halts the tree expansion process early if the chosen attribute selection measure falls below a specified threshold, or if the sample count within a node becomes too small. The node is immediately terminated as a leaf holding the majority class.
 - **Drawback:** High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification. .



2. **Postpruning (Trimming After Growth):** The tree is allowed to grow to its maximum size (until all leaves are pure or empty). The algorithm then works backward from the bottom up, evaluating internal nodes. If removing a subtree and replacing it with a simple leaf node does not significantly increase estimated error, the subtree is pruned.
 - **Cost-Complexity Pruning (CART):** Evaluates a trade-off between the tree's size (number of leaves) and its error rate.
 - **Pessimistic Error Pruning (C4.5):** Uses a statistical correction factor to estimate errors on unseen data directly from the training set, eliminating the absolute necessity of maintaining a distinct validation dataset.



An example of (a) subtree **repetition**, where an attribute is repeatedly tested along a given branch of the tree (e.g., age), and (b) subtree **replication**, where duplicate subtrees exist within a tree (e.g., the subtree headed by the node “ $credit_rating?$ ”).

Bayes Classification Methods

Bayesian classifiers are statistical classifiers that calculate the posterior probability that a given data tuple belongs to a specific class.

✿ Bayes' Theorem

Let X be a data tuple whose class label is unknown. Let H be the hypothesis that X belongs to a specific class C .

$P(H|X)$: Probability that the hypothesis H holds given the “evidence” or observed data tuple X . This is the **posterior probability**. “Then $P(H|X)$ reflects the probability that customer X will buy a computer given that we know the customer’s age and income.”

$P(H)$: **Prior probability**. “This is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter.”

$P(X)$: **Prior probability** of X . “En este caso, serían las probabilidades de que el cliente X tenga las características que aseguran que el cliente va a comprar una computadora”

$P(X|H)$: **Conditional probability**. “Probabilidad de que el cliente que va a comprar una computadora tenga las características”

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

In theory, Bayes classifier is optimal in the sense that it has the smallest classification error rate compared to all other classifiers. However, since Bayes classifier always predicts the class with the maximum posterior probability, the probability that its prediction is wrong for a given tuple X (which is often called risk) is the lowest in comparison to all other classifiers.

✿ Naïve Bayesian Classification

The Naïve Bayesian classifier assumes class-conditional independence. It assumes that the effect of an attribute value on a given class is completely independent of the values of all other attributes. This assumption is considered "naïve" because it ignores potential dependencies among attributes in real-world data.

✿ Mathematical Formulation

Given a set of m classes $C_1, C_2, \dots, C_m$, the classifier predicts that an unlabelled tuple $X = (x_1, x_2, \dots, x_n)$ belongs to the class C_i that maximizes the posterior probability in $P(C_i|X)$.

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}$$

Using the class-conditional independence assumption, the conditional probability $P(X|C_i)$ is simplified to the product of individual attribute probabilities:

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i)$$

✿ Probability Estimation Rules

- **Categorical Attributes:** $P(x_k|C_i)$ is estimated as the number of tuples in class C_i that have the value x_k , divided by the total number of tuples belonging to class C_i .
- **Continuous Attributes:** The attribute is modeled as a continuous variable following a Gaussian (Normal) distribution with a mean μ and standard deviation σ :

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i})$$

✿ The Zero-Probability Problem & Laplacian Correction

If an attribute value x_k does not appear alongside a class label C_i anywhere in the training set, the resulting conditional probability calculation yields $P(x_k|C_i) = 0$. Because the overall product is multiplicative, this single zero zero-out the entire joint probability product $P(X|C_i)$, destroying the influence of all other predictive attributes.

To fix this, the Laplacian Correction (or Laplace smoothing) adds a small integer count to both the numerator and denominator during probability estimation. Assuming the training set for class C_i has q tuples, and the attribute has v distinct states, we adjust the probability calculation:

$$P(x_k|C_i) = \frac{\text{count}(x_k) + 1}{q + v}$$

Lazy Learners (or Learning from Your Neighbors)

The classifiers discussed so far (such as decision trees and Bayesian networks) are eager learners. Eager learners use the training tuples to construct a generalized model *before* receiving any new or unseen test instances.

In contrast, **lazy learners** store the training data tuples and defer model generalization until an explicit unlabeled test tuple needs to be classified.

✿ k-Nearest-Neighbor Classifiers

Nearest-neighbor classifiers are **instance-based learners** that represent tuples as points in an n-dimensional Euclidean pattern space.

“comparing a given test tuple with training tuples that are similar to it.”

✿ Operational Method

When presented with an unlabeled test tuple X, the classifier searches the pattern space for the k training tuples that are **closest to X** based on a distance metric, such as the **Euclidean distance**.

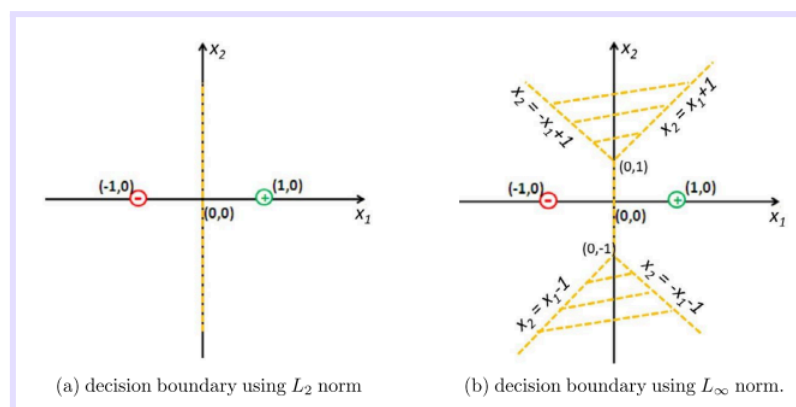
The Euclidean distance between two multi-dimensional points $X_1 = (x_{11}, x_{12}, \dots, x_{1n})$ and $X_2 = (x_{21}, x_{22}, \dots, x_{2n})$ is calculated as:

$$\text{dist}(X_1, X_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

Funciona como cube en SQL

Attribute Processing Restrictions

- **Normalization Requirement:** Distance calculations can be dominated by attributes with large initial numeric ranges (e.g., **annual_income** vs. **age**). To prevent this, numeric features must be normalized to a uniform scale (such as 0.0 to 1.0) prior to distance computation.
- **Handling Categorical Attributes:** For nominal attributes, a simple matching metric is applied: the distance is set to 0 if the values match exactly, and 1 if they differ.



✿ Case-Based Reasoning (CBR)

Case-based reasoning (CBR) extends instance-based learning by using structured, complex symbolic representations rather than simple numeric vectors.

Operational Workflow

1. When a new problem case is encountered, the CBR engine searches its database for stored cases that match or resemble the new instance.
2. If a closely matching case is found, its solution is retrieved. If the match is not perfect, the system attempts to adapt the historical solution to fit the nuances of the new case.
3. If the adapted solution proves successful in practice, the new experience is saved as a new case in the database, expanding the system's knowledge base.

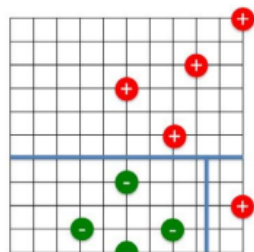
Linear Classifiers

Linear classifiers make class predictions by computing a linear combination of the input attributes' values.

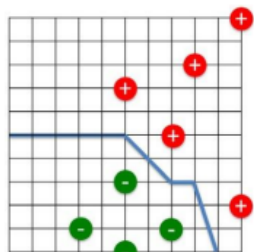
✿ Linear Regression

Although traditionally used for numeric prediction, linear regression can be adapted for binary classification. It models a straight-line relationship between a continuous independent variable x and a dependent variable y : $y = w x + b$

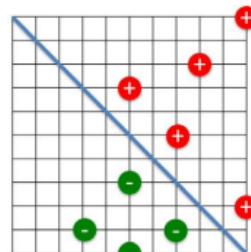
Where w represents the regression coefficient (weight) and b is the bias or intercept. These parameters are optimized using the least-squares method, which minimizes the sum of squared errors between the actual data points and the modeled line.



(a) Decision Tree Classifier



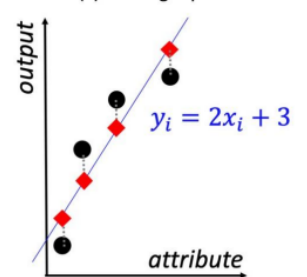
(b) 1-Nearest-Neighbor Classifier



(c) Linear Classifier

Index (i)	1	2	3	4
Attribute (x_i)	1	3	5	7
Output (y_i)	4	10	14	16

(a) Training tuples



(b) Least square regression

Para los curiosos:

- To find the slope (w):

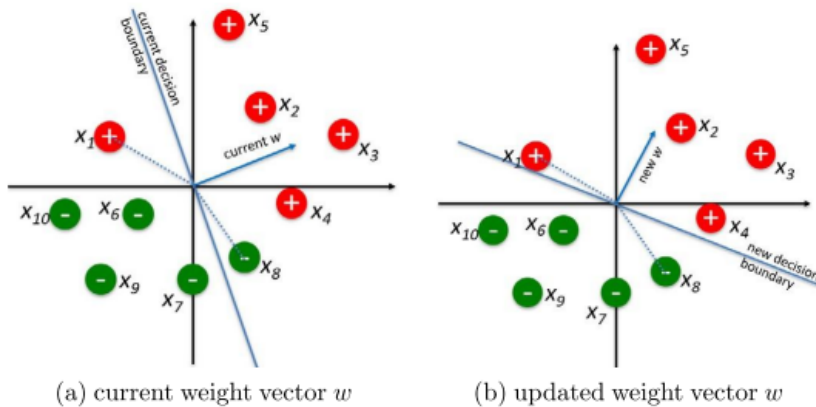
$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

- To find the intercept (b):

$$b = \bar{y} - w\bar{x}$$

✿ Perceptron

The **perceptron** is a simple, **single-layer neural network** structure used for **binary classification**. It takes an input vector x , applies a weight vector w , and adds a bias term b to compute a linear combination:



✿ Logistic Regression

Instead of predicting a discrete binary value directly like the perceptron, **logistic regression** models the **probability** that a given tuple belongs to a specific class. It passes the linear combination $z = w^T x + b$ through the **logistic function** (or sigmoid function):

$$P(\text{class} = 1|x) = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}.$$

The output $\sigma(z)$ is bound between 0 and 1, representing a continuous probability.

Model Evaluation and Selection

You may even have tried different methods to build more than one classifier and now wish to compare their accuracy. *But what is accuracy? How can we estimate it?* Are some measures of a classifier's accuracy more appropriate than others? How can we obtain a reliable accuracy estimate? These questions are addressed in this section.

✿ Metrics for Evaluating Classifier Performance

To comprehensively evaluate the performance of a classifier, its **predictions** are **cross-referenced with true labels** and organized into a **confusion matrix**.

Actual \ Predicted	Class = Yes (Positive)	Class = No (Negative)

Class = Yes (Positive)	TP (True Positive)	FN (False Negative)
Class = No (Negative)	FP (False Positive)	TN (True Negative)

P is the number of positive tuples, and N is the number of negative tuples.

✿ Performance Formulations

Measure	Formula
accuracy, recognition rate	$\frac{TP + TN}{P + N}$
error rate, misclassification rate	$\frac{FP + FN}{P + N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP + FP}$
F, F_1, F -score, harmonic mean of precision and recall	$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
F_β , where β is a nonnegative real number	$\frac{(1 + \beta^2) \times \text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$

✿ Holdout Method and Random Subsampling

- **Holdout Method:** The original labeled dataset is partitioned into two mutually exclusive, independent subsets: a training set (typically comprising 2/3 to 7/8 of the data) used to build the model, and a test set used exclusively to estimate accuracy.
- **Random Subsampling:** An extension of the holdout method where the data partitioning process is repeated k times. The final accuracy estimation is computed as the average of the accuracies obtained from each individual iteration.

✿ Cross-Validation

In k-fold cross-validation, the initial data are randomly partitioned into k mutually exclusive subsets or “folds” $D_1, D_2, \dots, D_k$, each of approximately equal size.

- The classification algorithm runs k times. In the i-th iteration, fold D_i serves as the validation test set, while the remaining k-1 folds are combined to form the training set.
- The final performance estimate is the average accuracy across all k iterations.
- **Stratified Cross-Validation:** Folds are balanced so that the class distribution in each subset matches the class distribution in the overall dataset.

✿ Bootstrap

The **bootstrap** method samples the training tuples uniformly with replacement from the original dataset D.

The most common implementation is the .632 bootstrap:

- For a dataset containing d tuples, the system samples d times with replacement to construct a training set.
- The probability of a specific tuple *not* being selected in any of the d draws is $(1-1/d)^d \approx e^{-1} \approx 0.368$.
- Consequently, roughly 63.2% of the original unique tuples end up in the training sample, while the remaining 36.8% form the test set.
- The final accuracy is weighted as follows:

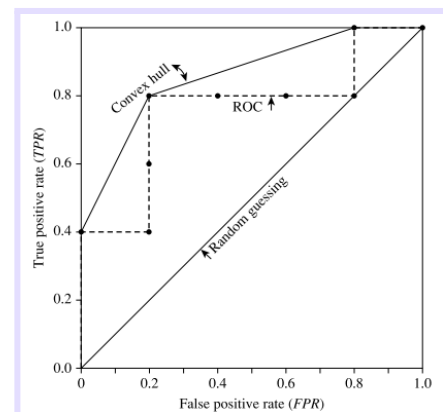
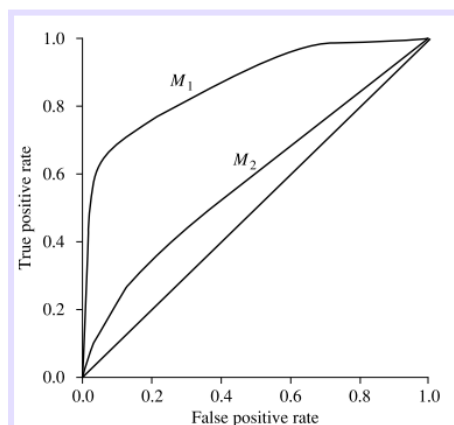
$$Acc(M) = \frac{1}{k} \sum_{i=1}^k (0.632 \times Acc(M_i)_{test_set} + 0.368 \times Acc(M_i)_{train_set}).$$

✿ Model Selection Using Statistical Tests of Significance

To determine whether the observed performance difference between two models is statistically meaningful or simply due to chance, researchers use significance tests. For cross-validation pairs, a paired t-test computes a t-statistic to evaluate the null hypothesis that the mean error difference between the two classifiers is zero.

✿ Comparing Classifiers Based on Cost-Benefit and ROC Curves

- **ROC Curves:** A Receiver Operating Characteristic (ROC) curve maps the trade-off between a classifier's true positive rate and false positive rate. The vertical axis plots Sensitivity (True Positive Rate), and the horizontal axis plots 1 - Specificity (False Positive Rate).

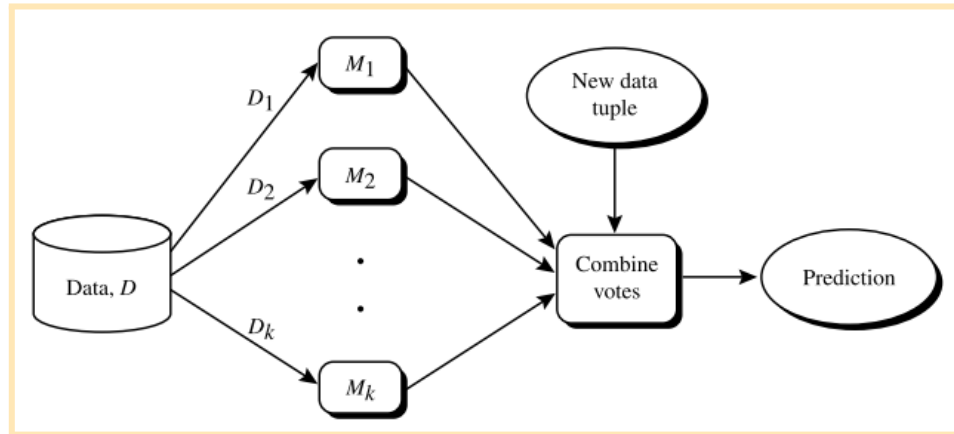


- **Area Under the Curve (AUC):** Measures the overall diagnostic quality of the model. A perfect classifier achieves an $AUC = 1.0$, while a completely random guess yields an $AUC = 0.5$.

Techniques to Improve Classification Accuracy

✿ Introducing Ensemble Methods

Ensemble methods combine a series of individual component models (base classifiers) to create a unified meta-classifier that often achieves higher overall accuracy than any of its components. Bagging, boosting, and random forests are examples of ensemble methods.



✿ Bagging

Bagging (Bootstrap Aggregation) trains multiple classifiers on different bootstrap samples of the data.

Operational Workflow

1. Given a training set D containing d tuples, the algorithm creates k independent bootstrap subsets ($D_1, D_2, \dots, D_k$) by sampling with replacement.
2. A base classifier (e.g., an unpruned decision tree) is trained on each subset D_i .
3. When classifying an unseen instance X , the ensemble aggregates the predictions of all k classifiers. For categorical labels, it uses a majority vote to determine the final class.

✿ Boosting

Unlike Bagging, where components are trained in parallel, Boosting is an iterative, sequential ensemble technique. Each new classifier focuses on the instances that were misclassified by the preceding models.

AdaBoost (Adaptive Boosting)

1. The algorithm begins by assigning an equal weight ($1/d$) to all training tuples in the dataset D .
2. In each iteration i , a new base classifier M_i is trained.
3. The algorithm computes the misclassification error rate of M_i by summing the weights of the tuples it classified incorrectly.
4. If the error rate exceeds 0.5, the training sequence stops. Otherwise, the algorithm increases the weights of the misclassified tuples and decreases the weights of the correctly classified ones.

5. When classifying an unseen instance, the final prediction is determined by a weighted vote of all component models, where each classifier's voting weight is inversely proportional to its error rate.

✿ Random Forests

A Random Forest is an ensemble composed specifically of decision tree classifiers. It introduces additional randomness during the tree-building process to decorrelate the individual trees:

1. Each tree is grown on an independent bootstrap sample of the training data.
2. Instead of choosing the best split point from all available features at each node, the algorithm randomly selects a small subset of m attributes and evaluates splits *only* within that subset.
3. The trees grow to their maximum size without any pruning. The ensemble determines final classifications using a majority vote.

✿ Improving Classification Accuracy of Class-Imbalanced Data

The class imbalance problem occurs when the main class of interest is represented by only a few tuples in the training dataset, while the alternative classes dominate the distribution.

Remediation Strategies

- **Oversampling:** Replicates minority class tuples to give them greater weight during model training.
- **Undersampling:** Randomly discards majority class tuples to balance the class distribution, though this risks losing valuable information.
- **Threshold Moving:** Adjusts the final classification decision boundary. Instead of using the standard 0.5 probability threshold, it shifts the boundary to favor predictions of the minority class.

6.8 Summary

- **Classification** maps data instances to categorical labels, whereas numeric prediction models continuous functions.
- **Decision tree induction** splits data top-down using heuristics like Information Gain, Gain Ratio, or the Gini Index. Overfitting is controlled using prepruning or postpruning techniques.
- **Naïve Bayesian classification** calculates class probabilities by assuming that attributes are conditionally independent given the class.
- **Linear classifiers** (such as the perceptron and logistic regression) separate classes using weighted linear combinations of the input features.
- Classifiers are evaluated using a **confusion matrix** to calculate metrics like accuracy, precision, recall, and F_1 score. Labeled data can be partitioned using holdout, cross-validation, or bootstrap methods.
- **Ensemble techniques** (such as bagging, boosting, and random forests) combine multiple base models to improve overall prediction accuracy.

Capítulo 8 - Data Mining

Cluster Analysis: Basic Concepts

Clustering is the process of grouping a set of data objects into multiple groups or clusters so that objects within a cluster have high similarity, but are dissimilar to objects in other clusters. Dissimilarities and similarities are assessed based on the attribute values describing the objects and often involve distance measures.

Overview of Cluster Analysis

✿ What Is Cluster Analysis?

Cluster analysis or simply clustering is the process of partitioning a set of data objects (or observations) into subsets. Each subset is a cluster, such that objects within a cluster are highly similar to one another, but highly dissimilar to objects in other clusters.

In the context of machine learning, clustering is an example of unsupervised learning. Unlike classification, clustering does not rely on a predefined set of class labels or a training set of labeled records. The objects are grouped based solely on information found in the data that describes the objects and their relationships.

Clustering can also be used for outlier detection.

✿ Applications of Cluster Analysis

Cluster analysis serves as both a standalone tool to gain insight into data distribution and a preprocessing step for other algorithms.

Major application areas include:

- **Business Intelligence & Marketing:** Segmenting a customer base into distinct groups for targeted marketing, or discovering distinct customer purchasing patterns.
- **Social Media Analysis:** Discovering communities, interest groups, or structural patterns within large social networks.
- **Biology & Medicine:** Taxonomy of plants and animals, isolation of functional gene structures with similar expression profiles, or identifying distinct sub-types of medical conditions.
- **Information Retrieval & Web Search:** Grouping thousands of search engine results into a small number of semantically meaningful thematic folders or taxonomies.
- **Data Summarization & Compression:** Using cluster prototypes (centroids) to represent large datasets, reducing storage requirements and optimizing nearest-neighbor lookups.

✿ Requirements for Cluster Analysis

To be effective in large real-world applications, a clustering algorithm must address the following highly challenging requirements:

1. **Scalability:** Handling datasets with millions of objects efficiently. Algorithms that work well on small samples may grind to a halt on massive databases.
2. **Ability to Deal with Different Types of Attributes:** The capability to analyze datasets containing complex mixtures of binary, nominal, ordinal, and numeric data types.
3. **Discovery of Clusters with Arbitrary Shape:** Moving beyond simple spherical shapes (common in center-based methods) to find long, drawn-out, or intertwined structural shapes.

4. Requirements for **Domain Knowledge** to Determine Input Parameters: Minimizing the need for users to input sensitive, hard-to-guess parameters (such as the exact number of clusters k).
5. Ability to **Deal with Noisy Data**: Real-world datasets contain missing, erroneous, or outlier values. Algorithms must be robust to this structural noise.
6. **Incremental Clustering** and Insensitivity to Input Order: Ensuring that the final cluster configuration does not radically shift based on the order in which data rows are fed into the system.
7. Capability of Clustering **High-Dimensional Data**: Dealing effectively with the curse of dimensionality, where traditional distance metrics lose meaning as the number of attributes grows.
8. **Constraint-Based Clustering**: Incorporating real-world operational constraints or background domain knowledge into the structural optimization process.
9. **Interpretability and Usability**: Ensuring that the final structural output is easily translatable into actionable insights or semantic schemas.

Basic Clustering Methods

To navigate the massive catalog of clustering algorithms, the textbook categorizes them into four **fundamental operational paradigms**:

✿ Partitioning Methods

Given a database of n objects, a partitioning method constructs k partitions of the data, where each partition represents a cluster and $k < n$. It must satisfy two objective criteria:

- Each group must contain at least one object.
- Each object must belong to exactly one group (note that this condition is relaxed in fuzzy partitioning frameworks).

The general technique operates via an **iterative relocation strategy**. It starts with an initial partitioning layout and **sequentially swaps elements between groups** to optimize a global objective function.

✿ Hierarchical Methods

A **hierarchical method** works by grouping data objects into a **tree-like hierarchy of clusters**. It can be executed via two directional philosophies:

- **Agglomerative (Bottom-up)**: Starts by treating every single object as its own isolated atomic cluster. It iteratively merges the closest pairs of clusters until all objects are unified under a single root node.
- **Divisive (Top-down)**: Begins with all objects contained within one monolithic cluster. It iteratively splits the dataset into smaller sub-clusters until every single object forms an isolated leaf node.

✿ Density-Based Methods

Most partitioning methods find it easy to isolate spherical or convex clusters, but struggle when confronted with irregular or intertwined spatial structures. **Density-based methods** overcome this limitation by modeling clusters as continuous, dense regions of points in space separated by regions of low density (which are classified as structural noise).

✿ Grid-Based Methods

Grid-based methods quantize the continuous multi-dimensional object space into a finite number of multi-dimensional cells that form a grid structure. All clustering operations are performed directly on this summarized grid architecture. The major advantage is its exceptional execution speed, which is independent of the absolute number of data objects and depends solely on the number of grid cells.

Partitioning Methods

✿ k-Means: A Centroid-Based Technique

The **k-means algorithm** is a centroid-based partitioning method. The parameter k must be specified explicitly by the user.

✿ The Centroid Optimization Problem

The algorithm defines the prototype of a cluster C_i using its **centroid**, which is the geometric mean of all objects within that cluster. The global quality of the partitioning is evaluated using the **Sum of Squared Error (SSE)**, also referred to as the Within-Cluster Variance:

$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(p, c_i)^2$$

Where:

- p is a multi-dimensional point in space.
- c_i is the mean centroid of cluster C_i .
- $\|p - c_i\|$ represents the standard Euclidean distance.

✿ The Algorithm

1. **Initialization**: Arbitrarily choose k objects from the dataset to serve as the initial cluster centers.
2. **Assignment**: Assign each remaining data object to its nearest cluster center based on Euclidean distance.
3. **Update**: Compute the new mean centroid vector for each individual cluster using its newly assigned member elements.
4. **Convergence Check**: Repeat the Assignment and Update steps until the centroids no longer change position, or the assignment of elements becomes completely stable.

✿ Computational Complexity

The time complexity of the k-means algorithm is $O(nkt)$

✿ Nuances, Limitations, and Fragilities

- **Sensitivity to Initialization**: k-means can easily get trapped in local optima. A bad selection of initial seeds can yield terrible cluster configurations. To combat this, practitioners use initialization enhancements like **k-means++**, which spaces out the initial centers probabilistically.
- **Vulnerability to Outliers**: Because it uses squared distances, a single extreme outlier can pull a cluster centroid far away from its true natural grouping.

- **Inability to handle non-convex shapes:** k-means is mathematically biased toward discovering spherical groupings of roughly uniform size and density. It fails when clusters have elongated or ring-like structures.

✿ **k-Medoids: A Representative Object-Based Technique**

To address k-means' vulnerability to outliers, the **k-medoid** algorithm replaces the abstract geometric mean vector with an **actual**, centrally-located data object chosen from the dataset, called a **medoid**.

✿ The Cost Function

The **quality** of a k-medoid partition is measured by the sum of absolute distances between every point and its respective cluster medoid:

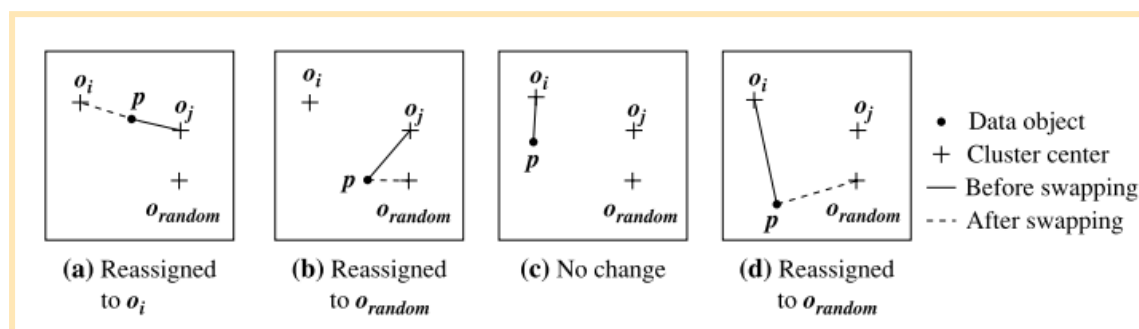
$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(p, o_i)$$

Where o_i is the designated representative object (medoid) of cluster C_i .

✿ The PAM (Partitioning Around Medoids) Algorithm

PAM (iterative and greedy way) is the foundational realization of the k-medoid approach:

1. Like the k-means algorithm, the initial representative objects are chosen arbitrarily.
2. We consider whether replacing a representative object by a nonrepresentative object would improve the clustering quality.
3. **All the possible replacements are tried out.**
4. The iterative process of replacing representative objects by other objects **continues** until the quality of the resulting clustering cannot be improved by any replacement.
5. This quality is measured by a cost function of the average dissimilarity



✿ Complexity Contrast

PAM is highly robust to noise and outliers because medoids are less affected by extreme values than centroids.

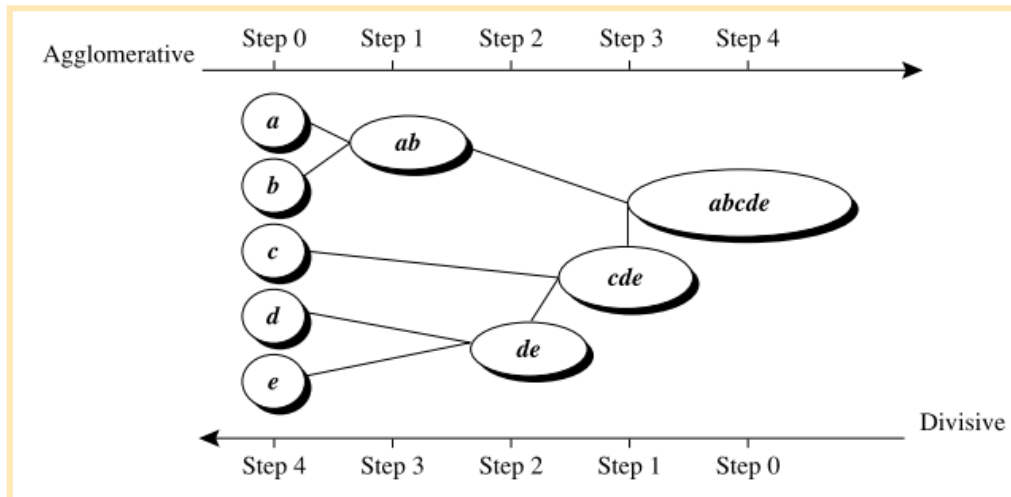
However, its computational cost is $O(k(n-k)^2)$ per iteration. This quadratic dependency on n makes PAM **highly inefficient for large datasets**. To scale k-medoid clustering to massive databases, advanced sampling algorithms like **CLARA** (Clustering Large Applications) and **CLARANS** (Clustering Large Applications based upon RANdomized Search) must be used.

Hierarchical Methods

✿ Agglomerative vs. Divisive Hierarchical Clustering

Hierarchical methods construct a **tree of clusters** called a **dendrogram**. A dendrogram illustrates how clusters are systematically split or merged at various proximity thresholds.

The user does not need to specify k upfront; instead, a desired partitioning configuration can be obtained by "cutting" the dendrogram horizontally at an appropriate distance threshold.



✿ Distance Measures Between Clusters

To decide which clusters should be merged (agglomerative) or split (divisive), we need a metric to evaluate the distance between two clusters C_i and C_j :

1. **Minimum Distance (Single Linkage)**: Measures the distance between the **two closest points** in the respective clusters. **Single-linkage algorithm**
Nuance: It can handle non-spherical shapes but suffers from chaining effects, where noisy data points act as a bridge and incorrectly merge two distinct clusters.
2. **Maximum Distance (Complete Linkage)**: Measures the distance between the **two furthest points** in the respective clusters. **Complete-linkage algorithm**
Nuance: Highly robust against noise, but strongly biased toward creating compact, tightly bounded spherical shapes.
3. **Mean Distance (Average Linkage)**: Computes the **average distance** between all possible pairs of points.
4. **Centroid Distance**: Computes the distance between the geometric centroids c_i and c_j of the two clusters.

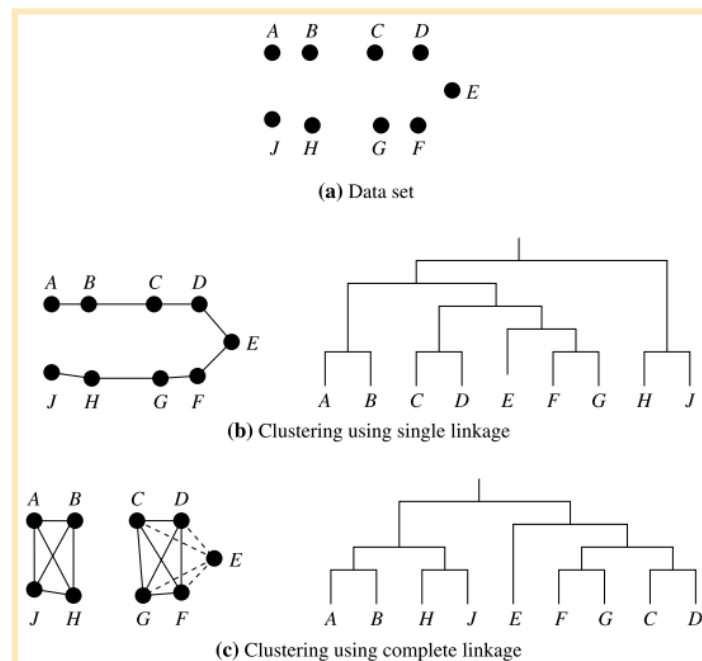
Minimum distance: $dist_{min}(C_i, C_j) = \min_{p \in C_i, p' \in C_j} \{\|p - p'\|\}$

Maximum distance: $dist_{max}(C_i, C_j) = \max_{p \in C_i, p' \in C_j} \{\|p - p'\|\}$

Mean distance: $dist_{mean}(C_i, C_j) = \|m_i - m_j\|$

Average distance: $dist_{avg}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i, p' \in C_j} \|p - p'\|$

Similar to the situation in the k-means method, single-linkage and complete-linkage methods are **sensitive to outliers**. The use of mean or average distance is a compromise between the minimum and maximum distances and overcomes the outlier sensitivity problem.



This example shows that by using single linkages we can find hierarchical clusters defined by local proximity, whereas complete linkage tends to find clusters opting for global closeness.

When we merge clusters in **agglomerative clustering**, we reduce the number of clusters.

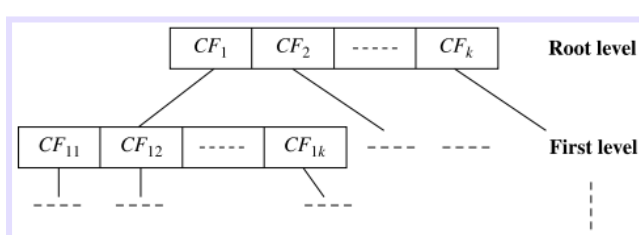
Which two clusters should we choose to merge?

Heuristically, we may want to merge two clusters so that the resulting clustering also minimizes the sum of squared errors, which is used as the criterion in partitioning methods like k-means.

The minimum spanning tree-based approach for divisive: Every cluster is a subset of nodes and is represented by the minimum spanning tree, which is a subtree of the minimum spanning tree of the whole data set.

✿ Algorithmic Enhancements for Scalability

Traditional hierarchical clustering has a steep computational complexity of at least $O(n^3)$, making it **impractical for large databases**. The textbook highlights three advanced scalable variations:



BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies): Introduces a highly compressed data structure called a **CF Tree (Clustering Feature Tree)**. A Clustering Feature is a summarized 3D vector tracking

basic cluster statistics: $CF = (n, LS, SS)$, where LS is the linear sum of the n points and SS is the square sum of the data points. BIRCH performs a fast linear scan to build this tree, reducing the memory footprint before running hierarchical operations on the summarized leaf components.

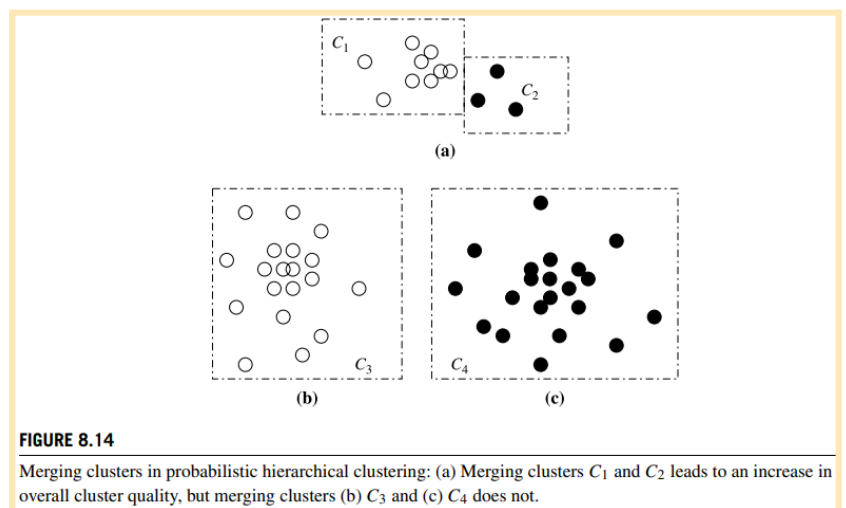
A CF-tree is a height-balanced tree that stores the clustering features for a hierarchical clustering.

Probabilistic Hierarchical Clustering

Algorithmic hierarchical clustering methods (such as single linkage, complete linkage, and BIRCH) rely on deterministic distance metrics to merge or split clusters. While efficient, these methods suffer from several key limitations:

- They lack a formal statistical foundation to evaluate the semantic quality or likelihood of the resulting tree structure.
- They struggle to handle data noise and overlapping clusters effectively.
- The choice of distance metric (e.g., minimum vs. maximum distance) is often arbitrary and can heavily bias the final shape of the clusters.

To overcome these challenges, probabilistic hierarchical clustering frames hierarchical clustering as a generative statistical modeling problem. Instead of measuring arbitrary spatial distances, it models the dataset using probabilistic mixture distributions (such as a Gaussian Mixture Model) at each level of the tree.



Density-Based Methods

Most of the partitioning and hierarchical methods are designed to find spherical-shaped clusters. They have difficulty finding clusters of arbitrary shape.

Density-based clustering algorithms locate clusters by identifying continuous, highly dense regions of points in space. This approach allows them to discover clusters of arbitrary shapes and filter out random background noise.

✿ DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN defines regional density based on two user-defined parameters:

- **epsilon(Eps)**: The maximum radius of the neighborhood around a point.

- **MinPts**: The minimum number of points required within that radius to form a dense region.

✿ Core Definitions and Classifications

- **epsilon-Neighborhood**: The volume of space surrounding an object within a radius of epsilon
- **Core Object**: An object whose epsilon-neighborhood contains at least MinPts objects
- **Border Object**: An object that is not a core object itself, but resides within the epsilon-neighborhood of a valid core object.
- **Noise Object**: Any object in the dataset that is neither a core object nor a border object.
- **Directly Density-Reachable**: An object q is directly density-reachable from a core object p if $d(p,q) \leq \epsilon$.
- **Density-Reachable**: An object q is density-reachable from an object p if there is a chain of intermediate core objects $p_1, p_2, \dots, p_n$ (where $p_1 = p$ and $p_n = q$) such that each element is directly density-reachable from the one preceding it.
- **Density-Connected**: Two distinct objects p and q are density-connected if there exists an intermediate anchor object o such that both p and q are density-reachable from o .

✿ The DBSCAN Core Process

1. Mark all objects in the database as unvisited.
2. Select an unvisited object p at random and check its epsilon-neighborhood.
3. If $|N_{\epsilon}(p)| > \text{MinPts}$, a new cluster is initiated. The algorithm expands this cluster by recursively discovering all density-reachable objects from p .
4. If p is not a core object, it is temporarily labeled as noise.
5. Repeat this process until every single object in the dataset has been visited and categorized.

✿ Strengths and Structural Weaknesses

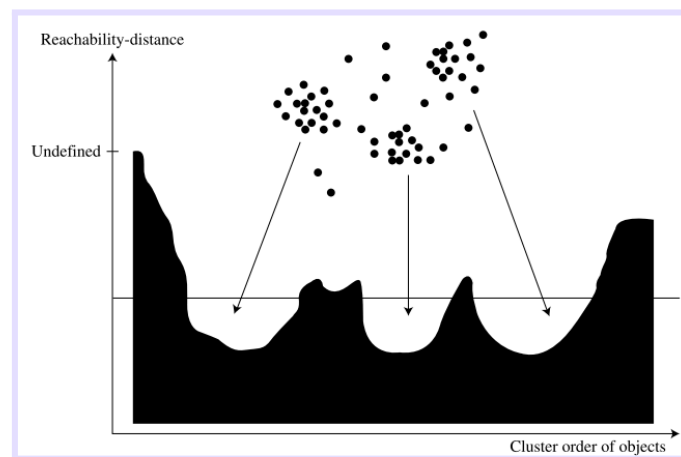
- **Advantage**: No need to specify the number of clusters k beforehand. It easily handles arbitrary shapes (like nested rings or crescent shapes) and filters out structural noise.
- **Limitation**: It fails when dealing with datasets that feature varying densities, as a single global pair of epsilon and MinPts cannot effectively capture both sparse and highly dense clusters simultaneously.

✿ OPTICS: Ordering Points To Identify the Clustering Structure

To overcome DBSCAN's difficulty with varying densities, OPTICS does not generate an explicit, rigid cluster partition. Instead, it computes an augmented cluster ordering for the dataset. It tracks two key metrics for each object:

- **Core-Distance**: The smallest distance ϵ' such that the ϵ' -neighborhood of an object contains at least MinPts points. If the point is not a core object at the maximum radius epsilon, its core-distance is undefined.
- **Reachability-Distance**: The reachability-distance of an object p relative to another core object o is the larger of either the core-distance of o , or the absolute Euclidean distance between p and o :

Using these metrics, OPTICS organizes the data linearly so that spatially close points are adjacent in the ordering. This structure allows practitioners to visualize and extract clusters of varying densities using a reachability plot.



✿ DENCLUE: Density-based Clustering

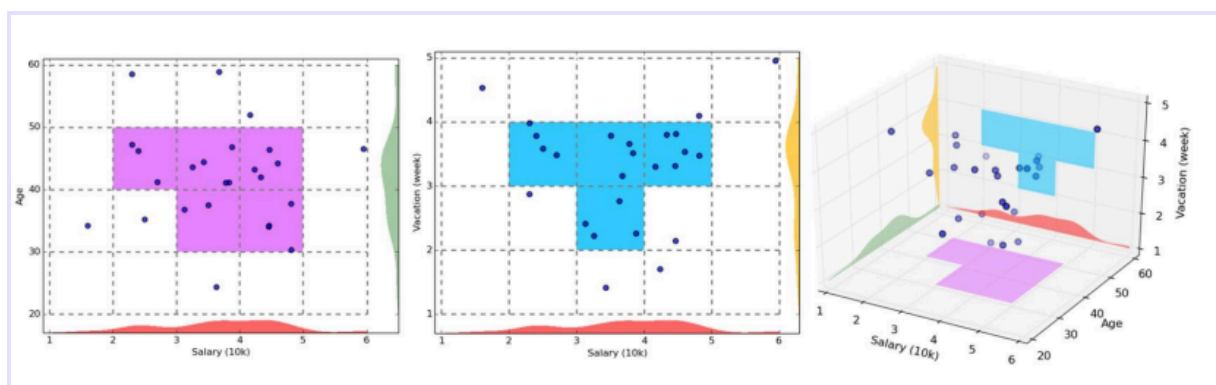
DENCLUE is an advanced clustering method based on a set of generalized density functions. It uses a rigorous mathematical framework built on kernel density estimation.

The overall density of any point in space is calculated as the sum of the influence functions of all data points around it. Clusters are identified by locating density attractors (local maxima of the global density function) using an optimization technique called hill-climbing.

DENCLUE is highly effective for high-dimensional data and can accurately model complex, mathematically precise cluster shapes.

Grid-Based Methods

Grid-based methods approach clustering from a space-partitioning perspective. Instead of performing distance computations between every pair of data objects, they divide the multi-dimensional search space into a finite grid of cells. All clustering calculations are then performed directly on these cell structures.



✿ STING: Statistical Information Grid

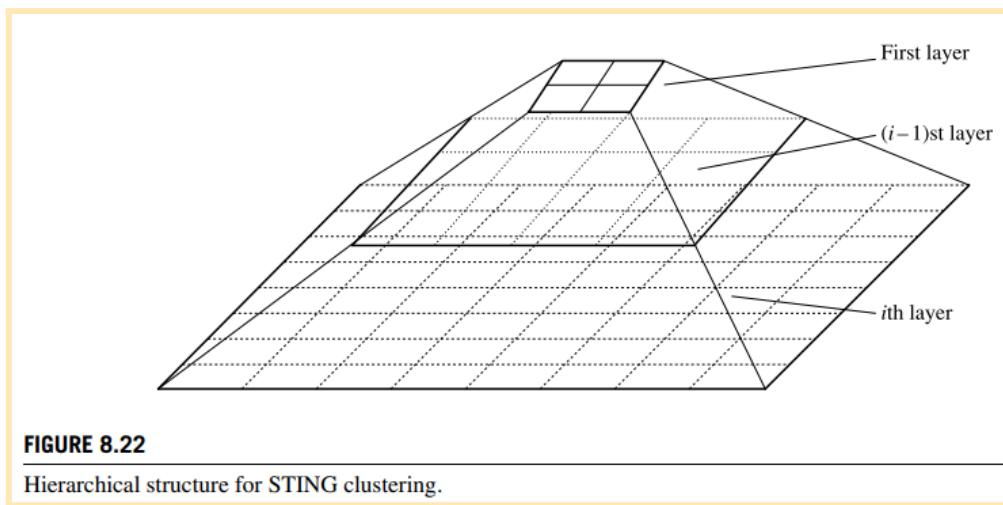
In STING, the spatial area is divided into a hierarchical structure of rectangular cells. There are multiple levels of these grid cells, where each higher-level cell is progressively partitioned into a set of smaller, lower-level child cells.

⌘ Statistical Parameters Maintained in Grid Cells

For each individual grid cell, STING computes and stores the following statistical parameters about the data points falling within its boundaries:

- **n**: The total number of points in the cell.
- **m**: The mean value of all points in the cell.
- **stdev**: The standard deviation of the points.
- **min**: The minimum attribute value within the cell.
- **max**: The maximum attribute value within the cell.
- **dist**: The type of distribution the data follows (e.g., normal, uniform, exponential, or unknown).

The parameters of higher-level cells can be calculated directly from the pre-computed parameters of their child cells, without reloading the raw data rows from disk.



⌘ Query Processing

To process a spatial data query, STING starts at the top level of the hierarchical grid architecture. Using the stored statistical parameters, it calculates a probability score for each cell to determine if it is relevant to the query. Cells that fall below a certain probability threshold are filtered out. This top-down traversal continues down to the lowest level of the grid. The remaining contiguous groups of relevant cells are then assembled to form the final return clusters.

⌘ **CLIQUE: Clustering In QUEst**

CLIQUE is a hybrid method that combines a grid-based approach with density-based concepts, designed specifically for clustering high-dimensional data. It utilizes an Apriori-like property to navigate high-dimensional spaces efficiently: *If a region is dense in a k -dimensional space, it must also be dense in all of its $(k-1)$ -dimensional projections.*

⌘ Algorithmic Execution Steps

1. **Identification of Subspaces That Contain Dense Clusters:** The algorithm discretizes each dimension into uniform intervals. It systematically scans the data to find dense

units, starting with 1-dimensional intervals and progressively building up to higher-dimensional dense units using the Apriori property.

2. **Identification of Clusters:** It connects adjacent dense units within the identified subspaces to form larger, contiguous dense regions, which represent the clusters.
3. **Generation of Minimal Descriptions:** For each cluster, CLIQUE computes a simplified, minimal bounding expression using hyper-rectangles to describe the cluster's boundaries to the user.

Evaluation of Clustering

Evaluating the quality of clustering results is often more challenging than evaluating classification accuracy because there are no definitive true class labels. This process is called **cluster validation** and relies on three main testing approaches.

✿ Assessing Clustering Tendency

Before applying any clustering algorithm to a dataset, we must first **determine** whether the data contains any **natural, non-random structural groupings**. If a dataset consists purely of uniformly distributed random points, forcing a clustering algorithm to run on it will still produce artificial groupings, which would be highly misleading.

CLUSTERING REQUIRES NONUNIFORM DISTRIBUTION OF DATA

✿ Hopkins Statistic

The **Hopkins Statistic** is a statistical test used to evaluate **spatial randomness**. The test is set up as follows:

1. Sample r points uniformly at random from the data space ($P_1, P_2, \dots, P_r$). For each sample point, compute its distance x_i to its nearest real neighbor in the original dataset.
2. Select r real data objects ($q_1, q_2, \dots, q_r$) at random from the dataset. For each object, compute its distance y_i to its nearest real neighbor in the dataset.
3. Calculate the Hopkins Statistic H :

$$H = \frac{\sum_{i=1}^n x_i^d}{\sum_{i=1}^n x_i^d + \sum_{i=1}^n y_i^d},$$

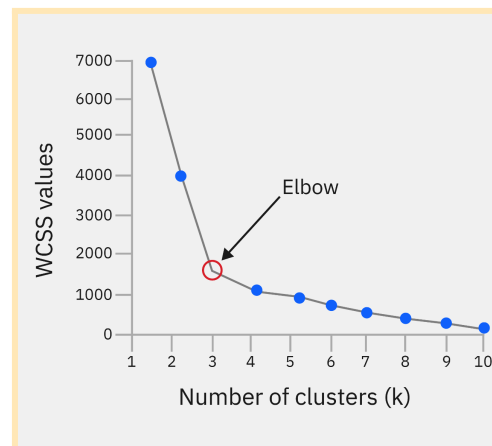
Interpretation:

- If H is close to 0.5, the data is **completely random** and uniformly distributed, meaning it has no clustering tendency.
- If H approaches 1.0, the data points are **highly clustered**, indicating a strong, non-random structural pattern.

✿ Determining the Number of Clusters (k)

For algorithms that require specifying the number of clusters k upfront, finding the optimal value is a key challenge. A common empirical technique is the **Elbow Method**.

Practitioners run the clustering algorithm multiple times, varying k from 1 up to a maximum value, and plot the resulting **Sum of Squared Errors (SSE)** for each run. As k increases, the SSE naturally decreases because the clusters become smaller and more compact. The optimal number of clusters is found at the "elbow" of the curve—the point where the rate of decrease drops sharply and flattens out, indicating diminishing returns for adding more clusters.



✿ Measuring Clustering Quality

Clustering quality can be assessed using either **internal metrics** (based solely on the data relationships) or **external metrics** (when independent ground-truth labels are available).

✿ External Measures

When ground-truth labels Y are available for evaluation, we compare our clustering results C against them using the following metrics:

- **BCubed Precision and Recall:** Evaluates precision and recall for each individual object in the dataset. The precision of an object is the proportion of its cluster companions that share its true class label. Its recall is the proportion of all objects with its true class label that are successfully grouped into its cluster.
- **Normalized Mutual Information (NMI):** An information-theoretic measure that quantifies the amount of shared information between the predicted cluster assignments and the true ground-truth classes, normalized to fall within a $[0, 1]$ range.

✿ Internal Measures

When no ground-truth labels are available, we must evaluate cluster quality using internal metrics that measure two key properties: **intra-cluster cohesion** (how tightly grouped the objects within a cluster are) and **inter-cluster separation** (how well-isolated distinct clusters are from each other).

The standard metric for this evaluation is the **Silhouette Coefficient**:

1. For an object i , calculate $a(i)$ as the average distance between i and all other objects within its same cluster.
2. Calculate $b(i)$ as the average distance between i and all objects in the closest neighboring cluster.
3. The Silhouette Coefficient $s(i)$ for object i is defined as:

$$s(o) = \frac{b(o) - a(o)}{\max\{a(o), b(o)\}}.$$

Interpretation:

- **s(i) approx +1**: Indicates that the object is well-clustered, as its distance to neighboring clusters is much larger than its distance to points within its own cluster.
- **s(i) approx 0**: Indicates that the object sits on the boundary between two clusters.
- **s(i) approx -1**: Indicates that the object has been assigned to the wrong cluster.

The overall quality of a clustering run is evaluated by computing the average Silhouette Coefficient across all objects in the dataset.